Introduction to Data Science in Non-Life Insurance

Pairote Satiracoo

Table of contents

Welcome to My Book **9**

1 Chapter 1: Introduction to Risk and Insurance **10**

Learning Outcomes . . . 10
1.1 Understanding Risk . . . 10
1.2 Perils and Hazards . . . 11
1.2.1 Peril . . . 11
1.2.2 Hazard . . . 12
1.2.3 Physical Hazard . . . 12
1.2.4 Moral Hazard . . . 12
1.2.5 Morale Hazard . . . 13
1.3 Components of Risk . . . 14
1.3.1 Peril . . . 14
1.3.2 Hazard . . . 14
1.3.3 Degree of Risk . . . 14
1.3.4 Uncertainty . . . 15
1.3.5 Relationship among the Components . . . 16
1.4 Classification of Risks . . . 16
1.4.1 Pure and Speculative Risks . . . 17
1.4.2 Particular and Fundamental Risks . . . 17
1.4.3 Static and Dynamic Risks . . . 18
1.4.4 Insurable and Uninsurable Risks . . . 18
1.5 Characteristics of Normally Insurable Risks . . . 18
1.5.1 Large Number of Homogeneous Exposure Units . . . 19
1.5.2 Accidental and Unintentional Loss . . . 19
1.5.3 Determinable and Measurable Loss . . . 19
1.5.4 Non-Catastrophic Loss . . . 20
1.5.5 Pure and Particular Risk . . . 20
1.5.6 Calculable Probability of Loss . . . 20
1.5.7 Financial Loss Requirement . . . 20
1.5.8 Reasonable Premium . . . 21
1.5.9 Risks Typically Not Insurable . . . 21
1.6 What Is Insurance? . . . 22
1.6.1 Risk Transfer . . . 22
1.6.2 Risk Pooling . . . 22
1.6.3 Indemnity . . . 23
1.6.4 Summary . . . 23
1.7 Insurance Mechanism . . . 24
1.7.1 Step 1: Risk Assessment . . . 24
1.7.2 Step 2: Premium Collection . . . 24

1.7.3 Step 3: Loss Occurrence and Claim Settlement . . . 24
1.7.4 Step 4: Risk Sharing . . . 25
1.7.5 The Insurance Mechanism . . . 25
1.7.6 Connection to Actuarial Science . . . 26
1.8 Insurance Contracts . . . 26
1.8.1 The Insured . . . 26
1.8.2 The Insurer . . . 26
1.8.3 The Beneficiary . . . 26
1.9 Benefits of Insurance . . . 27
1.10 Insurance in Thailand . . . 27
1.10.1 Overview of the Thai Insurance Industry . . . 27
1.10.2 Types of Insurance . . . 27
1.10.3 Characteristics of Life Insurance . . . 29
1.10.4 Characteristics of Non-Life Insurance . . . 29
1.10.5 Common Terminology in Life Insurance . . . 30
1.10.6 Insurance Regulation in Thailand . . . 30
1.11 Mathematical Foundations of Insurance . . . 31
1.11.1 Probability . . . 31
1.11.2 Law of Averages . . . 32
1.11.3 Law of Large Numbers . . . 32
1.11.4 Applications in Insurance . . . 33
1.11.5 Summary . . . 33
1.12 Insurance Premium Determination . . . 34
1.12.1 Principles of Premium Determination . . . 34
1.12.2 Components of an Insurance Premium . . . 35
1.12.3 Putting the Components Together . . . 37

2 Chapter 2: Introduction to Data Science 39
Learning Outcomes . . . 39
2.1 From Insurance to Data Science . . . 39
2.1.1 Data-Driven Insurance Decisions . . . 40
2.1.2 From Risk to Data . . . 40
2.1.3 Why Data Science Matters to Actuaries . . . 41
2.2 What Is Data Science? . . . 42
2.2.1 Data, Statistics, Computing and Domain Knowledge . . . 42
2.2.2 Data Science and Decision-Making . . . 44
2.3 The Role of Data in Insurance . . . 45
2.3.1 Insurance Data Sources . . . 45
2.3.2 Types of Data . . . 46
2.3.3 An Insurance Dataset . . . 47
2.4 The Data Science Life Cycle . . . 49
2.4.1 Define the Problem . . . 49
2.4.2 Acquire the Data . . . 50
2.4.3 Prepare the Data . . . 51
2.4.4 Explore the Data . . . 51
2.4.5 Model the Data . . . 52
2.4.6 Communicate the Results . . . 53
2.4.7 Deploy and Monitor . . . 53

2.5 Roles in a Data Science Team . . . 54
2.5.1 Data Analyst . . . 55
2.5.2 Data Scientist . . . 55
2.5.3 Data Engineer . . . 55
2.5.4 The Actuary in a Data Science Team . . . 56
2.6 Applications of Data Science in Insurance . . . 57
2.6.1 Pricing and Underwriting . . . 57
2.6.2 Claims Management . . . 58
2.6.3 Fraud Detection . . . 58
2.6.4 Customer Analytics . . . 59
2.6.5 Risk Management and Forecasting . . . 60
2.6.6 From Data to Insurance Decisions . . . 60

3 Chapter 3: Transforming and Summarizing Data 62
Learning Outcomes . . . 62
3.1 Why Data Transformation Matters . . . 62
3.2 Inspecting Data . . . 64
3.2.1 Observations and variables . . . 65
3.2.2 Variable types . . . 65
3.2.3 Inspecting the values . . . 65
3.2.4 Tidy data . . . 66
3.2.5 From inspection to transformation . . . 69
3.3 The Claims Experience Dataset . . . 69
3.3.1 Importing the data . . . 69
3.3.2 Inspecting the data . . . 70
3.3.3 Description of the rating variables . . . 71
3.4 Selecting and Filtering Data . . . 71
3.4.1 Selecting variables . . . 71
3.4.2 Filtering observations . . . 72
3.4.3 Selecting from a set of values . . . 75
3.4.4 Combining selection and filtering . . . 76
3.4.5 Why selection and filtering matter in actuarial analysis . . . 77
3.5 Creating and Transforming Variables . . . 77
3.5.1 Creating a new variable . . . 78
3.5.2 Transforming an existing variable . . . 79
3.5.3 Creating actuarial quantities . . . 79
3.5.4 Checking the result . . . 80
3.5.5 Transforming variables for analysis . . . 82
3.5.6 Transforming data as part of the workflow . . . 82
3.6 Combining Operations with Pipes . . . 83
3.7 Grouping and Summarizing Data . . . 85
3.7.1 Grouping observations . . . 86
3.7.2 Grouping by rating variables . . . 87
3.7.3 Summarizing insurance experience . . . 88
3.8 Insurance Summaries . . . 91
3.8.1 Claim count . . . 91
3.8.2 Aggregate claim amount . . . 91
3.8.3 Claim severity . . . 92

3.8.4 Exposure and claim frequency . . . 93
3.8.5 Frequency, severity, and risk premium . . . 93
3.9 Interpreting Insurance Summaries . . . 94
3.9.1 Comparing groups . . . 94
3.9.2 Experience and volume . . . 95
3.9.3 Claim count is not frequency . . . 96
3.9.4 Observed differences are not necessarily causal . . . 97
3.9.5 From summaries to actuarial judgment . . . 97

4 Chapter 4: Tidy Data **99**
Learning Outcomes . . . 99
4.1 Tidy Data . . . 99
4.1.1 Why Tidy Data Matters . . . 100
4.2 Common Forms of Untidy Data . . . 100
4.2.1 Column Headers Are Values . . . 101
4.2.2 Multiple Variables Stored in One Column . . . 102
4.2.3 Multiple Variables Stored in Column Names . . . 103
4.2.4 Variables Stored in Both Rows and Columns . . . 105
4.2.5 Recognising the Structure Before Reshaping . . . 106
4.3 Introducing `tidyr` . . . 106
4.3.1 From Wide to Long with `pivot_longer()` . . . 107
4.3.2 From Long to Wide with `pivot_wider()` . . . 108
4.3.3 Choosing the Appropriate Function . . . 109
4.4 Working with the Sample Insurance Data . . . 110
4.4.1 Claims Recorded by Year . . . 111
4.4.2 Policy Measures . . . 113
4.4.3 Claims by Year and Measure . . . 114
4.4.4 Portfolio Summary . . . 116
4.5 From Tidy Data to `dplyr` . . . 119

5 Chapter 5: Data Visualization with ggplot2 **121**
Learning Outcomes . . . 121
5.1 Why Data Visualization Matters . . . 121
5.2 Building Plots with ggplot2 . . . 122
5.2.1 A first plot . . . 122
5.2.2 Aesthetic mappings . . . 123
5.2.3 Geometric objects . . . 125
5.2.4 Layers and plot components . . . 126
5.2.5 Understanding the main components . . . 127
5.3 Visualizing Categorical Variables . . . 128
5.3.1 Bar charts of counts . . . 128
5.3.2 Comparing counts across groups . . . 129
5.3.3 Comparing proportions . . . 130
5.3.4 Comparing proportions across years . . . 132
5.3.5 Counts and proportions: choosing the appropriate display . . . 133
5.4 Visualizing Quantitative Distributions . . . 134
5.4.1 Histograms . . . 134
5.4.2 Histograms of claim amounts . . . 137

5.4.3 Density plots . . . 140
5.4.4 Boxplots . . . 142
5.4.5 Potential outliers and data errors . . . 143
5.4.6 Choosing a suitable plot . . . 145
5.5 Visualizing Relationships Between Variables . . . 145
5.5.1 Scatterplots . . . 145
5.5.2 Adding Aesthetic Mappings . . . 146
5.5.3 Adding Smooth Trend Lines . . . 147
5.5.4 Improving Plots with Skewed Data . . . 149
5.5.5 Nonlinear Relationships and Overplotting . . . 150
5.5.6 Interpreting Relationships in Actuarial Data . . . 151
5.6 Comparing Groups with Plots . . . 151
5.6.1 Comparing Distributions with Boxplots . . . 151
5.6.2 Improving Comparisons for Right-Skewed Outcomes . . . 152
5.6.3 Comparing Claim Frequency Across Groups . . . 154
5.6.4 Comparing Groups with Colour . . . 156
5.6.5 Using Facets to Separate Groups . . . 157
5.6.6 Creating a Grouping Variable from Indicator Columns . . . 160
5.6.7 Reshaping Wide Data into Long Format . . . 162
5.6.8 Choosing a Suitable Plot for Group Comparisons . . . 165
5.7 Scales, Labels, and Presentation . . . 166
5.7.1 Labels and Titles . . . 166
5.7.2 Legends and Aesthetic Scales . . . 167
5.7.3 Logarithmic Scales . . . 171
5.7.4 Themes and Presentation . . . 172
5.7.5 Coordinate Transformations . . . 174
5.7.6 Principles for Effective Actuarial Visualizations . . . 175
5.8 Actuarial Exploratory Visualization . . . 176
5.8.1 Exploring Claim Frequency Across Years . . . 176
5.8.2 Exploring Claim Severity . . . 179
5.8.3 Exploring the Relationship Between Coverage and Claim Amount . . . 181
5.8.4 From Visualization to Modelling . . . 183

6 Chapter 6: Linear Regression with tidymodels 184
Learning Outcomes . . . 184
6.1 Introduction . . . 184
6.1.1 Why `tidymodels`? . . . 185
6.2 Insurance Charges Data . . . 186
6.2.1 Data Visualization, Interpretation, and Feature Selection . . . 187
6.2.2 What Have We Learned? . . . 192
6.2.3 Exercises . . . 192
6.2.4 Solutions to Exercises . . . 193
6.2.5 Exercise: Examining the Distribution of Insurance Charges . . . 199
6.3 Build and Fit a Model . . . 201
6.3.1 Specifying and Fitting a Linear Regression Model for Insurance Data . . . 201
6.3.2 Use a Model to Predict Insurance Charges with Confidence Intervals . . . 208
6.3.3 Visualizing Model Predictions . . . 210
6.3.4 Exercise: Visualizing Model Predictions . . . 211

6.3.5 Exercise: Observed and Predicted Charges . . . 212
6.3.6 From Simple to Multiple Linear Regression . . . 213

7 Chapter 7: Assessing Model Performance 215
7.1 Learning Outcomes . . . 215
7.2 Introduction to Model Performance . . . 215
7.3 Training Performance and Test Performance . . . 216
7.3.1 Splitting Data with `rsample` . . . 217
7.4 Measuring Prediction Error . . . 218
7.4.1 Absolute Error . . . 219
7.4.2 Squared Error . . . 219
7.5 Common Regression Performance Metrics . . . 220
7.5.1 RMSE — Root Mean Squared Error . . . 220
7.5.2 MAE — Mean Absolute Error . . . 221
7.5.3 R^2 — Coefficient of Determination . . . 221
7.5.4 MAPE — Mean Absolute Percentage Error . . . 222
7.5.5 Comparing the Metrics . . . 222
7.6 Understanding R^2 . . . 222
7.7 Evaluating Model Predictions with `yardstick` . . . 224
7.7.1 Calculating RMSE . . . 225
7.7.2 Calculating MAE . . . 226
7.7.3 Calculating R^2 . . . 226
7.7.4 Calculating MAPE . . . 226
7.8 Computing Multiple Metrics with `metric_set()` . . . 227
7.9 Interpreting Model Performance . . . 229
7.10 Comparing Two Models . . . 230
7.11 Why Test Performance Matters . . . 233
7.12 From Model Evaluation to a Modelling Workflow . . . 234

8 Chapter 8: Preprocessing and Recipes 235
8.1 Introduction to Preprocessing and Recipes . . . 235
8.1.1 The Insurance Dataset . . . 235
8.1.2 Why Use Recipes? . . . 236
8.2 Preparing Data Before Creating a Recipe . . . 237
8.2.1 Removing Duplicate Observations . . . 237
8.2.2 Checking for Missing Values . . . 238
8.3 Creating and Understanding Recipes . . . 239
8.3.1 Specifying a Recipe . . . 239
8.3.2 Understanding `prep()` and `bake()` . . . 240
8.3.3 Selecting Variables in Recipes . . . 241
8.4 Handling Missing Values . . . 242
8.4.1 Creating a Dataset with Missing Values . . . 243
8.4.2 Specifying Imputation Steps in a Recipe . . . 244
8.4.3 Preparing the Recipe with `prep()` . . . 245
8.4.4 Applying Imputation with `bake()` . . . 246
8.4.5 Applying the Same Recipe to Test Data . . . 247
8.5 Preprocessing Numeric Predictors . . . 249
8.5.1 Why Normalize Numeric Predictors? . . . 249

8.5.2 Which Models Need Normalization? . . . 249
8.5.3 Normalizing Predictors with `step_normalize()` . . . 250
8.5.4 Why Must Normalization Be Learned from the Training Data? . . . 251
8.6 Preprocessing Categorical Predictors . . . 252
8.6.1 Creating Dummy Variables with `step_dummy()` . . . 252
8.6.2 Handling New Categorical Levels . . . 253
8.6.3 Practical Considerations . . . 254
8.7 Combining Preprocessing Steps into a Recipe . . . 254
8.7.1 Preparing and Applying the Recipe . . . 255
8.8 Exercise: Preprocessing with Recipes . . . 255
8.8.1 Tasks . . . 255
8.8.2 Expected Outcome . . . 256
8.9 Building Workflows with tidymodels . . . 256
8.9.1 Why Combine Preprocessing and Modelling? . . . 256
8.9.2 Creating a Workflow . . . 256
8.9.3 Example: Building a Linear Regression Workflow . . . 257
8.9.4 Fitting the Workflow . . . 258
8.9.5 9.5 Generating Predictions with `predict()` . . . 258
8.9.6 Key Idea . . . 259
8.10 9. Using Workflows for Predictive Modelling . . . 259
8.10.1 Fitting a Workflow on Training Data . . . 260
8.10.2 Generating Predictions on Test Data . . . 260
8.10.3 Assessing Predictive Performance . . . 261
8.10.4 Exercise: Fit a Workflow and Predict Test Data . . . 262

Welcome to My Book

1 Chapter 1: Introduction to Risk and Insurance

Learning Outcomes

After studying this chapter, students should be able to:

1. Define the fundamental concepts of risk, peril, hazard, insurance, and insurance premiums.
2. Identify the main types of insurance, the parties to an insurance contract, and the characteristics of normally insurable risks.
3. Explain the classifications of risk and the principles of risk transfer, risk pooling, and indemnity.
4. Describe the insurance mechanism, the benefits of insurance, and the structure and regulation of the insurance industry in Thailand.
5. Explain the roles of probability, the law of averages, and the law of large numbers in insurance.
6. Classify risks according to their characteristics and determine whether they are normally insurable.
7. Interpret the basic components and principles of insurance premium determination in simple insurance scenarios.

In everyday life, individuals, businesses, and governments face uncertainty regarding future events. Some of these events may result in financial losses, while others may create opportunities. Insurance is one of the most important mechanisms for managing the financial consequences of uncertain losses by pooling risks across many policyholders.

This chapter introduces the concept of risk, the causes of loss, the characteristics of insurable risks, and the role of insurance in modern society.

1.1 Understanding Risk

Risk refers to the possibility that the actual outcome of an event differs from the expected outcome. In insurance, the focus is primarily on outcomes that lead to financial loss.

More formally, **risk** is the uncertainty surrounding future outcomes together with the possibility that these outcomes may cause adverse financial consequences.

Two perspectives of risk are commonly encountered.

- **Negative perspective (downside risk):** Risk represents the possibility of loss or harm.
- **Positive perspective (upside risk):** Risk also creates opportunities for gain when uncertainty is managed appropriately.

Insurance primarily deals with **downside risk**, whereas investment and business decisions often consider both downside and upside risk.

> ❗ Risk versus Uncertainty
>
> The terms *risk* and *uncertainty* are often used interchangeably, but they are not identical.
>
> - **Risk** refers to uncertainty that can be measured or estimated using probability.
> - **Uncertainty** refers to situations where probabilities are unknown or difficult to estimate.
>
> Insurance relies on measurable risks rather than complete uncertainty.

Example 1.1 (Negative and Positive Perspectives of Risk)**.** Consider a farmer deciding whether to plant a new drought-resistant crop.

- From a **negative perspective**, the farmer may suffer financial losses if the crop fails.
- From a **positive perspective**, adopting the new crop may increase future profits if market demand is high.

Insurance can protect the farmer against certain losses (such as crop damage due to insured perils), but it cannot guarantee higher profits.

1.2 Perils and Hazards

A loss does not occur randomly without a cause. In insurance, it is useful to distinguish between the **peril** that directly causes a loss and the **hazards** that increase the likelihood or severity of that loss.

1.2.1 Peril

A **peril** is the direct cause of a loss. It is the event that triggers damage or financial loss.

Common examples of perils include

- fire,
- lightning,
- flood,
- windstorm,
- theft,
- motor vehicle collision, and
- earthquake.

For example, if a house is damaged by a fire, the **fire** is the peril.

1.2.2 Hazard

A **hazard** is a condition or circumstance that increases either

- the probability that a loss will occur (**loss frequency**), or
- the expected size of the loss (**loss severity**).

A hazard does **not** directly cause a loss. Instead, it makes a peril more likely to occur or increases the damage if the peril occurs.

Hazards are commonly classified into three categories.

1.2.3 Physical Hazard

A **physical hazard** is a tangible characteristic of a person, property, or environment that increases the likelihood or severity of loss.

Examples include

- faulty electrical wiring,
- worn vehicle tyres,
- storing flammable materials improperly,
- poor building maintenance, and
- slippery floors.

For example, faulty electrical wiring increases the likelihood of a fire occurring.

1.2.4 Moral Hazard

A **moral hazard** arises when a person intentionally behaves dishonestly because insurance provides financial protection.

Examples include

- intentionally damaging insured property,
- staging a motor vehicle accident,
- inflating the value of stolen property, and
- submitting fraudulent insurance claims.

Because these actions are intentional, moral hazards increase expected losses and are an important concern in underwriting and claims management.

1.2.5 Morale Hazard

A **morale hazard** arises when a person becomes less careful because insurance reduces the financial consequences of a loss. Unlike a moral hazard, there is **no intention to commit fraud**.

Examples include

- leaving a vehicle unlocked,
- neglecting routine maintenance of insured property,
- failing to install recommended safety devices, and
- driving less cautiously because comprehensive insurance is available.

Morale hazards often increase the frequency of accidental losses.

! Distinguishing Moral and Morale Hazards

Although their names are similar, **moral hazard** and **morale hazard** describe different behaviours.

- A **moral hazard** involves **intentional dishonesty** or fraud.
- A **morale hazard** involves **carelessness or indifference**, but without fraudulent intent.

This distinction is important because insurers use different methods to manage each type of hazard.

i Cause-and-Effect Relationship

The relationship among these concepts can be summarized as

$$\text{Hazard} \longrightarrow \text{Peril} \longrightarrow \text{Loss}.$$

A hazard increases the likelihood or severity of a peril, while the peril is the event that directly causes the loss.
For example,

- **Hazard:** Faulty electrical wiring
- **Peril:** Fire
- **Loss:** Damage to the building and its contents

Example 1.2 (Identifying Perils and Hazards)**.** Consider the following situations.

Situation	Peril	Hazard
A warehouse catches fire because of faulty wiring.	Fire	Faulty electrical wiring (physical hazard)
A policyholder deliberately burns an insured building to receive compensation.	Fire	Intentional fraud (moral hazard)
A driver leaves a car unlocked because it is fully insured, and the car is stolen.	Theft	Carelessness due to insurance (morale hazard)

Situation	Peril	Hazard
Heavy rainfall causes a river to overflow and damage nearby houses.	Flood	Houses located in a flood-prone area (physical hazard)

In each case, the **peril** is the immediate cause of the loss, while the **hazard** is the condition that increases the likelihood or severity of that loss.

1.3 Components of Risk

Insurers assess a risk by considering several related components. Together, these components help answer four fundamental questions:

Question	Component
What can cause a loss?	Peril
What makes the loss more likely or more severe?	Hazard
How often and how large might losses be?	Degree of risk (frequency and severity)
How certain are these estimates?	Uncertainty

This framework forms the basis of underwriting and premium determination.

1.3.1 Peril

The **peril** identifies the event that may result in a loss. Recognising the relevant peril helps insurers determine the appropriate policy coverage, exclusions, and methods of loss prevention.

1.3.2 Hazard

The **hazard** describes the conditions that influence the likelihood or severity of the peril. During underwriting, insurers evaluate hazards to estimate the expected level of risk and determine appropriate premiums or policy conditions.

1.3.3 Degree of Risk

The **degree of risk** describes the overall level of risk associated with a particular exposure. It is commonly assessed using two measures:

- **Frequency:** How often losses are expected to occur.
- **Severity:** The average financial impact of each loss.

These two measures provide valuable information for insurers because risks with similar expected losses may have very different patterns of claims.

Frequency	Severity	Typical Example
High	Low	Minor motor insurance claims
Low	High	Large industrial fire
High	High	Poorly maintained commercial building
Low	Low	Theft of inexpensive household items

For example, a portfolio of motor insurance policies may generate many small claims each year, whereas a commercial property portfolio may experience only a few claims but with very large losses.

1.3.4 Uncertainty

Even after identifying the peril, hazard, frequency, and severity, insurers cannot predict exactly

- whether a loss will occur,
- when it will occur, or
- how large the loss will be.

This lack of certainty is referred to as **uncertainty**.

Insurance companies reduce uncertainty by analysing historical claims data, identifying patterns, and applying statistical and actuarial methods. However, uncertainty can never be eliminated completely.

> **!** Risk Has More Than One Dimension
>
> Two risks may have the same expected loss but require different premiums because their **frequency**, **severity**, or **uncertainty** differs.
> Understanding these dimensions is fundamental to underwriting, pricing, and risk management.

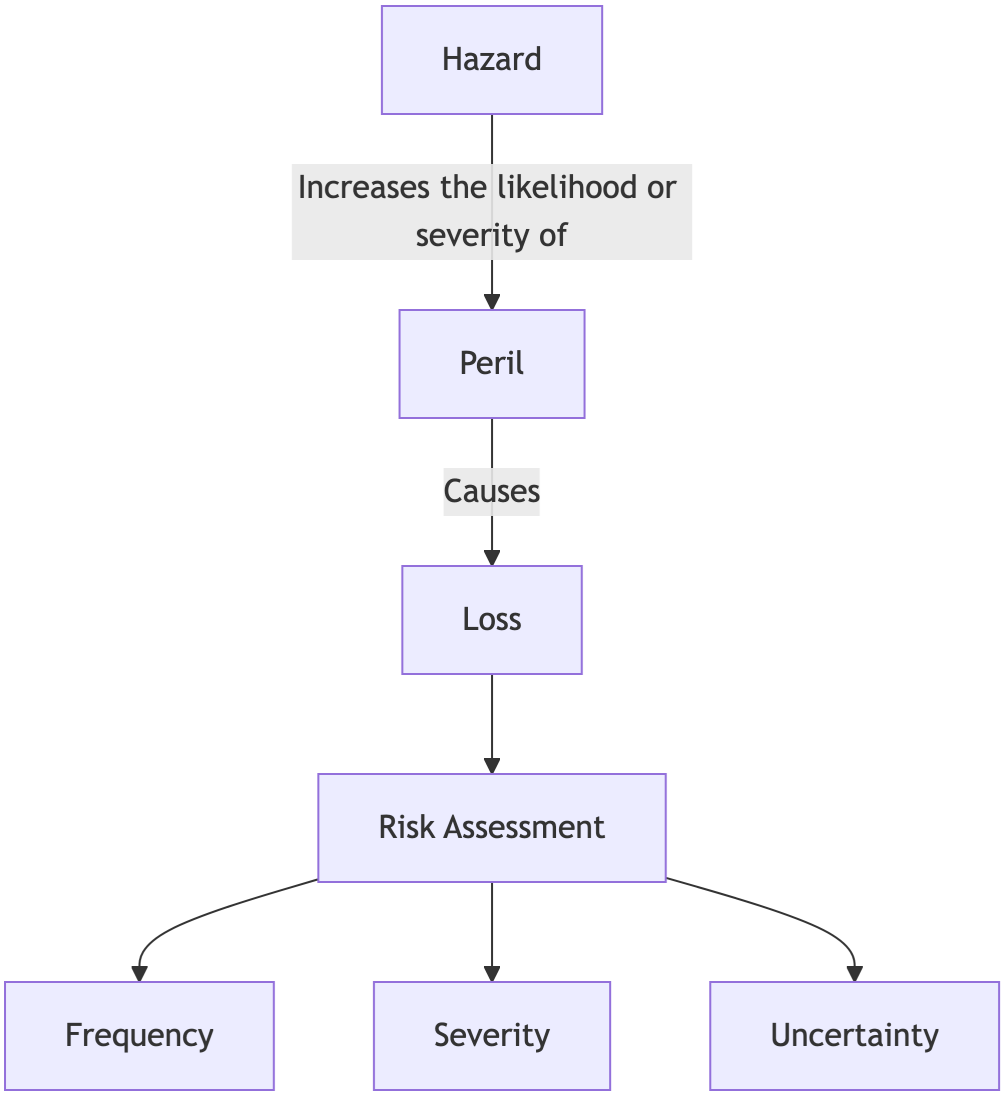
Example 1.3 (Analysing the Components of Risk)**.** Consider a homeowner living in an area prone to flooding.

Component	Description
Peril	Flood
Hazard	House located near a river with inadequate flood protection
Frequency	Flooding is expected once every 10 years
Severity	Each flood may cause substantial damage to the property
Uncertainty	The exact timing and extent of the next flood cannot be predicted

Although the homeowner knows that flooding is possible, the occurrence and magnitude of the next loss remain uncertain. Insurers use historical flood data and catastrophe models to estimate the likelihood and potential cost of future claims.

1.3.5 Relationship among the Components

The components of risk are closely related and can be viewed as a sequence:



The relationship between Hazards, Perils, and Risk Assessment.

Figure 1.1

This framework highlights how insurers assess risks. They identify the **peril**, evaluate the **hazards**, estimate the **frequency** and **severity** of potential losses, and quantify the remaining **uncertainty** before deciding whether to insure the risk and how much premium to charge.

1.4 Classification of Risks

Not all risks have the same characteristics. Classifying risks helps insurers understand the nature of potential losses, determine whether a risk can be insured, and select appropriate methods of risk

management. Several common classifications are used in insurance.

1.4.1 Pure and Speculative Risks

A **pure risk** is a situation in which the possible outcomes are either **loss** or **no loss**. There is no opportunity to make a profit from the occurrence of the risk. Most risks encountered in personal and commercial insurance, such as fire, theft, illness, and motor accidents, belong to this category.

In contrast, a **speculative risk** involves the possibility of either a **gain** or a **loss**. Investing in stocks, starting a business, or trading foreign currencies are examples of speculative risks. Although these activities may generate profits, they may also result in financial losses.

Because insurance is intended to compensate accidental losses rather than guarantee profits, private insurers generally insure **pure risks** but not **speculative risks**.

Example 1.4 (Pure and Speculative Risks)**.** The following situations illustrate the difference between pure and speculative risks.

Situation	Type of Risk
A house is damaged by fire.	Pure
A driver is involved in a motor accident.	Pure
An investor purchases company shares.	Speculative
A business launches a new product.	Speculative

1.4.2 Particular and Fundamental Risks

Risks may also be classified according to the number of people affected.

A **particular risk** affects a single individual, household, or business. Examples include the theft of a vehicle, damage to a factory caused by fire, or injuries resulting from a traffic accident. Since these losses are generally independent across policyholders, they can often be pooled effectively through insurance.

A **fundamental risk**, on the other hand, affects large groups of people or even entire economies. Examples include war, widespread inflation, pandemics, and severe economic recessions. Because many losses occur simultaneously, these risks are difficult for private insurers to diversify and often require government involvement or special insurance arrangements.

> **i** Why Fundamental Risks Are Difficult to Insure
>
> Insurance relies on pooling many independent risks. When a single event causes losses to a large number of policyholders at the same time, the insurer may face claims that exceed its financial capacity. This is one reason why risks such as war and widespread economic disruption are usually excluded from standard insurance policies.

1.4.3 Static and Dynamic Risks

Another useful classification distinguishes between **static** and **dynamic** risks.

A **static risk** exists regardless of changes in the economy or society. Many traditional insurance risks, such as fire, theft, and natural hazards, remain present over time even though their frequency may fluctuate.

A **dynamic risk** arises from changes in economic conditions, technological developments, social behaviour, or government policy. For example, advances in technology have increased exposure to cyber attacks, while climate change has altered the frequency and severity of certain natural hazards. Dynamic risks evolve over time and often require insurers to update their underwriting practices and pricing models.

1.4.4 Insurable and Uninsurable Risks

Not every risk can be transferred to an insurer. In general, an **insurable risk** has characteristics that allow its expected losses to be estimated with reasonable accuracy and its financial consequences to be shared among many policyholders.

Examples of insurable risks include damage to motor vehicles, residential property, and commercial buildings, as well as liability arising from accidents.

By contrast, **uninsurable risks** typically lack one or more of these characteristics. They may involve highly uncertain outcomes, affect many policyholders simultaneously, or depend primarily on business or investment decisions rather than accidental events. Examples include fluctuations in stock prices, changes in exchange rates, normal business competition, and reputational damage.

The characteristics of insurable risks will be discussed in greater detail later in this chapter.

> **!** Key Point
>
> The classifications introduced in this section are not independent. For example, most risks covered by general insurance are **pure**, **particular**, and **insurable**. Understanding these classifications helps explain why some risks are readily insured while others must be managed using alternative risk management techniques.

1.5 Characteristics of Normally Insurable Risks

Although many risks exist, not all of them are suitable for insurance. To operate sustainably, insurers must be able to estimate future losses with reasonable accuracy and charge premiums that are both fair and affordable. Consequently, a risk is normally considered **insurable** only if it possesses several desirable characteristics.

The following characteristics are commonly associated with risks that can be insured by private insurers.

1.5.1 Large Number of Homogeneous Exposure Units

Insurance relies on the **law of large numbers**, which states that the average outcome of many similar exposure units becomes more predictable as the number of observations increases.

To estimate future claims accurately, insurers require a large portfolio of policyholders with similar risk characteristics. The larger and more homogeneous the portfolio, the more reliable the estimates of claim frequency and severity.

For example, an insurer can estimate the expected number of motor insurance claims from thousands of similar vehicles more accurately than from only a few vehicles.

1.5.2 Accidental and Unintentional Loss

Insurance is designed to protect against losses that occur **fortuitously**, that is, by chance or accicent from the perspective of the insured.

If policyholders could deliberately cause insured losses, insurance would encourage fraud and become financially unsustainable. Consequently, losses resulting from intentional acts are generally excluded from insurance coverage.

> **!** Accidental Does Not Mean Unexpected
>
> An accidental loss does not necessarily mean that the event is impossible to anticipate. For example, insurers know that some houses will be damaged by fire each year. What is uncertain is **which** house will suffer the loss and **when** it will occur.

1.5.3 Determinable and Measurable Loss

For a claim to be settled fairly, the occurrence and amount of the loss must be objectively verifiable.

This means that insurers should be able to determine

- whether the loss occurred,
- when it occurred,
- what caused the loss, and
- the financial value of the loss.

For example, the repair cost of a damaged vehicle can usually be estimated by an assessor, whereas emotional distress is often much more difficult to measure objectively.

1.5.4 Non-Catastrophic Loss

Insurance works best when losses occur independently across policyholders. If a single event causes losses to a large proportion of insureds simultaneously, the insurer may be unable to meet all claim obligations.

Examples of catastrophic events include major earthquakes, widespread flooding, pandemics, and war.

Modern insurers manage catastrophic risks through geographical diversification, policy limits, and reinsurance. Nevertheless, some catastrophic risks remain difficult or impossible to insure in the private market.

1.5.5 Pure and Particular Risk

As discussed earlier, insurance primarily covers **pure risks**, where the possible outcomes are loss or no loss, rather than gain.

In addition, insured risks are usually **particular risks**, affecting individual policyholders rather than society as a whole. Risks such as house fires, theft, and motor accidents satisfy these characteristics and can therefore be pooled effectively.

By contrast, speculative investments and widespread economic recessions generally do not satisfy these conditions.

1.5.6 Calculable Probability of Loss

An insurer must be able to estimate the probability and expected cost of future losses using historical data, statistical analysis, and actuarial models.

Reliable estimation allows insurers to determine appropriate premiums and maintain adequate reserves for future claims.

When little or no historical data are available, estimating future losses becomes highly uncertain, making insurance difficult or prohibitively expensive.

1.5.7 Financial Loss Requirement

Insurance provides compensation only for **financial losses**. The insured must have an **insurable interest**, meaning that they would suffer a genuine financial loss if the insured event occurred.

For example, a homeowner has an insurable interest in their own house because its destruction would result in financial loss. However, the homeowner generally cannot insure a neighbour's house because they would not suffer a direct financial loss if it were damaged.

1.5.8 Reasonable Premium

Finally, the premium must be affordable relative to the protection provided.

If the expected loss is extremely high or highly uncertain, the premium required may become so expensive that few people are willing to purchase the insurance. In such cases, the risk is technically insurable but may not be commercially viable.

For example, a house located in an area with an exceptionally high probability of flooding may require a premium that exceeds what most homeowners are willing to pay.

Example 1.5 (Assessing Whether a Risk Is Insurable)**.** Consider the following situations.

Risk	Normally Insurable?	Reason
Fire damage to a residential house		Losses are accidental, measurable, and supported by extensive historical data.
Motor vehicle collision		Large numbers of similar exposure units allow reliable estimation of claims.
Loss from investing in company shares		This is a speculative risk with the possibility of gain or loss.
Losses caused by war		A single event may produce catastrophic losses affecting many policyholders simultaneously.
Damage to a neighbour's house		The policyholder has no insurable interest in the property.

This example illustrates that a risk is considered insurable only when it satisfies several characteristics simultaneously, rather than any single criterion alone.

1.5.9 Risks Typically Not Insurable

Some risks do not satisfy the characteristics described above and are therefore not normally covered by general insurers.

Market risk arises from fluctuations in market prices, such as changes in stock prices or commodity prices. **Financial risk** includes changes in interest rates, exchange rates, or credit conditions. **Production risk** refers to uncertainties associated with business operations, including changes in demand, production costs, or technological failures. **Political risk** arises from government actions or political instability, such as changes in legislation, expropriation of assets, or civil unrest.

These risks are generally excluded because they often possess one or more of the following characteristics:

- they are speculative rather than pure risks;
- they may affect many policyholders simultaneously, making diversification difficult;
- reliable historical data may be limited or unavailable;
- losses are difficult to measure objectively; or
- insuring such risks may create undesirable incentives or conflict with public policy.

Not Absolutely Uninsurable

Some risks traditionally regarded as uninsurable can be transferred through specialised insurance products or alternative risk transfer mechanisms. For example, political risk insurance, trade credit insurance, and weather derivatives are available in certain markets, although they typically involve specialised underwriting and pricing.

1.6 What Is Insurance?

Insurance is a financial arrangement in which an insurer agrees to compensate an insured for specified financial losses in exchange for the payment of a premium. By transferring individual risks to an insurer and pooling similar risks across many policyholders, insurance reduces the financial consequences of uncertain future events.

Insurance does not prevent losses from occurring. Instead, it provides financial protection by replacing the possibility of a large and uncertain loss with a relatively small and predictable premium.

Three fundamental principles underpin the operation of insurance: **risk transfer**, **risk pooling**, and **indemnity**.

1.6.1 Risk Transfer

The primary purpose of insurance is **risk transfer**. Individuals and businesses face many risks that could result in substantial financial losses. By purchasing insurance, they transfer the financial consequences of specified risks to an insurer in exchange for a premium.

For example, a homeowner purchases fire insurance to protect against the financial consequences of damage caused by fire. Although the risk of fire remains, the insurer assumes responsibility for compensating covered losses according to the policy terms.

1.6.2 Risk Pooling

Insurance operates by **pooling risks** from many policyholders who are exposed to similar types of risk. Each policyholder contributes a relatively small premium to a common fund. Since only a small proportion of policyholders are expected to experience losses during any policy period, the accumulated premiums are sufficient to compensate those who suffer covered losses.

Risk pooling enables uncertain and potentially large individual losses to be shared across many participants. The effectiveness of this mechanism depends on having a sufficiently large number of similar exposure units, allowing insurers to estimate future losses with reasonable accuracy.

Why Does Risk Pooling Work?

Insurance does not require every policyholder to experience a loss. Instead, premiums collected from the many who do **not** suffer losses are used to compensate the relatively few who do.

This principle enables insurers to spread financial losses across a large group of policyholders.

1.6.3 Indemnity

Most general insurance contracts are based on the **principle of indemnity**. The objective is to restore the insured, as nearly as possible, to the same financial position they occupied immediately before the loss.

Consequently, insurance is intended to compensate for financial loss rather than provide an opportunity for profit. If a vehicle worth THB 600,000 is completely destroyed in an accident, the insurer compensates the insured for the financial value of the loss, subject to the policy terms and conditions. The insured should not receive compensation exceeding the actual loss.

Certain types of insurance, such as life insurance and personal accident insurance, are **not** contracts of indemnity because the amount payable is specified in advance rather than determined by the actual financial loss.

! Insurance Does Not Eliminate Risk

Insurance transfers the **financial consequences** of specified losses; it does not eliminate the underlying risk itself. For example, purchasing fire insurance does not reduce the probability that a fire will occur. Instead, it protects the insured against the financial impact of a covered fire loss.

Example 1.6 (Insurance in Practice)**.** Suppose a homeowner purchases fire insurance for a house valued at THB 5 million.

If no fire occurs during the policy period, the homeowner receives no claim payment but benefits from financial protection throughout the year.

If a covered fire damages the house, the insurer compensates the homeowner according to the policy terms. The financial burden of the loss is therefore transferred from the homeowner to the insurer, while the cost of providing this protection is shared among many policyholders through risk pooling.

1.6.4 Summary

Insurance can be viewed as a system based on three interconnected principles:

Principle	Role in Insurance
Risk transfer	Transfers the financial consequences of specified losses from the insured to the insurer.
Risk pooling	Spreads the cost of losses among many policyholders through the collection of premiums.
Indemnity	Restores the insured to approximately the same financial position occupied before the loss.

This understanding provides the foundation for the next section, which explains how these principles are implemented through the insurance mechanism.

1.7 Insurance Mechanism

Insurance functions as a **risk-sharing mechanism** that enables individuals and businesses to protect themselves against uncertain financial losses. Instead of each person bearing the full cost of a potential loss, many policyholders contribute premiums to a common fund. Those who experience covered losses are then compensated from this fund.

This mechanism allows policyholders to replace an uncertain and potentially large financial loss with a relatively small and predictable premium.

The insurance mechanism operates through four basic steps.

1.7.1 Step 1: Risk Assessment

Before issuing an insurance policy, the insurer evaluates the risk presented by the applicant. This process, known as **underwriting**, considers factors that affect the likelihood and severity of future losses, such as the type of property being insured, its location, and the applicant's claims history.

Based on this assessment, the insurer determines whether to accept the risk and calculates an appropriate premium.

1.7.2 Step 2: Premium Collection

Policyholders who face similar risks pay premiums to the insurer. These premiums are pooled to form a common insurance fund that is used to pay future claims.

Although individual policyholders may pay different premiums according to their level of risk, all premiums contribute to the same risk-sharing mechanism.

1.7.3 Step 3: Loss Occurrence and Claim Settlement

During the policy period, some policyholders experience covered losses while others do not. When a covered loss occurs, the insured submits a claim to the insurer.

The insurer investigates the claim to verify that

- the loss is covered by the policy,
- the amount of the loss can be determined, and
- the claim satisfies the policy terms and conditions.

If the claim is valid, the insurer compensates the insured according to the policy provisions.

1.7.4 Step 4: Risk Sharing

Because only a relatively small proportion of policyholders experience losses during any policy period, the premiums collected from all policyholders are generally sufficient to compensate those who suffer covered losses.

This sharing of losses among many policyholders is the fundamental principle that makes insurance economically feasible.

> **Why Insurance Requires Many Policyholders**
>
> Insurance relies on the fact that losses occur randomly and independently across a large number of similar exposure units. If every policyholder suffered a loss at the same time, the insurer would be unable to spread the financial burden effectively.

1.7.5 The Insurance Mechanism

The operation of insurance can be summarized as follows.

The mechanism of risk pooling in insurance.

Figure 1.2

Although all policyholders contribute premiums, only those who experience covered losses receive compensation. In effect, the losses of the few are financed by the premiums collected from the many.

Example 1.7 (How the Insurance Mechanism Works)**.** Suppose an insurer provides fire insurance to **10,000 homeowners**.

Each homeowner pays an annual premium of **THB 4,000**, creating a total premium fund of **THB 40 million**.

During the year, only **80 homeowners** experience covered fire losses. After investigating the claims, the insurer compensates these policyholders from the premium fund.

Although most policyholders do not make claims, they benefit from knowing that financial protection is available should a covered loss occur. By pooling risks across many homeowners, the insurer is able to compensate the relatively few who suffer losses while charging each policyholder only a small fraction of the potential loss.

> **Insurance Is Based on Probability, Not Certainty**
>
> Insurers cannot predict which individual policyholders will experience losses. Instead, they estimate the **expected number** and **expected cost** of claims for a large group of similar policyholders using statistical and actuarial methods.

This reliance on probability is one of the fundamental differences between insurance and savings.

1.7.6 Connection to Actuarial Science

The insurance mechanism depends on accurate estimation of future claims. Actuaries use probability, statistics, and historical claims data to estimate claim frequency and severity, determine premiums, and ensure that insurers maintain sufficient funds to meet future claim obligations.

1.8 Insurance Contracts

An insurance policy is a legally enforceable contract between the insurer and the insured. The contract specifies the risks covered, the amount of insurance, the premium payable, the rights and obligations of each party, and the circumstances under which claims will be paid.

Under Section 862 of the Thai Civil and Commercial Code, an insurance contract may involve three parties.

1.8.1 The Insured

The **insured** is the person whose financial interest is protected by the insurance contract. The insured is responsible for paying premiums and disclosing material facts that may affect the insurer's assessment of the risk.

1.8.2 The Insurer

The **insurer** accepts the transferred risk and agrees to compensate the insured for covered losses in accordance with the policy terms. In Thailand, an insurer must be a public limited company licensed to conduct insurance business under the relevant legislation.

1.8.3 The Beneficiary

The **beneficiary** is the person entitled to receive policy proceeds. Depending on the type of insurance, the beneficiary may be

- the insured,
- another person designated in the policy, or
- a third party legally entitled to compensation.

Example 1.8 (Parties to an Insurance Contract)**.** A homeowner purchases fire insurance on a house financed by a bank.

- The homeowner is the **insured**.
- The insurance company is the **insurer**.

- If the house is mortgaged, the bank may be named as the **beneficiary** (or loss payee) to protect its financial interest.

1.9 Benefits of Insurance

Insurance provides benefits that extend beyond individual policyholders. By reducing financial uncertainty, it contributes to economic stability and supports business development.

For **individuals and households**, insurance provides financial protection against unexpected losses and promotes peace of mind. Instead of facing potentially devastating financial consequences, policyholders exchange uncertain losses for predictable premium payments.

For **businesses**, insurance protects assets, supports business continuity, and enables firms to undertake productive activities that might otherwise involve unacceptable levels of financial risk.

For **society and the economy**, insurance facilitates investment, encourages entrepreneurship, promotes long-term savings through certain life insurance products, and assists communities in recovering more quickly after major disasters.

> Insurance Supports Economic Growth
>
> Insurance does more than compensate individual losses. By reducing uncertainty, it encourages households and businesses to invest, borrow, and undertake economic activities that contribute to overall economic development.

1.10 Insurance in Thailand

1.10.1 Overview of the Thai Insurance Industry

The insurance industry plays an important role in Thailand's financial system by providing financial protection to individuals, businesses, and public organizations against uncertain losses. Through risk pooling and risk transfer, insurers help reduce the financial impact of accidents, property damage, illness, disability, and death.

In Thailand, insurance is governed by the **Civil and Commercial Code**, which recognizes two main categories of insurance contracts:

- **Life insurance**, and
- **Non-life insurance**.

These two sectors operate under separate legislation and are supervised by the **Office of Insurance Commission (OIC)**.

1.10.2 Types of Insurance

Thai insurance is broadly classified into **life insurance** and **non-life insurance**, reflecting the nature of the risks being insured and the contractual benefits provided.

1.10.2.1 Life Insurance

Life insurance provides financial protection against risks associated with human life. The payment of benefits depends on the survival or death of the insured person, as specified in the insurance contract.

The term **life insurance** is widely used in the United States and many other countries. Historically, the United Kingdom used the term **life assurance**, reflecting the certainty that death will eventually occur. Today, however, the term *life insurance* has become the more widely accepted international terminology.

Common examples of life insurance products include

- term life insurance,
- whole life insurance,
- endowment insurance, and
- annuity contracts.

1.10.2.2 Non-Life Insurance

Non-life insurance includes all forms of insurance other than life insurance. It protects policyholders against financial losses arising from damage to property, legal liability, or other specified risks.

Different countries use different terminology for this sector.

Term	Commonly Used In
Non-life insurance	Thailand and many Asian countries
General insurance	United Kingdom and Commonwealth countries
Property and casualty (P&C) insurance	United States and Japan

Examples of non-life insurance include

- motor insurance,
- fire insurance,
- marine insurance,
- engineering insurance,
- travel insurance, and
- liability insurance.

> **Different Names, Similar Concepts**
>
> Although the terms *non-life insurance*, *general insurance*, and *property and casualty insurance* differ across countries, they all refer broadly to insurance that is **not** life insurance. The scope of coverage may vary slightly among jurisdictions, but the underlying concept is the same.

1.10.3 Characteristics of Life Insurance

Life insurance differs from non-life insurance because the insured event is linked to the life of an individual rather than to damage or loss of property.

Some important characteristics of life insurance are:

- Benefits are payable upon the death or survival of the insured, depending on the policy.
- Policies are generally long-term contracts that may remain in force for many years.
- Premiums are determined using mortality assumptions together with interest rates and expense provisions.
- Many policies accumulate a cash value or provide savings and investment features.
- The policy benefit is specified in advance and is generally **not** based on the actual financial loss suffered.

Because the amount payable is predetermined, life insurance is generally **not** a contract of indemnity.

Example 1.9 (Example of a Life Insurance Policy)**.** A person purchases a 20-year term life insurance policy with a sum insured of THB 2 million.

If the insured dies during the policy term, the insurer pays THB 2 million to the designated beneficiary. If the insured survives beyond the policy term, no death benefit is payable unless otherwise specified by the policy.

1.10.4 Characteristics of Non-Life Insurance

Non-life insurance protects against accidental losses affecting property, liability, or other financial interests. Unlike life insurance, these contracts are generally short-term and are renewed periodically.

Typical characteristics include:

- Coverage applies only to specified accidental losses.
- Policies are usually written for one year and renewed if continued protection is required.
- Most contracts follow the **principle of indemnity**, compensating the insured for the actual financial loss up to the policy limits.
- Premiums are determined primarily from expected claim frequency and claim severity.

Since losses depend on uncertain events such as accidents, fires, or thefts, claims are paid only when a covered loss occurs.

Example 1.10 (Example of Non-Life Insurance)**.** A motor insurance policy provides coverage for accidental damage to a vehicle during a one-year policy period.

If the insured vehicle is damaged in a covered collision, the insurer compensates the policyholder for the repair cost, subject to the policy conditions, deductible, and policy limit.

1.10.5 Common Terminology in Life Insurance

Several terms are commonly used in life insurance and differ slightly from those used in non-life insurance.

Term	Meaning
Insured	The person whose life is covered by the policy.
Policyholder	The person who owns the insurance policy and is responsible for paying premiums.
Beneficiary	The person designated to receive the policy proceeds upon the insured's death.
Sum insured (sum assured)	The amount payable under the policy upon the occurrence of the insured event.
Premium	The amount paid to obtain insurance coverage.
Policy term	The period during which the insurance contract remains in force.

> **Insured and Policyholder Are Not Always the Same Person**
>
> In many life insurance contracts, the **policyholder** and the **insured** are the same person. However, they may differ. For example, a parent may purchase a life insurance policy on the life of a child, making the parent the policyholder and the child the insured.

1.10.6 Insurance Regulation in Thailand

Insurance companies operating in Thailand are supervised by the **Office of Insurance Commission (OIC)**, the national regulatory authority responsible for overseeing both life and non-life insurance businesses.

The OIC's primary responsibilities include

- licensing insurance companies,
- protecting the interests of policyholders,
- supervising insurers' financial soundness,
- promoting fair market conduct, and
- supporting the stable development of the insurance industry.

Life insurance and non-life insurance businesses are governed under separate legislation.

Insurance Business	Governing Law
Life insurance	*Life Insurance Act B.E. 2535 (1992)* and subsequent amendments
Non-life insurance	*Non-Life Insurance Act B.E. 2535 (1992)* and subsequent amendments

These laws establish the legal framework for licensing, corporate governance, capital requirements, and the conduct of insurance business in Thailand.

! Role of the Office of Insurance Commission (OIC)

The Office of Insurance Commission (OIC) is the primary regulator of Thailand's insurance industry. Its objective is to promote a stable, transparent, and trustworthy insurance market while protecting the interests of policyholders.

1.11 Mathematical Foundations of Insurance

Insurance relies on mathematical principles to estimate future losses and determine appropriate premiums. Although insurers cannot predict whether an individual policyholder will experience a loss, they can estimate the expected losses of a large group of similar policyholders using probability and statistical methods.

Three fundamental concepts underpin insurance operations:

- probability,
- the law of averages, and
- the law of large numbers.

Together, these concepts enable insurers to assess risks, calculate premiums, and maintain sufficient funds to pay future claims.

1.11.1 Probability

Probability measures the likelihood that an event will occur. It is expressed as a number between 0 and 1, where

- a probability of **0** indicates that the event cannot occur,
- a probability of **1** indicates that the event is certain to occur, and
- values between 0 and 1 represent varying degrees of likelihood.

In insurance, probability is used to estimate the likelihood of insured events such as fires, motor accidents, thefts, or deaths. These probabilities form the basis for estimating expected future claims.

For example, suppose historical data indicate that approximately **10 out of every 1,000 insured vehicles** are declared total losses each year. The estimated probability that an individual vehicle will suffer a total loss during the year is

$$\frac{10}{1000} = 0.01,$$

or **1%**.

This probability helps insurers estimate the expected number of claims within a portfolio and contributes to the calculation of insurance premiums.

Probability Describes Groups, Not Individuals

A probability of 1% does **not** mean that every insured vehicle has exactly a 1% chance of being involved in an accident in a particular year. Instead, it means that among many similar vehicles, approximately 1% are expected to experience the insured event.

1.11.2 Law of Averages

The **law of averages** is the principle that losses can be shared among many policyholders through **risk pooling**. Each policyholder contributes a relatively small premium to a common fund, from which the losses of the few who experience insured events are compensated.

Because individual losses are shared across many policyholders, insurance transforms uncertain and potentially large losses into relatively small and predictable premium payments.

In general, increasing the number of policyholders allows the cost of losses to be spread more widely. As administrative expenses and other factors are shared across a larger portfolio, the average cost borne by each policyholder may decrease.

Example 1.11 (Risk Pooling Through the Law of Averages)**.** Suppose that 100 homeowners purchase identical fire insurance policies.

During the year, only two houses are damaged by fire. Rather than requiring the two affected homeowners to bear the entire financial burden, all 100 policyholders contribute premiums that collectively cover the losses.

The financial burden of the losses is therefore shared among the entire group, illustrating the principle of risk pooling.

1.11.3 Law of Large Numbers

The **law of large numbers** states that as the number of similar exposure units increases, the observed average loss tends to approach the expected average loss.

This principle allows insurers to estimate future claims more accurately by analysing large portfolios of similar risks. While the occurrence of losses for an individual policyholder remains uncertain, the total number of losses across thousands of policyholders becomes increasingly predictable.

The law of large numbers is one of the most important mathematical foundations of insurance because it reduces the uncertainty associated with future claims.

Example 1.12 (Predicting Fire Losses)**.** Suppose an insurer covers **10,000 similar houses** against fire.

Although it is impossible to predict which individual houses will experience a fire during the coming year, historical data may indicate that approximately **25 houses** are expected to suffer fire damage.

Because the insurer has a large portfolio of similar risks, the actual number of claims is likely to be close to the expected number. This enables the insurer to estimate future claim costs and determine appropriate premium rates.

! The Law of Averages and the Law of Large Numbers Are Different

The **law of averages** explains **how losses are shared** among policyholders through risk pooling.
The **law of large numbers** explains **why insurers can estimate future losses more accurately** when many similar risks are insured.
Although closely related, the two principles serve different purposes in insurance.

1.11.4 Applications in Insurance

The mathematical principles discussed above play complementary roles in insurance operations.

Probability enables insurers to estimate the likelihood of insured events and the expected cost of future claims.

The **law of averages** provides the foundation for risk pooling, allowing the financial consequences of individual losses to be shared among many policyholders through premium contributions.

The **law of large numbers** improves the accuracy of these estimates by ensuring that actual claim experience becomes more stable as the number of similar exposure units increases.

Together, these principles support many actuarial and insurance activities, including

- underwriting and risk assessment,
- premium determination,
- reserving for future claims,
- reinsurance decisions, and
- capital management.

Why These Concepts Matter

Insurance companies cannot predict **who** will experience a loss, but they can estimate **how many** losses are likely to occur within a large portfolio. This ability allows insurers to charge fair premiums while maintaining sufficient funds to meet future claim obligations.

1.11.5 Summary

The three mathematical foundations of insurance are closely related but serve different purposes.

Concept	Main Purpose	Application in Insurance
Probability	Measures the likelihood of future events.	Estimates claim frequency and expected losses.
Law of averages	Spreads losses across many policyholders.	Forms the basis of risk pooling and premium sharing.
Law of large numbers	Improves the accuracy of loss predictions for large portfolios.	Enables reliable pricing, reserving, and financial planning.

This section naturally leads to the next topic, **Insurance Premium Determination**, where these mathematical principles are used to estimate expected losses and calculate premiums.

1.12 Insurance Premium Determination

An insurance premium is the amount paid by a policyholder in exchange for insurance protection. Determining an appropriate premium is one of the insurer's most important responsibilities. A premium that is too low may leave the insurer unable to pay future claims, whereas a premium that is too high may discourage customers from purchasing insurance.

To achieve a sustainable insurance system, insurers seek premiums that are **adequate**, **equitable**, and **reasonable**.

1.12.1 Principles of Premium Determination

1.12.1.1 Adequacy

A premium should be **adequate** to ensure that the insurer can meet its financial obligations. In particular, the premium should be sufficient to

- pay future claims,
- cover operating and administrative expenses,
- maintain an appropriate financial reserve for unexpected losses, and
- provide a reasonable return to the insurer.

If premiums are consistently inadequate, the insurer may experience financial difficulties and be unable to satisfy future claims.

1.12.1.2 Equity (Fairness)

A premium should be **equitable**, meaning that it reflects the level of risk presented by each policyholder. Individuals or businesses with higher expected losses should generally pay higher premiums, while those with lower expected losses should pay less.

This principle is sometimes referred to as **fair discrimination**, because premiums are differentiated according to measurable differences in risk rather than arbitrary factors.

For example, a building constructed with fire-resistant materials may qualify for a lower fire insurance premium than an otherwise similar building with a substantially higher fire risk.

1.12.1.3 Reasonableness

A premium should also be **reasonable** from the policyholder's perspective. Although premiums must cover the insurer's expected costs, they should not be so high that insurance becomes unaffordable or uncompetitive.

In a competitive insurance market, insurers must balance financial sustainability with affordability in order to attract and retain policyholders.

> **!** Balancing the Three Principles
>
> The three principles of premium determination must be considered together.
>
> - A premium that is **too low** may be fair to customers in the short term but financially unsustainable for the insurer.
> - A premium that is **too high** may be financially adequate but discourage customers from purchasing insurance.
> - A well-designed premium achieves an appropriate balance between adequacy, equity, and reasonableness.

1.12.2 Components of an Insurance Premium

The premium charged by an insurer consists of more than the expected cost of future claims. It also includes the expenses of operating the insurance business and a margin for uncertainty and profit.

Conceptually, the premium may be expressed as

$$\boxed{P = F \times S + E + C,}$$

where

- P = insurance premium,
- F = expected claim frequency,
- S = expected claim severity,
- E = operating expenses, and
- C = contingency provision and profit loading.

Although actual premium calculations are often more sophisticated, this expression illustrates the fundamental components of an insurance premium.

1.12.2.1 Expected Losses

The largest component of the premium is the **expected loss**, which represents the insurer's estimate of future claim payments.

Expected losses depend on two quantities:

- **frequency**, which measures how often claims are expected to occur, and

- **severity**, which measures the average cost of each claim.

Their product,

$$F \times S,$$

represents the **expected claim cost** for each insured exposure.

For example, if the expected claim frequency is 2% per year and the average claim amount is THB 100,000, the expected loss per policy is

$$0.02 \times 100,000 = \text{THB } 2,000.$$

1.12.2.2 Operating Expenses

Insurers incur various expenses in providing insurance services. These include

- underwriting expenses,
- policy administration,
- claim handling,
- commissions paid to agents or brokers,
- information technology and office operations, and
- reinsurance costs.

These expenses must be recovered through premiums to ensure the insurer can continue operating effectively.

1.12.2.3 Contingency Provision and Profit

Future losses are inherently uncertain. Even when expected losses are estimated carefully, actual claim experience may differ from expectations.

To protect against this uncertainty, insurers include a **contingency provision** in the premium. This provision provides financial protection against

- random fluctuations in claims,
- estimation errors,
- inflation in claim costs, and
- unexpected catastrophic events.

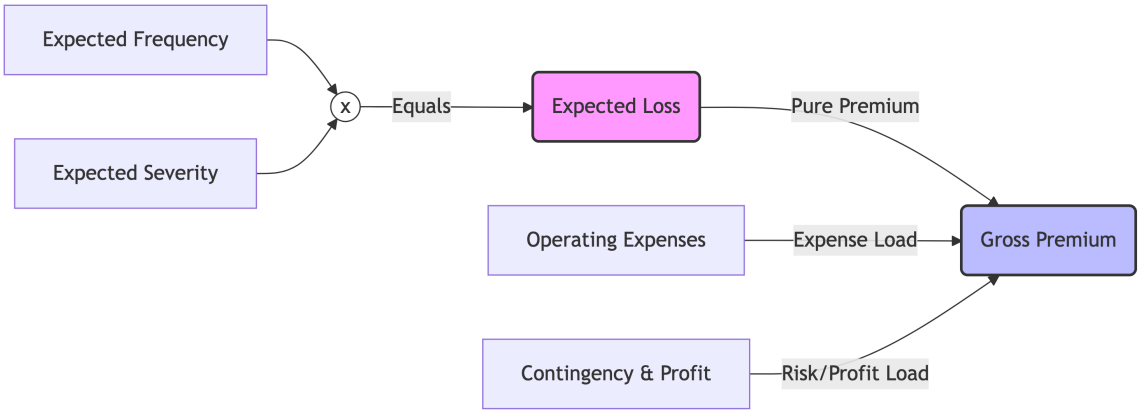
The premium also includes a **profit loading**, which compensates shareholders for providing capital and accepting the financial risks associated with the insurance business.

> **i** Why Is a Contingency Provision Needed?
>
> Historical data provide only estimates of future losses. Actual claims may be higher than expected because of unusually severe weather, economic changes, or random fluctuations. A contingency provision strengthens the insurer's financial resilience and helps ensure that

future claims can still be paid.

1.12.3 Putting the Components Together

The following diagram illustrates the conceptual structure of an insurance premium.



The mathematical components of an insurance premium.

Figure 1.3

The premium therefore consists of three broad components:

Component	Purpose
Expected losses	Pays future claims.
Operating expenses	Covers the cost of conducting insurance business.
Contingency provision and profit	Provides financial resilience and an appropriate return on capital.

Example 1.13 (Components of an Insurance Premium)**.** Suppose an insurer estimates the following values for a motor insurance policy.

Component	Amount (THB)
Expected claim frequency	2%
Expected claim severity	100,000
Expected losses ($F \times S$)	2,000
Operating expenses	600
Contingency provision and profit	400

The premium is therefore estimated as

$$P = 2,000 + 600 + 400 = \text{THB } 3,000.$$

This example illustrates that only part of the premium is used to pay claims. The remainder supports the operation and financial stability of the insurer.

> **Looking Ahead**
>
> In this chapter, the premium formula is presented as a **conceptual framework** for understanding the main components of an insurance premium. In later chapters, we will develop more rigorous actuarial methods for estimating expected losses and determining premiums using statistical models and insurance rating techniques.

2 Chapter 2: Introduction to Data Science

Learning Outcomes

By the end of this chapter, students should be able to:

1. Explain the role of data science in supporting insurance and actuarial decision-making.
2. Describe the relationship between data, statistics, computing, and domain knowledge in data science.
3. Identify common sources and types of data used in insurance.
4. Describe the main stages of the data science life cycle and explain how they relate to insurance problems.
5. Distinguish the roles of data analysts, data scientists, data engineers, and actuaries in a data science project.
6. Identify applications of data science in insurance, including pricing, underwriting, claims management, fraud detection, customer analytics, and risk management.
7. Explain how data-driven evidence is combined with actuarial judgement to support insurance decisions.

2.1 From Insurance to Data Science

Insurance is fundamentally concerned with making decisions under uncertainty. As discussed in Chapter 1, insurers pool risks, use information about past experience to assess future losses, and determine premiums that are appropriate for the risks being insured. These activities require more than an understanding of insurance principles and probability. They also require reliable data and systematic methods for analysing it.

For example, a motor insurer may want to determine whether claim frequency differs according to age, geographical area, vehicle characteristics, or other characteristics of policyholders. The insurer must obtain appropriate data, assess its quality, identify relevant patterns, and use suitable statistical methods to support the decision. These activities are part of the broader field of data science.

2.1.1 Data-Driven Insurance Decisions

An insurance decision is data-driven when information from relevant data is used systematically to support the decision. Data do not replace actuarial judgement; rather, they provide evidence that can be combined with statistical methods, insurance knowledge, and professional judgement.

Data are used throughout the insurance process. For example, insurers may use data to:

- assess and classify risks during underwriting;
- estimate claim frequency and claim severity;
- determine appropriate premiums;
- monitor claims and identify unusual patterns;
- evaluate the experience of an insurance portfolio; and
- support decisions about risk management and business strategy.

Consider a motor insurance portfolio. Suppose an insurer observes that policyholders in different age groups have different claim frequencies. The insurer can examine the observed experience, investigate whether the difference is supported by sufficient data, and determine whether age should be considered when assessing risk.

The process can be viewed as:

$$\text{Insurance problem} \rightarrow \text{Data} \rightarrow \text{Analysis} \rightarrow \text{Evidence} \rightarrow \text{Decision}.$$

The purpose of data science is therefore not simply to produce calculations or visualisations. Its purpose is to help transform data into useful information and evidence for decision-making.

> **Data do not speak for themselves**
>
> Observed patterns in insurance data must be interpreted in their context. A difference between two groups does not necessarily imply that the corresponding risk factor causes the difference. Data quality, exposure, other risk factors, and the method of analysis must also be considered.

2.1.2 From Risk to Data

Chapter 1 introduced risk as uncertainty concerning future outcomes and showed how insurance uses probability and pooling to manage this uncertainty. In practice, however, the probabilities and expected losses used by insurers are not known with certainty. They must be estimated from observations.

Suppose an insurer wants to estimate the expected number of claims for a portfolio. Historical observations may contain the number of claims and the corresponding exposure for each policy. A simple estimate of claim frequency is

$$\hat{\lambda} = \frac{\text{Number of claims}}{\text{Exposure}}.$$

This simple calculation illustrates an important transition from the concepts of Chapter 1 to the methods introduced in this chapter. The concept of risk is theoretical, but its measurement requires data.

The same principle applies to claim severity. If observed claims have amounts $X_1, \ldots, X_n$, their average provides a basic estimate of the average claim amount:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

In practice, insurers work with much richer data. A policy record may contain information about the policyholder, the insured object, geographical area, coverage, exposure period, and claims. These data can be used to investigate how insurance outcomes vary across different characteristics.

Thus, the relationship between risk and data can be represented as

$$\boxed{\text{Risk} \rightarrow \text{Observable outcomes} \rightarrow \text{Data} \rightarrow \text{Statistical analysis} \rightarrow \text{Risk assessment}}.$$

Data science provides a systematic framework for the stages involving data collection, preparation, exploration, analysis, modelling, and communication.

! Exposure matters in insurance data

Insurance experience cannot always be compared using raw counts alone. A portfolio with more policies or a longer period of exposure will generally produce more claims simply because it contains more opportunity for claims to occur. Measures such as claim frequency therefore relate observed claims to an appropriate measure of exposure.

2.1.3 Why Data Science Matters to Actuaries

Actuaries have traditionally used mathematics, probability, statistics, and financial theory to analyse uncertainty and make decisions about risk. Data science extends these capabilities by providing a broader framework for working with data throughout the analytical process.

For actuaries, data science is particularly relevant because modern insurance problems often involve large and complex datasets. An actuary may need to combine information from different sources, identify data-quality problems, explore relationships between variables, develop statistical models, and communicate the results to decision-makers.

The role of the actuary is not simply to apply an algorithm to a dataset. Actuarial knowledge is needed to formulate an appropriate problem, select meaningful variables, recognise relevant exposure measures, assess whether modelling assumptions are reasonable, and interpret the results in the context of insurance.

For example, a statistical model may indicate that claim frequency varies between two groups of policyholders. An actuary must consider whether the difference is credible, whether other risk factors may explain the observed relationship, whether the available exposure is sufficient, and whether the result is appropriate for the intended insurance decision.

This illustrates the complementary roles of data science and actuarial science:

Data science	Actuarial science
Collects and prepares data	Understands the insurance problem
Explores and visualises data	Interprets risk and uncertainty
Develops statistical and predictive models	Assesses actuarial assumptions and implications
Communicates analytical results	Applies professional judgement to decisions

Data science should therefore be viewed as an important part of the modern actuarial toolkit. The objective is not to replace actuarial reasoning with data or algorithms, but to combine **data, statistical methods, computational tools, and actuarial judgement** to make better-informed decisions.

This chapter develops the foundations for that process. It begins with the nature of data and the data science life cycle, before introducing methods for acquiring, preparing, exploring, modelling, and communicating data.

2.2 What Is Data Science?

Data science is concerned with extracting useful information and knowledge from data to support understanding, prediction, and decision-making. It combines statistical methods, computational tools, and knowledge of the subject area in which the data arise.

There is no single method that defines data science. Different problems require different combinations of data management, statistical analysis, programming, visualisation, and modelling. The common element is the systematic use of data to answer meaningful questions.

For an insurer, for example, data science may be used to investigate questions such as:

- How does claim frequency vary across different groups of policyholders?
- Which characteristics are associated with larger claims?
- Can future claims be predicted from available information?
- Are there unusual patterns in claims that require further investigation?

These questions require more than simply calculating statistics. They require an understanding of the insurance problem, appropriate data, suitable analytical methods, and computational tools.

2.2.1 Data, Statistics, Computing and Domain Knowledge

Data science brings together four closely related components: **data, statistics, computing, and domain knowledge**.

Data provide the observations from which information can be obtained. In insurance, data may include policy characteristics, exposure, claims, claim amounts, premiums, and other information relevant to risk.

Statistics provides methods for describing data, identifying patterns, quantifying uncertainty, and drawing conclusions from observations. For example, an insurer may use statistical methods to estimate claim frequency or compare the experience of different groups.

Computing provides the tools needed to store, manipulate, analyse, and visualise data. Modern datasets may contain thousands or millions of observations, making computational tools essential for carrying out analyses efficiently and reproducibly.

Domain knowledge provides an understanding of the context in which the data arise. For an actuary, this includes knowledge of insurance products, risk, exposure, claims, premiums, and the business decisions that the analysis is intended to support.

These components are complementary:

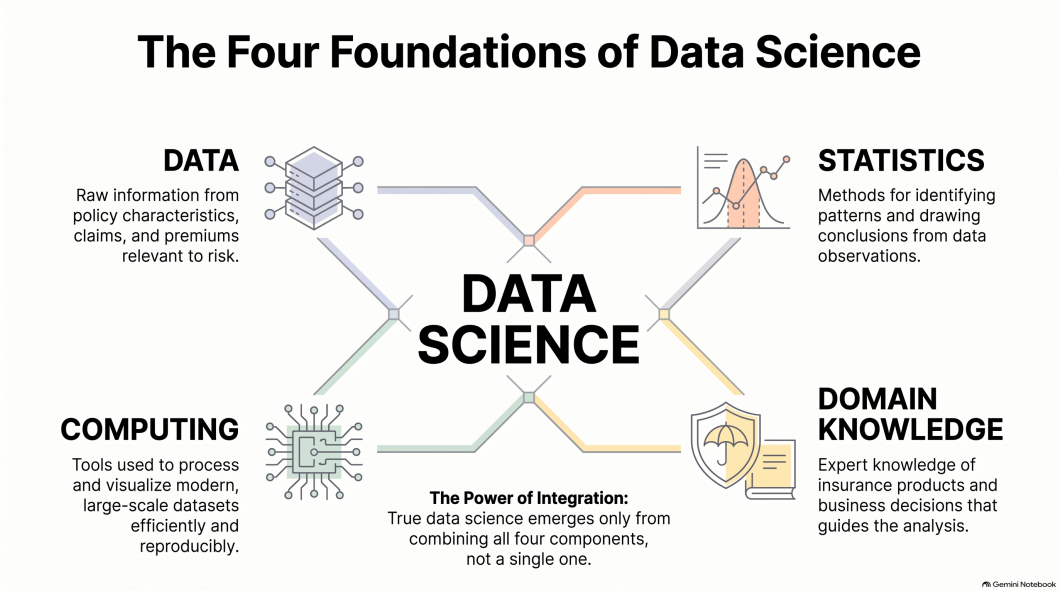


Figure 2.1: The Four Foundations of Data Science

The combination is important because weakness in any one component can affect the quality of the analysis. A sophisticated statistical model cannot compensate for poor-quality data, and a technically correct analysis may still lead to an inappropriate conclusion if the insurance context is misunderstood.

For example, suppose an insurer observes that one group of policyholders has a higher claim frequency than another. Statistics can quantify the difference, computing can process the underlying data, and domain knowledge can help determine whether the comparison is meaningful. The analysis may also need to account for exposure and other characteristics that differ between the groups.

> **!** Data science is more than programming
>
> Programming is an important tool in data science, but data science is not simply programming with data. A successful analysis requires an appropriate question, reliable data, suitable statistical methods, and an understanding of the context in which the results will be used.

The four components can therefore be viewed as complementary skills rather than separate stages:

Component	Main purpose	Example in insurance
Data	Provide observations	Policy and claims records
Statistics	Quantify and analyse patterns	Estimate claim frequency
Computing	Process and analyse data efficiently	Use R to manipulate and model data
Domain knowledge	Interpret results in context	Understand exposure and insurance risk

For an actuarial student, this framework is particularly useful. Actuarial training already provides a strong foundation in statistics, probability, and insurance knowledge. Data science adds computational and analytical practices that allow these foundations to be applied effectively to increasingly complex datasets.

2.2.2 Data Science and Decision-Making

The ultimate purpose of data science is not to produce data, tables, graphs, or models for their own sake. Its purpose is to provide evidence that can improve decisions.

A useful way to view the process is:

$$\text{Question} \rightarrow \text{Data} \rightarrow \text{Analysis} \rightarrow \text{Evidence} \rightarrow \text{Decision}.$$

Consider an insurer deciding whether a particular characteristic should be used in pricing. The analysis might begin with historical policy and claims data. The data are then prepared and explored, and an appropriate statistical model may be developed. The results provide evidence about the relationship between the characteristic and insurance outcomes. The insurer can then consider whether that evidence is sufficient and appropriate for a pricing decision.

The quality of the final decision depends on more than the predictive performance of a model. The analyst must also consider the quality and relevance of the data, the assumptions underlying the analysis, uncertainty in the results, and the consequences of using those results.

This is particularly important in insurance because decisions can affect both the insurer and its policyholders. For example, a pricing model may identify differences in expected claim costs between groups. An actuarial assessment must consider whether those differences are credible, whether the data support the conclusion, and whether the resulting rating structure is appropriate for the insurance product.

Data science therefore supports, rather than replaces, professional judgement. The role of the actuary is to connect analytical evidence with insurance principles and business decisions.

> **i** From analysis to action
>
> A useful data-science result should answer three questions:
>
> 1. **What did the data show?**
> 2. **How reliable is the evidence?**
> 3. **What decision, if any, should follow from it?**

> Keeping these questions in mind helps distinguish useful analysis from analysis that merely produces numerical results.

The distinction between **analysis** and **decision-making** will remain important throughout this book. Data science provides methods for obtaining evidence from data; actuarial science provides the risk, financial, and professional context needed to interpret that evidence and use it responsibly.

2.3 The Role of Data in Insurance

Data provide the empirical foundation for many insurance activities. Insurers collect and maintain information about policies, policyholders, exposures, premiums, claims, and payments. These data allow insurers to measure past experience, assess current risks, and support decisions about underwriting, pricing, claims management, and risk management.

The usefulness of insurance data depends on more than the amount of information available. Data must be relevant to the problem, recorded consistently, and sufficiently complete and accurate for the intended analysis. In addition, insurance data often need to be interpreted in relation to exposure and the characteristics of the insurance portfolio.

For example, the number of claims alone provides limited information about claim risk. A portfolio with 1,000 claims may have a higher risk than a portfolio with 500 claims, but it may simply have a much larger amount of exposure. Comparing claim frequency rather than claim counts can provide a more meaningful basis for comparison.

2.3.1 Insurance Data Sources

Insurance data come from many sources. Some are generated directly through the insurer's operations, while others may be obtained from external sources.

Common internal sources include:

- **Policy data** — information about policies, such as policy type, coverage, inception and expiry dates, insured characteristics, and premium.
- **Exposure data** — information describing the amount of time or risk exposure associated with a policy.
- **Claims data** — information about reported claims, including claim dates, claim types, claim status, and claim amounts.
- **Payment data** — information about premiums received, claim payments, expenses, and other financial transactions.
- **Customer data** — information used to understand policyholder characteristics and interactions with the insurer.

External sources may provide additional information relevant to risk assessment. Examples include geographical information, demographic information, weather data, economic indicators, and other publicly or commercially available data.

These sources may be stored separately. For example, policy information and claims information may be maintained in different systems and linked using a policy identifier. Combining these sources is often an important part of an insurance data-science project.

A simple representation is:

$$\text{Policy data} + \text{Exposure data} + \text{Claims data} \longrightarrow \text{Insurance analysis}.$$

The appropriate data sources depend on the question being investigated. A pricing analysis may require policy, exposure, claim, and premium information, whereas a claims-management analysis may focus more heavily on claim and payment records.

> **Start with the question**
>
> The objective of data collection is not to collect as much data as possible. The data should be relevant to the question being investigated and measured in a way that supports the intended analysis.

2.3.2 Types of Data

Insurance datasets contain variables with different characteristics. Recognising the type of each variable is important because the type determines how the variable can be summarised, visualised, and analysed.

One useful distinction is between **categorical** and **numerical** data.

Categorical data describe membership in a group or category. Examples include:

- type of insurance;
- geographical area;
- vehicle type;
- policy status; and
- claim type.

A categorical variable may have a small number of possible categories. For example, a policy may be classified as `Urban` or `Rural`.

Numerical data represent quantities for which arithmetic operations are meaningful. Examples include:

- age;
- exposure;
- premium;
- number of claims; and
- claim amount.

Numerical variables can be further distinguished as **discrete** or **continuous**.

A discrete variable takes values that can be counted. The number of claims, for example, is a discrete variable:

$$N \in 0, 1, 2, ...$$

A continuous variable can take values within an interval. Claim amount and exposure are examples, depending on how they are measured and recorded.

Another useful distinction is between **structured** and **unstructured** data. Structured data are organised in a predefined format, such as rows and columns in a database. Traditional policy and claims records are usually structured data. Unstructured data do not follow such a predefined tabular structure and may include documents, images, audio, or text.

For introductory actuarial analysis, structured data are particularly important because many insurance models are developed from datasets in which each row represents an observation and each column represents a variable.

Variable	Example	Type
Age	35	Numerical, continuous/discrete depending on recording
Area	Urban	Categorical
Exposure	0.75	Numerical, continuous
Number of claims	2	Numerical, discrete
Claim amount	18,500	Numerical, continuous
Policy status	Active	Categorical

The classification of a variable is not merely a technical label. It affects the methods used to analyse the variable. For example, the distribution of a numerical variable may be examined using a histogram, whereas the frequencies of categories may be displayed using a bar chart.

> **!** A variable's role depends on the analysis
>
> The same variable can serve different roles in different analyses. For example, claim amount may be an outcome variable when modelling claim severity, but it may also be summarised as part of an exploratory analysis. Variable type and variable role should therefore be considered separately.

2.3.3 An Insurance Dataset

To illustrate the data-science process, consider a simplified motor insurance dataset. Each row represents an insurance policy, while each column records a characteristic of that policy or its observed experience.

Policy	Age	Area	Exposure	Claims	Claim Amount
1	24	Urban	1.00	1	12,500

Policy	Age	Area	Exposure	Claims	Claim Amount
2	42	Rural	1.00	0	0
3	31	Urban	0.50	2	8,400
4	56	Rural	1.00	0	0
5	27	Urban	0.75	1	21,300
6	45	Urban	1.00	0	0

This small dataset contains several types of insurance information. `Age` and `Area` describe characteristics of the policyholder or risk, while `Exposure` describes the amount of risk observed during the period. `Claims` records the number of observed claims, and `Claim Amount` records the corresponding aggregate claim amount.

The dataset can be used to illustrate several questions.

Descriptive question: How does claim experience vary across different areas?

Frequency question: What is the observed claim frequency?

Severity question: What is the average claim amount among policies with claims?

Comparative question: Does claim experience appear to differ between younger and older policyholders?

These questions illustrate an important feature of insurance data: different variables describe different aspects of the risk. In particular, the distinction between **frequency** and **severity** is fundamental in insurance analysis.

For a portfolio, claim frequency can be represented by

$$\text{Claim frequency} = \frac{\text{Number of claims}}{\text{Exposure}},$$

while average claim severity can be represented by

$$\text{Average severity} = \frac{\text{Total claim amount}}{\text{Number of claims}},$$

provided that the denominator is positive.

Combining the two gives the basic relationship

$$\text{Expected loss per unit exposure} \approx \text{Claim frequency} \times \text{Average severity}.$$

This relationship provides a simple bridge between the insurance concepts introduced in Chapter 1 and the data-science methods introduced in this chapter. The quantities are estimated from observed data, and more advanced methods can then be used to examine how they vary with risk characteristics.

2.4 The Data Science Life Cycle

A data science project typically follows a sequence of activities that transforms a practical problem into an evidence-based decision. Although individual projects differ, most involve several common stages:

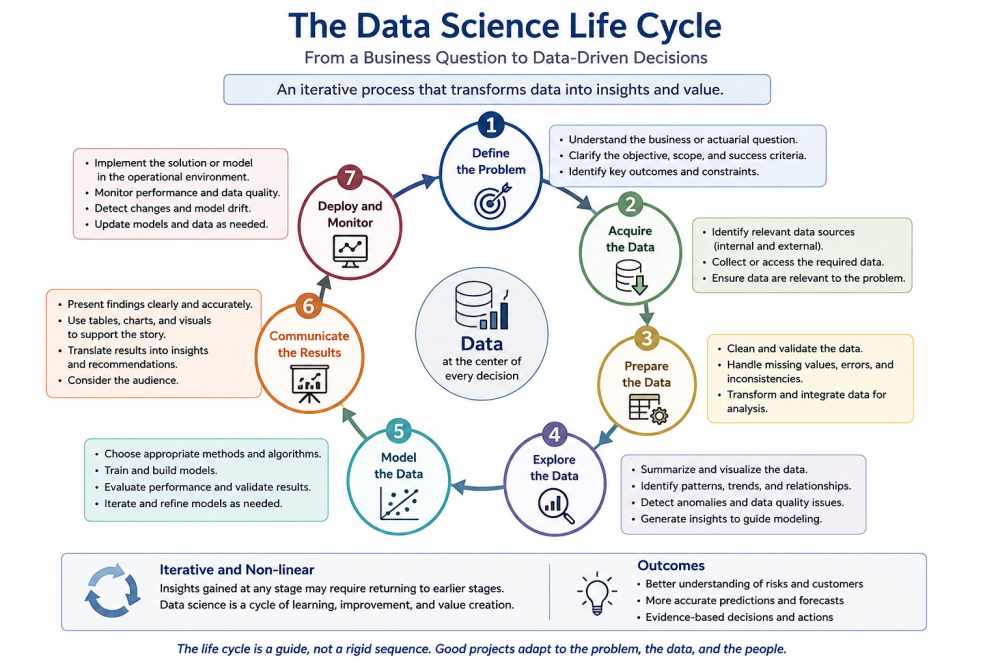


Figure 2.2: Data Science Life Cycle Flowchart

These stages provide a useful framework for organising a data science project. They should not, however, be viewed as a strictly linear process. Findings from one stage may require the analyst to return to an earlier stage. For example, exploratory analysis may reveal data-quality problems that require additional data preparation, while modelling results may indicate that the original question needs to be reconsidered.

In insurance, the cycle may begin with a question about claim experience, proceed through the analysis of policy and claims data, and ultimately produce information that supports underwriting, pricing, claims management, or risk management.

2.4.1 Define the Problem

The first stage is to clearly define the problem that the analysis is intended to address. A data science project should begin with a question or decision, rather than with a dataset or a particular modelling technique.

For an insurance problem, the analyst should establish:

- what decision needs to be supported;
- what outcome or quantity is of interest;
- which population or portfolio is being studied;

- what information may be relevant; and
- how the results will be used.

For example, an insurer may observe differences in motor insurance claim experience and ask:

Which characteristics are associated with differences in claim frequency?

This question is more useful than simply stating that the insurer wants to "analyse the claims data." It identifies the outcome of interest and provides a direction for the analysis.

The problem should also be translated into measurable quantities. For example, if claim frequency is being studied, the analysis needs information about both the number of claims and the corresponding exposure.

> ! Start with the decision, not the model
>
> The choice of statistical or machine-learning method should follow from the problem being addressed. A sophisticated model is not useful if it does not answer a relevant question or support a meaningful decision.
> A well-defined problem provides a clear objective for the remaining stages of the data science life cycle.

2.4.2 Acquire the Data

Once the problem has been defined, the next stage is to obtain the data required for the analysis.

Insurance data may come from several sources, including policy administration systems, claims systems, financial records, customer databases, and external sources. Data from different systems may need to be combined using a common identifier such as a policy or claim number.

The data required depend on the question being investigated. For example, analysing claim frequency may require:

- the number of claims;
- exposure;
- policy characteristics; and
- relevant risk characteristics.

Data acquisition may involve importing files, querying databases, accessing application programming interfaces (APIs), or obtaining data from external sources.

At this stage, it is important to distinguish between **available data** and **appropriate data**. A large dataset is not necessarily useful if it does not measure the quantities relevant to the problem.

2.4.3 Prepare the Data

Raw data are rarely ready for analysis. Data preparation involves checking, cleaning, transforming, and organising the data so that they are suitable for the intended analysis.

Common problems include:

- missing values;
- invalid or impossible values;
- inconsistent data types;
- duplicated observations;
- inconsistent categories;
- incorrect dates; and
- records that fall outside the relevant observation period.

Insurance data provide many examples. A policy may have a missing inception date, an exposure period may be incorrectly recorded, a claim may be associated with a policy that was not active at the claim date, or a categorical variable may contain several different labels for the same geographical area.

Data preparation may also require transformations. For example, a date may need to be converted into a standard date format, or several claim records may need to be aggregated to obtain the total claim amount for each policy.

The objective is not simply to make the dataset look tidy. Each transformation should have a clear reason based on the meaning of the underlying data and the requirements of the analysis.

> **i** Data cleaning is part of the analysis
>
> Cleaning data is not merely a technical task. Decisions about missing, invalid, or unusual observations can affect the results of the analysis. Data preparation should therefore be documented and carried out with an understanding of the underlying insurance process.

2.4.4 Explore the Data

After the data have been prepared, the next stage is **exploratory data analysis (EDA)**. The purpose of EDA is to understand the main characteristics of the data before formal modelling.

EDA can be used to examine:

- the distribution of individual variables;
- differences between groups;
- relationships between variables;
- unusual or extreme observations;
- missing values; and
- possible data-quality problems.

For example, an insurer may compare claim frequency across geographical areas or examine the distribution of claim amounts.

Both numerical summaries and graphical methods can be useful. Means, medians, standard deviations, and other descriptive statistics provide numerical summaries, while histograms, boxplots, bar charts, and scatterplots can reveal patterns that may not be apparent from numerical summaries alone.

EDA can also influence subsequent modelling. It may reveal variables that require transformation, groups that should be examined separately, or potential relationships that require further investigation.

Thus, exploration is not simply a preliminary presentation of the data. It helps the analyst understand what the data can reasonably support.

2.4.5 Model the Data

Once the data have been understood, an appropriate statistical or computational model can be developed when modelling is required.

A model provides a mathematical representation of relationships in the data. Depending on the problem, the objective may be to explain observed patterns, estimate an unknown quantity, or predict future outcomes.

In insurance, examples include:

- modelling claim frequency;
- modelling claim severity;
- predicting the probability of a claim;
- forecasting future claims; and
- estimating expected losses.

The choice of model depends on the nature of the outcome, the available data, the purpose of the analysis, and the assumptions that can reasonably be made.

For example, the number of claims is a count variable, whereas claim amount is a monetary variable. These different outcomes may require different modelling approaches.

Model development should also include evaluation. A model that fits historical data well may not necessarily provide reliable predictions for new observations. The analyst therefore needs to assess whether the model is appropriate for its intended purpose.

! A model is an approximation

A statistical model does not reproduce reality exactly. It provides a simplified representation of relationships observed in the data. Its usefulness depends on whether the assumptions and results are appropriate for the problem being considered.

2.4.6 Communicate the Results

The results of an analysis must be communicated clearly to the people who will use them. A technically correct analysis has limited value if its findings cannot be understood or connected to the original decision.

Communication may involve:

- tables;
- charts and visualisations;
- written reports;
- presentations; or
- interactive dashboards.

The form of communication should reflect the audience. An actuarial report may require technical details about assumptions and model performance, whereas senior management may need a concise summary of the main findings and their implications.

Good communication should connect the results back to the original question. For example, rather than simply reporting that two groups have different observed claim frequencies, the analyst should explain the size of the difference, the evidence supporting it, relevant limitations, and its potential implications for the insurance decision.

A useful structure is:

$$\text{Problem} \rightarrow \text{Evidence} \rightarrow \text{Interpretation} \rightarrow \text{Implication}.$$

This helps ensure that analysis leads to understanding rather than simply producing additional information.

2.4.7 Deploy and Monitor

When an analysis produces a model intended for operational use, the final stage is to deploy the model and monitor its performance.

Deployment means making the model available within the process for which it was developed. For example, a predictive model might be incorporated into an underwriting system to estimate risk when a new policy is quoted.

Deployment should normally be preceded by appropriate testing. The model should be checked in an environment that reflects its intended use before it is introduced into production.

Once deployed, the model should not simply be left unchanged. Insurance portfolios, customer behaviour, economic conditions, claim patterns, and data-generation processes can change over time. A model that performed well when developed may therefore become less appropriate later.

Monitoring involves checking whether the model and its underlying data continue to perform as expected. This may include monitoring predictive performance, data quality, unusual changes in input variables, and differences between predicted and observed outcomes.

For example, if a claim prediction model consistently underestimates the frequency of claims in a particular segment, the model may need to be investigated and potentially recalibrated or redeveloped.

The life cycle is iterative

The final stage may lead back to earlier stages. Changes in model performance may require new data, additional data preparation, further exploration, or redevelopment of the model.

Data science is a cycle, not a one-time process

The objective is to develop solutions that remain useful as the data, environment, and decision-making requirements change.

2.5 Roles in a Data Science Team

Data science projects often involve several people with different areas of expertise. Although the responsibilities may overlap, different roles contribute different skills to the data science life cycle.

The composition of a data science team depends on the size and nature of the organisation. In a small organisation, one person may perform several roles. In a larger organisation, responsibilities may be distributed among specialists.

Three common roles are **data analyst**, **data scientist**, and **data engineer**. In an insurance organisation, these roles may work closely with actuaries and other insurance professionals.

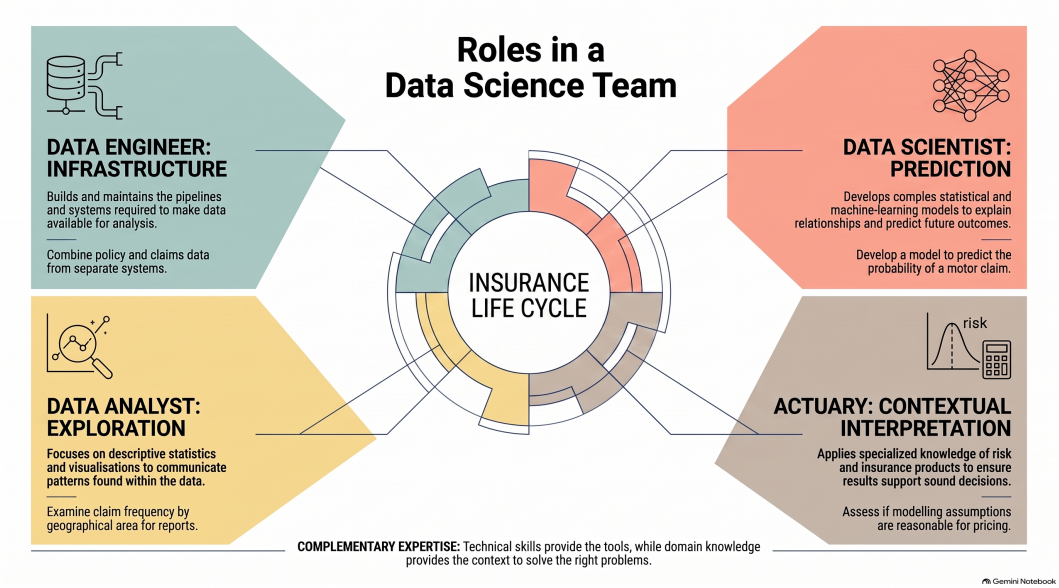


Figure 2.3: Core Roles and Responsibilities in a Data Science Team

2.5.1 Data Analyst

A data analyst focuses on understanding and communicating information contained in data. Typical activities include:

- collecting and preparing data for analysis;
- producing descriptive statistics;
- identifying patterns and trends;
- creating tables and visualisations; and
- communicating findings to stakeholders.

For example, a data analyst in an insurance company might examine claim frequency by geographical area and prepare a report showing differences in observed experience.

The role is closely associated with the **exploration and communication** stages of the data science life cycle.

2.5.2 Data Scientist

A data scientist typically works on more complex analytical and predictive problems. In addition to preparing and exploring data, a data scientist may develop statistical or machine-learning models to explain relationships or make predictions.

For example, a data scientist might develop a model to predict the probability that a motor insurance policy will generate a claim.

The role commonly involves:

- statistical modelling;
- predictive modelling;
- machine learning;
- model evaluation; and
- translating analytical results into useful information for decision-making.

The precise responsibilities vary between organisations, and the distinction between a data analyst and a data scientist is not always strict.

2.5.3 Data Engineer

A data engineer focuses on the infrastructure and processes required to make data available for analysis. This may include:

- collecting data from different systems;
- building and maintaining data pipelines;
- storing and organising data;
- integrating data from multiple sources; and
- ensuring that data can be accessed reliably.

For example, an insurance company may have separate systems for policies, claims, and payments. A data engineer may develop processes that extract and combine information from these systems so that it can be used for analysis.

The work of a data engineer is particularly important in the **data acquisition and preparation** stages of the data science life cycle.

2.5.4 The Actuary in a Data Science Team

For an actuarial student, an important question is where the **actuary** fits within a data science team.

Actuaries contribute specialised knowledge of risk, uncertainty, insurance products, financial consequences, and actuarial methods. They also understand the decisions that analytical results are intended to support.

For example, when developing a model for motor insurance pricing, an actuary may contribute by:

- defining the insurance problem;
- identifying appropriate measures of exposure;
- selecting relevant risk characteristics;
- assessing whether modelling assumptions are reasonable;
- interpreting model results in an insurance context; and
- considering the implications for pricing and risk management.

This illustrates the complementary nature of the different roles.

Role	Main focus	Example in insurance
Data analyst	Describe and communicate data	Analyse claim experience
Data scientist	Develop analytical and predictive models	Predict claim frequency
Data engineer	Build and manage data infrastructure	Combine policy and claims data
Actuary	Apply risk and insurance expertise	Interpret results for pricing

The boundaries between these roles are not fixed. An actuary may perform data analysis and statistical modelling, while a data scientist may need substantial knowledge of the insurance problem. In smaller organisations, a single person may perform several of these functions.

What matters is not the job title but the combination of skills required to complete the data science life cycle.

> **Data science is collaborative**
>
> A successful data science project usually requires more than one type of expertise. Technical skills are needed to work with data and develop models, while domain knowledge is needed to ensure that the analysis addresses the right problem and that its results are interpreted appropriately.
> For actuarial students, developing data-science skills therefore does not mean becoming a data engineer or replacing actuarial methods with machine learning. Instead, these skills

provide additional tools for applying actuarial knowledge to increasingly complex data and decision-making problems.

2.6 Applications of Data Science in Insurance

Data science can be applied throughout the insurance value chain. Insurers collect data at many stages of the insurance process, from underwriting and policy administration to claims settlement and portfolio management. Analysing these data can help insurers understand risk, improve operational processes, and support better-informed decisions.

The applications of data science in insurance can be grouped into several broad areas:

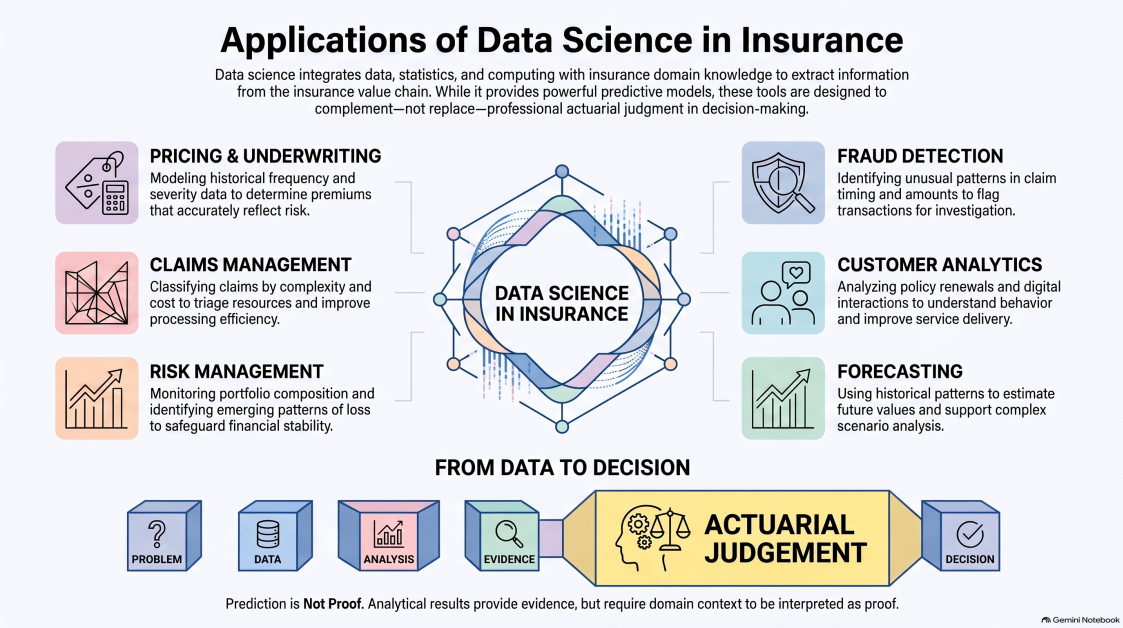


Figure 2.4: Data Science Applications in the Insurance Industry

The methods used in each area may differ, but they generally follow the same data-science principles introduced earlier in this chapter: define a problem, obtain and prepare relevant data, explore the data, develop an appropriate analysis or model, and communicate the results.

2.6.1 Pricing and Underwriting

Pricing is one of the most important applications of data science in insurance. Insurers need to estimate the expected cost of claims and determine premiums that appropriately reflect the risks being insured.

Historical policy and claims data can be used to investigate how insurance outcomes vary with characteristics such as age, geographical area, vehicle characteristics, coverage, or other risk factors.

For example, a motor insurer may analyse claim experience to investigate whether claim frequency varies with the age of the policyholder. A statistical model can then be used to estimate expected claim frequency for different combinations of risk characteristics.

A simple representation is

$$\text{Expected loss} \approx \text{Expected frequency} \times \text{Expected severity}.$$

Data science can support both components. Historical data can be used to model the number of claims and the amount of individual claims, while other variables can be used to explain differences in risk.

Data can also support underwriting decisions. An insurer may use information available at the time a policy is requested to assess the characteristics of the risk and determine whether additional information or underwriting review is required.

> **Data science does not replace actuarial pricing**
>
> A predictive model provides estimates based on data and modelling assumptions. Pricing decisions also require consideration of expenses, capital, reinsurance, regulatory requirements, market conditions, and professional judgement.

2.6.2 Claims Management

Claims generate a substantial amount of information that can be analysed to improve the claims process.

Data science can be used to understand claim patterns, estimate expected claim costs, predict claims outcomes, and support claims triage. For example, claims may be classified according to characteristics such as type, estimated cost, complexity, or likelihood of requiring further investigation.

Predictive models can help identify claims that are likely to become large or require additional attention. This can allow claims staff to allocate resources more effectively.

Data can also be used to monitor the development of claims over time. Comparing observed claim experience with previous experience or expected outcomes can provide information for claims management and portfolio monitoring.

The objective is not necessarily to automate the entire claims process. Instead, analytical results can provide additional information to claims professionals when making decisions.

2.6.3 Fraud Detection

Insurance fraud can result in financial losses and can affect the overall cost of insurance. Data science can assist insurers in identifying claims or transactions that have unusual characteristics.

For example, an insurer may analyse relationships among:

- policy characteristics;
- claim characteristics;
- claim timing;
- claim amounts; and
- previous claims.

Statistical or machine-learning methods can be used to identify patterns associated with claims that require further investigation.

An important distinction is that a model identifying an unusual claim does **not** establish that fraud has occurred. It identifies a case that may deserve additional investigation.

> **!** Prediction is not proof
>
> A data-science model can identify patterns or assign a probability to an outcome, but the result should not automatically be interpreted as evidence that a particular policyholder has committed fraud.

This distinction illustrates the importance of combining analytical results with domain knowledge and appropriate professional procedures.

2.6.4 Customer Analytics

Data science can also be used to understand customer behaviour and interactions with insurers.

Examples include analysing:

- policy renewals;
- cancellations and lapses;
- customer enquiries;
- product preferences; and
- patterns of interaction with digital services.

For example, an insurer may wish to identify policies that have a high probability of not being renewed. Historical policy data can be used to develop a predictive model and identify characteristics associated with higher lapse rates.

The resulting information can support decisions about customer communication, product design, and service delivery.

Customer analytics therefore extends the use of insurance data beyond the traditional assessment of claims risk. It considers how policyholders interact with the insurer and how these behaviours may affect the business.

2.6.5 Risk Management and Forecasting

Data science can support broader risk-management and forecasting activities.

Insurers need to monitor changes in their portfolios and anticipate future outcomes. Historical data can be analysed to identify trends, changes in risk characteristics, and unusual patterns.

Examples include:

- forecasting claim volumes;
- monitoring changes in claim severity;
- analysing portfolio composition;
- identifying emerging patterns of loss; and
- supporting scenario analysis.

Forecasting methods can be used to estimate future values based on historical patterns and other relevant information. However, forecasts are subject to uncertainty, particularly when future conditions differ from those observed in the historical data.

For example, changes in economic conditions, technology, regulation, customer behaviour, or the underlying risk environment may affect future insurance experience.

2.6.6 From Data to Insurance Decisions

The applications described above illustrate that data science is not a single insurance technique. It is a collection of methods that can support different types of insurance decisions.

The common process can be represented as:

Insurance problem → Relevant data → Analysis or model → Evidence → Actuarial judgement → Decision.

For example, in pricing, data may provide evidence about differences in expected claims between groups of policyholders. In claims management, data may help identify claims requiring additional attention. In fraud detection, data may identify unusual patterns for investigation.

In each case, the analytical result is only one part of the decision-making process. The result must be interpreted in the context of the insurance problem, the quality of the data, the assumptions of the analysis, and the consequences of the decision.

This is why data science and actuarial science are complementary. Data science provides methods for extracting information from data, while actuarial knowledge provides the context needed to interpret that information and apply it to insurance decisions.

> The same data-science principles apply across applications
>
> Although pricing, claims, fraud detection, and customer analytics address different problems, they follow the same fundamental process:
>
> Question → Data → Analysis → Evidence → Decision.

Learning the data-science process therefore provides a foundation for many different actuarial applications.

3 Chapter 3: Transforming and Summarizing Data

Learning Outcomes

By the end of this chapter, students should be able to:

1. Explain the principles of tidy data and recognize common untidy structures.
2. Import, inspect, and describe an insurance dataset in R.
3. Select, filter, and transform variables using `dplyr`.
4. Combine data-manipulation operations using pipes.
5. Group and summarize insurance data by rating variables.
6. Calculate and interpret basic claims summaries, including claim count, aggregate claims, and severity.
7. Distinguish claim count, frequency, severity, and risk premium.
8. Interpret claims experience in relation to the volume of data supporting it.
9. Recognize the limitations of unadjusted group comparisons in actuarial analysis.

3.1 Why Data Transformation Matters

Insurance data are rarely collected in exactly the form needed for analysis. A dataset may contain individual policy records, claim records, exposure measures, and characteristics of policyholders or insured objects. Before these data can be used to answer an actuarial question, they often need to be **transformed, organized, and summarized**.

Data transformation refers to changing the structure or representation of data so that they are suitable for a particular analysis. This may involve creating new variables, changing the units of measurement, grouping observations, filtering records, or combining information from different variables.

For example, consider a motor insurance dataset containing the policyholder's age, ZIP code, number of claims, exposure, and aggregate claim amount. The raw variables may be sufficient for storing the data, but they do not immediately provide measures such as claim frequency or average claim severity. These measures must be calculated from the underlying data.

$$\text{Raw data} \longrightarrow \text{Transformed data} \longrightarrow \text{Summary} \longrightarrow \text{Actuarial insight.}$$

The purpose of transformation is therefore not simply to make the dataset look cleaner. It is to create variables and summaries that help answer a specific insurance question.

For example, an insurer may want to investigate whether claim experience differs by age group. The original dataset may contain the exact age of each policyholder, but an analysis may require

age groups such as 18–25, 26–35, 36–45, and so on. Similarly, comparing the number of claims between two groups without considering exposure can be misleading. A group with more policies will generally have more claims simply because it has more opportunities for claims to occur.

This is why actuarial analysis often transforms the original data into measures that account for the volume of business. For example, claim frequency can be calculated as

$$\text{Claim Frequency} = \frac{\text{Number of Claims}}{\text{Exposure}},$$

while average claim severity can be calculated as

$$\text{Average Severity} = \frac{\text{Aggregate Claim Amount}}{\text{Number of Claims}}.$$

These quantities provide more meaningful information about the underlying risk than claim counts or aggregate claim amounts alone.

Transformation depends on the question

There is no single transformation that is appropriate for every analysis. The same raw dataset may be transformed in different ways depending on the actuarial question being investigated. For example, age may be used as an exact numerical variable when studying how risk changes with age, or it may be grouped into age bands when constructing an easily interpretable rating structure.

Data summarization is the next step. A large dataset may contain thousands of individual observations, making it difficult to identify patterns by inspecting individual records. Summaries condense the data into a smaller set of informative quantities, such as counts, totals, averages, medians, and measures of variability.

In insurance, summaries are often produced separately for different risk groups. For example, an insurer might compare exposure, claim frequency, and average severity across ZIP codes:

ZIP	Exposure	Claims	Claim Frequency	Average Severity
0	206.84	29	0.140	28,328
1	11,080.63	1,593	0.144	72,931
2	7,782.63	1,008	0.130	59,278
3	7,587.56	1,038	0.137	56,829

Such a table is much easier to interpret than the underlying individual policy records. It allows an analyst to compare both **risk experience** and **portfolio volume** across groups.
However, a summary is not itself an explanation. Differences between groups may arise for many reasons, including differences in the underlying risk characteristics, random variation, or differences in the composition of the portfolio. Data transformation and summarization therefore provide the evidence needed for further analysis; they do not replace actuarial judgement.
The central idea of this chapter is therefore:

Transform the data to represent the question, then summarize it to reveal the evidence.

These skills form an essential foundation for exploratory data analysis, statistical modelling, and insurance rating. Before fitting a model, an actuary should understand what the data contain, how the variables have been constructed, and what the basic summaries reveal about the portfolio.

3.2 Inspecting Data

Before transforming or summarizing a dataset, we first need to understand its structure and the meaning of the information it contains. This initial inspection is an important part of the data-science workflow because subsequent calculations are meaningful only when the underlying data have been correctly understood.

A dataset consists of **observations** and **variables**. An observation represents the unit being studied, while a variable records a characteristic of that unit. In insurance data, an observation might represent a policy, a policy-year, or another defined unit of exposure. Variables may describe the policyholder, the insured object, the exposure, or the claims arising during the observation period.

For example, a claims experience dataset may contain variables such as:

Variable	Description	Possible type
`age_policyholder`	Age of the policyholder	Numeric
`nclaims`	Number of claims	Numeric
`exposure`	Amount of exposure during the observation period	Numeric
`amount`	Aggregate claim amount	Numeric
`power`	Vehicle power	Numeric
`bm`	Bonus-malus level	Categorical or ordinal
`zip`	Geographic area	Categorical

Some of these variables describe the **outcome of insurance experience**, while others describe characteristics that may be associated with that experience. In actuarial applications, variables that may be used to differentiate risks for rating purposes are commonly called **rating variables**. They may include characteristics such as age, geographic area, vehicle characteristics, or previous claims experience.

3.2.1 Observations and variables

The number of observations is one of the first things we should establish. It tells us how many units are represented in the dataset and provides context for all subsequent calculations.

The number of observations, however, is not necessarily the same as the amount of insurance business represented by the data. In insurance, **exposure** provides an important measure of portfolio volume. For example, two groups may contain the same number of policy records but represent different amounts of exposure because policies may have been in force for different lengths of time.

This distinction becomes particularly important when comparing claims experience. A group with more claims does not necessarily have a higher underlying claim risk if it also has substantially greater exposure.

3.2.2 Variable types

Before performing calculations, each variable should be classified according to its meaning.

A **numeric variable** represents a quantity for which numerical operations are meaningful. Examples include age, exposure, number of claims, and claim amount. Such variables can be summarized using quantities such as means, medians, minima, maxima, and standard deviations.

A **categorical variable** represents membership in a group or category. Examples include vehicle type, geographic area, and rating class.

A variable should be classified according to its **meaning**, rather than simply according to the way its values are stored. For example, a ZIP code may be recorded using numbers such as `0`, `1`, `2`, and `3`, but these numbers identify geographic groups rather than quantities. Calculating their arithmetic mean would therefore have no useful interpretation.

> **!** Numeric values are not necessarily numeric variables
>
> The fact that a variable contains numbers does not mean that arithmetic operations on those numbers are meaningful.
> For example, if `zip` takes the values `0`, `1`, `2`, and `3`, it should normally be treated as a categorical variable. The value `3` does not represent three units of geographic area, nor is the average ZIP code an actuarial quantity of interest.

Some variables may also have an **ordinal** interpretation. For example, a bonus-malus level may consist of ordered categories in which higher or lower levels indicate different positions in a rating system. Although the categories have an order, the numerical codes should not automatically be treated as measurements with equal numerical distances.

3.2.3 Inspecting the values

Variable names and types are not sufficient. We should also examine the values recorded in each variable.

For numeric variables, useful initial questions include:

- What are the minimum and maximum values?
- Are there missing values?
- Are there zero or negative values?
- Are there unusually large or small observations?
- What units are being used?

For categorical variables, we should consider:

- What categories occur?
- How many observations belong to each category?
- Are there unexpected categories?
- Are category labels recorded consistently?

These checks are particularly important for insurance data. For example, a negative claim amount may indicate a data-entry or coding issue, while a zero claim count may be perfectly valid because most policyholders may have no claim during a given observation period.

3.2.4 Tidy data

Once the basic structure of the data has been identified, the next question is whether the data are organized in a form that is convenient for analysis.

A useful standard is **tidy data**. A dataset is tidy when:

1. each variable has its own column;
2. each observation has its own row; and
3. each value has its own cell.

This structure separates the concepts of observations and variables clearly. It also makes subsequent transformation, summarization, and visualization more systematic.

For example, suppose a claims dataset records the policy number, year, and number of claims. A tidy representation might look like this:

Policy	Year	Claims
120002	2009	0
120002	2010	1
120003	2009	2
120003	2010	0

Here, `Policy`, `Year`, and `Claims` are variables, while each row represents one policy-year observation. Each value occupies a single cell.

The same information can, however, be stored in several other forms. These representations may be convenient for reporting or data entry, but they are less convenient for data analysis.

3.2.4.1 Example 1: Variables stored in column names

Consider the following representation:

Policy	Claims_2009	Claims_2010
120002	0	1
120003	2	0

The values for `Year` have been incorporated into the column names. This means that `Year` is no longer an explicit variable in the dataset.

This structure is often convenient for displaying results, but it becomes less convenient when we want to perform an operation such as:

```
claims %>%
  group_by(Year) %>%
  summarise(total_claims = sum(Claims))
```

In the tidy representation, `Year` is a variable and can therefore be used directly for grouping and summarization. In the representation above, the years must first be extracted from the column names.

This type of transformation is commonly referred to as converting data from **wide** to **long** format. In `tidyr`, the `pivot_longer()` function is used for this purpose.

3.2.4.2 Example 2: Multiple variables stored in one column

Suppose claims information is recorded as follows:

Policy	Year	Information
120002	2009	Claims:0
120002	2010	Claims:1
120003	2009	Claims:2
120003	2010	Claims:0

Here, the column `Information` contains both the name of the variable (`Claims`) and its value. The actual claim count is therefore not stored as a separate variable.

A more useful representation is:

Policy	Year	Claims
120002	2009	0
120002	2010	1
120003	2009	2
120003	2010	0

Separating the variable name from its values makes the data easier to filter, summarize, and visualize.

3.2.4.3 Example 3: Multiple observations stored in one row

Consider a dataset in which a policy's claims experience is recorded in a single row:

Policy	Claims_1	Claims_2	Claims_3
120002	0	1	0
120003	2	0	1

Suppose `Claims_1`, `Claims_2`, and `Claims_3` represent claims in three successive years. The year information is again encoded in the column names rather than represented by a separate variable.

A tidy representation would instead have one row for each policy-year observation:

Policy	Year	Claims
120002	1	0
120002	2	1
120002	3	0
120003	1	2
120003	2	0
120003	3	1

This structure makes it straightforward to perform operations such as calculating total claims by year or comparing claims experience across policies.

The process of converting data from the first representation to the second is another example of transforming **wide data into long data**.

> **i** Why tidy data matter
>
> Tidy data provide a consistent structure for subsequent operations. Once each observation, variable, and value has a clear place in the dataset, operations such as filtering observations, creating variables, grouping records, and calculating summaries become easier to express and interpret.
> Tidy data are not necessarily the only possible way to store data. Wide formats can be useful for reporting and presentation. The purpose of tidy data is to provide a structure that is particularly convenient for data manipulation and analysis.

In practice, data imported from spreadsheets, databases, or other sources may not initially be tidy. A substantial part of data preparation therefore involves reshaping the data into a suitable structure before analysis. The `tidyr` package provides functions such as `pivot_longer()` and `pivot_wider()` for converting between long and wide representations.

3.2.5 From inspection to transformation

Inspecting the data does not mean that we have finished preparing them. Rather, it gives us the information needed to decide **what transformation is appropriate**.

For example, after inspecting a motor insurance dataset, we might decide to:

- retain only variables relevant to the analysis;
- select policies from a particular period;
- create a claim frequency variable;
- group policyholders into age bands;
- classify ZIP codes as rating groups; or
- calculate claim severity for observations with at least one claim.

These operations correspond to the next stage of the workflow:

$$\text{Inspect} \longrightarrow \text{Transform} \longrightarrow \text{Summarize}.$$

In R, the `dplyr` package provides a set of functions designed for these common operations. The most important functions introduced in this chapter are `select()`, `filter()`, `mutate()`, `group_by()`, and `summarise()`.

> **i** Further reference: `dplyr`
>
> The `dplyr` package provides a consistent set of tools for transforming and summarizing data in R. This chapter introduces its core functions for common data-manipulation tasks.
> For the complete reference and additional examples, see the official `dplyr` documentation.

We begin by loading and briefly inspecting the claims dataset that will be used throughout this section. We then consider how data should be structured for analysis before introducing the operations used to manipulate and transform the data step by step.

3.3 The Claims Experience Dataset

Throughout this chapter, we will use a claims dataset to demonstrate how R can be used to import, inspect, transform, and analyse insurance data. The dataset contains information on insured risks and their claims experience.

3.3.1 Importing the data

The data are stored in the `data` folder of the project directory. We can therefore import the dataset using a relative file path without changing the working directory manually.

```
library(dplyr)
library(readr)

claims <- read_csv("../data/ClaimsExperienceData.csv")
```

3.3.2 Inspecting the data

After importing a dataset, it is good practice to inspect its basic structure before beginning any analysis. This helps us understand the observations and variables available and whether R has assigned appropriate data types to the variables.

```
glimpse(claims)
```

```
Rows: 5,639
Columns: 23
$ PolicyNum     <dbl> 120002, 120002, 120002, 120002, 120002, 120003, 120003, ~
$ Year          <dbl> 2006, 2007, 2008, 2009, 2010, 2006, 2007, 2008, 2009, 20~
$ LnCoverage    <dbl> 3.123002, 3.220740, 3.037427, 3.084324, 3.157489, 4.5986~
$ BCcov         <dbl> 22714456, 25046646, 20851525, 21852696, 23511493, 993533~
$ Premium       <dbl> 9313, 8767, 7090, 8522, 7994, 40735, 39728, 42280, 36893~
$ Freq          <dbl> 0, 0, 0, 0, 1, 0, 5, 1, 2, 1, 3, 1, 0, 3, 1, 0, 5, 0, 2,~
$ Deduct        <dbl> 1000, 1000, 1000, 1000, 1000, 5000, 5000, 5000, 5000, 50~
$ y             <dbl> 0.00, 0.00, 0.00, 0.00, 6838.87, 0.00, 15500.00, 8775.00~
$ lny           <dbl> 0.000000, 0.000000, 0.000000, 0.000000, 8.830378, 0.0000~
$ yAvg          <dbl> 0.000, 0.000, 0.000, 0.000, 6838.870, 0.000, 3100.000, 8~
$ lnDeduct      <dbl> 6.907755, 6.907755, 6.907755, 6.907755, 6.907755, 8.5171~
$ Fire5         <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ NoClaimCredit <dbl> 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ TypeCity      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ TypeCounty    <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ TypeMisc      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ TypeSchool    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ TypeTown      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ TypeVillage   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ AC00          <dbl> 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ AC05          <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ AC10          <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ AC15          <dbl> 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
```

The `glimpse()` function provides a compact overview of the dataset, including the number of observations and variables, variable names, data types, and example values.

We can also use `head()` to view the first few observations:

```
head(claims)
```

```
# A tibble: 6 x 23
  PolicyNum  Year LnCoverage    BCcov Premium  Freq Deduct     y   lny  yAvg
      <dbl> <dbl>      <dbl>    <dbl>   <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>
1    120002  2006       3.12 22714456    9313     0   1000     0   0       0
2    120002  2007       3.22 25046646    8767     0   1000     0   0       0
3    120002  2008       3.04 20851525    7090     0   1000     0   0       0
```

```
4    120002  2009       3.08 21852696    8522     0   1000    0   0       0
5    120002  2010       3.16 23511493    7994     1   1000 6839.  8.83 6839.
6    120003  2006       4.60 99353377   40735     0   5000    0   0       0
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

3.3.3 Description of the rating variables

The following variables represent characteristics of the insured risks that may be related to claims outcomes.

Variable	Description
`EntityType`	Categorical variable with six types: Village, City, County, Misc, School, or Town
`LnCoverage`	Total building and content coverage, in logarithmic millions of dollars
`LnDeduct`	Deductible, in logarithmic dollars
`AlarmCredit`	Categorical variable with four levels (0, 5, 10, or 15), representing credits for automatic smoke alarms in main rooms
`NoClaimCredit`	Binary variable indicating whether there were no claims in the past two years
`Fire5`	Binary variable indicating whether the fire class is below 5; the fire class ranges from 0 to 10

These variables are potential **rating factors** because they describe characteristics of the insured risks that may help explain differences in claims experience. Their roles will become clearer as we use the dataset for data manipulation and analysis.

3.4 Selecting and Filtering Data

Once a dataset has been inspected and organized in a tidy form, the next step is to work with the observations and variables relevant to the question being investigated. A dataset may contain many variables and observations, but an analysis rarely requires all of them at once.

Two common operations are therefore **selecting variables** and **filtering observations**. In `dplyr`, these operations are performed using `select()` and `filter()`.

3.4.1 Selecting variables

The `select()` function is used to choose particular variables from a data frame. This is useful when a dataset contains many columns but only a subset is relevant to the current analysis.

For example, suppose a claims dataset contains information on policy number, year, coverage, claim frequency, and several other variables. We may select only the variables needed to examine claims experience:

```
claims %>%
  select(PolicyNum, Year, LnCoverage, y, Freq)
```

```
# A tibble: 5,639 x 5
   PolicyNum  Year LnCoverage      y  Freq
       <dbl> <dbl>      <dbl>  <dbl> <dbl>
 1    120002  2006       3.12     0      0
 2    120002  2007       3.22     0      0
 3    120002  2008       3.04     0      0
 4    120002  2009       3.08     0      0
 5    120002  2010       3.16  6839.     1
 6    120003  2006       4.60     0      0
 7    120003  2007       4.65 15500      5
 8    120003  2008       4.71  8775      1
 9    120003  2009       4.78 37471.     2
10    120003  2010       4.74  9711.     1
# i 5,629 more rows
```

The result contains the selected columns while retaining all observations.

The names supplied to `select()` refer to variables in the data frame. We can therefore use `select()` to create a simpler view of a dataset without changing the original data.

For example,

```
new_table <- claims %>%
  select(PolicyNum, Year, LnCoverage, y, Freq)
```

creates a new data frame containing only the selected variables. The original `claims` data remain unchanged.

> **i** Selecting variables is not the same as deleting data
>
> `select()` determines which variables are included in the resulting data frame. Unless the result is assigned back to the original object, the original dataset is not modified.
> Selecting variables is particularly useful when checking whether related variables have been recorded consistently or when preparing a smaller table for further analysis.

3.4.2 Filtering observations

While `select()` works with variables, `filter()` works with **observations**. It retains only the rows that satisfy a specified condition.

For example, suppose we want to examine claims experience for the year 2010:

```
claims %>%
  filter(Year == 2010)
```

```
# A tibble: 1,110 x 23
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2010       3.16  23511493    7994     1   1000  6839.  8.83  6839.
 2    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
 3    120004  2010       3.46  31853574   17839     1   1000 10324.  9.24 10324.
 4    120005  2010       3.49  32713698   12431     0   5000     0   0        0
 5    120008  2010       3.86  47321724   16090     1    500  3470.  8.15  3470.
 6    120009  2010       4.28  72286976   15180     0  25000     0   0        0
 7    120010  2010       4.60  99135847   15861     1  25000 35000  10.5  35000
 8    120012  2010       6.34 569366096  222052    19    500 83935. 11.3   4418.
 9    120013  2010       5.10 163526511   58869     5    500 63530. 11.1  12706.
10    120015  2010       4.68 107382645   38657    11   1000 38117. 10.5   3465.
# i 1,100 more rows
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

The condition `Year == 2010` is evaluated for each observation. Only observations for which the condition is `TRUE` are retained.

The equality operator in R is `==`. It should not be confused with `=`, which is commonly used to assign a value to an object or to specify an argument.

Filtering can also be used with other logical conditions. For example, to retain observations with positive claim counts:

```
claims %>%
  filter(Freq > 0)
```

```
# A tibble: 1,679 x 23
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2010       3.16  23511493    7994     1   1000 6.84e3  8.83  6839.
 2    120003  2007       4.65 104546824   39728     5   5000 1.55e4  9.65  3100
 3    120003  2008       4.71 111265009   42280     1   5000 8.78e3  9.08  8775
 4    120003  2009       4.78 119005219   36893     2   5000 3.75e4 10.5  18735.
 5    120003  2010       4.74 114646079   36687     1   5000 9.71e3  9.18  9711.
 6    120004  2006       3.25  25879634   16823     3    500 1.30e4  9.47  4334.
 7    120004  2007       3.39  29720359   20507     1    500 3.27e4 10.4  32733.
 8    120004  2009       3.50  33064745   18185     3   1000 4.26e4 10.7  14200.
 9    120004  2010       3.46  31853574   17839     1   1000 1.03e4  9.24 10324.
10    120005  2007       3.50  33128746   14245     5   5000 1.44e5 11.9  28801.
```

```
# i 1,669 more rows
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

This is useful when the analysis concerns claim severity, because observations with no claims do not have an observed claim severity.

Multiple conditions can be combined. For example, we may want observations from 2010 that also have at least one claim:

```
claims %>%
  filter(Year == 2010 & Freq > 0)
```

```
# A tibble: 403 x 23
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2010       3.16  23511493    7994     1   1000  6839.  8.83  6839.
 2    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
 3    120004  2010       3.46  31853574   17839     1   1000 10324.  9.24 10324.
 4    120008  2010       3.86  47321724   16090     1    500  3470.  8.15  3470.
 5    120010  2010       4.60  99135847   15861     1  25000 35000  10.5  35000
 6    120012  2010       6.34 569366096  222052    19    500 83935. 11.3   4418.
 7    120013  2010       5.10 163526511   58869     5    500 63530. 11.1  12706.
 8    120015  2010       4.68 107382645   38657    11   1000 38117. 10.5   3465.
 9    120017  2010       4.38  79719144   22321     7   5000  9132.  9.12  1305.
10    120018  2010       4.75 115589417   39301     1   5000  9649.  9.17  9649.
# i 393 more rows
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

The operator `&` means **and**: both conditions must be satisfied.

Similarly, `|` means **or**. For example,

```
claims %>%
  filter(Year == 2009 | Year == 2010)
```

```
# A tibble: 2,222 x 23
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2009       3.08  21852696    8522     0   1000      0   0        0
 2    120002  2010       3.16  23511493    7994     1   1000  6839.  8.83  6839.
 3    120003  2009       4.78 119005219   36893     2   5000 37471. 10.5  18735.
```

```
 4    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
 5    120004  2009       3.50  33064745   18185     3   1000 42600. 10.7  14200.
 6    120004  2010       3.46  31853574   17839     1   1000 10324.  9.24 10324.
 7    120005  2009       3.61  36974202   14050     2   5000 27481. 10.2  13740.
 8    120005  2010       3.49  32713698   12431     0   5000     0   0        0
 9    120008  2009       3.81  45360920   18144     2    500 12015.  9.39  6007.
10    120008  2010       3.86  47321724   16090     1    500  3470.  8.15  3470.
# i 2,212 more rows
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

retains observations from either 2009 or 2010.

3.4.3 Selecting from a set of values

When a variable needs to be compared with several possible values, the `%in%` operator provides a convenient alternative.

For example,

```
claims %>%
  filter(Year %in% c(2009, 2010))
```

```
# A tibble: 2,222 x 23
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2009       3.08  21852696    8522     0   1000     0   0        0
 2    120002  2010       3.16  23511493    7994     1   1000  6839.  8.83  6839.
 3    120003  2009       4.78 119005219   36893     2   5000 37471. 10.5  18735.
 4    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
 5    120004  2009       3.50  33064745   18185     3   1000 42600. 10.7  14200.
 6    120004  2010       3.46  31853574   17839     1   1000 10324.  9.24 10324.
 7    120005  2009       3.61  36974202   14050     2   5000 27481. 10.2  13740.
 8    120005  2010       3.49  32713698   12431     0   5000     0   0        0
 9    120008  2009       3.81  45360920   18144     2    500 12015.  9.39  6007.
10    120008  2010       3.86  47321724   16090     1    500  3470.  8.15  3470.
# i 2,212 more rows
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

retains observations for which `Year` is either 2009 or 2010. The expression

```
Year %in% c(2009, 2010)
```

checks whether the value of `Year` belongs to the specified set of values. This is often more readable than writing several equality conditions joined by `|`. The source material uses the same approach when filtering the claims data by multiple years.

3.4.4 Combining selection and filtering

In practice, selecting variables and filtering observations are often performed together.

For example, we may want to examine only the policy number, year, coverage, and claim variables for policies observed in 2009 and 2010:

```
claims %>%
  select(PolicyNum, Year, LnCoverage, y, Freq) %>%
  filter(Year %in% c(2009, 2010))
```

```
# A tibble: 2,222 x 5
   PolicyNum  Year LnCoverage      y  Freq
       <dbl> <dbl>      <dbl>  <dbl> <dbl>
 1    120002  2009       3.08     0      0
 2    120002  2010       3.16  6839.     1
 3    120003  2009       4.78 37471.     2
 4    120003  2010       4.74  9711.     1
 5    120004  2009       3.50 42600.     3
 6    120004  2010       3.46 10324.     1
 7    120005  2009       3.61 27481.     2
 8    120005  2010       3.49     0      0
 9    120008  2009       3.81 12015.     2
10    120008  2010       3.86  3470.     1
# i 2,212 more rows
```

The operations are performed from left to right. First, the relevant variables are selected. The resulting data are then filtered to retain the required observations.

This illustrates an important principle of data transformation:

$$\text{Select relevant variables} \longrightarrow \text{Filter relevant observations} \longrightarrow \text{Continue the analysis.}$$

The same operations could be performed separately by creating intermediate data frames. However, combining them into a sequence makes the analytical process easier to follow.

3.4.5 Why selection and filtering matter in actuarial analysis

Selection and filtering are not merely programming operations. They determine **which part of the portfolio is being analyzed**.

For example, an analyst might investigate:

- policies written during a particular year;
- policyholders within a particular age range;
- policies belonging to a particular ZIP code;
- observations with positive claims;
- a particular set of rating classes.

The resulting summaries describe only the observations that have been retained. Therefore, the filtering criteria should always be stated clearly when reporting an analysis.

This is especially important when analyzing claims. If zero-claim observations are removed before calculating average severity, the resulting statistic describes the severity **conditional on a claim occurring**. It does not describe the average claim amount across all policyholders.

> **!** Always ask: what observations are being analyzed?
>
> A summary statistic is meaningful only when we know which observations were included in its calculation.
> For example,
>
> $$\frac{\text{Aggregate Claim Amount}}{\text{Number of Claims}}$$
>
> describes average claim severity, whereas excluding observations with zero claims before calculating this quantity does not mean that zero claims have been ignored from the underlying portfolio; they simply do not contribute to the denominator because they contain no claim whose severity can be measured.

Selection and filtering therefore provide the basic tools for defining the **scope of an analysis**. Once the relevant observations and variables have been identified, the next step is often to create new variables that express quantities more directly relevant to the actuarial question.

3.5 Creating and Transforming Variables

After selecting the relevant observations and variables, the next step is often to create variables that are more useful for the analysis. A raw dataset may contain the information needed to calculate an actuarial quantity, but that quantity may not yet exist as a variable in the dataset.

For example, a claims dataset may contain the total claim amount and the number of claims, but not the average claim severity. We can create a new variable from the existing variables:

$$\text{Average Severity} = \frac{\text{Aggregate Claim Amount}}{\text{Number of Claims}}.$$

In `dplyr`, the `mutate()` function is used to create new variables or transform existing ones. The basic form is:

```
data %>%
  mutate(new_variable = expression)
```

The expression on the right-hand side determines the values of the new variable. Existing variables can be used directly in the expression.

3.5.1 Creating a new variable

Suppose the claims dataset contains `y`, representing aggregate claim amount, and `Freq`, representing the number of claims. We can create a variable called `yAvg` representing average claim severity:

```
claims %>%
  mutate(yAvg = y / Freq)
```

```
# A tibble: 5,639 x 23
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2006       3.12  22714456    9313     0   1000     0   0      NaN
 2    120002  2007       3.22  25046646    8767     0   1000     0   0      NaN
 3    120002  2008       3.04  20851525    7090     0   1000     0   0      NaN
 4    120002  2009       3.08  21852696    8522     0   1000     0   0      NaN
 5    120002  2010       3.16  23511493    7994     1   1000  6839.  8.83  6839.
 6    120003  2006       4.60  99353377   40735     0   5000     0   0      NaN
 7    120003  2007       4.65 104546824   39728     5   5000 15500   9.65  3100
 8    120003  2008       4.71 111265009   42280     1   5000  8775   9.08  8775
 9    120003  2009       4.78 119005219   36893     2   5000 37471. 10.5  18735.
10    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
# i 5,629 more rows
# i 13 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>
```

The new variable is calculated separately for each observation. The original variables remain available in the resulting data frame.

We can also save the transformed data as a new object:

```
claims_new <- claims %>%
  mutate(yAvg = y / Freq)
```

This creates `claims_new` with the additional variable `yAvg`, while leaving `claims` unchanged.

The source material similarly uses `mutate()` to create `LnCoverageChecked` from the underlying coverage variable and `yAvgChecked` from claim amount divided by claim frequency.

3.5.2 Transforming an existing variable

Creating a new variable is not limited to arithmetic operations. A transformation may also change the scale or representation of an existing quantity.

For example, the source claims data contains coverage represented on a logarithmic scale. The underlying coverage variable can be transformed using

```
claims %>%
  mutate(LnCoverageChecked = log(BCcov / 10^6))
```

```
# A tibble: 5,639 x 24
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2006       3.12  22714456    9313     0   1000      0   0        0
 2    120002  2007       3.22  25046646    8767     0   1000      0   0        0
 3    120002  2008       3.04  20851525    7090     0   1000      0   0        0
 4    120002  2009       3.08  21852696    8522     0   1000      0   0        0
 5    120002  2010       3.16  23511493    7994     1   1000  6839.   8.83  6839.
 6    120003  2006       4.60  99353377   40735     0   5000      0   0        0
 7    120003  2007       4.65 104546824   39728     5   5000 15500    9.65  3100
 8    120003  2008       4.71 111265009   42280     1   5000  8775    9.08  8775
 9    120003  2009       4.78 119005219   36893     2   5000 37471.  10.5  18735.
10    120003  2010       4.74 114646079   36687     1   5000  9711.   9.18  9711.
# i 5,629 more rows
# i 14 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>, LnCoverageChecked <dbl>
```

Here, the new variable is calculated by first converting coverage into millions of dollars and then taking its logarithm.

The important principle is that a transformation should have a clear interpretation. A variable should not be transformed simply because a particular function is available. The analyst should understand what the resulting quantity represents and why it is appropriate for the analysis.

3.5.3 Creating actuarial quantities

Many useful insurance variables are derived from existing information.

For example, suppose an observation contains claim count N and exposure E. Claim frequency can be represented by

$$\text{Frequency} = \frac{N}{E}.$$

Similarly, if Y denotes aggregate claim amount and N denotes the number of claims, average claim severity is

$$\text{Severity} = \frac{Y}{N}, \qquad N > 0.$$

These quantities can be derived from the variables in the dataset. In particular, the average claim severity can be calculated as the total claim amount divided by the number of claims:

```
claims %>%
  mutate(
    severity = y / Freq
  )
```

```
# A tibble: 5,639 x 24
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2006       3.12  22714456    9313     0   1000     0   0        0
 2    120002  2007       3.22  25046646    8767     0   1000     0   0        0
 3    120002  2008       3.04  20851525    7090     0   1000     0   0        0
 4    120002  2009       3.08  21852696    8522     0   1000     0   0        0
 5    120002  2010       3.16  23511493    7994     1   1000  6839.  8.83  6839.
 6    120003  2006       4.60  99353377   40735     0   5000     0   0        0
 7    120003  2007       4.65 104546824   39728     5   5000 15500   9.65  3100
 8    120003  2008       4.71 111265009   42280     1   5000  8775   9.08  8775
 9    120003  2009       4.78 119005219   36893     2   5000 37471. 10.5  18735.
10    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
# i 5,629 more rows
# i 14 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>, severity <dbl>
```

The precise variable names and definitions depend on the dataset. The important idea is that **derived variables express information already contained in the data in a form that is more directly relevant to the actuarial question**.

3.5.4 Checking the result

Creating a variable does not guarantee that the resulting values are meaningful. Derived variables should be checked, particularly when the calculation involves division, logarithms, or other operations with restrictions.

For example, the calculation

$$\frac{y}{\text{Freq}}$$

is not defined when `Freq = 0`. In R, dividing zero claim observations may produce `NaN` when both the numerator and denominator are zero. The source material identifies this issue when calculating average claims and shows that the resulting `NaN` values arise from division by zero.

A better approach is usually to consider the meaning of the quantity before calculating it. Average claim severity is defined for observations where a claim has occurred, so observations with zero claims should be handled explicitly rather than simply replacing an undefined result after the calculation.

For example:

```
claims %>%
  mutate(
    severity = if_else(Freq > 0, y / Freq, NA_real_)
  )
```

```
# A tibble: 5,639 x 24
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2006       3.12  22714456    9313     0   1000      0   0        0
 2    120002  2007       3.22  25046646    8767     0   1000      0   0        0
 3    120002  2008       3.04  20851525    7090     0   1000      0   0        0
 4    120002  2009       3.08  21852696    8522     0   1000      0   0        0
 5    120002  2010       3.16  23511493    7994     1   1000  6839.   8.83  6839.
 6    120003  2006       4.60  99353377   40735     0   5000      0   0        0
 7    120003  2007       4.65 104546824   39728     5   5000 15500    9.65  3100
 8    120003  2008       4.71 111265009   42280     1   5000  8775    9.08  8775
 9    120003  2009       4.78 119005219   36893     2   5000 37471.  10.5  18735.
10    120003  2010       4.74 114646079   36687     1   5000  9711.   9.18  9711.
# i 5,629 more rows
# i 14 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>, severity <dbl>
```

This records the severity as missing when there are no claims, reflecting the fact that there is no observed claim severity for such an observation.

> **!** Undefined is not the same as zero
>
> An observation with zero claims does not have a claim severity of zero.
> If
>
> $$N = 0,$$
>
> then
>
> $$\frac{Y}{N}$$

is undefined. A zero claim count means that no claim occurred; it does not mean that the severity of a claim was zero.

3.5.5 Transforming variables for analysis

Variable transformation can also make the data easier to analyze. A continuous variable may, for example, be converted into groups when the analysis requires comparisons between categories.

Suppose age is recorded as an exact numerical variable. We might create age groups such as 18–25, 26–35, 36–45, and 46 or older. The resulting categorical variable can then be used to compare claims experience across age groups.

The transformation should be guided by the purpose of the analysis. Grouping a continuous variable simplifies interpretation but also discards some information about the exact values. Consequently, grouping should be justified rather than performed automatically.

3.5.6 Transforming data as part of the workflow

Creating variables is most useful when it is viewed as part of a sequence of operations rather than as an isolated calculation. For example, we may first select the relevant variables, filter the observations, and then create a derived variable:

```
claims %>%
  select(PolicyNum, Year, y, Freq) %>%
  filter(Year == 2010) %>%
  mutate(
    severity = if_else(Freq > 0, y / Freq, NA_real_)
  )
```

```
# A tibble: 1,110 x 5
   PolicyNum  Year      y  Freq severity
       <dbl> <dbl>  <dbl> <dbl>    <dbl>
 1    120002  2010  6839.     1    6839.
 2    120003  2010  9711.     1    9711.
 3    120004  2010 10324.     1   10324.
 4    120005  2010     0      0      NA
 5    120008  2010  3470.     1    3470.
 6    120009  2010     0      0      NA
 7    120010  2010 35000      1   35000
 8    120012  2010 83935.    19    4418.
 9    120013  2010 63530.     5   12706.
10    120015  2010 38117.    11    3465.
# i 1,100 more rows
```

This sequence expresses the analytical logic directly:

$$\text{Select} \longrightarrow \text{Filter} \longrightarrow \text{Transform}.$$

The resulting data contain only the variables and observations needed for the analysis, together with the newly derived quantity.

This approach becomes increasingly useful as the analysis becomes more complex. Rather than repeatedly creating intermediate datasets, we can combine several operations into a readable sequence using the **pipe**. The next section introduces this way of expressing a data-transformation workflow.

3.6 Combining Operations with Pipes

Data transformation often involves several operations performed in sequence. For example, we may first select the variables of interest, then filter the observations, and finally create a new variable. Rather than writing each operation separately and creating intermediate data frames, we can connect the operations using the **pipe operator %>%**.

The pipe passes the result of the operation on its left to the function on its right. For example,

```
256 %>% sqrt()
```

```
[1] 16
```

is equivalent to

```
sqrt(256)
```

```
[1] 16
```

The result of `sqrt(256)` can then be passed to another function:

```
256 %>% sqrt() %>% log2()
```

```
[1] 4
```

This is equivalent to

```
log2(sqrt(256))
```

```
[1] 4
```

The pipe is particularly useful with `dplyr` because its data-manipulation functions take the data frame as their first argument. Thus,

```
claims %>%
  select(PolicyNum, Year, LnCoverage, yAvg) %>%
  filter(Year %in% c(2009, 2010))
```

```
# A tibble: 2,222 x 4
   PolicyNum  Year LnCoverage   yAvg
       <dbl> <dbl>      <dbl>  <dbl>
 1    120002  2009       3.08     0
 2    120002  2010       3.16  6839.
 3    120003  2009       4.78 18735.
 4    120003  2010       4.74  9711.
 5    120004  2009       3.50 14200.
 6    120004  2010       3.46 10324.
 7    120005  2009       3.61 13740.
 8    120005  2010       3.49     0
 9    120008  2009       3.81  6007.
10    120008  2010       3.86  3470.
# i 2,212 more rows
```

is equivalent to performing the operations separately:

```
new_table <- select(claims, PolicyNum, Year, LnCoverage, yAvg)

new_table <- filter(new_table, Year %in% c(2009, 2010))
```

The pipe version makes the sequence of operations more explicit. The data flow can be read from left to right:

$$\text{claims} \longrightarrow \text{select variables} \longrightarrow \text{filter observations.}$$

We can extend the sequence by adding further transformations:

```
claims %>%
  select(PolicyNum, Year, y, Freq) %>%
  filter(Year == 2010) %>%
  mutate(yAvg = y / Freq)
```

```
# A tibble: 1,110 x 5
   PolicyNum  Year      y  Freq   yAvg
       <dbl> <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2010  6839.     1  6839.
 2    120003  2010  9711.     1  9711.
```

```
 3    120004  2010 10324.     1 10324.
 4    120005  2010     0      0   NaN
 5    120008  2010  3470.     1  3470.
 6    120009  2010     0      0   NaN
 7    120010  2010 35000      1 35000
 8    120012  2010 83935.    19  4418.
 9    120013  2010 63530.     5 12706.
10    120015  2010 38117.    11  3465.
# i 1,100 more rows
```

Each step receives the result of the previous step. The complete workflow can therefore be read as:

$$\text{Select} \longrightarrow \text{Filter} \longrightarrow \text{Transform}.$$

This style is particularly useful when the analysis involves several transformations because each step corresponds to a recognizable analytical action. The source material uses the same principle to combine `select()`, `filter()`, and `head()` in a single sequence.

> **Reading a pipe**
>
> When reading a sequence of piped operations, start from the left and proceed to the right.
>
> ```
> data %>%
>   operation_1() %>%
>   operation_2() %>%
>   operation_3()
> ```
>
> means that the data are passed through `operation_1`, then the resulting data are passed through `operation_2`, and finally through `operation_3`.

The pipe does not introduce a new statistical method. Its purpose is to make a sequence of data operations easier to write and understand. The resulting code should still be evaluated according to the actuarial question being addressed.

Once the relevant observations have been selected and the required variables have been created, we can move from individual observations to **summaries of the data**.

3.7 Grouping and Summarizing Data

A large dataset is often too detailed to interpret by inspecting individual observations. **Summary statistics** provide a way to reduce many observations to a small number of quantities that describe important features of the data.

Common summary statistics include the mean, standard deviation, minimum, maximum, median, and quantiles. In actuarial analysis, other summaries such as total exposure, number of claims, aggregate claim amount, claim frequency, and average claim severity are also important.

The `dplyr` package provides the `summarise()` function for calculating summary statistics.

For example, suppose we want to calculate the mean and standard deviation of average claim severity for observations in 2010 where at least one claim occurred:

```
summary_table <- claims %>%
  filter(Year == 2010 & Freq > 0) %>%
  summarise(
    Mean = mean(yAvg),
    SD = sd(yAvg)
  )
```

The resulting data frame contains one row with the two requested summaries. The names `Mean` and `SD` are chosen by the analyst.

The filtering step is important here. Average claim severity is meaningful only for observations with positive claim counts. The source material uses the same sequence of filtering to calculate the mean and standard deviation of average severity for 2010.

3.7.1 Grouping observations

A single summary for the entire dataset may hide important differences between groups. In exploratory analysis, we often want to calculate the same summaries separately for different levels of a variable.

The `group_by()` function divides the data into groups. A subsequent `summarise()` then calculates the requested summaries separately within each group.

For example, instead of calculating average severity for only 2010, we can calculate it separately for each year:

```
summary_table_all_years <- claims %>%
  group_by(Year) %>%
  filter(Freq > 0) %>%
  summarise(
    Mean = mean(yAvg),
    SD = sd(yAvg)
  )
```

The result contains one row for each year, with the mean and standard deviation calculated using the observations belonging to that year. The source material demonstrates this use of `group_by(Year)` followed by `summarise()`.

The conceptual difference is:

```
summarise(...)
```

calculates a summary for the selected data as a whole, whereas

```
group_by(variable) %>%
  summarise(...)
```

calculates the summary separately for each level of `variable`.

This distinction is fundamental to actuarial analysis because risk experience is rarely homogeneous across an entire portfolio.

3.7.2 Grouping by rating variables

Grouping becomes particularly useful when investigating **rating variables**. For example, we may want to compare claims experience across characteristics such as entity type, alarm credit, no-claim credit, or fire class. The same principle applies to insurance variables such as age group, ZIP code, vehicle type, or bonus-malus level.

In the claims dataset, entity type is represented by several binary indicator variables, such as `TypeCounty`, `TypeMisc`, `TypeSchool`, and `TypeTown`, rather than by a single categorical variable. If we want to analyse claims experience by entity type, we can first combine these indicators into a single categorical variable.

For example:

```
claims <- claims %>%
  mutate(
    EntityType = case_when(
      TypeCounty == 1 ~ "County",
      TypeMisc == 1 ~ "Misc",
      TypeSchool == 1 ~ "School",
      TypeTown == 1 ~ "Town",
      TypeVillage == 1 ~ "Village",
      TRUE ~ "City"
    )
  )

claims
```

```
# A tibble: 5,639 x 24
   PolicyNum  Year LnCoverage     BCcov Premium  Freq Deduct      y   lny   yAvg
       <dbl> <dbl>      <dbl>     <dbl>   <dbl> <dbl>  <dbl>  <dbl> <dbl>  <dbl>
 1    120002  2006       3.12  22714456    9313     0   1000      0   0        0
 2    120002  2007       3.22  25046646    8767     0   1000      0   0        0
 3    120002  2008       3.04  20851525    7090     0   1000      0   0        0
 4    120002  2009       3.08  21852696    8522     0   1000      0   0        0
 5    120002  2010       3.16  23511493    7994     1   1000  6839.   8.83  6839.
 6    120003  2006       4.60  99353377   40735     0   5000      0   0        0
 7    120003  2007       4.65 104546824   39728     5   5000 15500   9.65  3100
 8    120003  2008       4.71 111265009   42280     1   5000  8775   9.08  8775
```

```
 9    120003  2009       4.78 119005219   36893     2   5000 37471. 10.5  18735.
10    120003  2010       4.74 114646079   36687     1   5000  9711.  9.18  9711.
# i 5,629 more rows
# i 14 more variables: lnDeduct <dbl>, Fire5 <dbl>, NoClaimCredit <dbl>,
#   TypeCity <dbl>, TypeCounty <dbl>, TypeMisc <dbl>, TypeSchool <dbl>,
#   TypeTown <dbl>, TypeVillage <dbl>, AC00 <dbl>, AC05 <dbl>, AC10 <dbl>,
#   AC15 <dbl>, EntityType <chr>
```

The `case_when()` function checks the conditions in order and assigns the corresponding category. Here, the remaining observations are classified as `City`.

Once `EntityType` has been created, we can use it with `group_by()` to compare claims experience across the different entity types. For example, if `yAvg` represents average claim severity:

```
claims %>%
  group_by(EntityType) %>%
  summarise(
    Mean = mean(yAvg, na.rm = TRUE),
    SD = sd(yAvg, na.rm = TRUE)
  )
```

```
# A tibble: 6 x 3
  EntityType   Mean      SD
  <chr>       <dbl>   <dbl>
1 City        8707.  44794.
2 County     11590.  33285.
3 Misc        5229.  54625.
4 School     20791. 366484.
5 Town        1491.  32629.
6 Village     2874.  18389.
```

This produces a summary of the mean and standard deviation of claim severity for each entity type. The same approach can be applied to other rating variables once they are represented in a form suitable for grouping.

The resulting table allows us to compare the observed experience of different groups. However, differences in averages should always be considered together with the amount of data supporting each group. A group with very little exposure or few claims may have an extreme observed value simply because of random variation.

3.7.3 Summarizing insurance experience

The same grouping principle can be used to produce more directly actuarial summaries.

In this dataset, `Freq` represents the number of claims and `y` represents the aggregate claim amount. We can therefore calculate the total number of claims and aggregate claim amount for each rating group:

```r
claims %>%
  group_by(EntityType) %>%
  summarise(
    claims = sum(Freq),
    amount = sum(y)
  )
```

```
# A tibble: 6 x 3
  EntityType claims    amount
  <chr>       <dbl>     <dbl>
1 City         1539 18609205.
2 County       1607 25491545.
3 Misc          113  3401011.
4 School       2290 42999241.
5 Town          100  1605103.
6 Village       606  5376997.
```

These totals provide a simple description of the volume of claims and the total cost of claims within each group.

If `yAvg` represents the average claim severity for each observation, we can also summarize severity by rating group:

```r
claims %>%
  group_by(EntityType) %>%
  summarise(
    mean_severity = mean(yAvg, na.rm = TRUE),
    sd_severity = sd(yAvg, na.rm = TRUE)
  )
```

```
# A tibble: 6 x 3
  EntityType mean_severity sd_severity
  <chr>              <dbl>       <dbl>
1 City               8707.      44794.
2 County            11590.      33285.
3 Misc               5229.      54625.
4 School            20791.     366484.
5 Town               1491.      32629.
6 Village            2874.      18389.
```

Alternatively, when `y` is the aggregate claim amount and `Freq` is the corresponding claim count, the overall average severity for each group can be calculated as:

```r
claims %>%
  group_by(EntityType) %>%
  summarise(
    claims = sum(Freq),
    amount = sum(y),
    severity = amount / claims
  )
```

```
# A tibble: 6 x 4
  EntityType claims    amount severity
  <chr>       <dbl>     <dbl>    <dbl>
1 City         1539 18609205.   12092.
2 County       1607 25491545.   15863.
3 Misc          113  3401011.   30097.
4 School       2290 42999241.   18777.
5 Town          100  1605103.   16051.
6 Village       606  5376997.    8873.
```

Here, `severity` represents the average amount per claim within each `EntityType`. The calculation is based on the total claim amount divided by the total number of claims, rather than taking a simple average of individual observations.

These summaries illustrate two important dimensions of claims experience:

$$\text{Claims experience} = \text{Frequency} \times \text{Severity}.$$

In this dataset, we can examine claim counts and severity directly. A conventional claim frequency, however, requires an appropriate measure of exposure. Since exposure is not available in this dataset, we do not calculate frequency as claims divided by exposure here. Exposure-based frequency measures will be introduced when a dataset containing exposure information is used.

> **!** Volume and experience
>
> When comparing groups, do not look only at the calculated risk measure. It is also important to consider the **volume of observations or claims** supporting the measure.
> For example, two groups may have similar average severities but very different numbers of claims. Conversely, a group with an unusually high average severity may contain only a small number of claims, making the observed estimate relatively unstable.
> The number of observations, claim counts, and aggregate claim amounts therefore provide useful context when interpreting differences between rating groups.

Grouping and summarizing provide a systematic way to compare claims experience across risk factors. These summaries form a useful starting point for actuarial analysis before more formal frequency, severity, and premium models are considered.

3.8 Insurance Summaries

The grouping and summarizing operations introduced above allow us to move from individual observations to quantities that are more directly relevant to actuarial analysis. In insurance, we are often interested in the number and cost of claims and in measures that allow claims experience to be compared across groups.

Several basic quantities are particularly important.

3.8.1 Claim count

The **claim count** is the total number of claims observed within a group. In the current dataset, `Freq` represents the number of claims for each observation. We can therefore calculate the total claim count for each `EntityType` as follows:

```
claims %>%
  group_by(EntityType) %>%
  summarise(
    claims = sum(Freq)
  )
```

```
# A tibble: 6 x 2
  EntityType claims
  <chr>       <dbl>
1 City         1539
2 County       1607
3 Misc          113
4 School       2290
5 Town          100
6 Village       606
```

The resulting value represents the total number of claims observed in each group.

3.8.2 Aggregate claim amount

The **aggregate claim amount** is the total amount paid or incurred for claims within a group. If `y` represents the aggregate claim amount for each observation, it can be obtained by summing `y`:

```
claims %>%
  group_by(EntityType) %>%
  summarise(
    claims = sum(Freq),
    amount = sum(y)
  )
```

```
# A tibble: 6 x 3
  EntityType claims    amount
  <chr>       <dbl>     <dbl>
1 City         1539 18609205.
2 County       1607 25491545.
3 Misc          113  3401011.
4 School       2290 42999241.
5 Town          100  1605103.
6 Village       606  5376997.
```

The claim count and aggregate claim amount describe two different aspects of claims experience. A group may have many claims but relatively low total claim costs, or relatively few claims but a high total claim amount.

3.8.3 Claim severity

Claim severity measures the average amount associated with a claim. At the group level, it can be calculated as aggregate claim amount divided by the total number of claims:

$$\text{Severity}_j = \frac{\text{Aggregate claims}_j}{\text{Claims}_j},$$

where j denotes a particular rating group.

For example:

```
claims %>%
  group_by(EntityType) %>%
  summarise(
    claims = sum(Freq),
    amount = sum(y),
    severity = amount / claims
  )
```

```
# A tibble: 6 x 4
  EntityType claims    amount severity
  <chr>       <dbl>     <dbl>    <dbl>
1 City         1539 18609205.   12092.
2 County       1607 25491545.   15863.
3 Misc          113  3401011.   30097.
4 School       2290 42999241.   18777.
5 Town          100  1605103.   16051.
6 Village       606  5376997.    8873.
```

This calculation gives the average claim amount for each `EntityType`. It is preferable to simply taking the average of individual claim amounts when observations may represent different numbers of claims, because the group-level calculation uses the total claim amount and total number of claims.

3.8.4 Exposure and claim frequency

In many insurance applications, claim counts are interpreted relative to the amount of **exposure** underlying the business. Exposure measures the amount of risk insured or the period for which the risk was covered. For example, one policy-year may represent one unit of exposure.

When exposure is available, the claim frequency for group j can be calculated as

$$\text{Frequency}_j = \frac{\text{Claims}_j}{\text{Exposure}_j}.$$

This is different from claim count. A group with 100 claims does not necessarily have a higher claim frequency than a group with 50 claims if the first group has substantially more exposure.

The current claims dataset does not contain an exposure variable. Therefore, we do not calculate exposure-based claim frequency from this dataset. Instead, we focus on claim counts, aggregate claim amounts, and severity. Exposure-based frequency will be introduced when we work with a dataset containing an appropriate exposure measure.

3.8.5 Frequency, severity, and risk premium

In actuarial analysis, frequency and severity are often combined to describe the expected claim cost per unit of exposure. Conceptually,

$$\text{Risk premium} = \text{Frequency} \times \text{Severity}.$$

Thus, when exposure is available,

$$\text{Risk premium}_j = \frac{\text{Claims}_j}{\text{Exposure}_j} \times \frac{\text{Aggregate claims}_j}{\text{Claims}_j} = \frac{\text{Aggregate claims}_j}{\text{Exposure}_j}.$$

This relationship provides an important connection between the basic summaries and later actuarial pricing models. Frequency describes how often claims occur, severity describes the average cost of a claim, and their product gives the expected claim cost per unit of exposure.

For the current dataset, however, the quantities that can be directly summarized are claim count, aggregate claim amount, and severity:

```
claims %>%
  group_by(EntityType) %>%
  summarise(
    claims = sum(Freq),
    amount = sum(y),
    severity = amount / claims
  )
```

```
# A tibble: 6 x 4
  EntityType claims    amount severity
  <chr>       <dbl>     <dbl>    <dbl>
1 City         1539 18609205.   12092.
2 County       1607 25491545.   15863.
3 Misc          113  3401011.   30097.
4 School       2290 42999241.   18777.
5 Town          100  1605103.   16051.
6 Village       606  5376997.    8873.
```

These summaries provide a first description of claims experience across rating groups. They do not, by themselves, establish that one group should be charged a higher premium than another. Such a conclusion requires consideration of the amount of business represented by each group and, ultimately, an appropriate statistical model.

Key actuarial quantities

When exposure is available, the basic relationships can be summarized as

$$\text{Frequency} = \frac{\text{Claims}}{\text{Exposure}},$$

$$\text{Severity} = \frac{\text{Aggregate claims}}{\text{Claims}},$$

and

$$\text{Risk premium} = \text{Frequency} \times \text{Severity}.$$

Equivalently,

$$\text{Risk premium} = \frac{\text{Aggregate claims}}{\text{Exposure}}.$$

The current dataset allows us to examine claims and severity, but not exposure-based frequency or risk premium.

3.9 Interpreting Insurance Summaries

Calculating insurance summaries is only the first step. In actuarial analysis, the more important question is what these summaries tell us about the underlying claims experience. A difference between two groups does not automatically imply that one group represents a higher-risk class. The size and composition of each group must also be considered.

3.9.1 Comparing groups

Suppose we summarize claims experience by `EntityType`:

```r
claims %>%
  group_by(EntityType) %>%
  summarise(
    observations = n(),
    claims = sum(Freq),
    amount = sum(y),
    severity = amount / claims
  )
```

```
# A tibble: 6 x 5
  EntityType observations claims    amount severity
  <chr>             <int>  <dbl>     <dbl>    <dbl>
1 City                793   1539 18609205.   12092.
2 County              328   1607 25491545.   15863.
3 Misc                609    113  3401011.   30097.
4 School             1597   2290 42999241.   18777.
5 Town                971    100  1605103.   16051.
6 Village            1341    606  5376997.    8873.
```

The resulting table allows us to compare groups using several measures. For example, one group may have a higher aggregate claim amount simply because it contains more observations. Another group may have fewer claims but a substantially higher average severity.

It is therefore useful to consider **experience** and **volume** together.

3.9.2 Experience and volume

The term **experience** refers to the observed claims results, such as the number or cost of claims. **Volume** refers to the amount of business supporting those results, such as the number of observations, claims, or exposure.

For example, consider two groups:

Group	Observations	Claims	Aggregate claims	Severity
A	1,000	100	500,000	5,000
B	20	5	40,000	8,000

Group B has a higher observed severity, but this result is based on only five claims. Group A has considerably more claims and therefore provides more observations from which to assess the underlying claims experience.

This does not mean that the result for Group B should be ignored. Rather, the difference should be interpreted with appropriate caution.

! Volume matters

A calculated summary should always be considered together with the amount of data supporting it.
A group with a small number of observations or claims can produce an extreme observed result simply through random variation. Conversely, a result based on a large volume of business may provide more stable evidence of an underlying difference in risk.
Therefore, when comparing rating groups, ask both:

- **What is the observed claims experience?**
- **How much data support that experience?**

3.9.3 Claim count is not frequency

It is also important to distinguish **claim count** from **claim frequency**.

A claim count measures the number of claims:

$$\text{Claims} * j = \sum_i N * ij.$$

A claim frequency relates the number of claims to the amount of exposure:

$$\text{Frequency}_j = \frac{\text{Claims}_j}{\text{Exposure}_j}.$$

Consequently, comparing claim counts between groups can be misleading when the groups contain different amounts of business.

For example, a group with 200 claims may appear to have worse experience than a group with 100 claims. However, if the first group has 10,000 exposure units while the second has only 1,000, their claim frequencies are

$$\frac{200}{10{,}000} = 0.02$$

and

$$\frac{100}{1{,}000} = 0.10,$$

respectively. The group with fewer claims actually has the higher claim frequency.

3.9.4 Observed differences are not necessarily causal

Differences between rating groups should also be interpreted carefully. A group may have higher observed severity because of another characteristic that is associated with both the group and claim severity.

For example, suppose one `EntityType` has higher observed claim severity. This does not by itself establish that the type of entity causes higher claim severity. The group may differ from other groups in coverage, deductible, location, or other risk characteristics.

This is one reason why simple grouped summaries are useful for **exploratory analysis**, but are not sufficient on their own for determining rating relativities or premiums.

3.9.5 From summaries to actuarial judgment

Grouped summaries provide an initial view of the claims experience associated with different rating variables. They help identify patterns, unusual observations, and potential differences between groups.

However, actuarial analysis requires more than identifying which group has the largest observed value. A sound interpretation considers:

1. the observed claims experience;
2. the volume of business supporting the experience;
3. the definition of the measure being compared;
4. other characteristics that may differ between groups; and
5. the uncertainty associated with the observed results.

These considerations provide the bridge from **descriptive analysis** to **actuarial modelling**. Statistical models can then be used to assess the relationship between claims outcomes and multiple rating variables simultaneously, rather than relying only on unadjusted comparisons between groups.

> **i** From data to actuarial decision
>
> The purpose of data manipulation and summarization is not simply to produce tables. The objective is to obtain evidence that can support actuarial judgment.
> A useful progression is:
>
> $$\text{Data} \rightarrow \text{Summary} \rightarrow \text{Comparison} \rightarrow \text{Interpretation} \rightarrow \text{Actuarial decision}.$$
>
> The summaries developed in this chapter provide the descriptive foundation for the statistical and actuarial models considered later.

> **i** Beyond the examples in this chapter
>
> The operations introduced in this chapter cover the main `dplyr` tools needed for the analyses that follow, but they are not an exhaustive list. `dplyr` also provides functions for joining datasets, arranging observations, counting records, handling missing values, working with

distinct observations, and performing more advanced grouped operations.
The purpose here is to establish the core principles of data manipulation rather than to cover every available function. The same principles can be extended to more complex datasets and analyses as needed.

4 Chapter 4: Tidy Data

Learning Outcomes

By the end of this chapter, students should be able to:

1. Explain the three principles of tidy data and identify common forms of untidy data.
2. Distinguish between identifier variables and variables that need to be reshaped.
3. Use `pivot_longer()` and `pivot_wider()` to transform untidy insurance data into a tidy format.
4. Explain how tidy data facilitates data transformation and summarization using `dplyr`.

4.1 Tidy Data

Before transforming or summarizing data, it is important to consider how the data are organized. **Tidy data** provides a consistent structure that makes data easier to inspect, transform, summarize, and analyse.

A dataset is **tidy** when its structure follows three principles:

1. **Each variable is a column.** A variable represents an attribute being measured, such as age, exposure, claim count, or premium.
2. **Each observation is a row.** An observation represents one unit being studied, such as a policy, policyholder, or claim.
3. **Each value is a cell.** Each cell contains one value for one variable and one observation.

For example, a dataset of motor insurance policies might be organized as follows:

policy_id	age	exposure	nclaims	amount
1	35	1.0	0	0
2	42	0.8	1	12500
3	29	1.0	2	8400

Here, `age`, `exposure`, `nclaims`, and `amount` are variables, while each row represents one policy. Each cell contains a single value.

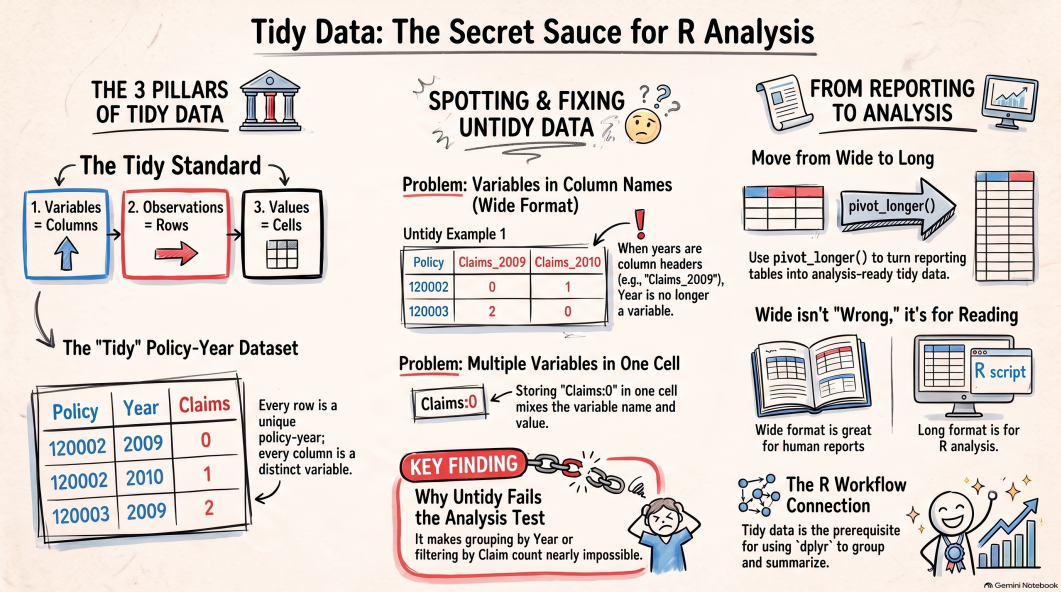


Figure 4.1: Guide to Tidy Data Principles

4.1.1 Why Tidy Data Matters

Tidy data is particularly useful when working with R because many data manipulation functions are designed around this structure. For example, with `dplyr`:

- `select()` works with variables represented as columns;
- `filter()` works with values within observations;
- `mutate()` creates or transforms variables;
- `group_by()` defines groups of observations; and
- `summarise()` calculates summaries of variables within those groups.

This gives a natural workflow:

$$\text{Tidy data} \longrightarrow; \text{Transform} \longrightarrow; \text{Group and summarise} \longrightarrow; \text{Analyse}.$$

When data are not organized in this way, the analysis may still be possible, but additional steps are often needed to reshape the data first. The **`tidyr`** package provides tools for doing this. In the next sections, we will examine common forms of untidy data and learn how to reshape them into a structure that is easier to work with using `dplyr`.

4.2 Common Forms of Untidy Data

Untidy data can arise in several ways. The issue is not simply whether data are stored in a *wide* or *long* format. A dataset is untidy when its structure does not clearly represent the variables and observations it contains.

The following examples use small synthetic insurance datasets stored in the project's `data/` folder. Each example illustrates a different structural problem.

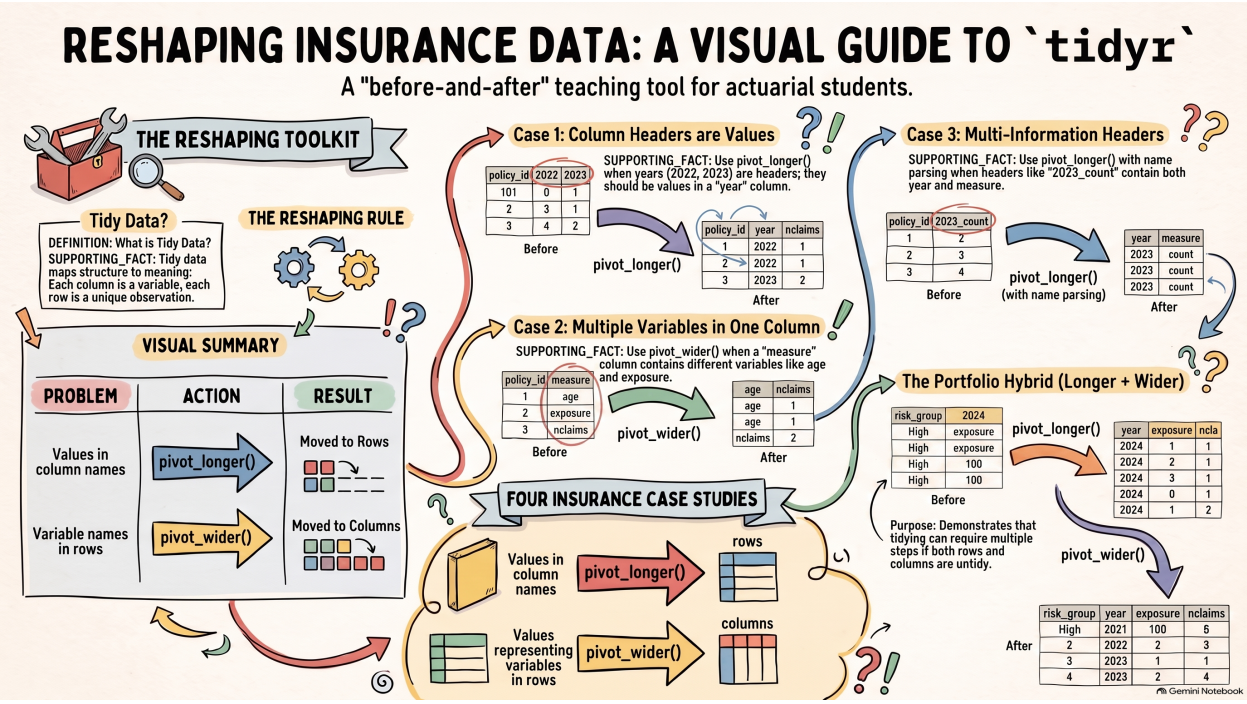


Figure 4.2: Guide to Reshaping Insurance Data

4.2.1 Column Headers Are Values

The file `tidyr_01_claims_by_year.csv` records the number of claims for each policy across several years.

The main variables are:

Variable	Description
`policy_id`	Unique identifier for the insurance policy
`risk_group`	Risk classification of the policyholder
2022–2026	Number of claims recorded in each year

The data have a structure such as:

```
policy_id  risk_group  2022  2023  2024  2025  2026
1001       Young          1     0     2     1     0
1002       Standard       0     1     0     0     1
1003       Young          2     1     1     0     2
```

The problem is that 2022, 2023, …, 2026 are not really different variables. They are **values of the same variable, `year`**.

The underlying variables are:

- `policy_id`
- `risk_group`
- `year`
- `nclaims`

The tidy structure would therefore be:

```
policy_id  risk_group  year  nclaims
1001       Young       2022       1
1001       Young       2023       0
1001       Young       2024       2
...
```

The key question to ask is:

> **Are the column names actually variable names, or are they values of a variable?**

Here, the years are values, so the yearly columns need to be reshaped into rows. This is the type of problem addressed by `pivot_longer()`.

4.2.2 Multiple Variables Stored in One Column

The file `tidyr_02_policy_measures.csv` records several characteristics of each policy. However, the name of the characteristic is stored as a value in the `measure` column.

The variables are:

Variable	Description
`policy_id`	Unique identifier for the insurance policy
`risk_group`	Risk classification of the policyholder
`measure`	Type of measurement being recorded
`value`	Value of the corresponding measurement

The data have a structure such as:

```
policy_id  risk_group  measure    value
1001       Young       age         25
1001       Young       exposure     1
1001       Young       nclaims      2
1002       Standard    age         45
1002       Standard    exposure     1
1002       Standard    nclaims      0
```

Here, `age`, `exposure`, and `nclaims` are not merely categories of a variable called `measure`. They represent **different variables** that should become separate columns.

The tidy structure would be:

```
policy_id  risk_group  age  exposure  nclaims
1001       Young        25       1.0        2
1002       Standard     45       1.0        0
```

The key question is:

> **Are values in a column actually the names of different variables?**

If so, the values in that column may need to become column names. This is the type of problem addressed by `pivot_wider()`.

This example is also important because it shows that **`pivot_wider()` can be used to make data tidy**. Tidy data are not necessarily produced by making data longer.

4.2.3 Multiple Variables Stored in Column Names

The file `tidyr_03_claims_year_measure.csv` contains claims information for several years and measures.

The dataset includes columns such as:

```
policy_id  risk_group  vehicle_type  2023_count  2023_amount  2024_count  2024_amount  ...
1001       Young       Car                    0            0           0            0  ...
```

The main variables are:

Variable	Description
`policy_id`	Unique identifier for the insurance policy
`risk_group`	Risk classification of the policyholder
`vehicle_type`	Type of vehicle insured
`2023_count`, `2024_count`, …	Number of claims in each year
`2023_amount`, `2024_amount`, …	Total claim amount in each year

The problem is that a column name such as `2023_count` contains **two pieces of information**:

```
2023_count

       measure
     year
```

Thus, the column names contain values for two variables:

- `year`
- `measure`

The tidy structure would separate these pieces of information:

policy_id	risk_group	vehicle_type	year	measure	value
1001	Young	Car	2023	count	0
1001	Young	Car	2023	amount	0
1001	Young	Car	2024	count	0
1001	Young	Car	2024	amount	0

The key question is:

> **Does a column name contain more than one piece of information?**

If it does, those pieces may represent separate variables that need to be extracted from the column names.

For example, `2023_count` contains both a year and a measure. `pivot_longer()` can move the column names into rows, while `names_to` can separate the different pieces into variables.

The important idea is that `pivot_longer()` can create **multiple variables from column names**. When column names contain several pieces of information, `names_to` can be used to separate those pieces into different variables.

After reshaping, `measure` is a legitimate variable. It does **not** need to be converted into separate columns simply because its values are `count` and `amount`.

This leads to an important principle:

> **Tidy data do not require one row per policy.**

For this dataset, an observation can be identified by the combination of:

$$(\text{policy_id},\ \text{year},\ \text{measure})$$

Therefore, a policy can legitimately appear on several rows.

Tidy Does Not Mean One Row per Policy

The number of rows in a tidy dataset depends on what an **observation** represents.
In this example, the observation is a particular measure for a particular policy and year.
Therefore, `policy_id` alone does not identify a row.
The combination

$$(\text{policy_id},\ \text{year},\ \text{measure})$$

identifies the observation.
Do not reshape data simply to obtain one row per policy. First determine what an observation represents.

4.2.4 Variables Stored in Both Rows and Columns

The file `tidyr_04_portfolio_summary.csv` combines two of the structural problems seen above.

The data have a structure such as:

```
risk_group  measure    2024  2025  2026
Young       exposure   ...   ...   ...
Young       nclaims    ...   ...   ...
Young       premium    ...   ...   ...
Standard    exposure   ...   ...   ...
Standard    nclaims    ...   ...   ...
Standard    premium    ...   ...   ...
```

The main variables are:

Variable	Description
`risk_group`	Risk classification of the policyholder group
`measure`	Type of portfolio measure
2024–2026	Year for which the measure is reported
`exposure`	Total exposure for the risk group and year
`nclaims`	Number of claims for the risk group and year
`premium`	Premium for the risk group and year

There are two structural problems.

First, `measure` contains values such as `exposure`, `nclaims`, and `premium`. These values represent different variables.

Second, 2024, 2025, and 2026 are column names but represent values of the variable `year`.

The tidy structure should therefore be:

```
risk_group  year  exposure  nclaims  premium
Young       2024     ...       ...      ...
Young       2025     ...       ...      ...
Young       2026     ...       ...      ...
Standard    2024     ...       ...      ...
```

The key question is:

> **Are there structural problems in both the rows and the columns?**

Here, both directions need to be considered:

- the year values need to move from columns into rows;
- the measure values need to move from rows into columns.

Consequently, tidying this dataset requires a combination of `pivot_longer()` and `pivot_wider()`.

This is the most complex of the four examples because it demonstrates that **tidying is not determined by a single reshaping operation**. The appropriate sequence depends on the structure of the original data.

4.2.5 Recognising the Structure Before Reshaping

The four examples illustrate a common way of thinking about untidy data. Before deciding which function to use, first identify what the data are trying to represent.

For each dataset, ask:

1. **What is an observation?**
2. **Which variables describe or identify that observation?**
3. **Which values have been placed in column names or in inappropriate rows?**
4. **What should the tidy variables be?**
5. **Which reshaping operation would put those variables in the correct places?**

For the insurance examples, `policy_id` identifies an individual policy, while variables such as `risk_group` and `vehicle_type` describe that policy. In an aggregated portfolio dataset, a combination such as `risk_group` and `year` may identify an observation.

The reasoning process can be summarised as:

Identify the observation → Identify the variables → Identify the structural problem → Choose the appropriat

The goal is therefore not simply to make data *long* or *wide*. The goal is to make the rows and columns correspond to the observations and variables represented by the data.

The complete R workflows for these four datasets are developed in **Working with the Sample Insurance Data**, where the datasets are reshaped and then used with the `dplyr` operations introduced earlier.

4.3 Introducing `tidyr`

The `tidyr` package provides tools for reshaping data so that its structure better reflects the variables and observations it contains. In practice, this is particularly useful when data have been recorded in a form that is convenient for data entry or presentation but does not follow the principles of tidy data.

The two functions used most often for reshaping data are `pivot_longer()` and `pivot_wider()`.

4.3.1 From Wide to Long with `pivot_longer()`

`pivot_longer()` makes a dataset **longer** by turning selected columns into rows.

The basic idea is:

$$\text{Wide} \xrightarrow{\text{pivot_longer()}} \text{Long}$$

For example, suppose claims are recorded in separate columns for each year:

```
policy_id  2022  2023  2024
1001          1     0     2
1002          0     1     0
```

After reshaping, the year becomes a variable:

```
policy_id  year  nclaims
1001       2022        1
1001       2023        0
1001       2024        2
1002       2022        0
...
```

The general form is:

```
data |>
  tidyr::pivot_longer(
    cols = ...,
    names_to = ...,
    values_to = ...
  )
```

The main arguments are:

Argument	Purpose
`cols`	Columns to reshape
`names_to`	Name of the new variable containing the original column names
`values_to`	Name of the new variable containing the values

For example:

```
data |>
  tidyr::pivot_longer(
    cols = `2022`:`2024`,
    names_to = "year",
    values_to = "nclaims"
  )
```

Here, the column names become values of `year`, while the cell values become values of `nclaims`.

4.3.1.1 Separating information contained in column names

Sometimes a column name contains more than one piece of information. For example:

```
2023_count
2023_amount
2024_count
2024_amount
```

The names contain both `year` and `measure`.

`pivot_longer()` can separate these pieces into different variables:

```
data |>
  tidyr::pivot_longer(
    cols = ...,
    names_to = c("year", "measure"),
    names_sep = "_",
    values_to = "value"
  )
```

The additional argument `names_sep` specifies the character used to separate the pieces of the column name.

Thus,

```
2023_count
```

becomes:

```
year       measure
2023       count
```

This is useful when column names contain several pieces of information that should become separate variables.

4.3.2 From Long to Wide with `pivot_wider()`

`pivot_wider()` makes a dataset **wider** by turning values in a column into new columns.

The basic idea is:

$$\text{Long} \xrightarrow{\text{pivot_wider()}} \text{Wide}$$

For example, suppose the data contain:

```
policy_id  measure    value
1001       age           25
1001       exposure       1
1001       nclaims        2
1002       age           45
1002       exposure       1
1002       nclaims        0
```

The values in `measure` represent different variables. We can turn them into columns:

```
policy_id  age  exposure  nclaims
1001        25       1.0        2
1002        45       1.0        0
```

The general form is:

```
data |>
  tidyr::pivot_wider(
    names_from = ...,
    values_from = ...
  )
```

The main arguments are:

Argument	Purpose
`names_from`	Column containing the values that will become new column names
`values_from`	Column containing the values that will fill the new columns

For example:

```
data |>
  tidyr::pivot_wider(
    names_from = measure,
    values_from = value
  )
```

Here, the values in `measure` become column names, and the corresponding values in `value` fill those columns.

4.3.3 Choosing the Appropriate Function

The choice between the two functions depends on **where the variables are currently stored**.

If values representing a variable have been stored in **column names**, `pivot_longer()` is usually appropriate:

Values in column names → pivot_longer()

If values representing variables have been stored in **rows**, `pivot_wider()` may be appropriate:

Variable names stored as values in rows → pivot_wider()

In some datasets, both problems occur at the same time. The fourth insurance dataset demonstrates how `pivot_longer()` and `pivot_wider()` can be combined.

> **!** Long and Wide Do Not Define Tidy Data
>
> The terms **long** and **wide** describe the shape of a dataset. They do not, by themselves, determine whether the data are tidy.
> A wide dataset can be tidy, and a long dataset can be untidy.
> The goal of `tidyr` is not simply to make data longer or wider. The goal is to reshape the data so that **each variable is a column, each observation is a row, and each value is a cell**.

In the next section, these ideas are applied to the four synthetic insurance datasets. The examples show how to identify the columns that should remain unchanged and how to select the appropriate `pivot_*()` operation for each structure.

4.4 Working with the Sample Insurance Data

The four synthetic datasets introduced earlier are stored in the project's `data/` folder. We now use them to practise the reshaping operations introduced above.

The purpose of this section is not to introduce new types of untidy data. Instead, it is to apply the same reasoning to actual data:

1. inspect the data;
2. identify the observation;
3. identify the variables that should remain unchanged;
4. identify the columns that need to be reshaped;
5. apply the appropriate `pivot_*()` function; and
6. check the resulting tidy structure.

> A useful habit
>
> Before writing a `pivot_*()` command, do not start with the function. First ask:
>
> **What does one row represent in the data I want to obtain?**
>
> Once the intended observation is clear, it becomes much easier to decide which columns should remain and which information needs to be reshaped.

4.4.1 Claims Recorded by Year

Start with `tidyr_01_claims_by_year.csv`.

```
claims_by_year <- readr::read_csv(
  "../data/tidyr_01_claims_by_year.csv",
  show_col_types = FALSE
)

claims_by_year
```

```
# A tibble: 100 x 9
   policy_id risk_group vehicle_type exposure `2022` `2023` `2024` `2025` `2026`
       <dbl> <chr>      <chr>           <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
 1      1001 Young      Car              0.83      0      0      0      0      0
 2      1002 Young      Car              0.81      0      0      1      0      0
 3      1003 Older      SUV              0.87      0      0      0      0      0
 4      1004 Older      SUV              0.58      0      0      0      0      0
 5      1005 Middle     Pickup           0.67      1      0      0      1      0
 6      1006 Middle     Car              0.58      0      0      0      0      0
 7      1007 Young      Pickup           0.92      0      0      1      0      0
 8      1008 Young      SUV              0.87      0      0      0      0      0
 9      1009 Middle     SUV              0.63      0      0      0      0      0
10      1010 Older      Pickup           0.64      0      0      0      0      0
# i 90 more rows
```

The yearly claim counts are stored in separate columns. We want one observation for each **policy and year**.

The columns `policy_id` and `risk_group` describe the policy and therefore remain unchanged. The columns containing years are reshaped using `pivot_longer()`.

```
claims_by_year_tidy <- claims_by_year |>
  tidyr::pivot_longer(
    cols = `2022`:`2026`,
    names_to = "year",
    values_to = "nclaims"
  )

claims_by_year_tidy
```

```
# A tibble: 500 x 6
   policy_id risk_group vehicle_type exposure year  nclaims
       <dbl> <chr>      <chr>           <dbl> <chr>   <dbl>
 1      1001 Young      Car              0.83 2022        0
 2      1001 Young      Car              0.83 2023        0
```

```
 3      1001 Young      Car              0.83 2024        0
 4      1001 Young      Car              0.83 2025        0
 5      1001 Young      Car              0.83 2026        0
 6      1002 Young      Car              0.81 2022        0
 7      1002 Young      Car              0.81 2023        0
 8      1002 Young      Car              0.81 2024        1
 9      1002 Young      Car              0.81 2025        0
10      1002 Young      Car              0.81 2026        0
# i 490 more rows
```

The resulting data have the variables:

```
dplyr::glimpse(claims_by_year_tidy)
```

```
Rows: 500
Columns: 6
$ policy_id    <dbl> 1001, 1001, 1001, 1001, 1001, 1002, 1002, 1002, 1002, 100~
$ risk_group   <chr> "Young", "Young", "Young", "Young", "Young", "Young", "Yo~
$ vehicle_type <chr> "Car", "Car", "Car", "Car", "Car", "Car", "Car", "Car", "~
$ exposure     <dbl> 0.83, 0.83, 0.83, 0.83, 0.83, 0.81, 0.81, 0.81, 0.81, 0.8~
$ year         <chr> "2022", "2023", "2024", "2025", "2026", "2022", "2023", "~
$ nclaims      <dbl> 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
```

The `year` variable is initially read as character because the original column names are text. If we want `year` to be numeric, we can convert it explicitly:

```
claims_by_year_tidy <- claims_by_year_tidy |>
  dplyr::mutate(year = as.integer(year))
```

The tidy data can now be used directly with `dplyr`. For example, we can calculate the total number of claims by year:

```
claims_by_year_tidy |>
  dplyr::group_by(year) |>
  dplyr::summarise(
    nclaims = sum(nclaims),
    .groups = "drop"
  )
```

```
# A tibble: 5 x 2
   year nclaims
  <int>   <dbl>
1  2022      10
2  2023       5
3  2024      15
4  2025       4
5  2026       8
```

The important point is that `pivot_longer()` has not performed an analysis. It has **put the data into a structure in which the analysis is straightforward**.

4.4.2 Policy Measures

Next, consider `tidyr_02_policy_measures.csv`.

```
policy_measures <- readr::read_csv(
  "../data/tidyr_02_policy_measures.csv",
  show_col_types = FALSE
)

policy_measures
```

```
# A tibble: 400 x 5
   policy_id risk_group vehicle_type measure     value
       <dbl> <chr>      <chr>        <chr>       <dbl>
 1      1001 Young      Car          age         19
 2      1001 Young      Car          exposure     0.95
 3      1001 Young      Car          nclaims      1
 4      1001 Young      Car          premium  11875
 5      1002 Young      Car          age         19
 6      1002 Young      Car          exposure     0.98
 7      1002 Young      Car          nclaims      0
 8      1002 Young      Car          premium  12250
 9      1003 Older      SUV          age         72
10      1003 Older      SUV          exposure     0.98
# i 390 more rows
```

Here, `measure` contains values such as `age`, `exposure`, and `nclaims`. These values represent different variables.

The intended observation is one **policy**, so we want one row per `policy_id`. The values in `measure` should become column names.

```
policy_measures_tidy <- policy_measures |>
  tidyr::pivot_wider(
    names_from = measure,
    values_from = value
  )

policy_measures_tidy
```

```
# A tibble: 100 x 7
   policy_id risk_group vehicle_type   age exposure nclaims premium
       <dbl> <chr>      <chr>        <dbl>    <dbl>   <dbl>   <dbl>
```

```
 1      1001 Young      Car             19     0.95       1   11875
 2      1002 Young      Car             19     0.98       0   12250
 3      1003 Older      SUV             72     0.98       0    8820
 4      1004 Older      SUV             61     0.74       0    6660
 5      1005 Middle     Pickup          37     0.8        0    8000
 6      1006 Middle     Car             31     0.97       0    9700
 7      1007 Young      Pickup          27     0.76       0    9500
 8      1008 Young      SUV             20     0.6        0    7500
 9      1009 Middle     SUV             35     0.82       0    8200
10      1010 Older      Pickup          72     0.71       0    6390
# i 90 more rows
```

We can now use the variables directly in `dplyr`. For example, calculate the total number of claims by risk group:

```
policy_measures_tidy |>
  dplyr::group_by(risk_group) |>
  dplyr::summarise(
    exposure = sum(exposure),
    nclaims = sum(nclaims),
    .groups = "drop"
  )
```

```
# A tibble: 3 x 3
  risk_group exposure nclaims
  <chr>         <dbl>   <dbl>
1 Middle         31.6       1
2 Older          18.8       2
3 Young          24.7       4
```

This example demonstrates why the direction of reshaping cannot be chosen simply by looking at whether the data appear long or wide. We first identify what the variables are and what one row should represent.

4.4.3 Claims by Year and Measure

Now consider `tidyr_03_claims_year_measure.csv`.

```
claims_year_measure <- readr::read_csv(
  "../data/tidyr_03_claims_year_measure.csv",
  show_col_types = FALSE
)

claims_year_measure
```

```
# A tibble: 100 x 11
   policy_id risk_group vehicle_type `2023_count` `2023_amount` `2024_count`
       <dbl> <chr>      <chr>                <dbl>         <dbl>         <dbl>
 1      1001 Young      Car                      0             0             0
 2      1002 Young      Car                      1         47486             0
 3      1003 Older      SUV                      0             0             0
 4      1004 Older      SUV                      0             0             0
 5      1005 Middle     Pickup                   0             0             0
 6      1006 Middle     Car                      0             0             0
 7      1007 Young      Pickup                   0             0             0
 8      1008 Young      SUV                      0             0             0
 9      1009 Middle     SUV                      0             0             0
10      1010 Older      Pickup                   0             0             0
# i 90 more rows
# i 5 more variables: `2024_amount` <dbl>, `2025_count` <dbl>,
#   `2025_amount` <dbl>, `2026_count` <dbl>, `2026_amount` <dbl>
```

The column names contain both the year and the measure, for example `2023_count` and `2023_amount`.

The observation we want is one **policy, year, and measure**. Therefore, `policy_id`, `risk_group`, and `vehicle_type` remain unchanged.

We can separate the information contained in each column name using `names_to` and `names_sep`:

```
claims_year_measure_tidy <- claims_year_measure |>
  tidyr::pivot_longer(
    cols = -c(policy_id, risk_group, vehicle_type),
    names_to = c("year", "measure"),
    names_sep = "_",
    values_to = "value"
  )

claims_year_measure_tidy
```

```
# A tibble: 800 x 6
   policy_id risk_group vehicle_type year  measure value
       <dbl> <chr>      <chr>        <chr> <chr>   <dbl>
 1      1001 Young      Car          2023  count       0
 2      1001 Young      Car          2023  amount      0
 3      1001 Young      Car          2024  count       0
 4      1001 Young      Car          2024  amount      0
 5      1001 Young      Car          2025  count       0
 6      1001 Young      Car          2025  amount      0
 7      1001 Young      Car          2026  count       0
 8      1001 Young      Car          2026  amount      0
 9      1002 Young      Car          2023  count       1
10      1002 Young      Car          2023  amount  47486
```

```
# i 790 more rows
```

We can check the resulting structure:

```
dplyr::glimpse(claims_year_measure_tidy)
```

```
Rows: 800
Columns: 6
$ policy_id    <dbl> 1001, 1001, 1001, 1001, 1001, 1001, 1001, 1001, 1002, 100~
$ risk_group   <chr> "Young", "Young", "Young", "Young", "Young", "Young", "Yo~
$ vehicle_type <chr> "Car", "Car", "Car", "Car", "Car", "Car", "Car", "Car", "~
$ year         <chr> "2023", "2023", "2024", "2024", "2025", "2025", "2026", "~
$ measure      <chr> "count", "amount", "count", "amount", "count", "amount", ~
$ value        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 1, 47486, 0, 0, 0, 0, 0, 0, 0, 0,~
```

Again, `measure` remains a variable. There is no need to widen it further because the intended observation includes the measure.

For example, we can examine total claim amounts by year:

```
claims_year_measure_tidy |>
  dplyr::filter(measure == "amount") |>
  dplyr::group_by(year) |>
  dplyr::summarise(
    claim_amount = sum(value),
    .groups = "drop"
  )
```

```
# A tibble: 4 x 2
  year  claim_amount
  <chr>        <dbl>
1 2023        204972
2 2024        168787
3 2025        271468
4 2026        222881
```

This illustrates an important advantage of tidy data. We can use the value of `measure` to select the type of information we want to analyse.

4.4.4 Portfolio Summary

Finally, consider `tidyr_04_portfolio_summary.csv`.

```
portfolio_summary <- readr::read_csv(
  "../data/tidyr_04_portfolio_summary.csv",
  show_col_types = FALSE
)

portfolio_summary
```

```
# A tibble: 9 x 5
  risk_group measure    `2024`   `2025`   `2026`
  <chr>      <chr>       <dbl>    <dbl>    <dbl>
1 Young      exposure    135.     146.     160.
2 Young      nclaims      22        5       15
3 Young      premium  288403   206554   228693
4 Middle     exposure    123.     128.     137
5 Middle     nclaims      21       22       19
6 Middle     premium  157140   268681   318099
7 Older      exposure     88.6     92.9     97.1
8 Older      nclaims       9       10       15
9 Older      premium  214187   259423   300269
```

This dataset contains both structural problems encountered earlier:

- the years are stored in column names; and
- `measure` contains values representing different variables.

The intended observation is one **risk group and year**. We therefore need to reshape the years into rows and then turn the measures into separate columns.

First, use `pivot_longer()`:

```
portfolio_long <- portfolio_summary |>
  tidyr::pivot_longer(
    cols = `2024`:`2026`,
    names_to = "year",
    values_to = "value"
  )

portfolio_long
```

```
# A tibble: 27 x 4
   risk_group measure  year    value
   <chr>      <chr>    <chr>   <dbl>
 1 Young      exposure 2024     135.
 2 Young      exposure 2025     146.
 3 Young      exposure 2026     160.
 4 Young      nclaims  2024      22
 5 Young      nclaims  2025       5
```

```
 6 Young      nclaims  2026      15
 7 Young      premium  2024  288403
 8 Young      premium  2025  206554
 9 Young      premium  2026  228693
10 Middle     exposure 2024     123.
# i 17 more rows
```

The data are now longer, but `measure` still contains values that represent variables. We can use `pivot_wider()` to complete the transformation:

```
portfolio_tidy <- portfolio_long |>
  tidyr::pivot_wider(
    names_from = measure,
    values_from = value
  )

portfolio_tidy
```

```
# A tibble: 9 x 5
  risk_group year  exposure nclaims premium
  <chr>      <chr>    <dbl>   <dbl>   <dbl>
1 Young      2024     135.       22  288403
2 Young      2025     146.        5  206554
3 Young      2026     160.       15  228693
4 Middle     2024     123.       21  157140
5 Middle     2025     128.       22  268681
6 Middle     2026     137        19  318099
7 Older      2024      88.6       9  214187
8 Older      2025      92.9      10  259423
9 Older      2026      97.1      15  300269
```

The final structure contains:

```
risk_group  year  exposure  nclaims  premium
Young       2024     ...       ...      ...
Young       2025     ...       ...      ...
Young       2026     ...       ...      ...
```

We can now use the resulting variables naturally with `dplyr`. For example, calculate the claim frequency for each risk group and year:

```
portfolio_tidy |>
  dplyr::mutate(
    frequency = nclaims / exposure
  )
```

```
# A tibble: 9 x 6
  risk_group year  exposure nclaims premium frequency
  <chr>      <chr>    <dbl>   <dbl>   <dbl>     <dbl>
1 Young      2024     135.       22  288403    0.163
2 Young      2025     146.        5  206554    0.0341
3 Young      2026     160.       15  228693    0.0938
4 Middle     2024     123.       21  157140    0.170
5 Middle     2025     128.       22  268681    0.172
6 Middle     2026     137        19  318099    0.139
7 Older      2024      88.6       9  214187    0.102
8 Older      2025      92.9      10  259423    0.108
9 Older      2026      97.1      15  300269    0.154
```

Here, `tidyr` has prepared the structure, while `dplyr` performs the transformation.

> **!** Reshaping Is About Structure
>
> The four examples use different reshaping operations, but the reasoning is the same:
>
> Identify the observation →; Identify the variables →; Reshape the data →; Check the tidy structure
>
> Only after the data have the appropriate structure should we proceed with transformations, summaries, or analysis.

The four datasets therefore illustrate a progression from a single reshaping problem to a combination of reshaping operations:

Dataset	Structural problem	Reshaping
`tidyr_01_claims_by_year.csv`	Years stored in column names	`pivot_longer()`
`tidyr_02_policy_measures.csv`	Variables stored as values	`pivot_wider()`
`tidyr_03_claims_year_measure.csv`	Multiple pieces of information in column names	`pivot_longer()` with `names_to` and `names_sep`
`tidyr_04_portfolio_summary.csv`	Problems in both rows and columns	`pivot_longer()` + `pivot_wider()`

The resulting tidy datasets can now be handled using the `dplyr` verbs introduced earlier in the chapter.

4.5 From Tidy Data to `dplyr`

Tidying data is usually not the final goal. Once the data have been reshaped into a tidy structure, we can use the `dplyr` functions introduced earlier to transform, summarise, and analyse the data.

For example, after tidying the portfolio data, we can calculate claim frequency by risk group and year:

```r
portfolio_tidy |>
  dplyr::mutate(
    frequency = nclaims / exposure
  ) |>
  dplyr::group_by(risk_group, year) |>
  dplyr::summarise(
    frequency = sum(nclaims) / sum(exposure),
    .groups = "drop"
  )
```

```
# A tibble: 9 x 3
  risk_group year  frequency
  <chr>      <chr>     <dbl>
1 Middle     2024     0.170
2 Middle     2025     0.172
3 Middle     2026     0.139
4 Older      2024     0.102
5 Older      2025     0.108
6 Older      2026     0.154
7 Young      2024     0.163
8 Young      2025     0.0341
9 Young      2026     0.0938
```

The roles of the two packages are therefore complementary:

Package	Main role
`tidyr`	Reshape and organise the structure of data
`dplyr`	Transform, filter, group, and summarise data

The overall workflow is:

Recognise structure → Tidy → Transform → Summarise → Analyse

Tidy data provide the structure that allows the subsequent `dplyr` operations to work naturally.

5 Chapter 5: Data Visualization with ggplot2

Learning Outcomes

By the end of this chapter, students should be able to:

1. Explain how data visualization supports exploratory data analysis and actuarial decision-making.
2. Construct plots using the grammar of graphics implemented in `ggplot2`.
3. Visualize categorical variables using bar charts and quantitative variables using histograms, density plots, and boxplots.
4. Use scatterplots and other suitable visualizations to explore relationships between variables.
5. Compare distributions and relationships across groups using aesthetic mappings and faceting.
6. Customize plots using appropriate scales, labels, legends, and themes.
7. Interpret visual patterns in insurance data and identify features that may warrant further investigation or statistical modelling.

5.1 Why Data Visualization Matters

Insurance data often contain patterns that are difficult to identify from tables of observations or summary statistics alone. A table may show the average claim amount for a portfolio, for example, but it may not reveal whether most claims are relatively small with a few exceptionally large claims. Similarly, a summary of average insurance charges by smoking status may indicate a difference between groups without showing how much their distributions overlap.

Data visualization complements numerical summaries by presenting the distribution of observations, differences between groups, and relationships between variables in a graphical form. It can help analysts identify unusual observations, detect possible data-quality problems, and develop questions for further investigation.

In actuarial work, these tasks are important because insurance data often exhibit characteristics such as skewed claim amounts, many policies with no claims, variation across risk groups, and relationships between premiums and policy characteristics. Visualizations help analysts explore these features before selecting an appropriate statistical model.

Consider the `ClaimsExperienceData.csv` dataset used in the earlier chapters. The variable `Freq` records claim frequency, while `yAvg` represents average claim amount for policies with claims. Numerical summaries can describe their means, medians, and ranges. However, plots can reveal additional features, such as the concentration of policies with low claim frequency and the presence of a small number of policies with unusually large average claim amounts.

Visualization is therefore part of the exploratory data analysis process. It does not replace numerical summaries or statistical modelling; rather, it helps determine which patterns deserve closer examination and which modelling approaches may be appropriate.

5.2 Building Plots with ggplot2

The `ggplot2` package provides a systematic approach to data visualization based on the grammar of graphics. A plot is constructed by combining a dataset with aesthetic mappings, geometric objects, and other components that determine how the data are displayed.

The central function is `ggplot()`. A basic plot has the following structure:

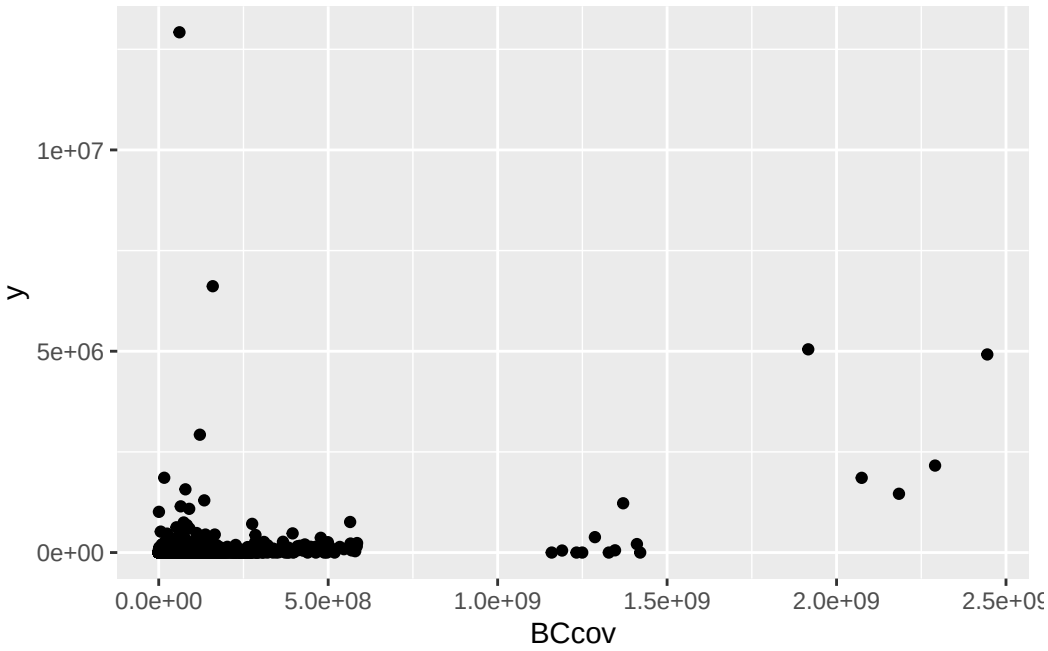
```
library(ggplot2)
ggplot(data, aes(x = variable_x, y = variable_y)) +
  geom_point()
```

Here, `data` identifies the dataset, `aes()` specifies how variables are mapped to visual properties, and `geom_point()` displays the observations as points. The `+` operator adds components to the plot.

5.2.1 A first plot

We begin by loading the `ClaimsExperienceData.csv` dataset used in the earlier chapters. Suppose we want to explore the relationship between insurance coverage and claim amount.

```
library(readr)
library(ggplot2)

claims <- read_csv("../data/ClaimsExperienceData.csv")

ggplot(claims, aes(x = BCcov, y = y)) +
  geom_point()
```



This plot places `BCcov` on the horizontal axis and `y` on the vertical axis. Each point represents an observation in the dataset. The plot provides an initial view of how claim amount varies with coverage, although the concentration of observations near zero or a small number of extreme values may make some patterns difficult to see.

The example illustrates the basic structure of a ggplot: the data and aesthetic mappings are supplied to `ggplot()`, while a geometric object determines how the observations are drawn.

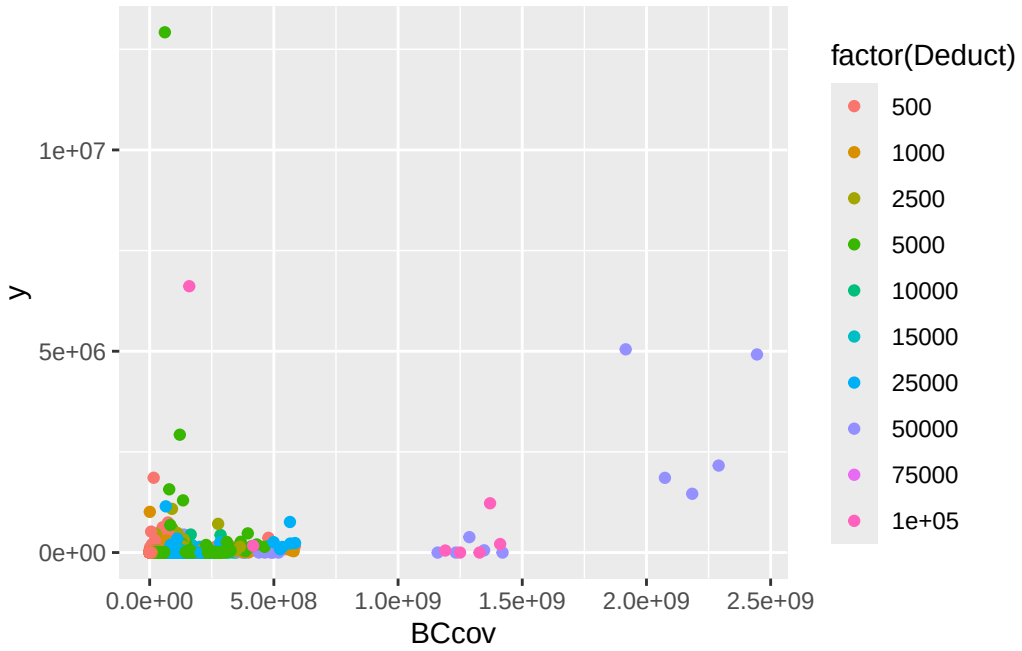
5.2.2 Aesthetic mappings

Aesthetics describe the visual properties used to represent data. Common aesthetics include:

- `x` and `y` for horizontal and vertical positions;
- `colour` for the colour of points, lines, or outlines;
- `fill` for the interior colour of bars and other filled shapes;
- `size` for the size of points or lines;
- `shape` for the shape of points.

The `aes()` function maps variables to these visual properties. For example, we can use the deductible to colour the observations:

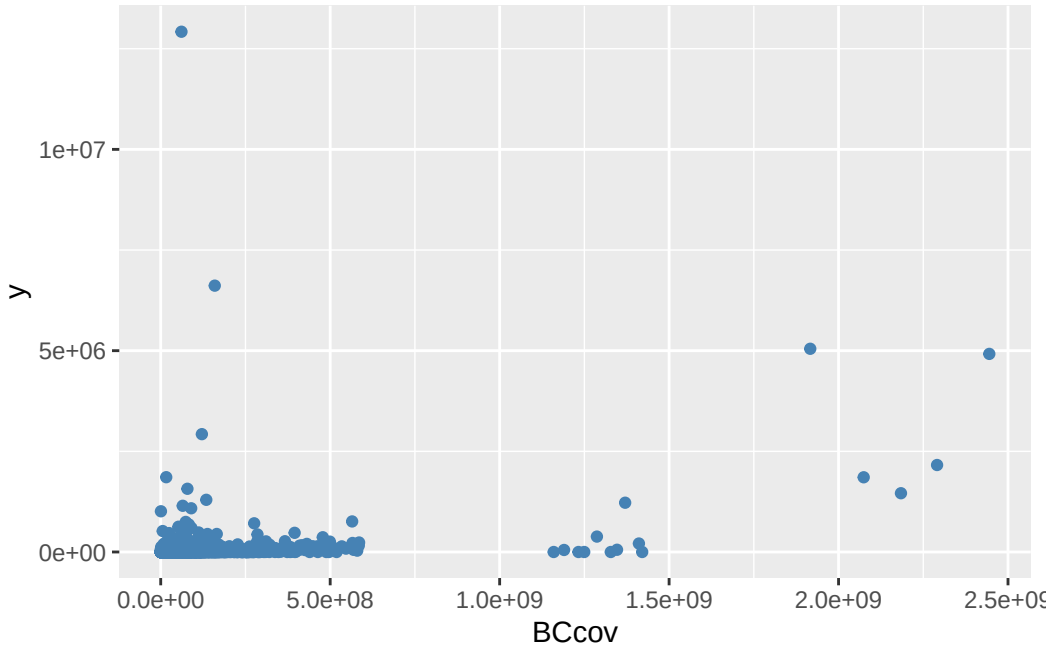
```
ggplot(claims, aes(x = BCcov, y = y, colour = factor(Deduct))) +
  geom_point()
```



Using `factor(Deduct)` treats the deductible as a categorical variable for plotting, even though it is stored numerically. This is appropriate when the objective is to distinguish deductible categories rather than represent the deductible as a continuous numerical scale.

Aesthetic mappings should be distinguished from fixed settings. If every point should have the same colour, specify it outside `aes()`:

```
ggplot(claims, aes(x = BCcov, y = y)) +
  geom_point(colour = "steelblue")
```



In the first example, colour represents a variable and a legend is generated. In the second, all points use the same specified colour, so no legend is required.

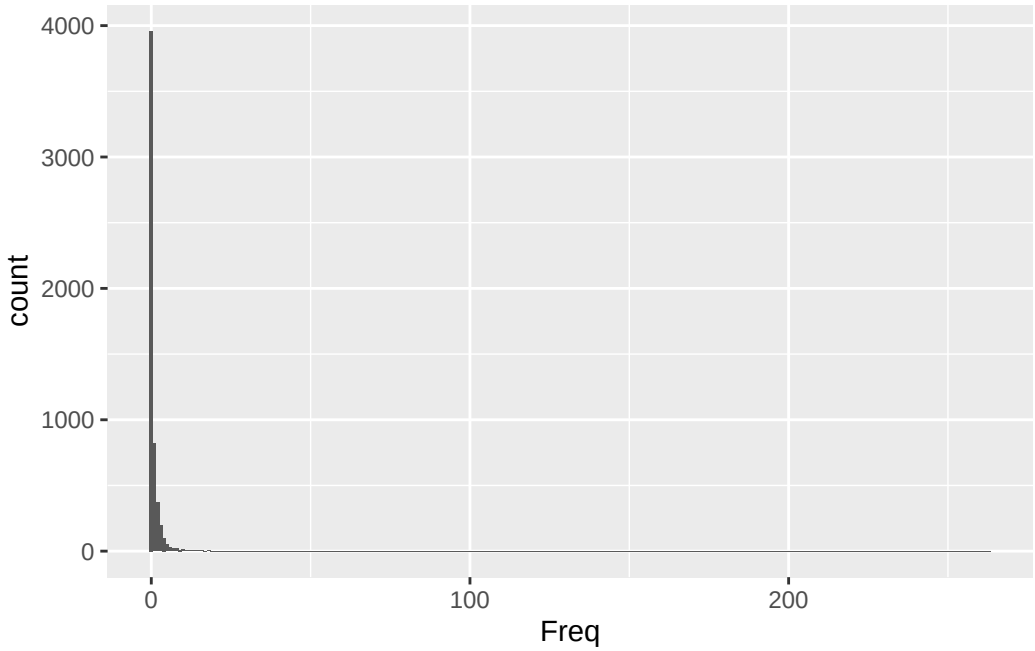
5.2.3 Geometric objects

Geometric objects, or geoms, determine how observations are represented. Different types of data and analytical questions call for different geoms.

Geom	Typical purpose
`geom_point()`	Explore relationships between two quantitative variables
`geom_bar()`	Display counts for categories
`geom_col()`	Display precomputed values for categories
`geom_histogram()`	Examine the distribution of a quantitative variable
`geom_density()`	Display a smoothed estimate of a distribution
`geom_boxplot()`	Compare distributions across groups
`geom_smooth()`	Show a smoothed trend in a relationship

For example, we can examine the distribution of claim frequency using a histogram:

```
ggplot(claims, aes(x = Freq)) +
  geom_histogram(binwidth = 1)
```



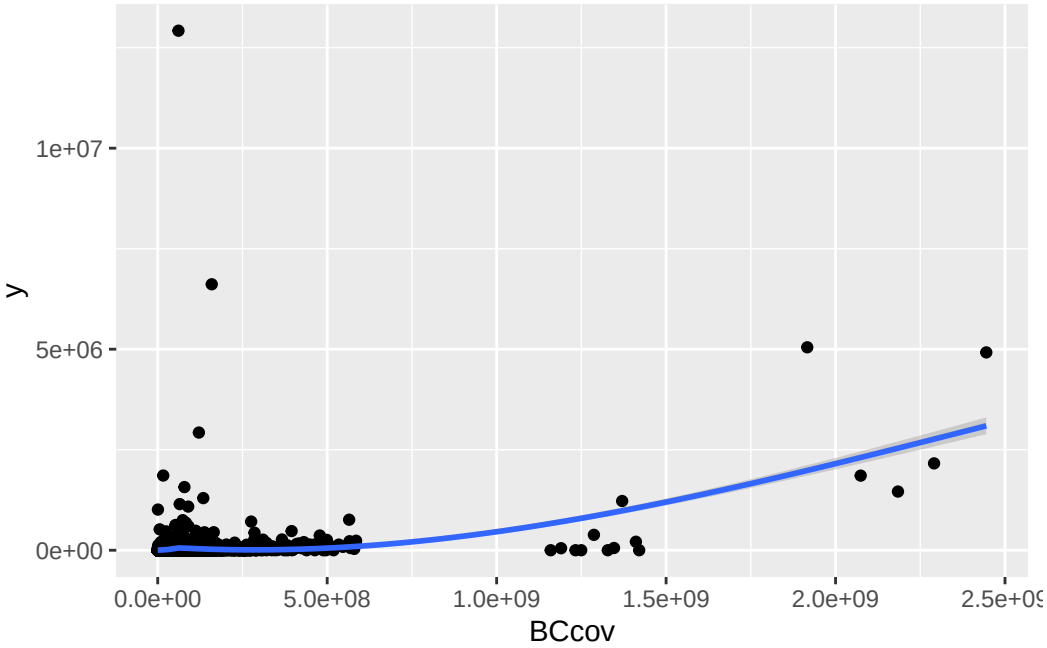
The histogram groups observations into intervals and displays the number of observations in each interval. Here, a bin width of 1 is suitable for exploring a discrete count variable such as `Freq`. Because claim frequency is a count, the resulting bars correspond to intervals of claim counts; later in the chapter, we will consider more suitable ways to display discrete frequency distributions.

The choice of geom depends on the question being investigated. A scatterplot is useful for examining relationships, while a histogram is intended to show a distribution. Choosing an appropriate geom is therefore an important part of constructing a meaningful visualization.

5.2.4 Layers and plot components

A ggplot can contain multiple layers. For example, we can add a smooth trend to the scatterplot of coverage and claim amount:

```
ggplot(claims, aes(x = BCcov, y = y)) +
  geom_point() +
  geom_smooth()
```



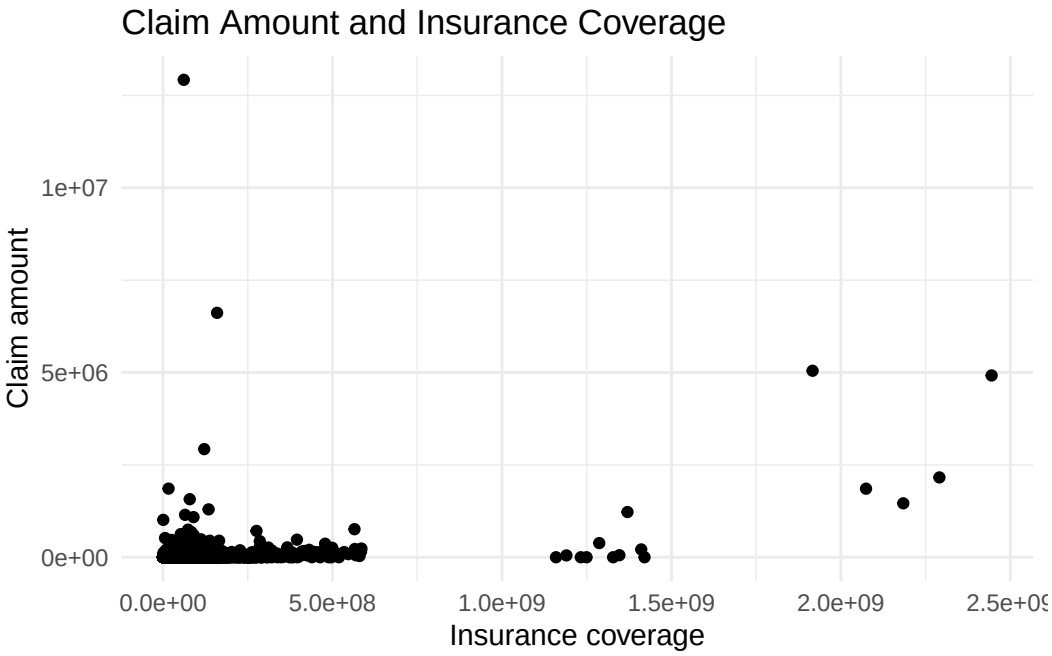
The first layer displays the observations. The second adds a smoothed trend, helping us see the general relationship between the two variables. By default, `geom_smooth()` chooses a smoothing method based on the size of the dataset and the type of relationship being modelled. The shaded region represents uncertainty around the estimated trend.

This trend is exploratory: it does not establish a causal relationship or replace a fitted regression model. Formal regression modelling will be considered in a later chapter.

A plot can also include labels and presentation settings:

```
ggplot(claims, aes(x = BCcov, y = y)) +
  geom_point() +
  labs(
    title = "Claim Amount and Insurance Coverage",
    x = "Insurance coverage",
    y = "Claim amount"
```

```
) +
theme_minimal()
```



The `labs()` function specifies the title and axis labels, while `theme_minimal()` changes the overall appearance of the plot. Clear labels help readers understand what is being displayed without having to refer back to the code.

5.2.5 Understanding the main components

The following table summarizes the components introduced in this section and how they contribute to a plot.

Component	Main role	Example
Data	Supplies the observations	`claims`
Aesthetics	Maps variables to visual properties	`aes(x = BCcov, y = y)`
Geoms	Determines how observations are drawn	`geom_point()`
Scales	Controls how data values are represented visually	`scale_colour_manual()`
Statistics	Computes quantities used in a layer	Counting observations or smoothing a trend
Facets	Splits a plot into panels by groups	`facet_wrap(~Year)`
Labels and themes	Improves communication and appearance	`labs()` and `theme_minimal()`

These components do not all need to be specified in every plot. A basic visualization may require only data, aesthetic mappings, and a geom. Scales, statistical transformations, and facets can be added when they serve a clear analytical purpose.

In the following sections, we will introduce these components in context. Bar charts will be used to explore categorical variables, distribution plots will examine quantitative variables, and scatterplots and facets will help investigate relationships and differences across insurance risk groups.

5.3 Visualizing Categorical Variables

Categorical variables describe groups or classes, such as deductible levels, policy types, and calendar years. Bar charts are useful for comparing the number of observations in each category and examining how the distribution of categories changes across groups.

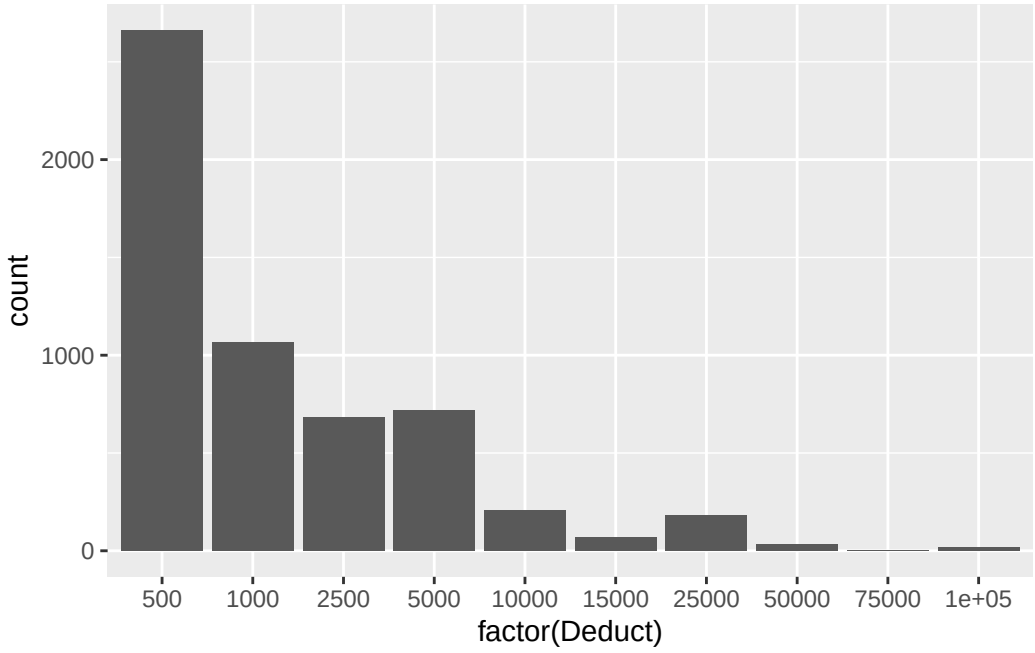
In `ggplot2`, two functions are particularly useful for constructing bar charts:

- `geom_bar()` counts observations in each category automatically.
- `geom_col()` displays values that have already been calculated.

5.3.1 Bar charts of counts

We begin by examining the distribution of deductible levels in the `claims` dataset. Although `Deduct` is stored numerically, we treat it as a categorical variable because the objective is to compare the number of observations at each deductible level.

```
library(ggplot2)

ggplot(claims, aes(x = factor(Deduct))) +
  geom_bar()
```



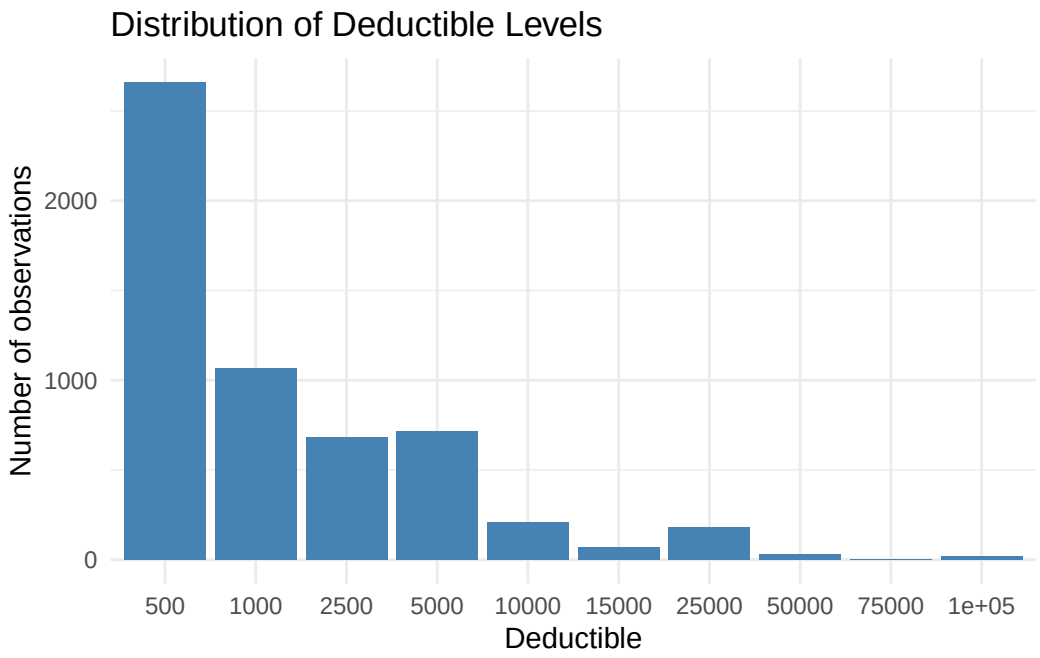
The horizontal axis shows the deductible categories, while the vertical axis shows the number of observations in each category. By default, `geom_bar()` counts the observations associated with each category.

This plot provides a simple summary of the portfolio's deductible distribution. For example, it can reveal whether most observations are concentrated in a small number of deductible categories.

The `factor()` function converts `Deduct` into a categorical variable for plotting. This is useful when the numerical values represent distinct levels rather than a continuous scale.

We can improve the presentation by adding a title and descriptive axis labels.

```
ggplot(claims, aes(x = factor(Deduct))) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Distribution of Deductible Levels",
    x = "Deductible",
    y = "Number of observations"
  ) +
  theme_minimal()
```

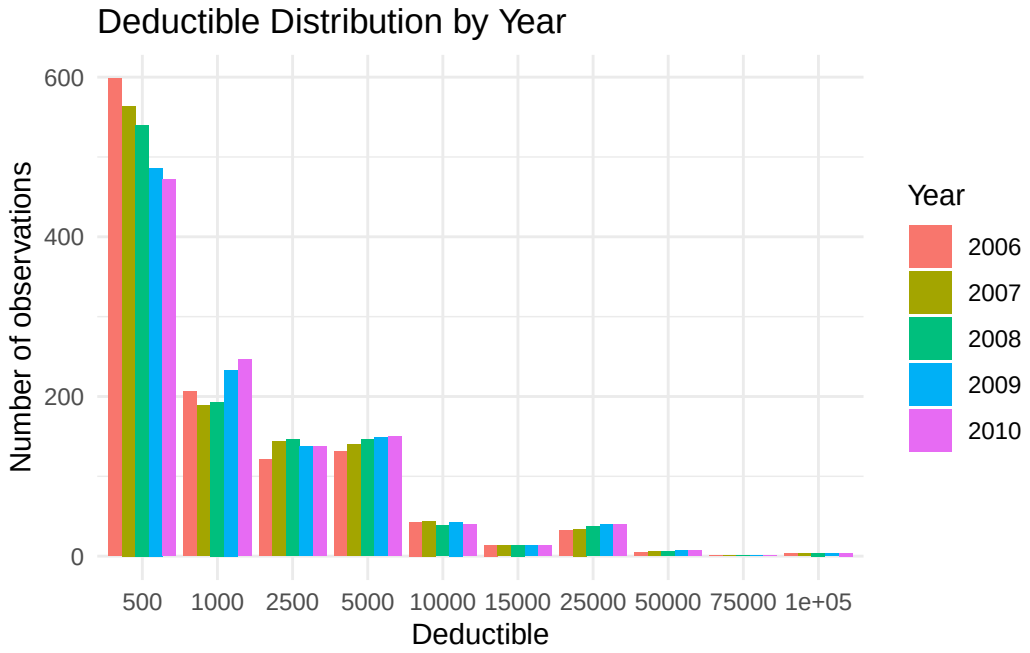


Here, `fill = "steelblue"` is a fixed setting rather than an aesthetic mapping. All bars have the same colour, so no legend is needed.

5.3.2 Comparing counts across groups

Bar charts can also compare categories across groups. For example, we can examine the number of observations at each deductible level in different years.

```
ggplot(claims, aes(x = factor(Deduct), fill = factor(Year))) +
  geom_bar(position = "dodge") +
  labs(
    title = "Deductible Distribution by Year",
    x = "Deductible",
    y = "Number of observations",
    fill = "Year"
  ) +
  theme_minimal()
```



The argument `position = "dodge"` places bars for different years side by side within each deductible category. This makes it easier to compare counts across years than when the bars overlap.

However, differences in counts do not necessarily imply changes in the portfolio's deductible mix. The number of observations may also change from year to year. Comparing proportions can help distinguish changes in portfolio size from changes in the relative distribution of deductible levels.

5.3.3 Comparing proportions

Counts describe the number of observations in each category. Proportions describe the share of observations belonging to each category.

We first calculate the number of observations at each deductible level and then divide by the total number of observations.

```
library(dplyr)

deductible_summary <- claims |>
  filter(!is.na(Deduct)) |>
```

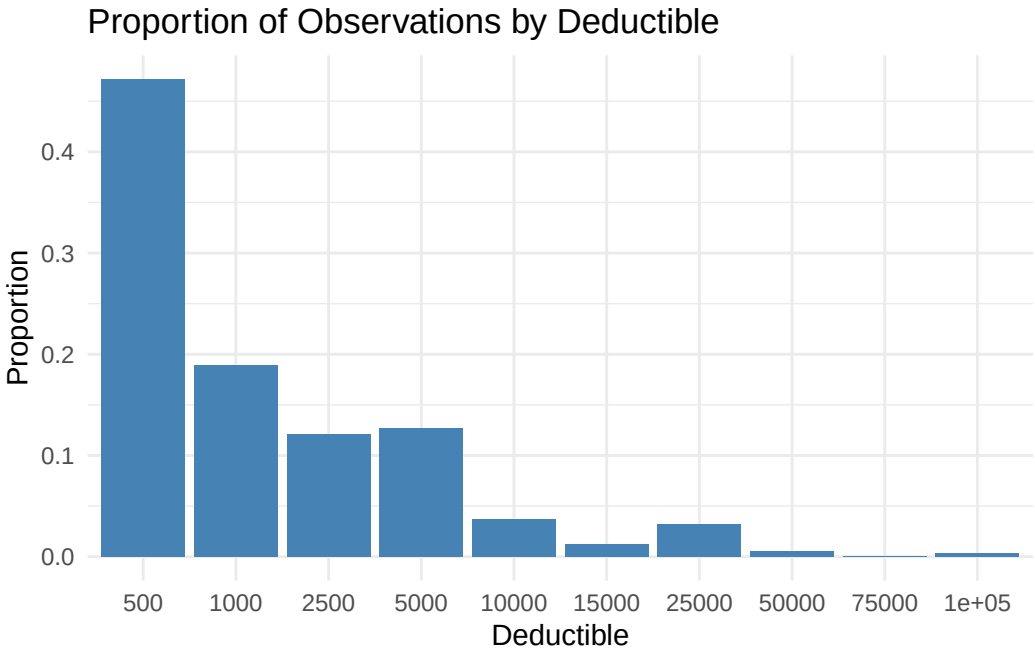
```
  count(Deduct) |>
  mutate(proportion = n / sum(n))

deductible_summary
```

```
# A tibble: 10 x 3
   Deduct     n proportion
    <dbl> <int>      <dbl>
 1    500  2660   0.472
 2   1000  1067   0.189
 3   2500   685   0.121
 4   5000   716   0.127
 5  10000   208   0.0369
 6  15000    67   0.0119
 7  25000   183   0.0325
 8  50000    31   0.00550
 9  75000     5   0.000887
10 100000    17   0.00301
```

The variable `n` contains the number of observations in each deductible category, while `proportion` gives the corresponding share of the observed portfolio. The proportions sum to 1, apart from possible rounding when displayed.

We can use `geom_col()` to display these precomputed proportions.

```
ggplot(deductible_summary,
       aes(x = factor(Deduct), y = proportion)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Proportion of Observations by Deductible",
    x = "Deductible",
    y = "Proportion"
  ) +
  theme_minimal()
```



Unlike `geom_bar()`, `geom_col()` does not count observations. It uses the values supplied through the `y` aesthetic, which in this example are the calculated proportions.

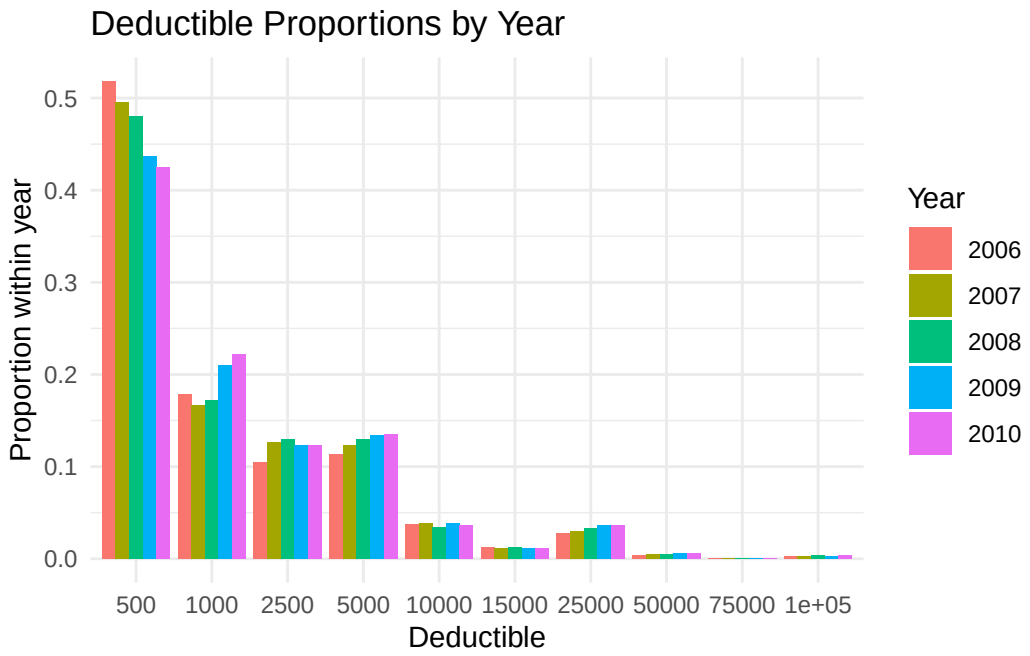
Proportions are particularly useful when comparing groups of different sizes. For example, two years may have different numbers of policy observations but similar proportions across deductible categories.

5.3.4 Comparing proportions across years

To examine whether the deductible mix changes over time, we calculate the proportion of observations in each deductible category separately for each year.

```
deductible_by_year <- claims |>
  filter(!is.na(Deduct)) |>
  count(Year, Deduct) |>
  group_by(Year) |>
  mutate(proportion = n / sum(n)) |>
  ungroup()

ggplot(deductible_by_year,
       aes(x = factor(Deduct),
           y = proportion,
           fill = factor(Year))) +
  geom_col(position = "dodge") +
  labs(
    title = "Deductible Proportions by Year",
    x = "Deductible",
    y = "Proportion within year",
    fill = "Year"
```

```
) +
theme_minimal()
```



Within each year, the proportions across the observed deductible categories sum to 1. The plot therefore compares the relative composition of the portfolio rather than the total number of observations.

If there are many years, however, the grouped bars can become difficult to read. A faceted plot or a proportional stacked bar chart may be more appropriate when the objective is to compare the composition of several groups.

5.3.5 Counts and proportions: choosing the appropriate display

The choice between counts and proportions depends on the question being investigated.

Objective	Suitable display	Interpretation
Compare the number of observations in each category	`geom_bar()`	Shows category counts
Display calculated counts or proportions	`geom_col()`	Uses values already computed
Compare portfolio size across years	Bar chart of counts	Shows differences in the number of observations
Compare the relative composition of groups	Bar chart of proportions	Shows differences in category shares

A count is appropriate when portfolio size is itself of interest. A proportion is more informative when the objective is to compare the relative composition of groups of different sizes.

In actuarial applications, this distinction is important. An increase in the number of policies with a particular deductible may reflect portfolio growth rather than a change in policyholder preferences.

5.4 Visualizing Quantitative Distributions

Quantitative variables describe numerical measurements, such as claim frequency, claim amount, and insurance coverage. Visualizing their distributions helps us understand where observations are concentrated, how much variation exists, whether the data are skewed, and whether some observations are unusually large or small.

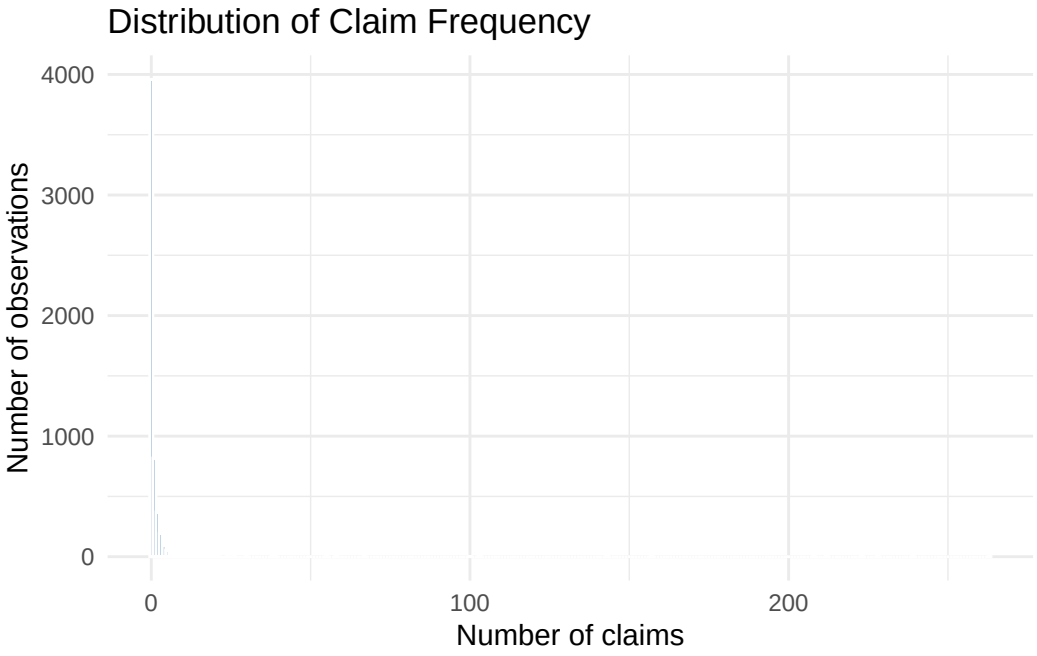
In actuarial work, these features matter because insurance data often contain many small or zero values alongside a small number of large claims. Different plots emphasize different aspects of a distribution. We will use histograms, density plots, and boxplots to explore these features.

5.4.1 Histograms

A histogram divides a quantitative variable into intervals, called *bins*, and displays the number of observations falling within each interval. It is useful for examining the shape and spread of a distribution.

We begin with `Freq`, which records claim frequency in the `claims` dataset.

```
ggplot(claims, aes(x = Freq)) +
  geom_histogram(
    binwidth = 1,
    boundary = -0.5,
    fill = "steelblue",
    colour = "white"
  ) +
  labs(
    title = "Distribution of Claim Frequency",
    x = "Number of claims",
    y = "Number of observations"
  ) +
  theme_minimal()
```



Because claim frequency is a discrete count, a bin width of 1 with the boundary set at -0.5 places each integer claim count at the centre of a bin. This makes the plot easier to interpret than a histogram whose intervals divide the integer values inconsistently.

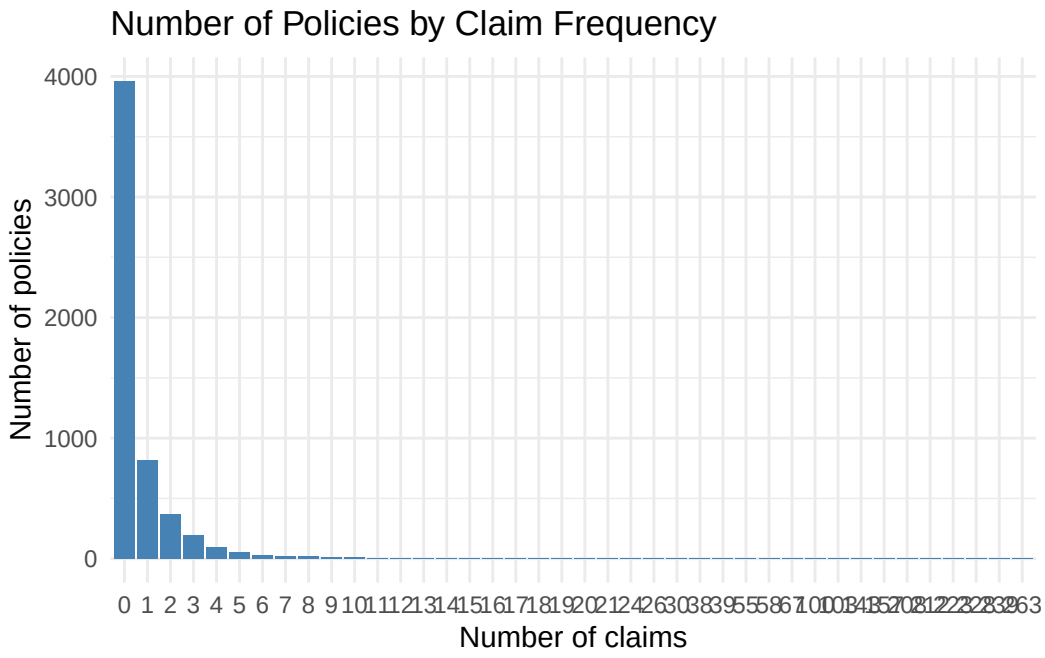
The distribution shows how observations are spread across different claim counts. In insurance portfolios, zero claims are often common, while higher claim counts occur less frequently. The histogram helps reveal this concentration and the extent of the upper tail.

For a discrete variable such as `Freq`, a bar chart using `geom_bar()` is also appropriate. The histogram is useful here for introducing the general idea of a distribution and the role of bin width, but the bar chart more directly represents the distinct claim counts.

Example 5.1. Example: Displaying claim frequency

A histogram of claim frequency may show a large concentration of observations near zero. Because claim frequency takes non-negative integer values, a bar chart provides a direct comparison of the number of observations at each count.

```
ggplot(claims, aes(x = factor(Freq))) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Number of Policies by Claim Frequency",
    x = "Number of claims",
    y = "Number of policies"
  ) +
  theme_minimal()
```



Each bar represents a distinct claim count. A tall bar at zero indicates that many observations have no claims, while shorter bars at higher counts show how frequently multiple claims occur.

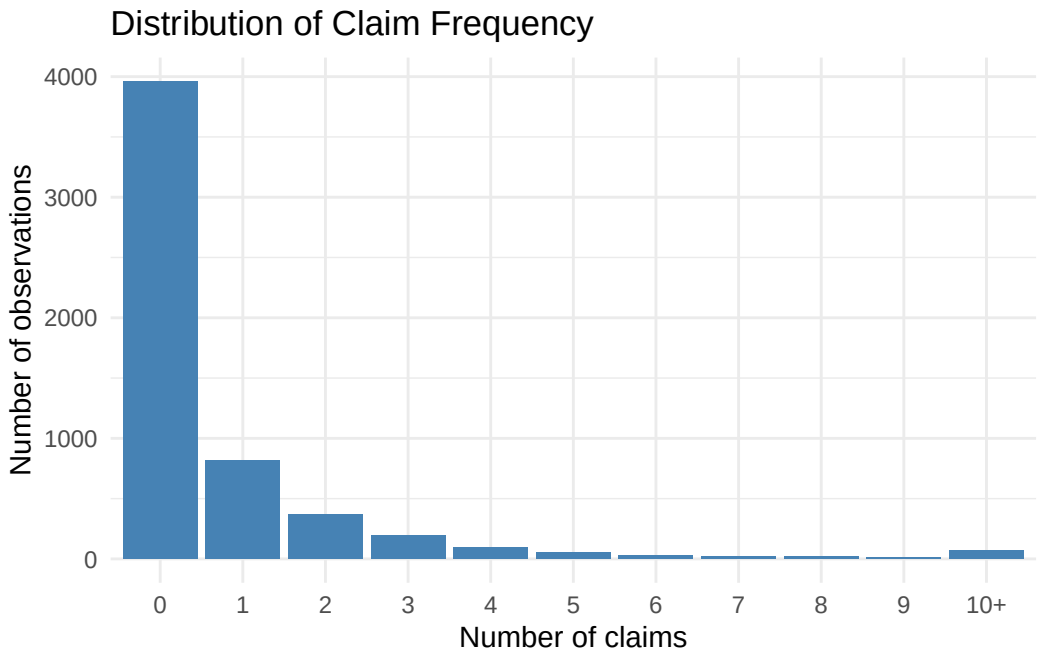
This example illustrates why a bar chart is often appropriate for discrete count data.

Example 5.2. Example: Grouping high claim frequencies

A small number of observations may have very high claim frequencies. To make the distribution easier to read, we group all observations with 10 or more claims into a single category, labelled `10+`.

```
library(dplyr)

claims_grouped <- claims |>
  filter(!is.na(Freq)) |>
  mutate(
    Freq_group = if_else(
      Freq >= 10,
      "10+",
      as.character(Freq)
    ),
    Freq_group = factor(
      Freq_group,
      levels = c(as.character(0:9), "10+")
    )
  )

ggplot(claims_grouped, aes(x = Freq_group)) +
  geom_bar(fill = "steelblue") +
  labs(
    title = "Distribution of Claim Frequency",
```

```
    x = "Number of claims",
    y = "Number of observations"
  ) +
  theme_minimal()
```



The plot displays separate bars for claim frequencies from 0 to 9 and one bar for all frequencies of 10 or more. This makes the main part of the distribution easier to read while retaining the information that a small number of observations have high claim frequencies.

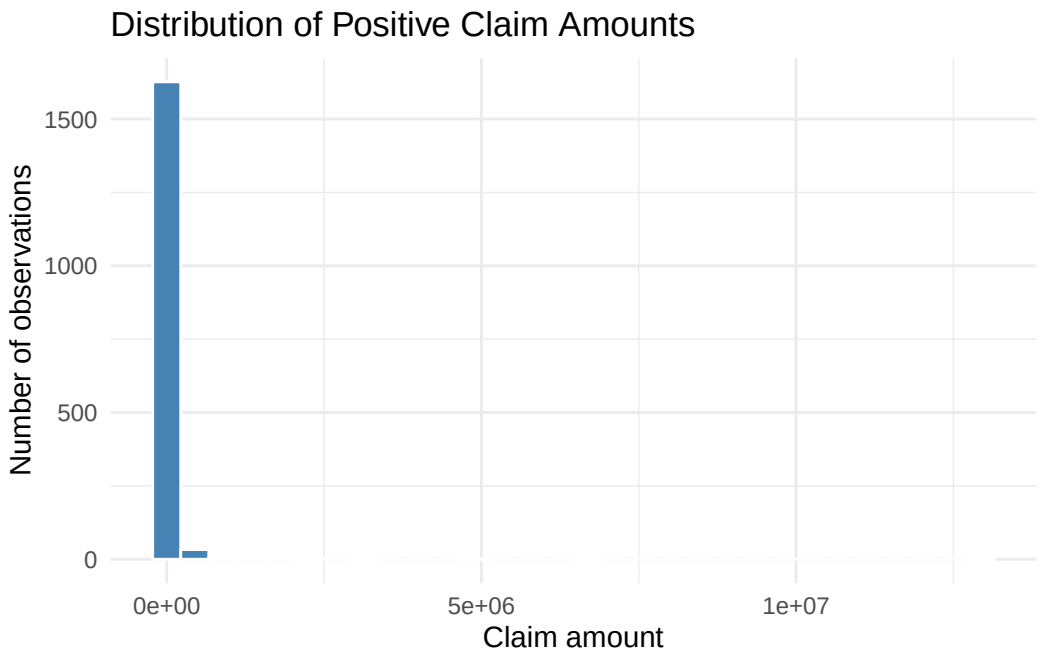
Grouping is a presentation choice: the original `Freq` values remain unchanged in `claims`. The grouped variable should be used for this visualization, not as a replacement for the original claim frequency in statistical analysis.

5.4.2 Histograms of claim amounts

We next examine the distribution of `y`, which records claim amount in the dataset. For this example, we focus on positive claim amounts to study the distribution of amounts when a positive amount is recorded.

```
claims_positive <- claims |>
  filter(!is.na(y), y > 0)

ggplot(claims_positive, aes(x = y)) +
  geom_histogram(
    bins = 30,
    fill = "steelblue",
    colour = "white"
  ) +
```

```
  labs(
    title = "Distribution of Positive Claim Amounts",
    x = "Claim amount",
    y = "Number of observations"
  ) +
  theme_minimal()
```



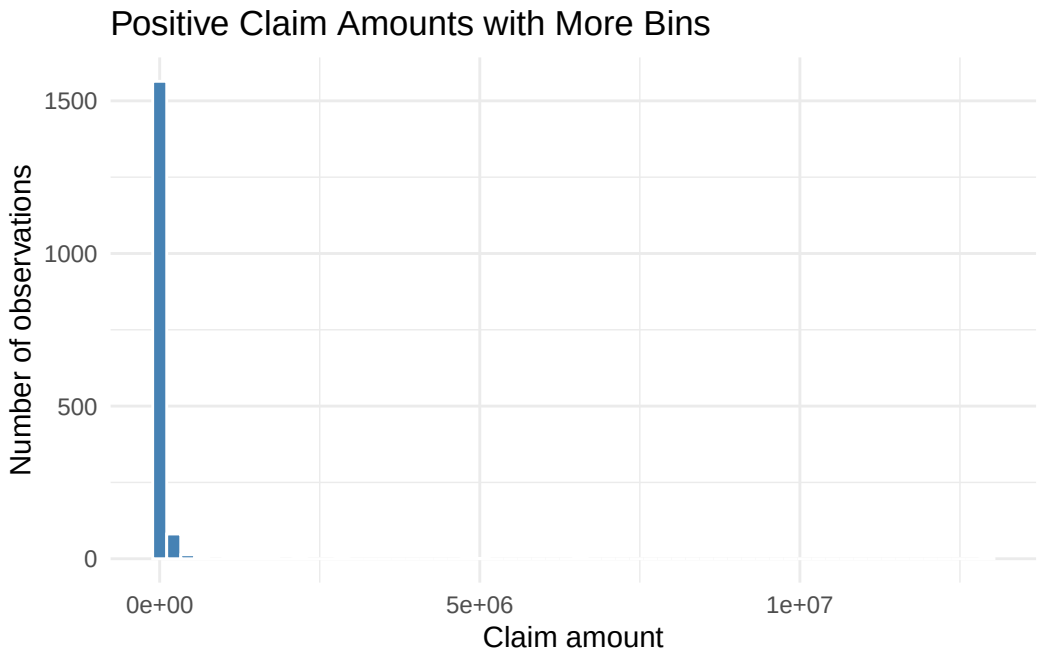
The histogram provides an initial view of the distribution of positive claim amounts. Insurance claim amounts commonly exhibit right skewness: many claims have relatively small amounts, while a smaller number have much larger amounts. These large claims extend the distribution's right tail.

The appearance of a histogram depends on the choice of bin width or number of bins. Too few bins can conceal important features, while too many can make random variation appear more prominent than it is. It is therefore useful to try different settings and check whether the main features of the distribution remain visible.

For example, we can increase the number of bins to examine the distribution in greater detail.

```
ggplot(claims_positive, aes(x = y)) +
  geom_histogram(
    bins = 60,
    fill = "steelblue",
    colour = "white"
  ) +
  labs(
    title = "Positive Claim Amounts with More Bins",
    x = "Claim amount",
    y = "Number of observations"
```

```
  ) +
  theme_minimal()
```

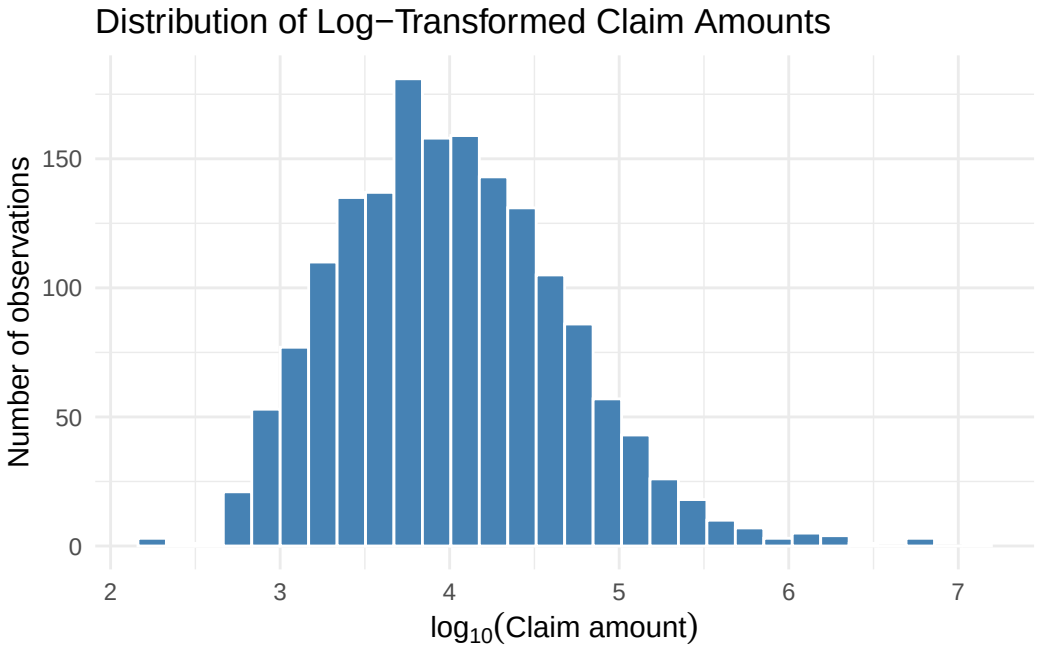


The two histograms may reveal different levels of detail, but neither is automatically better. The choice should help readers understand the overall distribution without obscuring its important features.

Example 5.3. Example: Examining claim amounts on a logarithmic scale

When a few large claims stretch the horizontal axis, smaller claim amounts can become difficult to distinguish. A logarithmic transformation can make the distribution easier to examine.

```
ggplot(claims_positive, aes(x = log10(y))) +
  geom_histogram(
    bins = 30,
    fill = "steelblue",
    colour = "white"
  ) +
  labs(
    title = "Distribution of Log-Transformed Claim Amounts",
    x = expression(log[10]("Claim amount")),
    y = "Number of observations"
  ) +
  theme_minimal()
```



For example, claim amounts of 100, 1,000, and 10,000 become 2, 3, and 4 on the logarithmic scale. This can reveal differences among smaller and moderate claims that are less apparent on the original scale.

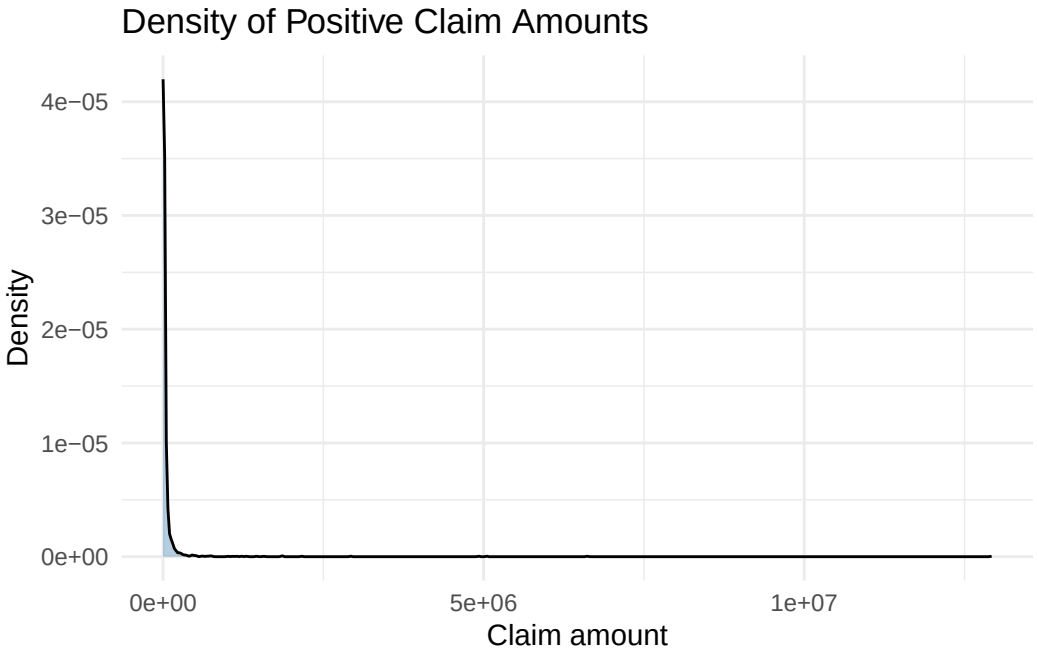
The horizontal axis now represents log-transformed amounts rather than monetary values. The transformation changes the appearance of the distribution but does not remove extreme claims.

5.4.3 Density plots

A density plot provides a smoothed representation of a quantitative distribution. Unlike a histogram, which groups observations into bins, a density plot estimates how the observations are distributed across the range of values.

We can use `geom_density()` to examine the distribution of positive claim amounts.

```
ggplot(claims_positive, aes(x = y)) +
  geom_density(fill = "steelblue", alpha = 0.4) +
  labs(
    title = "Density of Positive Claim Amounts",
    x = "Claim amount",
    y = "Density"
  ) +
  theme_minimal()
```



The density curve provides a smooth view of where claim amounts are concentrated. Peaks indicate regions with relatively high estimated density, while low regions indicate values that occur less frequently relative to nearby values.

Density is not the same as the number or proportion of observations at a particular value. The total area under a density curve is 1, so the area over an interval represents the estimated proportion of observations within that interval. The height of the curve at a single point is not itself a probability.

The degree of smoothing depends on a parameter called the *bandwidth*. A smaller bandwidth produces a more detailed curve, while a larger bandwidth produces a smoother curve. As with histogram bin width, the smoothing choice affects the appearance of the plot.

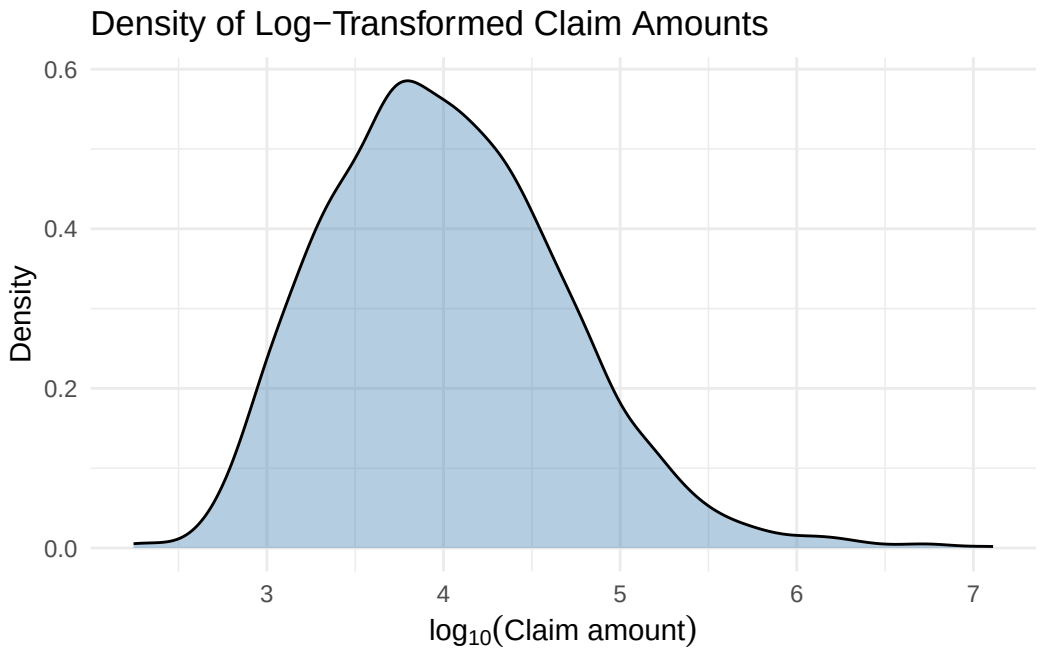
Density plots are particularly useful for comparing distributions on a common scale. However, they should be interpreted carefully for discrete variables or variables with a large concentration of observations at zero. For claim frequency, a bar chart is generally more suitable.

Example 5.4. Example: Density of log-transformed claim amounts

A density plot of log-transformed claim amounts provides a smoothed view of the distribution on the logarithmic scale.

```
ggplot(claims_positive, aes(x = log10(y))) +
  geom_density(
    fill = "steelblue",
    alpha = 0.4
  ) +
  labs(
    title = "Density of Log-Transformed Claim Amounts",
    x = expression(log[10]("Claim amount")),
    y = "Density"
```

```
  ) +
  theme_minimal()
```



The transformed plot may make the central structure of the distribution easier to see. The total area under the density curve remains 1, but the density is now measured on the logarithmic scale, not per unit of the original claim amount.

5.4.4 Boxplots

A boxplot summarizes a distribution using its median, quartiles, and potential outliers. It is particularly useful for comparing a quantitative variable across groups.

The box represents the interquartile range (IQR), from the first quartile Q_1 to the third quartile Q_3. The line inside the box marks the median, Q_2. The IQR is

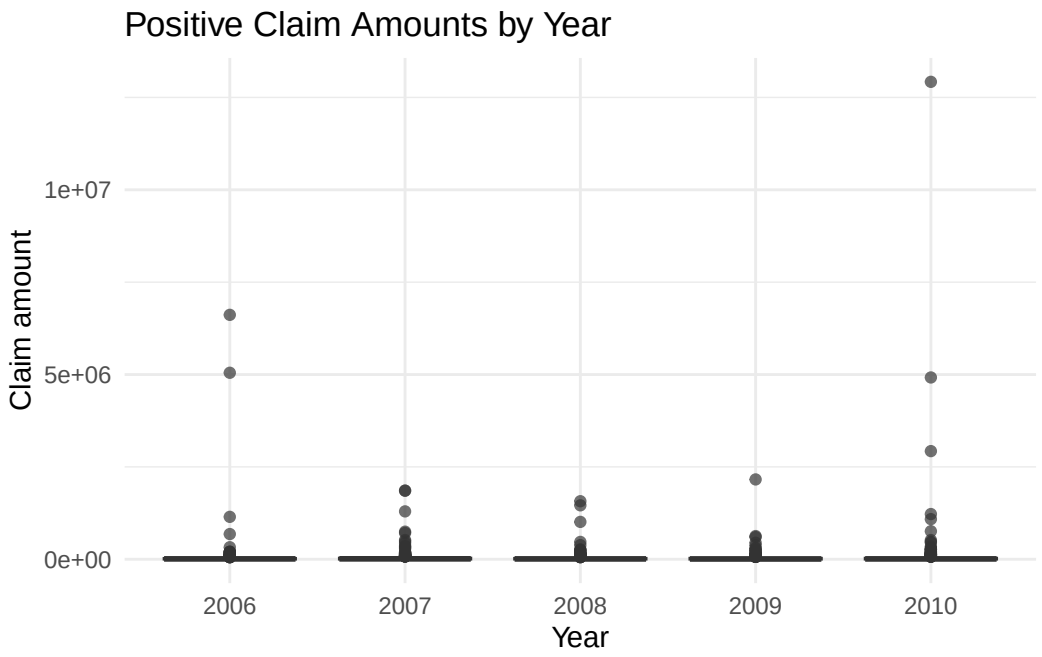
$$\text{IQR} = Q_3 - Q_1.$$

The whiskers typically extend to the most extreme observations within $1.5 \times \text{IQR}$ of the corresponding quartile. Observations beyond the whiskers are plotted individually and flagged as potential outliers.

We can compare positive claim amounts across years using a boxplot.

```
ggplot(
  claims_positive,
  aes(x = factor(Year), y = y)
) +
  geom_boxplot(fill = "steelblue", alpha = 0.7) +
```

```
  labs(
    title = "Positive Claim Amounts by Year",
    x = "Year",
    y = "Claim amount"
  ) +
  theme_minimal()
```



This plot allows us to compare the median claim amount, the spread of the central half of the observations, and the presence of unusually large claims across years. If the distributions are strongly right-skewed, a small number of large claims may appear far above the boxes and make differences in the central distributions difficult to see.

A boxplot provides a compact summary, but it does not show the full shape of a distribution. Histograms and density plots can reveal features that a boxplot may conceal, such as multiple peaks or concentrations near particular values.

5.4.5 Potential outliers and data errors

An unusual observation is not necessarily an error. In insurance data, an unusually large claim may be genuine and important for understanding the insurer's exposure to severe losses.

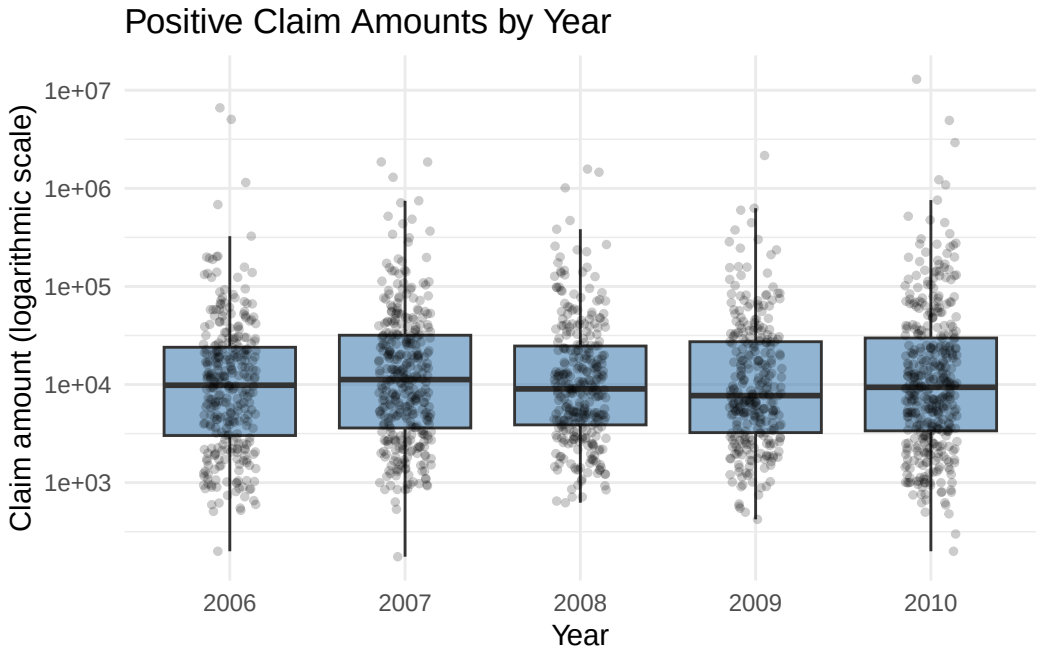
Boxplots identify potential outliers according to a statistical rule. They do not establish that an observation is incorrect, nor do they identify every observation that deserves investigation.

When an unusual claim amount is found, an actuary should consider whether it reflects a genuine large loss, a data-entry error, an incorrect unit, or another recording issue. The appropriate response depends on the context and the purpose of the analysis.

Example 5.5. Example: Comparing claim amounts using boxplots

Claim amounts can be strongly right-skewed, with a small number of large claims making differences between the central portions of the distributions difficult to see. A logarithmic vertical scale can make these differences easier to examine.

```
ggplot(
  claims_positive,
  aes(x = factor(Year), y = y)
) +
  geom_boxplot(
    fill = "steelblue",
    alpha = 0.6,
    outlier.shape = NA
  ) +
  geom_jitter(
    width = 0.15,
    alpha = 0.2,
    size = 1
  ) +
  scale_y_log10() +
  labs(
    title = "Positive Claim Amounts by Year",
    x = "Year",
    y = "Claim amount (logarithmic scale)"
  ) +
  theme_minimal()
```



The boxplots compare the median and interquartile range of positive claim amounts across years.

The jittered points show individual observations, with a small horizontal displacement to reduce overlap.

The logarithmic scale makes variation among smaller claims easier to see when claim amounts span a wide range. Because the scale is logarithmic, equal vertical distances represent equal ratios rather than equal differences in claim amounts.

The boxplot's potential outliers are suppressed in this example to avoid plotting them twice: the individual observations are already displayed by `geom_jitter()`. These points may overlap, so the plot should be interpreted as an exploratory display rather than an exact count of observations at each value.

5.4.6 Choosing a suitable plot

The following table summarizes the main uses of the three plots introduced in this section.

Plot	Main purpose	Important consideration
Histogram	Examine the shape and spread of a distribution	Appearance depends on bin width
Density plot	View a smoothed distribution and compare shapes	Depends on bandwidth; total area equals 1
Boxplot	Summarize a distribution and compare groups	Does not show the full distribution shape

These plots complement one another. A histogram or density plot helps reveal the overall distribution, while a boxplot provides a compact summary and makes comparisons across groups convenient. In actuarial applications, using them together can help identify important distributional features before fitting a statistical model.

5.5 Visualizing Relationships Between Variables

Actuaries often need to investigate how one variable is associated with another. For example, claim amounts may vary with insurance coverage, while insurance charges may differ with age. Visualizing these relationships helps identify patterns, unusual observations, and potential nonlinear relationships before fitting a statistical model.

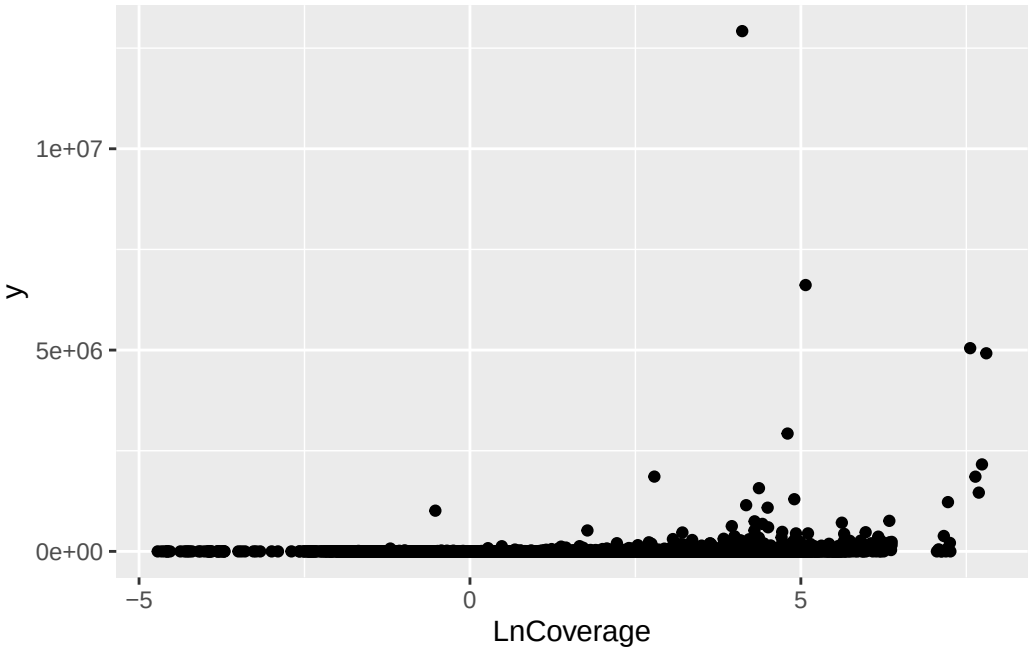
5.5.1 Scatterplots

A scatterplot displays the relationship between two quantitative variables. Each point represents one observation, with its horizontal and vertical positions determined by the values of the two variables.

In `ggplot2`, `geom_point()` creates a scatterplot.

We begin with the `claims` dataset introduced earlier in the course. Suppose we want to examine the relationship between insurance coverage and claim amount.

```r
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point()
```



Here, `LnCoverage` represents the logarithm of insurance coverage, while `y` represents the claim amount. Each point corresponds to an observation in the dataset.

The scatterplot allows us to examine whether larger coverage amounts tend to be associated with larger claims. However, the presence of a few very large claims may make the overall pattern difficult to see. Many observations may also overlap, particularly when claim amounts are concentrated near zero.

Example 5.6. Example: Examining claim amounts

Suppose the scatterplot shows that most claims are small, while a few observations have very large claim amounts. These extreme observations may dominate the vertical scale and make the majority of the data appear compressed near the bottom of the plot.

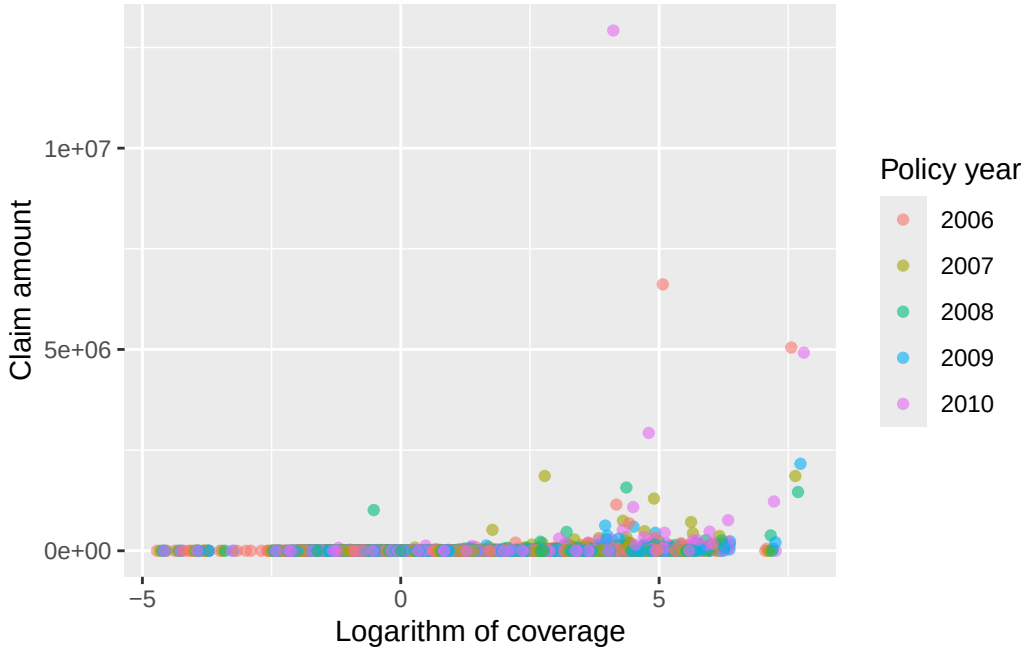
The first step is to examine the original data rather than immediately remove extreme observations. Large claims may be genuine insurance losses and can be important for actuarial analysis.

5.5.2 Adding Aesthetic Mappings

Scatterplots can display additional information by mapping other variables to visual properties such as colour, shape, or size. For example, colour can distinguish observations belonging to different policy groups.

Suppose we want to examine whether the relationship between coverage and claim amount differs across policy years.

```
ggplot(claims, aes(x = LnCoverage, y = y, colour = factor(Year))) +
  geom_point(alpha = 0.6) +
  labs(
    x = "Logarithm of coverage",
    y = "Claim amount",
    colour = "Policy year"
  )
```



The mapping `colour = factor(Year)` treats `Year` as a categorical variable, assigning a separate colour to each year. The `alpha` argument makes the points partially transparent, which can help reveal areas where observations overlap.

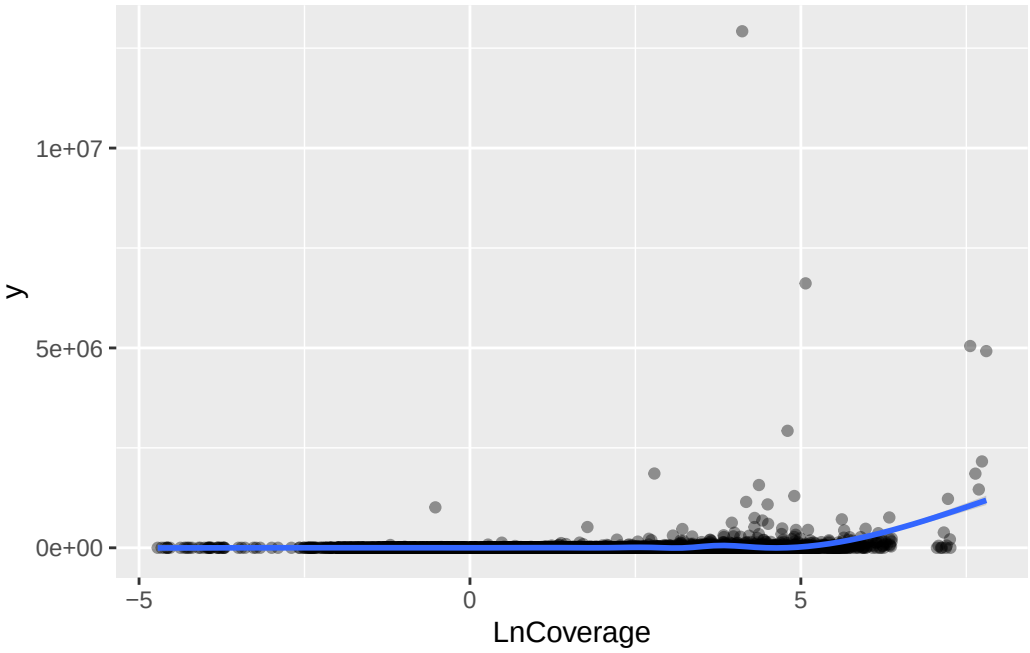
Aesthetic mappings should be selected carefully. Adding too many colours or shapes can make a plot difficult to interpret. When the main objective is to investigate the relationship between two quantitative variables, additional aesthetics should provide information relevant to the question.

5.5.3 Adding Smooth Trend Lines

A scatterplot shows individual observations, but the overall relationship may be difficult to identify when the data are noisy or heavily concentrated in certain regions. A smooth trend line can help reveal the general pattern.

The function `geom_smooth()` adds a fitted trend line to a plot. By default, `ggplot2` selects a smoothing method based on the size of the dataset. For a small or moderate dataset, it commonly uses a locally estimated smooth curve; for larger datasets, it commonly uses a generalized additive model.

```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.4) +
  geom_smooth()
```

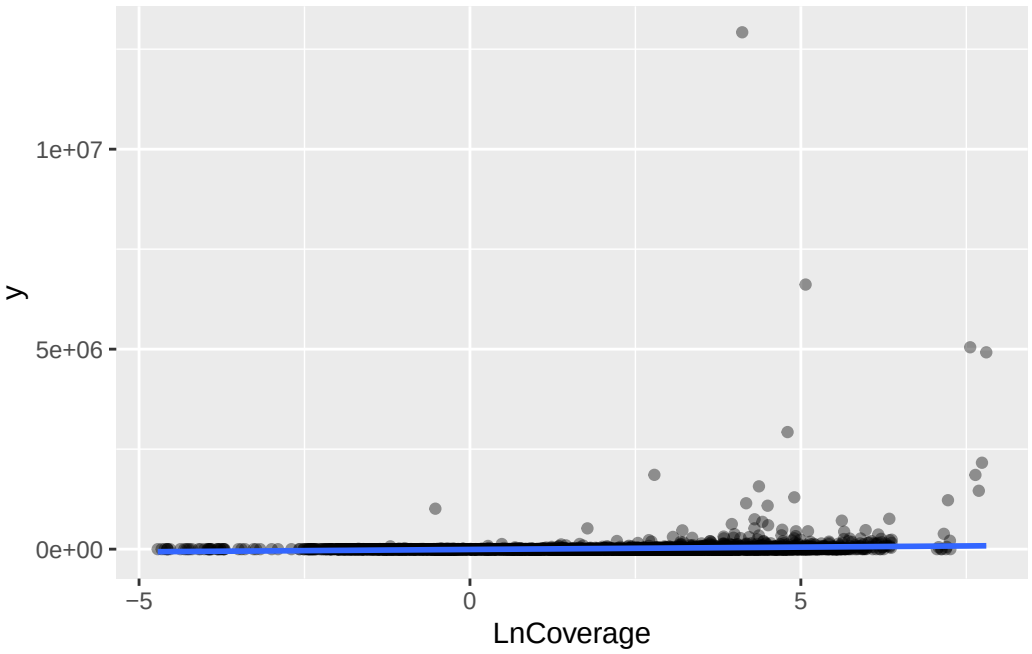


The shaded region around the line represents an approximate confidence interval for the estimated mean response. It describes uncertainty in the fitted trend, not the range within which individual claim amounts are expected to fall.

The smooth line helps us assess whether the relationship appears approximately linear or whether it exhibits curvature. However, it does not establish a causal relationship between coverage and claim amount.

When a straight-line trend is appropriate, we can request a linear fit explicitly.

```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = TRUE)
```



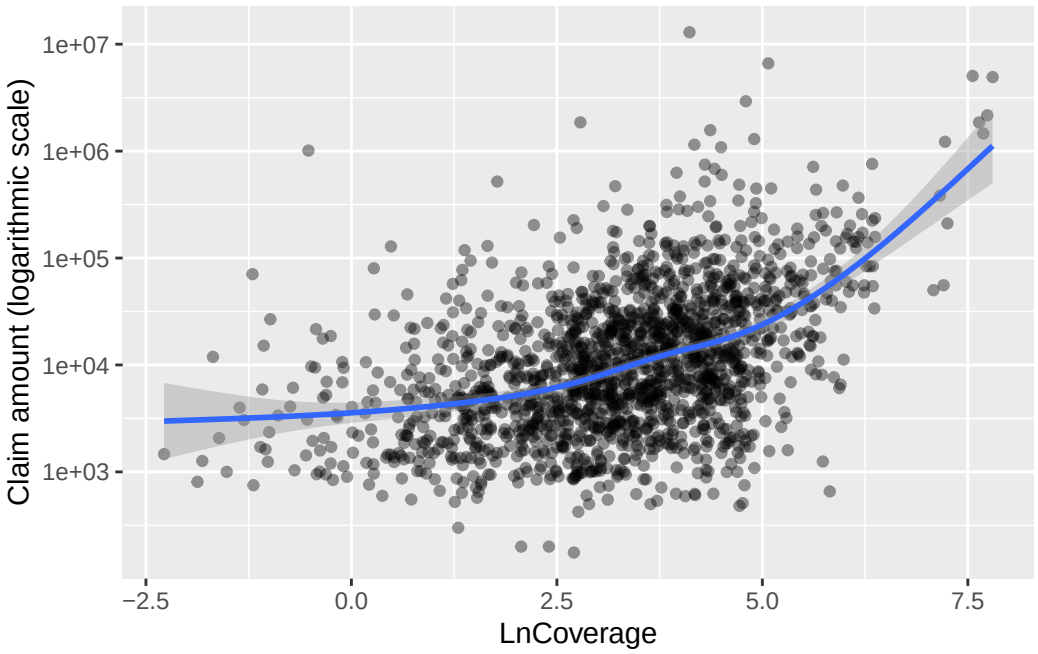
The argument `method = "lm"` fits a linear regression line, while `se = TRUE` displays its confidence band. Comparing a linear fit with a flexible smooth curve can help identify departures from linearity before choosing a regression model.

5.5.4 Improving Plots with Skewed Data

Claim amounts and insurance charges are often right-skewed: most observations are relatively small, while a small number are much larger. When plotted on their original scales, these large values can dominate the axes and obscure patterns among the majority of observations.

One option is to use a logarithmic axis. For example, if claim amounts are positive, we can display them on a logarithmic scale.

```
ggplot(filter(claims, y > 0), aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.4) +
  geom_smooth() +
  scale_y_log10() +
  labs(y = "Claim amount (logarithmic scale)")
```



The condition `y > 0` excludes zero and negative values because a logarithmic scale cannot display them. This is appropriate only if the purpose is to study positive claim amounts; the excluded observations should still be considered when interpreting the dataset as a whole.

The function `scale_y_log10()` changes the display scale of the vertical axis. It does not replace the original values in the dataset. Large claims remain visible, but their distances from smaller claims are compressed, making variation among smaller and moderate claims easier to examine.

Important distinction: Using a logarithmic axis is not the same as fitting a model to logarithmically transformed claim amounts. The former changes how the data are displayed, while the latter changes the response variable and therefore the model being fitted.

5.5.5 Nonlinear Relationships and Overplotting

Not all relationships are linear. A scatterplot may reveal curvature, a change in variability, or a pattern that differs across ranges of the explanatory variable. A smooth trend line can help identify these features.

Overplotting occurs when multiple observations occupy the same or similar positions, causing points to obscure one another. This is common when variables take only a limited number of values or when observations are concentrated in a narrow region.

Several techniques can improve readability:

- Use `alpha` to make points transparent and reveal areas of high concentration.
- Use `geom_jitter()` when observations overlap because of repeated or discrete values. Jitter adds small random displacements, so it should be used cautiously when exact positions matter.
- Use a logarithmic axis when the data are strongly right-skewed and the variable is strictly positive.
- Use grouping or faceting when relationships may differ across policy characteristics.

Extreme observations also require careful interpretation. They may represent genuine large claims, data-entry errors, or unusual policy characteristics. A plot can identify observations worth investigating, but it cannot determine their cause by itself.

5.5.6 Interpreting Relationships in Actuarial Data

When examining a scatterplot, consider the direction, form, and strength of the relationship. A positive association means that larger values of one variable tend to occur with larger values of the other; a negative association means that larger values of one tend to occur with smaller values of the other. The form may be approximately linear or nonlinear, while the strength describes how closely the observations follow the overall pattern.

For actuarial applications, these observations can guide subsequent modelling. For example, a curved relationship between age and insurance charges may suggest that a simple linear model is insufficient. A strong concentration of small claims alongside a few very large claims may indicate that the distribution and scale of the response deserve further attention.

Visual evidence should be interpreted alongside numerical summaries and knowledge of the insurance context. Association does not imply causation, and a visible pattern alone does not establish which statistical model is appropriate.

5.6 Comparing Groups with Plots

Actuaries often compare claim experience across groups of policyholders or policies. For example, claim frequency may differ by policy type, while claim amounts may vary across policy characteristics. Visualizations help reveal differences in distributions, central values, variability, and unusual observations.

The appropriate plot depends on the types of variables involved. Bar charts are useful for comparing counts or proportions of categorical outcomes, while boxplots and other distribution plots are useful for comparing quantitative outcomes across groups.

5.6.1 Comparing Distributions with Boxplots

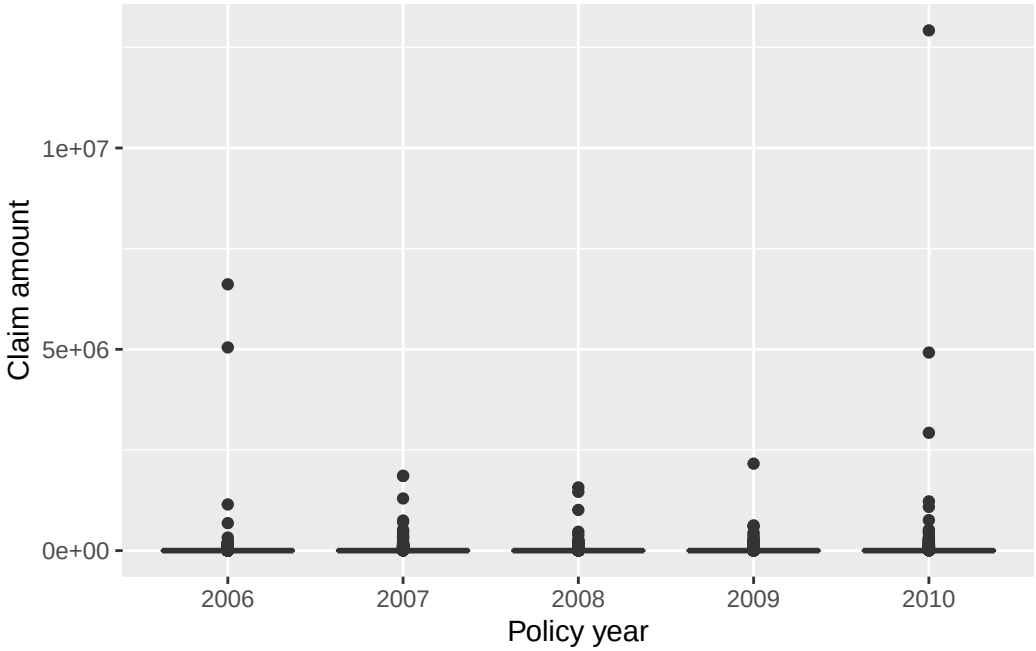
A boxplot summarizes the distribution of a quantitative variable. The box extends from the first quartile (Q_1) to the third quartile (Q_3), with a line marking the median. The interquartile range is

$$\text{IQR} = Q_3 - Q_1.$$

The whiskers extend to the most extreme observations within 1.5 IQR of the quartiles. Observations beyond the whiskers are plotted individually and are often described as potential outliers. They are not necessarily data errors.

Suppose we want to compare claim amounts across policy years using the `claims` dataset.

```
ggplot(claims, aes(x = factor(Year), y = y)) +
  geom_boxplot() +
  labs(
    x = "Policy year",
    y = "Claim amount"
  )
```



The function `factor(Year)` treats policy year as a categorical variable, allowing us to compare the distributions across years rather than treating year as a continuous explanatory variable.

The plot helps us compare the median claim amount, the spread of the central half of observations, and the presence of unusually large claims. However, claim amounts are often strongly right-skewed. A small number of large claims can make the boxes appear compressed near the bottom of the plot.

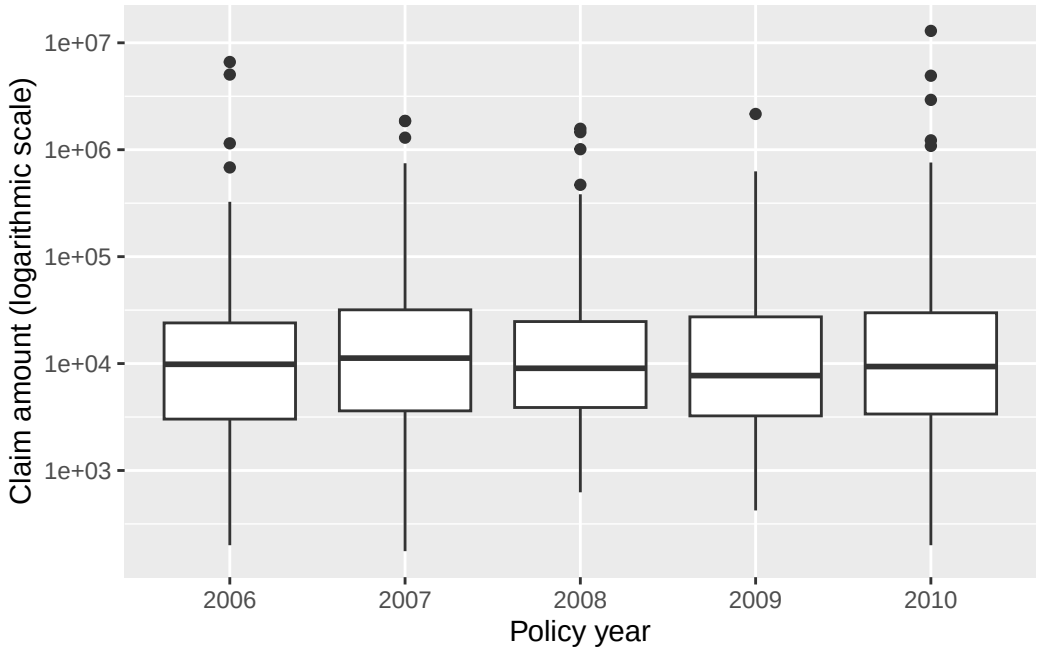
A boxplot also does not show the full shape of a distribution. Two groups can have similar medians and interquartile ranges but differ in other respects, such as skewness or the presence of multiple concentrations of observations.

5.6.2 Improving Comparisons for Right-Skewed Outcomes

When claim amounts are right-skewed, plotting them on the original scale remains useful for understanding the magnitude of losses. However, it may be difficult to compare the distributions of typical claims if a few very large claims dominate the vertical scale.

If the claim amounts are positive, a logarithmic axis can make differences among smaller and moderate claims easier to see.

```
ggplot(
  filter(claims, y > 0),
  aes(x = factor(Year), y = y)
) +
  geom_boxplot() +
  scale_y_log10() +
  labs(
    x = "Policy year",
    y = "Claim amount (logarithmic scale)"
  )
```

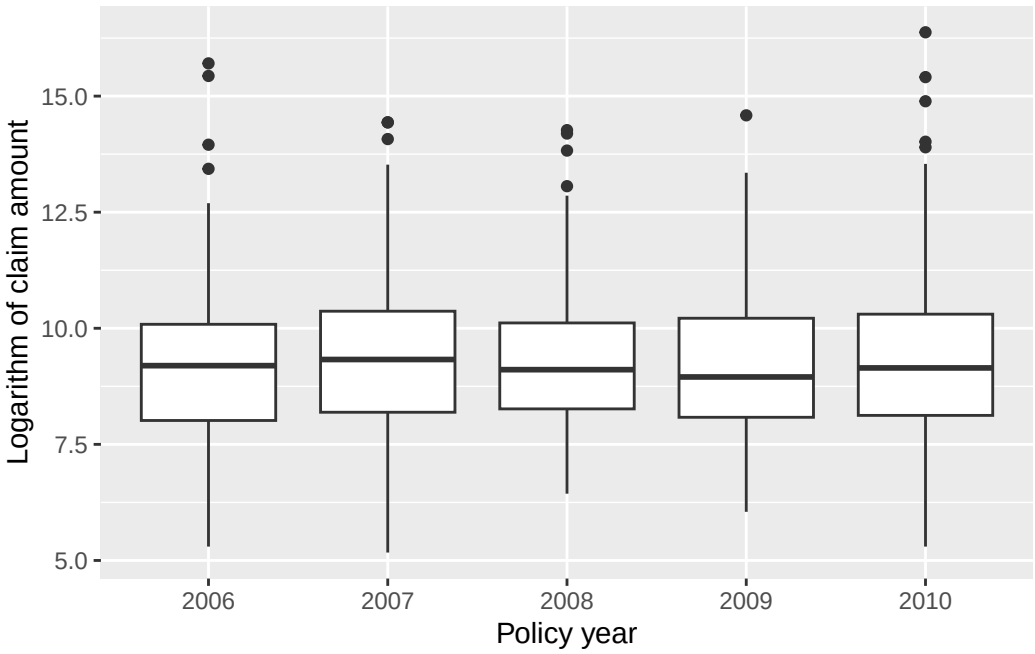


The logarithmic axis compresses large values and expands the visual separation among smaller values. The observations themselves are not changed in the dataset.

This plot excludes zero and negative values, if any. Consequently, it describes the distribution of positive claim amounts rather than the complete claim outcome. The original-scale plot should be retained when the absolute magnitude of large losses is important.

Another option is to examine the logarithm of positive claim amounts directly:

```
ggplot(
  filter(claims, y > 0),
  aes(x = factor(Year), y = log(y))
) +
  geom_boxplot() +
  labs(
    x = "Policy year",
    y = "Logarithm of claim amount"
  )
```



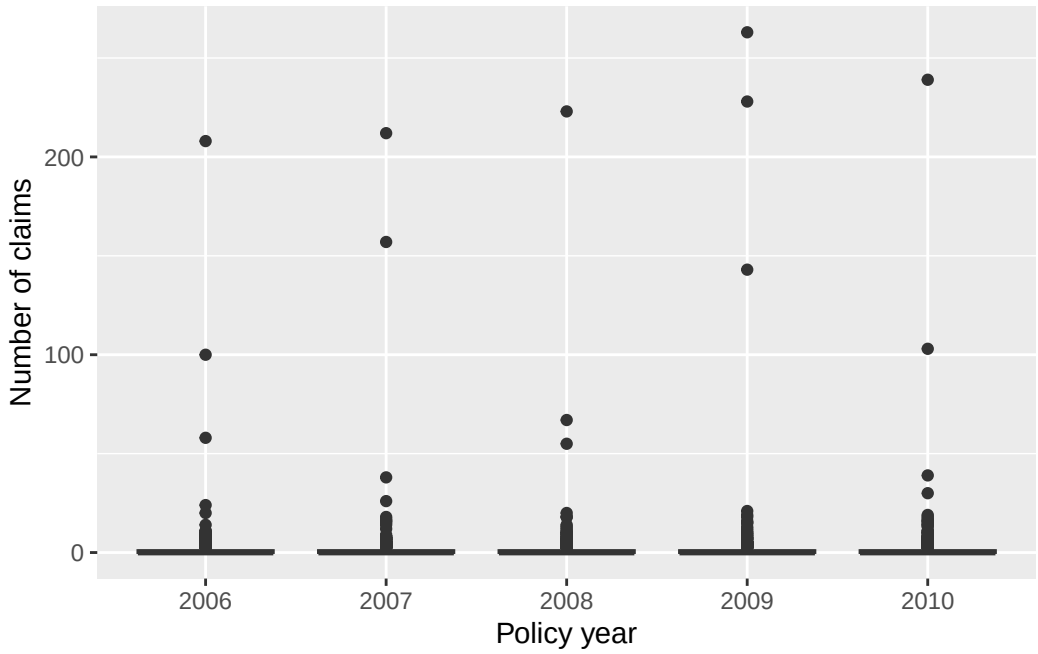
Here, `log(y)` transforms the variable used to construct the boxplot, whereas `scale_y_log10()` changes the display scale. Both can help reveal patterns in right-skewed data, but the resulting summaries are expressed on different scales. The logarithm used by `log()` is the natural logarithm, while `scale_y_log10()` uses base 10.

5.6.3 Comparing Claim Frequency Across Groups

Claim frequency is a count variable, often taking values such as zero, one, or two claims. A boxplot can be used to compare claim counts across groups, but it may be less informative when most policies have no claims and only a small proportion have positive counts.

A boxplot of claim frequency by policy year provides a starting point:

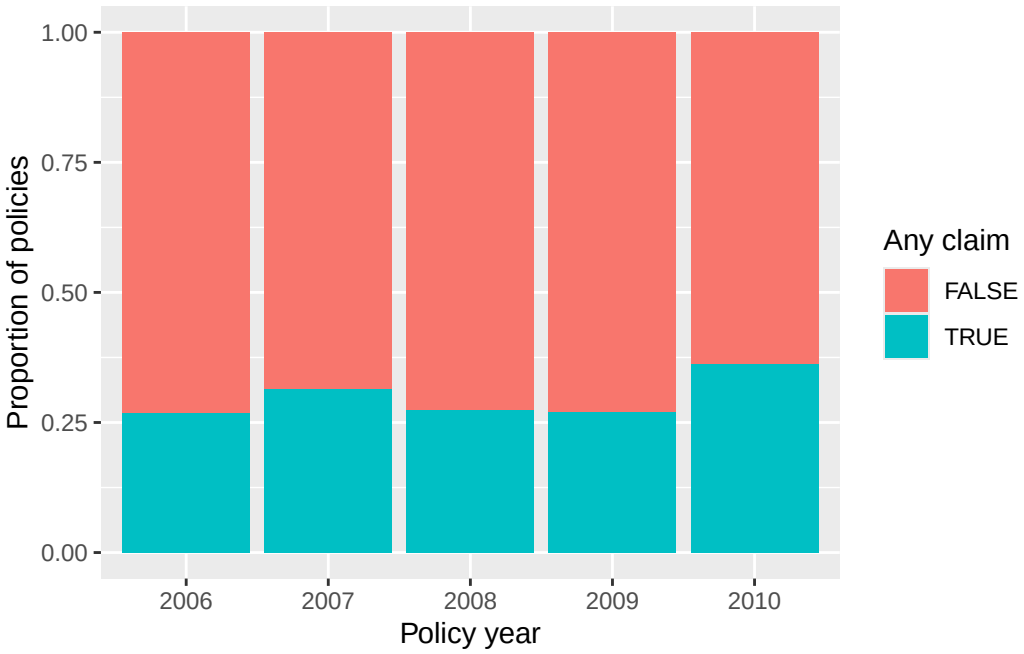
```
ggplot(claims, aes(x = factor(Year), y = Freq)) +
  geom_boxplot() +
  labs(
    x = "Policy year",
    y = "Number of claims"
  )
```



If the distribution is concentrated at zero, the box and median may look similar across years even when the proportions of policies with claims differ. In this situation, comparing the proportion of policies with at least one claim may be more useful.

We can create a binary indicator for whether a policy has any claims and compare the proportions across years.

```
claims %>%
  mutate(HasClaim = Freq > 0) %>%
  ggplot(aes(x = factor(Year), fill = HasClaim)) +
  geom_bar(position = "fill") +
  labs(
    x = "Policy year",
    y = "Proportion of policies",
    fill = "Any claim"
  )
```



The argument `position = "fill"` scales each bar to one, so the plot displays proportions rather than counts. This makes it easier to compare the proportion of policies with and without claims across years, even when the number of policies differs between years.

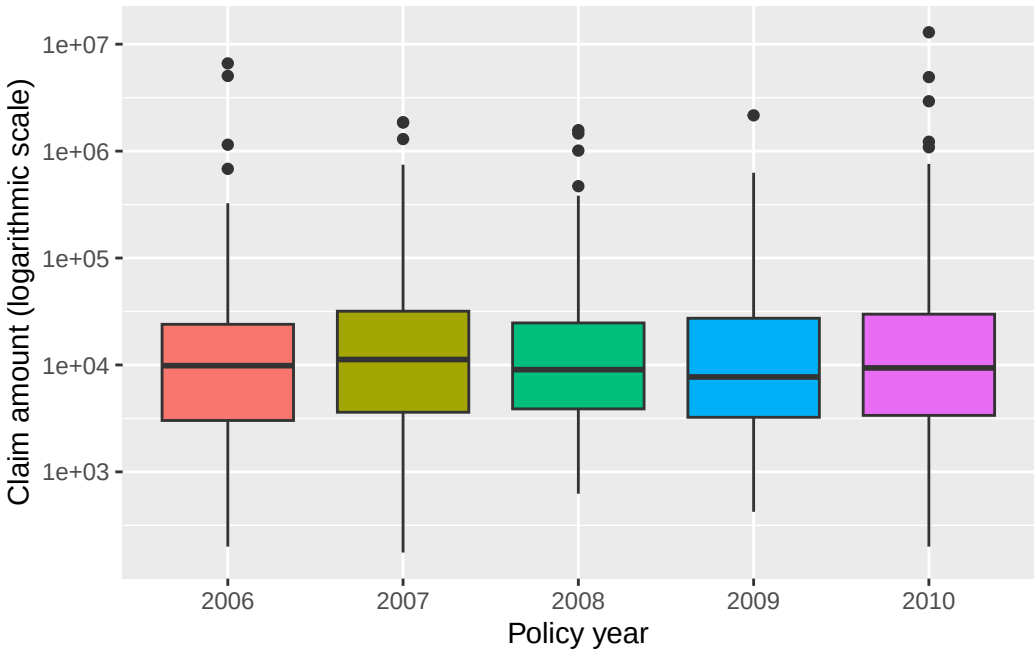
This comparison concerns the probability of observing at least one claim, not the average number of claims per policy. These are different measures of claim frequency and should not be interpreted interchangeably.

5.6.4 Comparing Groups with Colour

Colour can distinguish groups within a single plot. For example, we can compare claim amount distributions across policy years by mapping `Year` to `fill` in a boxplot.

Because claim amounts are often strongly right-skewed, a few large claims may compress the boxes on the original scale. We can use a logarithmic vertical axis to make differences among smaller and moderate positive claims easier to see.

```
ggplot(
  filter(claims, y > 0),
  aes(x = factor(Year), y = y, fill = factor(Year))
) +
  geom_boxplot(show.legend = FALSE) +
  scale_y_log10() +
  labs(
    x = "Policy year",
    y = "Claim amount (logarithmic scale)"
  )
```



The logarithmic scale makes it easier to compare the distributions of positive claim amounts across years. The observations themselves are unchanged, but differences are displayed proportionally rather than as equal absolute increments. Policies with zero or negative values of `y`, if any, are excluded from this plot because they cannot be displayed on a logarithmic scale.

In this example, colour distinguishes the years visually. Because the horizontal axis already identifies the groups, the legend is unnecessary and is suppressed with `show.legend = FALSE`.

When comparing many groups, however, using a different colour for every category can make a plot harder to read. Colour is most useful when it answers a specific question, such as whether claim amounts differ across fire-risk ratings or policy types.

5.6.5 Using Facets to Separate Groups

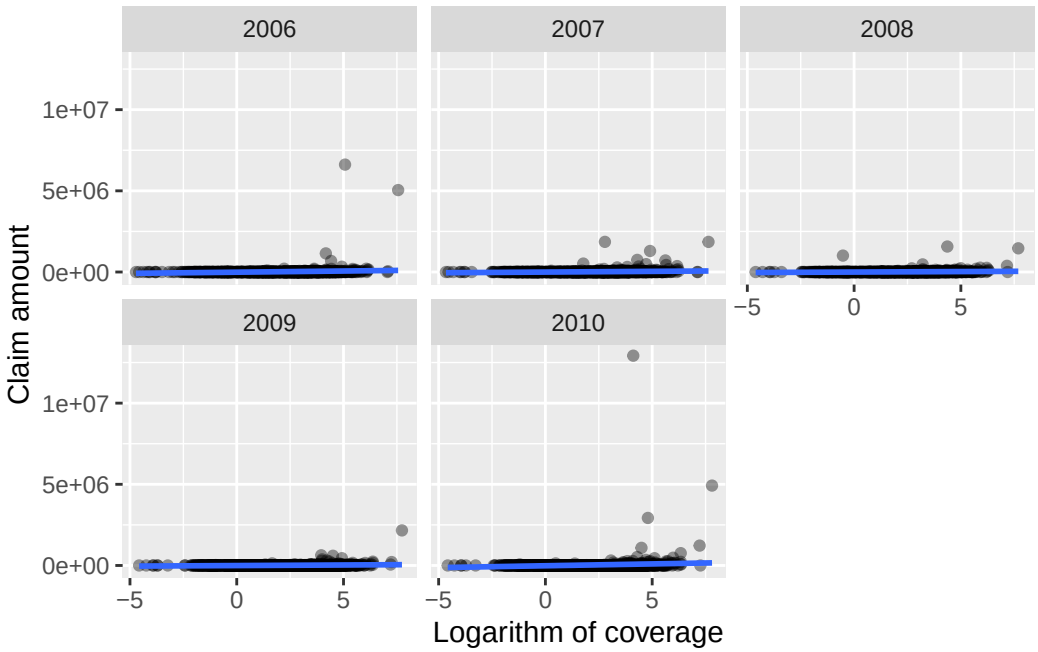
When a plot contains several groups, their observations may overlap or the display may become cluttered. Faceting creates separate panels for different subsets of the data while keeping the same basic plot structure.

The function `facet_wrap()` arranges panels for the levels of one grouping variable. In the `claims` dataset, `Year` can be used to compare the relationship between insurance coverage and claim amount across policy years.

We begin with the original claim amounts to see how the large claims affect the overall pattern.

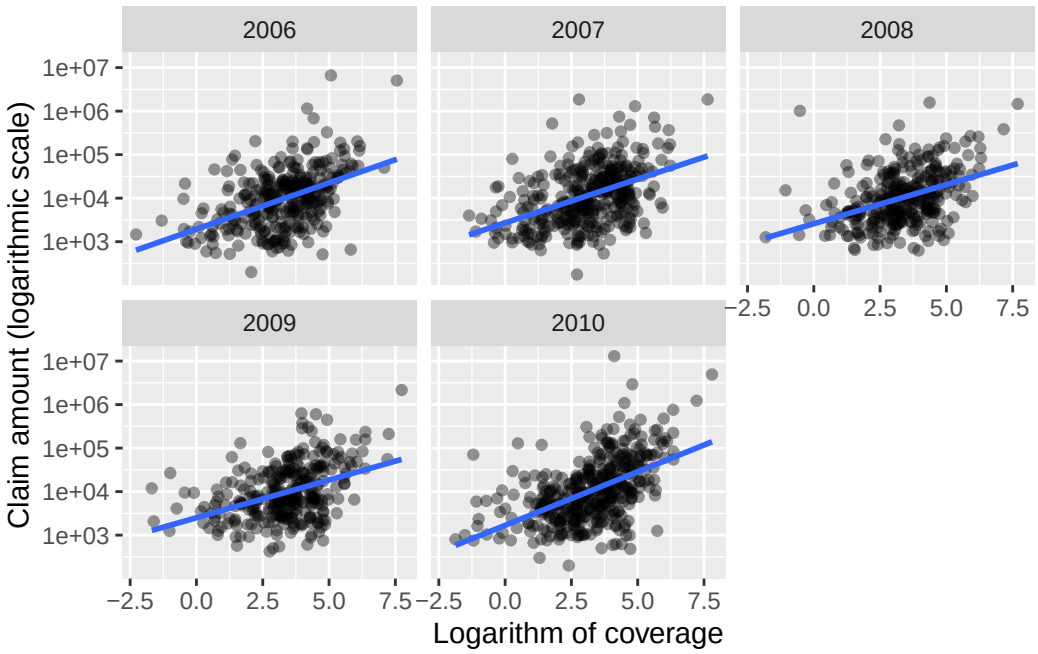
```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  facet_wrap(~ Year) +
  labs(
    x = "Logarithm of coverage",
```

```
    y = "Claim amount"
  )
```



Large claims may dominate the vertical scale, making patterns among smaller claims difficult to see. If the objective is to examine the relationship among positive claim amounts, we can improve the display using a logarithmic vertical axis.

```
ggplot(
  filter(claims, y > 0),
  aes(x = LnCoverage, y = y)
) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  facet_wrap(~ Year) +
  scale_y_log10() +
  labs(
    x = "Logarithm of coverage",
    y = "Claim amount (logarithmic scale)"
  )
```

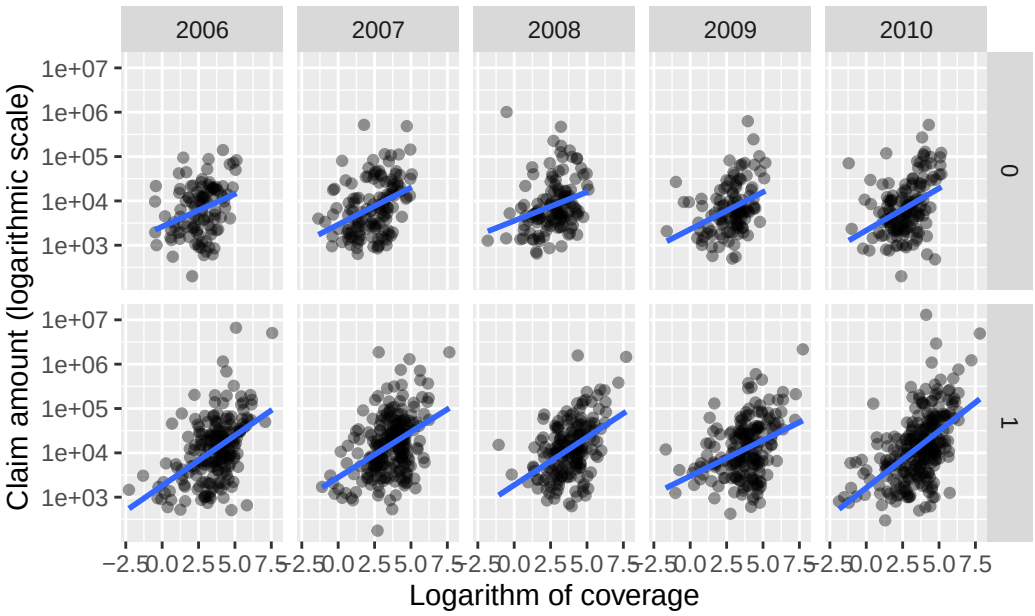


Each panel displays the relationship between coverage and positive claim amount for a particular year. The fitted lines allow us to compare the direction of the relationships across years without placing all observations in the same plotting area.

The panels use a common scale by default, making direct visual comparison easier. The logarithmic axis compresses large claim amounts, but the fitted lines are still estimated on the original response scale because only the display scale has changed. Thus, these lines are linear regression fits for `y`, not for `log(y)`. If the objective is to model the logarithm of claim amount, the response must be transformed explicitly.

The function `facet_grid()` arranges panels according to two grouping variables, typically using rows for one variable and columns for another. The `claims` dataset contains `Fire5`, which records the fire-risk rating, and `Year`. We can use these variables to examine whether the relationship between coverage and positive claim amount differs across years and fire-risk ratings.

```
ggplot(
  filter(claims, y > 0),
  aes(x = LnCoverage, y = y)
) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  facet_grid(Fire5 ~ Year) +
  scale_y_log10() +
  labs(
    x = "Logarithm of coverage",
    y = "Claim amount (logarithmic scale)",
    caption = "Fire5 is the fire-risk rating."
  )
```



This creates a grid with fire-risk ratings in rows and policy years in columns. It allows us to compare patterns across combinations of these characteristics. However, if there are many distinct ratings or years, the plot may contain too many small panels. In that case, using `facet_wrap(~ Year)` may be easier to interpret.

Faceting is particularly useful when a single plot becomes cluttered. It also makes it easier to identify whether a pattern is consistent across groups or appears only within particular subgroups. Nevertheless, differences observed in these plots are descriptive and do not establish that a policy characteristic causes differences in claim amounts.

5.6.6 Creating a Grouping Variable from Indicator Columns

A dataset may represent a categorical variable using several indicator columns, with each column indicating whether an observation belongs to a particular category. For example, the `claims` dataset contains six policy-type variables: `TypeCity`, `TypeCounty`, `TypeMisc`, `TypeSchool`, `TypeTown`, and `TypeVillage`.

If each policy belongs to exactly one entity type, we can combine these indicators into a single categorical variable called `EntityType`. The `case_when()` function from `dplyr` allows us to specify the category corresponding to each indicator.

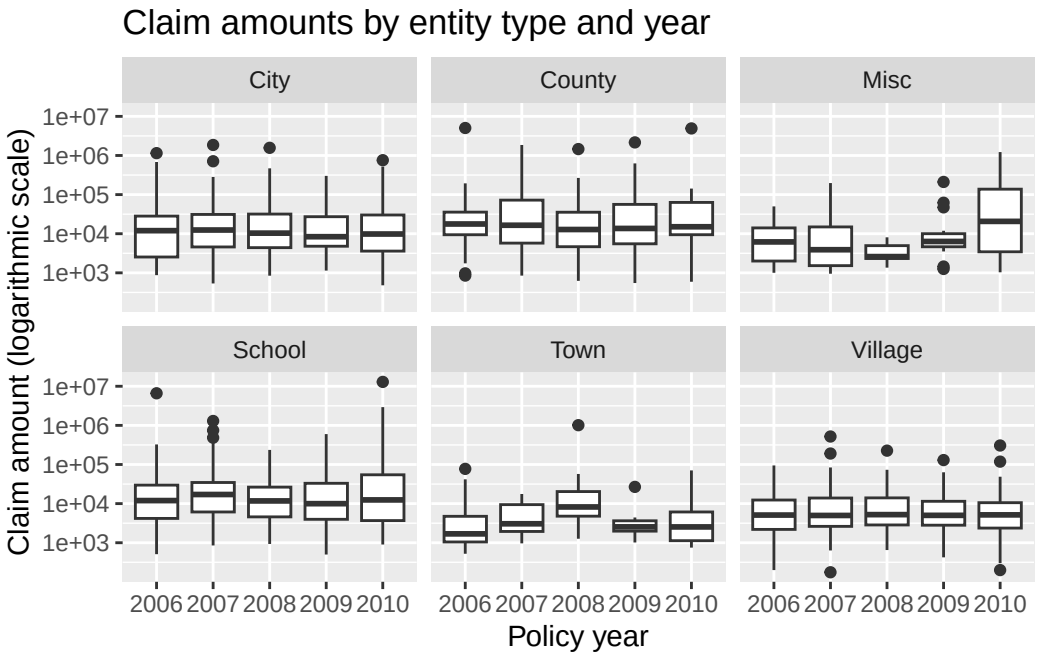
```
claims <- claims %>%
  mutate(
    EntityType = case_when(
      TypeCity == 1     ~ "City",
      TypeCounty == 1   ~ "County",
      TypeMisc == 1     ~ "Misc",
      TypeSchool == 1   ~ "School",
      TypeTown == 1     ~ "Town",
```

```
      TypeVillage == 1 ~ "Village",
      TRUE             ~ NA_character_
    )
  )
```

The new `EntityType` variable identifies the policy's entity type. Unlike `pivot_longer()`, this operation does not expand the dataset: each policy remains one row, and the number of observations is unchanged.

We can now compare claim amounts across entity types using `facet_wrap()`. Since claim amounts may be strongly right-skewed, we use a logarithmic vertical axis to make differences among positive claim amounts easier to examine.

```
ggplot(
  filter(claims, y > 0),
  aes(x = factor(Year), y = y)
) +
  geom_boxplot() +
  facet_wrap(~ EntityType) +
  scale_y_log10() +
  labs(
    x = "Policy year",
    y = "Claim amount (logarithmic scale)",
    title = "Claim amounts by entity type and year"
  )
```



Each panel represents an entity type, while the horizontal axis compares policy years. The common vertical scale supports comparisons across panels. Because the plot uses a logarithmic scale, policies with zero or negative claim amounts are excluded.

This example illustrates how preparing a suitable grouping variable can make a plot easier to construct and interpret. Before using `case_when()`, however, we should verify that the indicator columns are mutually exclusive and that each policy has exactly one active entity-type indicator. Otherwise, the resulting category may be missing or may not represent the policy correctly.

Tip

Check the new grouping variable
Use `count()` to examine the number of policies in each entity type.

```
count(claims, EntityType)
```

```
# A tibble: 6 x 2
  EntityType     n
  <chr>      <int>
1 City         793
2 County       328
3 Misc         609
4 School      1597
5 Town         971
6 Village     1341
```

Then compare the number of rows before and after creating `EntityType`.

```
nrow(claims)
```

```
[1] 5639
```

Does every policy have an assigned entity type? Does the total number of policies remain unchanged? If `EntityType` contains missing values, investigate whether some policies have no active type indicator.

5.6.7 Reshaping Wide Data into Long Format

Data are sometimes stored in wide format, where repeated measurements appear in separate columns. For example, a dataset may have one row per policy and a separate column for the claim amount recorded in each year. Although this format can be convenient for viewing data, long format is often more suitable for visualization and analysis with `ggplot2`.

We will use the `claims` dataset to demonstrate how to reshape selected variables from wide format to long format without averaging or discarding the original claim amounts.

5.6.7.1 Creating a wide-format dataset

First, select the policy identifier, year, and claim amount. We then use `pivot_wider()` to create a separate column for each year.

```
library(tidyverse)

claims_wide <- claims %>%
  select(PolicyNum, Year, y) %>%
  pivot_wider(
    names_from = Year,
    values_from = y,
    names_prefix = "Year_"
  )

claims_wide
```

```
# A tibble: 1,227 x 6
   PolicyNum Year_2006 Year_2007 Year_2008 Year_2009 Year_2010
       <dbl>     <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1    120002         0         0         0         0      6839.
 2    120003         0     15500      8775     37471.     9711.
 3    120004    13003.    32733.         0     42600.    10324.
 4    120005         0    144005.         0     27481.        0
 5    120008    48061.     2250.       713.     12015.     3470.
 6    120009         0         0         0      5474.        0
 7    120010        NA        NA        NA     20110.    35000
 8    120012    55273.   366403.    47562.   157402.    83935.
 9    120013    20169.    13102.    41266.         0     63530.
10    120015   194133.    26820.    52867.    36411.    38117.
# i 1,217 more rows
```

Each row represents a policy, while columns such as `Year_2010` and `Year_2011` contain the corresponding claim amounts. Some cells may contain `NA` because a policy has no recorded observation for a particular year.

5.6.7.2 Converting to long format

We can use `pivot_longer()` to combine the year-specific columns into a single year variable and a single claim-amount variable.

```
claims_long <- claims_wide %>%
  pivot_longer(
    cols = starts_with("Year_"),
    names_to = "Year",
    names_prefix = "Year_",
    values_to = "ClaimAmount"
  ) %>%
  mutate(Year = as.integer(Year))

claims_long
```

```
# A tibble: 6,135 x 3
   PolicyNum  Year ClaimAmount
       <dbl> <int>       <dbl>
 1    120002  2006          0
 2    120002  2007          0
 3    120002  2008          0
 4    120002  2009          0
 5    120002  2010       6839.
 6    120003  2006          0
 7    120003  2007      15500
 8    120003  2008       8775
 9    120003  2009      37471.
10    120003  2010       9711.
# i 6,125 more rows
```

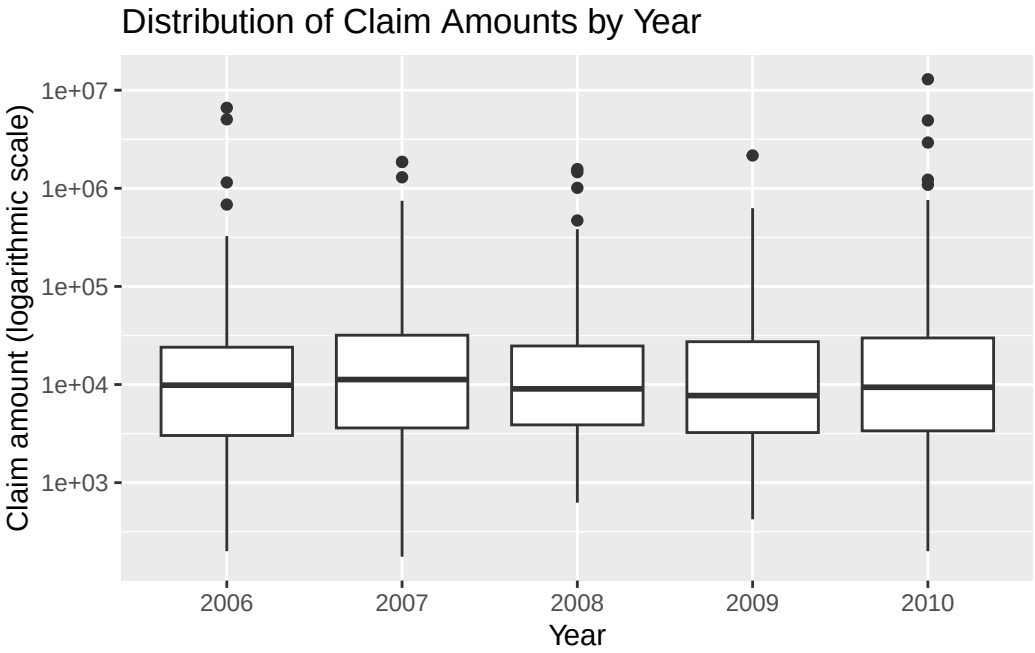
The resulting dataset contains three variables: `PolicyNum`, `Year`, and `ClaimAmount`. Each row represents a policy-year combination. The original claim amounts are retained, and missing values remain `NA`.

> **Note**
>
> **Why reshape data into long format?**
> When measurements for different years are stored in separate columns, comparing their distributions with `ggplot2` can require additional data manipulation or separate plotting commands. Reshaping the data into long format places the measurements in a single column and identifies their groups in another column. This structure makes it easier to compare distributions across years using a single plotting command, such as `geom_boxplot()`, with `Year` mapped to the x-axis.

5.6.7.3 Visualizing claim amounts by year

We can now compare the distributions of positive claim amounts across years using boxplots.

```
ggplot(
  filter(claims_long, ClaimAmount > 0),
  aes(x = factor(Year), y = ClaimAmount)
) +
  geom_boxplot() +
  scale_y_log10() +
  labs(
    x = "Year",
    y = "Claim amount (logarithmic scale)",
    title = "Distribution of Claim Amounts by Year"
  )
```



Claim amounts are often strongly right-skewed. A logarithmic display scale makes it easier to examine variation among smaller and larger positive claims. The filter excludes zero, negative, and missing claim amounts from this particular plot.

> **!** Important
>
> **Reshaping does not change the underlying measurements.**
> `pivot_wider()` and `pivot_longer()` rearrange the data but do not calculate averages or otherwise summarize claim amounts. Missing values remain missing unless they are handled explicitly.

> Tip
>
> **Handling missing values**
> Before replacing `NA` with zero, determine why the value is missing. A missing policy-year combination may indicate that no observation was recorded for that year; it does not automatically mean that the claim amount was zero.
> If a missing combination is known to represent zero claims, `pivot_wider()` can fill these combinations with zero using `values_fill = 0`. Otherwise, retain `NA` or handle missing values according to the purpose of the analysis. Also distinguish absent policy-year combinations from `NA` values already present in the original data.

5.6.8 Choosing a Suitable Plot for Group Comparisons

The following guidelines summarize the main choices.

Objective	Suitable plot
Compare counts across categories	Bar chart using `geom_bar()`
Compare proportions across categories	Proportional bar chart using `position = "fill"`
Compare the distributions of claim amounts across years or risk groups	Boxplot using `geom_boxplot()`
Compare relationships between coverage and claim amount across years	Scatterplot with colour mappings or facets
Compare patterns across two policy characteristics	`facet_grid()`

The most informative plot depends on the question being asked. For example, comparing the average number of claims per policy is different from comparing the proportion of policies with at least one claim. Similarly, comparing typical claim amounts is different from comparing overall claim costs, which may be strongly influenced by a few large losses.

In actuarial analysis, plots should therefore be selected to match the measure of interest and interpreted in the context of the underlying insurance process. These comparisons provide useful exploratory evidence for deciding which variables and group differences may warrant further investigation through statistical models.

5.7 Scales, Labels, and Presentation

A plot should communicate its main message clearly. Appropriate labels, scales, colours, and themes help readers understand what is being displayed without distracting them from the data. In actuarial work, these choices are particularly important when comparing claim experience across groups or displaying variables with highly skewed distributions.

5.7.1 Labels and Titles

Axis labels should describe the variables being displayed, including their units where appropriate. A descriptive title can communicate the purpose of the plot, while a subtitle can provide additional context.

The `labs()` function sets the plot title, subtitle, axis labels, legend title, and caption.

```
library(ggplot2)
library(readr)


ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  labs(
    title = "Insurance Coverage and Claim Amount",
    subtitle = "Observed claims in the insurance portfolio",
```

```
    x = "Log Insurance Coverage",
    y = "Claim Amount",
    caption = "Source: ClaimsExperienceData.csv"
  )
```

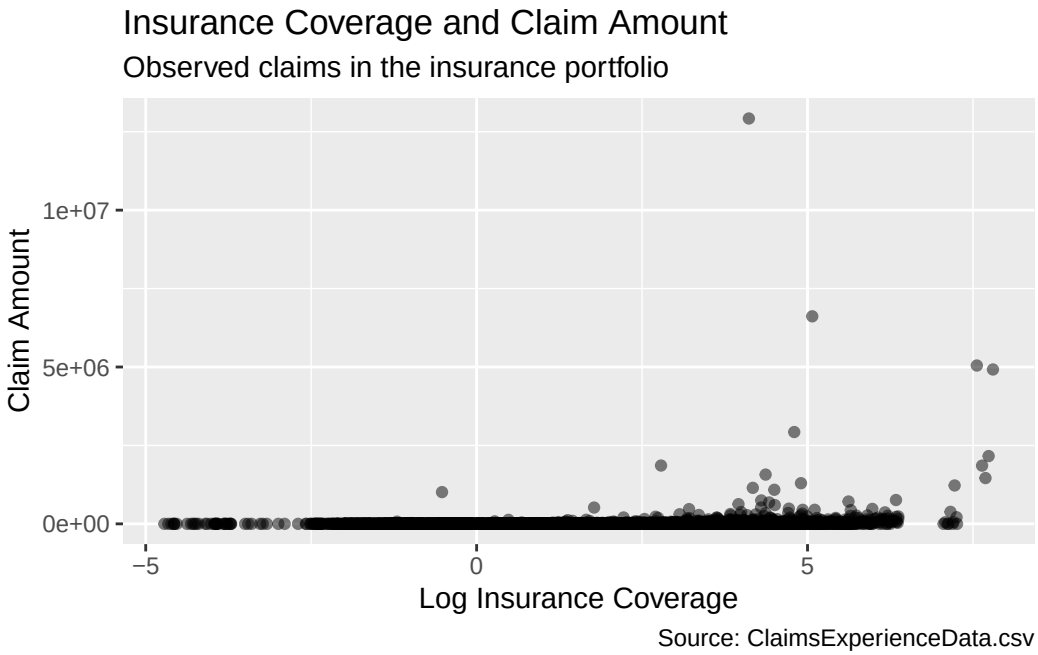


Figure 5.1: Relationship between insurance coverage and claim amount.

The axis labels should reflect the variables actually plotted. Here, `LnCoverage` represents the logarithm of coverage, so labelling it simply as "Insurance Coverage" would be potentially misleading. Similarly, if `y` is measured in monetary units, the vertical axis should identify the currency when this information is available.

Titles and captions should add useful context rather than repeat information already obvious from the plot.

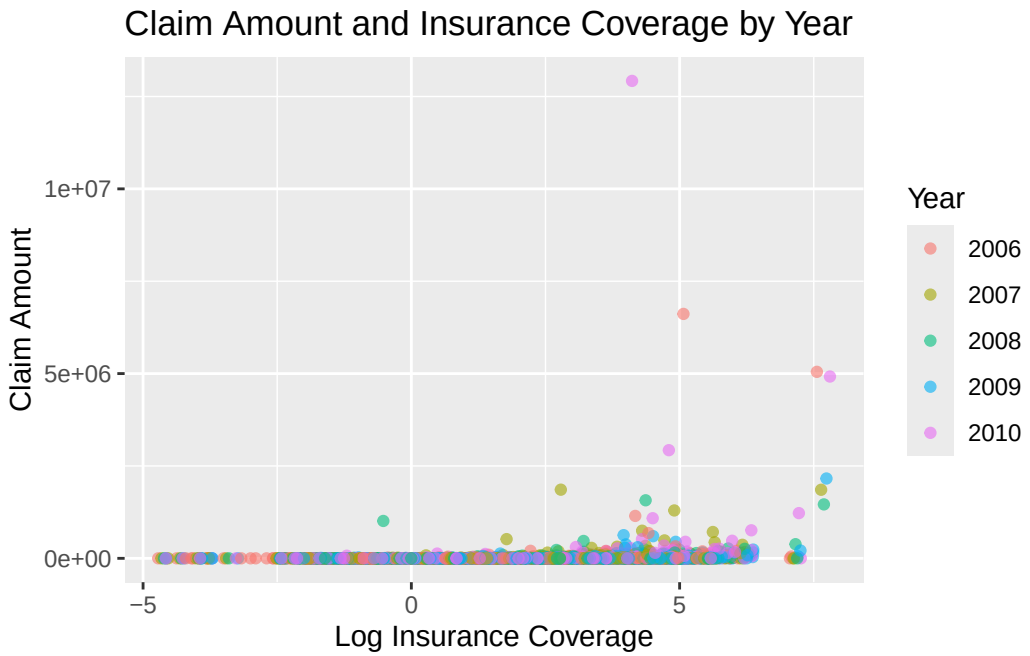
5.7.2 Legends and Aesthetic Scales

When an aesthetic is mapped to a variable, ggplot2 usually creates a legend automatically. The legend explains how visual properties such as colour, fill, shape, or size represent values in the data.

For example, we can use colour to distinguish observations by year.

```
ggplot(claims, aes(x = LnCoverage, y = y, colour = factor(Year))) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Claim Amount and Insurance Coverage by Year",
    x = "Log Insurance Coverage",
```

```
    y = "Claim Amount",
    colour = "Year"
  )
```



Converting `Year` to a factor treats it as a categorical variable, giving each year a separate colour. This is appropriate when the purpose is to compare years as distinct groups. If the objective is to emphasize a continuous progression over time, a continuous colour scale may be more suitable.

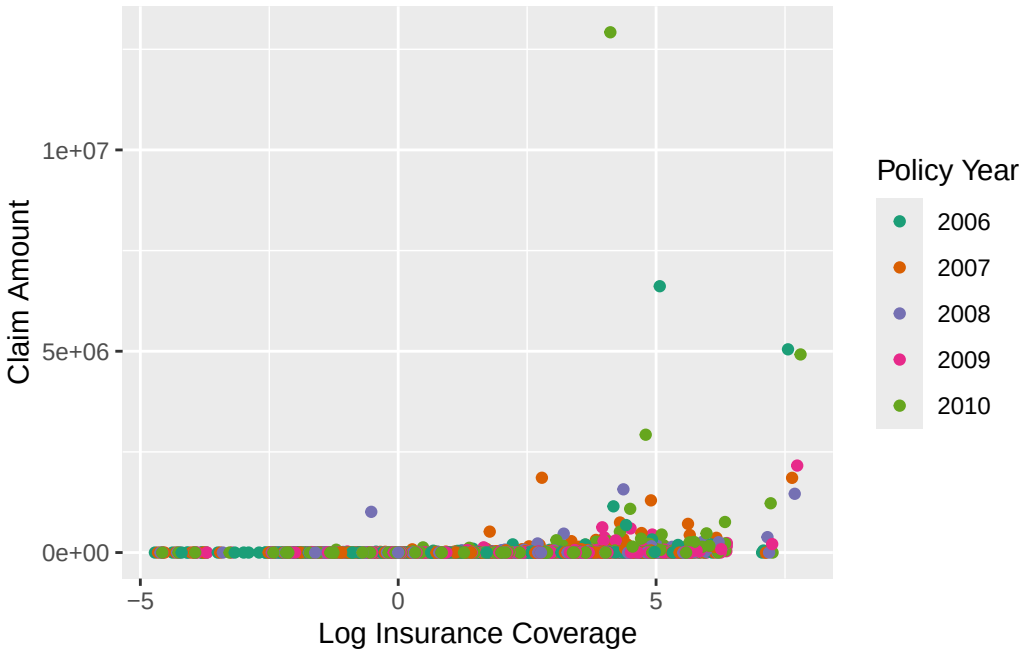
The `labs()` function can also rename the legend title. When a legend is unnecessary, it can be removed with `theme(legend.position = "none")`, provided that the meaning of the visual encoding remains clear.

5.7.2.1 Discrete and Continuous Scales

Scales determine how data values are translated into visual properties and how those properties are labelled. Discrete scales are used for categorical values, whereas continuous scales are used for numerical values.

For example, the following plot uses a discrete colour scale to distinguish policy years.

```
ggplot(claims, aes(x = LnCoverage, y = y, colour = factor(Year))) +
  geom_point() +
  scale_colour_brewer(palette = "Dark2") +
  labs(
    x = "Log Insurance Coverage",
    y = "Claim Amount",
    colour = "Policy Year"
  )
```

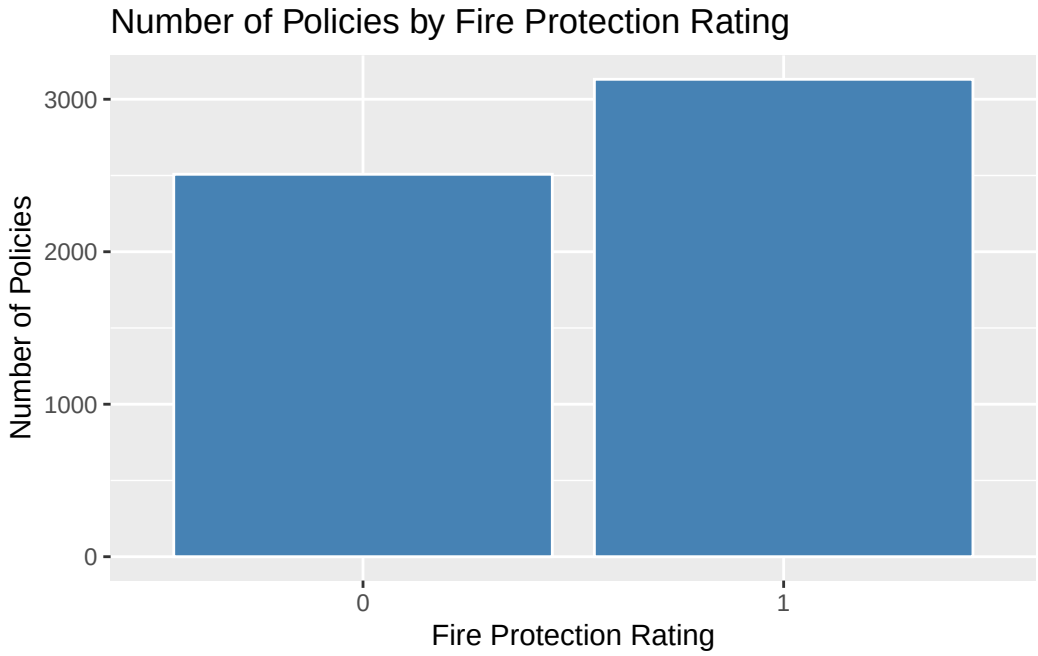


For a categorical variable, `scale_colour_brewer()` provides a palette designed for distinguishing groups. Other options include `scale_colour_manual()`, which allows specific colours to be assigned to categories.

> **Note**
>
> **Choosing a colour scale**
> Use `scale_colour_brewer(palette = "Dark2")` to customize the colours used to distinguish categories, such as different policy years. Choose a colour palette that makes the groups easy to distinguish.

The distinction between `colour` and `fill` is also useful. The `colour` aesthetic controls outlines and lines, as well as the colour of points, whereas `fill` controls the interior of filled shapes and areas. For example, `geom_bar()` uses `fill` to colour the bars and `colour` to control their outlines.

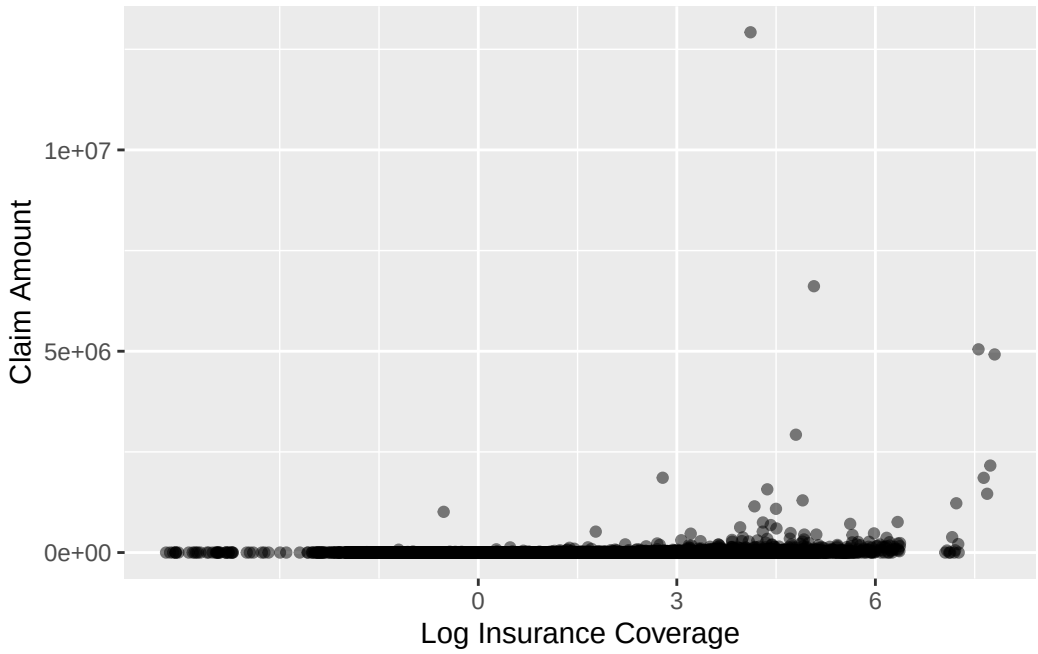
```
ggplot(claims, aes(x = factor(Fire5))) +
  geom_bar(
    fill = "steelblue",
    colour = "white"
  ) +
  labs(
    x = "Fire Protection Rating",
    y = "Number of Policies",
    title = "Number of Policies by Fire Protection Rating"
  )
```



Here, `fill = "steelblue"` and `colour = "white"` are fixed settings, not mappings to variables. All bars have the same fill and outline colours, so no legend is produced.

Continuous scales can be adjusted using functions such as `scale_x_continuous()` and `scale_y_continuous()`. For example, breaks can be specified to make the axis easier to read.

```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  scale_x_continuous(
    breaks = seq(0, 15, by = 3)
  ) +
  labs(
    x = "Log Insurance Coverage",
    y = "Claim Amount"
  )
```



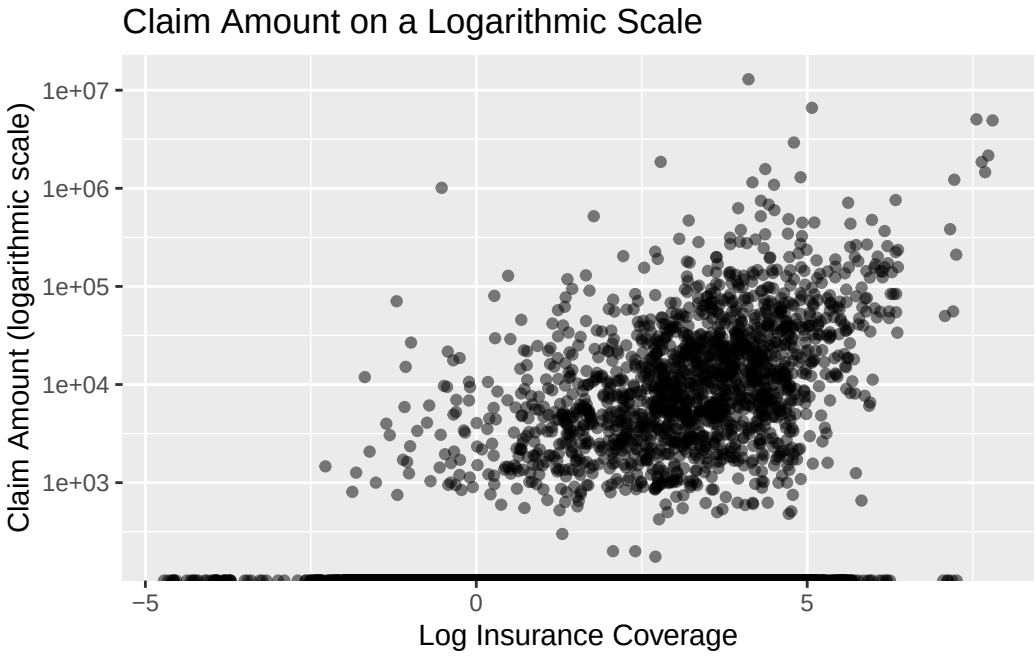
The chosen breaks should suit the observed range of the data. Explicitly specifying breaks is useful when consistent axis markings are required across several plots.

5.7.3 Logarithmic Scales

Insurance variables such as claim amounts and coverage often have right-skewed distributions. A small number of very large observations can make the majority of observations difficult to distinguish on the original scale. A logarithmic scale can help reveal patterns across a wide range of values.

For example, we can compare claim amounts using the original and logarithmic vertical scales.

```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  scale_y_log10() +
  labs(
    title = "Claim Amount on a Logarithmic Scale",
    x = "Log Insurance Coverage",
    y = "Claim Amount (logarithmic scale)"
  )
```



The function `scale_y_log10()` transforms the vertical axis to a base-10 logarithmic scale. Equal distances on this axis represent equal multiplicative changes in claim amount, rather than equal additive changes.

A logarithmic scale can make variation among smaller and medium-sized claims easier to see when some claims are much larger than others. However, it changes the visual interpretation of distances and does not remove extreme observations.

Important: A logarithmic scale requires positive values. Zero and negative claim amounts cannot be represented on a standard logarithmic scale. If zero claims are present, they must be handled according to the analytical objective rather than silently removed. For example, a plot of positive claim amounts may be appropriate when studying claim severity, but it does not describe the distribution of claim amounts including zero observations.

A logarithmic scale also differs from transforming the variable itself. With `scale_y_log10()`, the axis is transformed for display while the original variable remains in the data. By contrast, using `aes(y = log10(y))` plots the transformed values directly, so the axis labels and interpretation change accordingly.

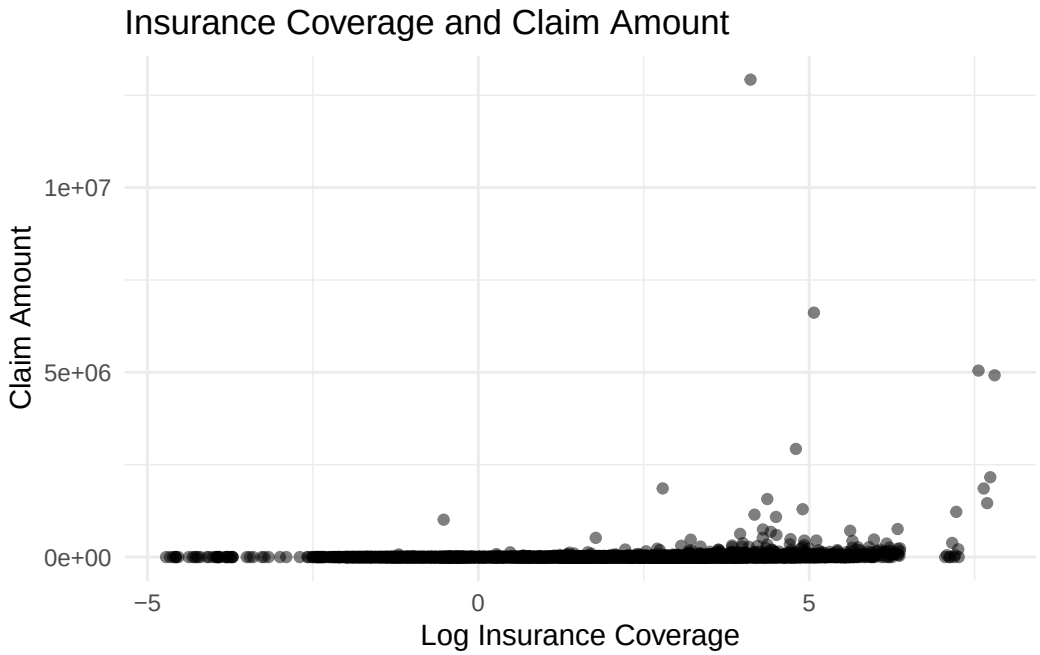
5.7.4 Themes and Presentation

Themes control the non-data elements of a plot, including background, grid lines, text, and legend placement. They help maintain a consistent appearance across figures.

ggplot2 provides several built-in themes, including `theme_minimal()`, `theme_classic()`, and `theme_bw()`.

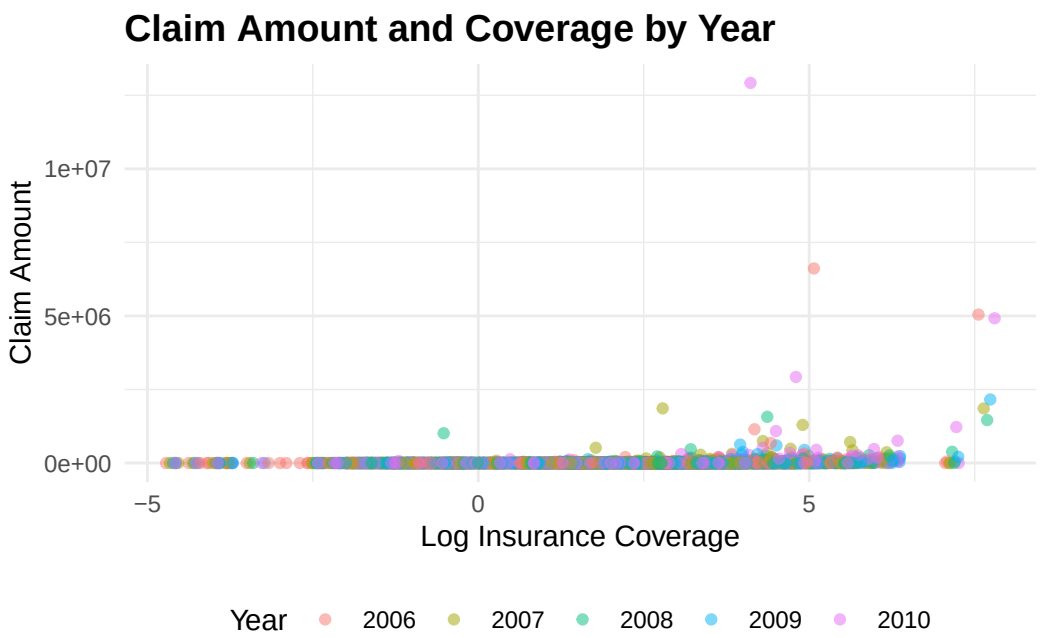
```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  labs(
```

```
    title = "Insurance Coverage and Claim Amount",
    x = "Log Insurance Coverage",
    y = "Claim Amount"
  ) +
  theme_minimal()
```



A theme can be modified further using `theme()`. For example, we can increase the title size and position the legend below the plot.

```
ggplot(claims, aes(x = LnCoverage, y = y, colour = factor(Year))) +
  geom_point(alpha = 0.5) +
  labs(
    title = "Claim Amount and Coverage by Year",
    x = "Log Insurance Coverage",
    y = "Claim Amount",
    colour = "Year"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "bottom"
  )
```



The `theme()` function changes presentation without altering the underlying data or statistical calculations. The same theme can be applied to multiple plots to maintain consistency throughout a report.

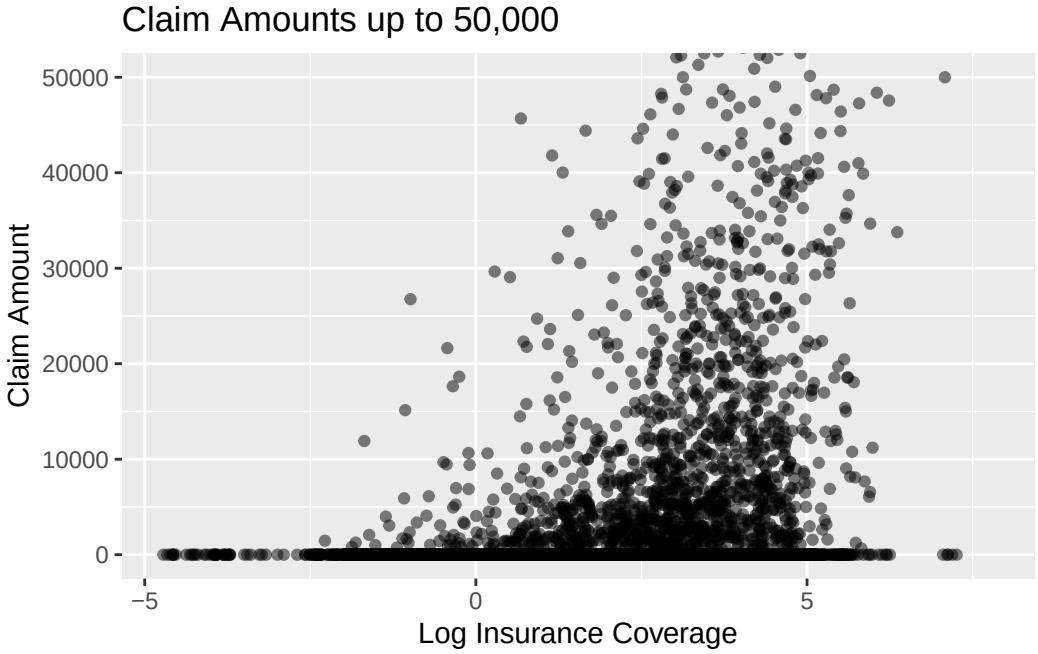
Good presentation should support interpretation. Excessive grid lines, unnecessary colours, oversized titles, and decorative effects can compete with the data. Use visual emphasis only when it helps readers identify important features.

5.7.5 Coordinate Transformations

Coordinate systems control how a plot is displayed. The `coord_cartesian()` function allows us to focus on a particular region of a plot without removing observations from the underlying data used in statistical layers.

For example, suppose we want to inspect the relationship among claims below a particular amount.

```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  coord_cartesian(ylim = c(0, 50000)) +
  labs(
    x = "Log Insurance Coverage",
    y = "Claim Amount",
    title = "Claim Amounts up to 50,000"
  )
```



This restricts the displayed vertical range but does not remove observations above 50,000 from the data used to construct the plot. Such observations are outside the visible region.

This differs from filtering the data before plotting. Filtering removes observations from subsequent calculations, whereas `coord_cartesian()` changes the visible region. This distinction matters when a plot includes statistical summaries, such as a smoothed curve or a fitted trend.

Coordinate transformations should be used when they improve the comparison or interpretation of the data. They should not be used to conceal unusual observations or to create a misleading impression of the results.

5.7.6 Principles for Effective Actuarial Visualizations

When preparing a plot, consider the following principles:

- **Label clearly.** Identify the variables, units, groups, and time periods being displayed.
- **Choose appropriate scales.** Use discrete scales for categories and continuous scales for numerical variables. Consider logarithmic scales when the data span a wide range of positive values.
- **Use colour purposefully.** Distinguish groups only when doing so helps answer the analytical question.
- **Maintain consistency.** Use comparable scales, labels, and themes when figures are intended to be compared.
- **Preserve the data's meaning.** Do not remove observations or change axis limits without understanding the effect on interpretation.

These principles help ensure that a plot is not merely attractive but also statistically informative. In the next section, we bring the techniques together in an actuarial exploratory analysis, using visualizations to formulate questions, investigate claim experience, and identify relationships that may warrant further study through regression modelling.

5.8 Actuarial Exploratory Visualization

In this section, we bring together the visualization techniques introduced in this chapter to explore an insurance portfolio. The aim is to use plots to investigate claim experience, identify patterns, and formulate questions for further analysis.

We will use the `claims` dataset and focus on three questions:

1. How is claim frequency distributed across policies?
2. What does the distribution of claim amounts tell us about claim severity?
3. Is claim amount associated with insurance coverage?

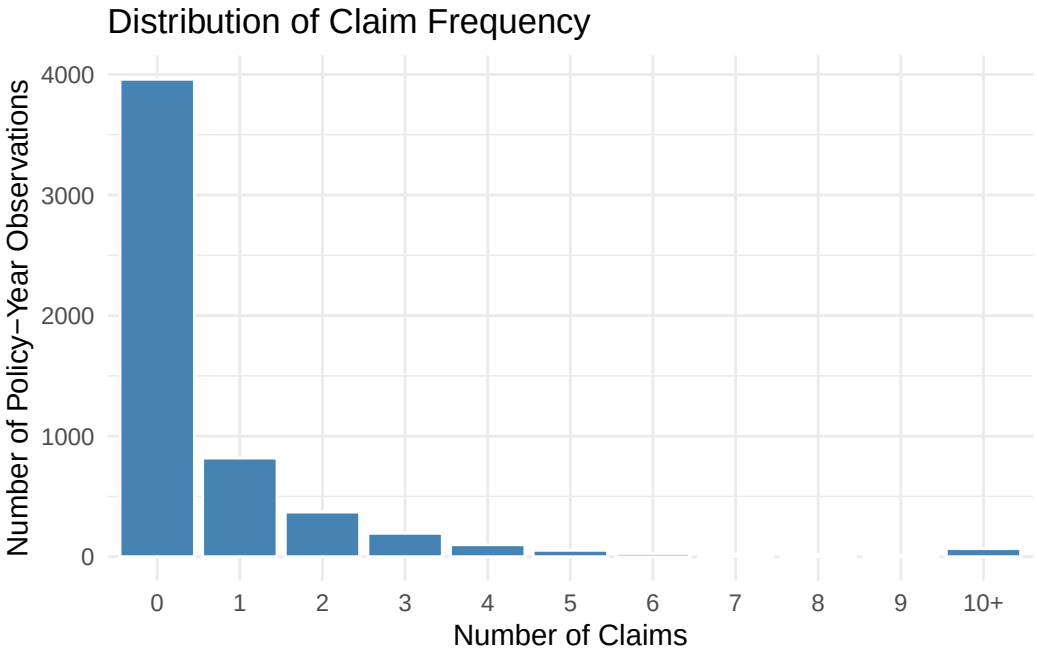
The purpose is not to establish causal relationships or build a predictive model at this stage. Instead, exploratory visualization helps us understand the data and identify relationships worth investigating through regression modelling.

5.8.1 Exploring Claim Frequency Across Years

Claim frequency describes the number of claims associated with a policy during a specified period. We begin by examining `Freq`, which records the number of claims for each policy-year observation.

A bar chart shows the distribution of claim frequency across the portfolio. Since most policy-year observations may have no claims and relatively few may have many claims, we group frequencies of 10 or more into a single category.

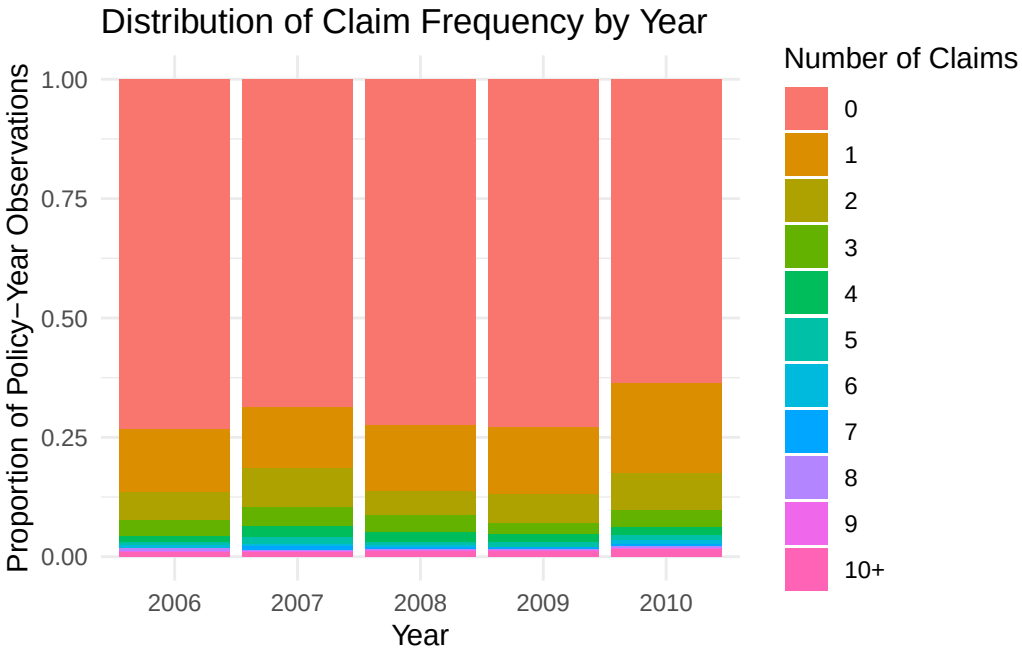
```
library(dplyr)

claims %>%
  mutate(
    FreqGroup = if_else(Freq >= 10, "10+", as.character(Freq)),
    FreqGroup = factor(
      FreqGroup,
      levels = c(as.character(0:9), "10+")
    )
  ) %>%
  ggplot(aes(x = FreqGroup)) +
  geom_bar(fill = "steelblue", colour = "white") +
  labs(
    title = "Distribution of Claim Frequency",
    x = "Number of Claims",
    y = "Number of Policy-Year Observations"
  ) +
  theme_minimal()
```



The 10+ category combines all policy-year observations with at least 10 claims. This improves readability while retaining the distinction between common claim counts and the upper tail of the distribution.

We can then investigate whether the distribution of claim frequency differs across years. Because the number of observations may vary by year, comparing counts alone may be misleading. A proportional bar chart allows us to compare the relative distribution of claim frequency in each year.

```
claims %>%
  mutate(
    FreqGroup = if_else(Freq >= 10, "10+", as.character(Freq)),
    FreqGroup = factor(
      FreqGroup,
      levels = c(as.character(0:9), "10+")
    )
  ) %>%
  ggplot(aes(x = factor(Year), fill = FreqGroup)) +
  geom_bar(position = "fill") +
  labs(
    title = "Distribution of Claim Frequency by Year",
    x = "Year",
    y = "Proportion of Policy-Year Observations",
    fill = "Number of Claims"
  ) +
  theme_minimal()
```

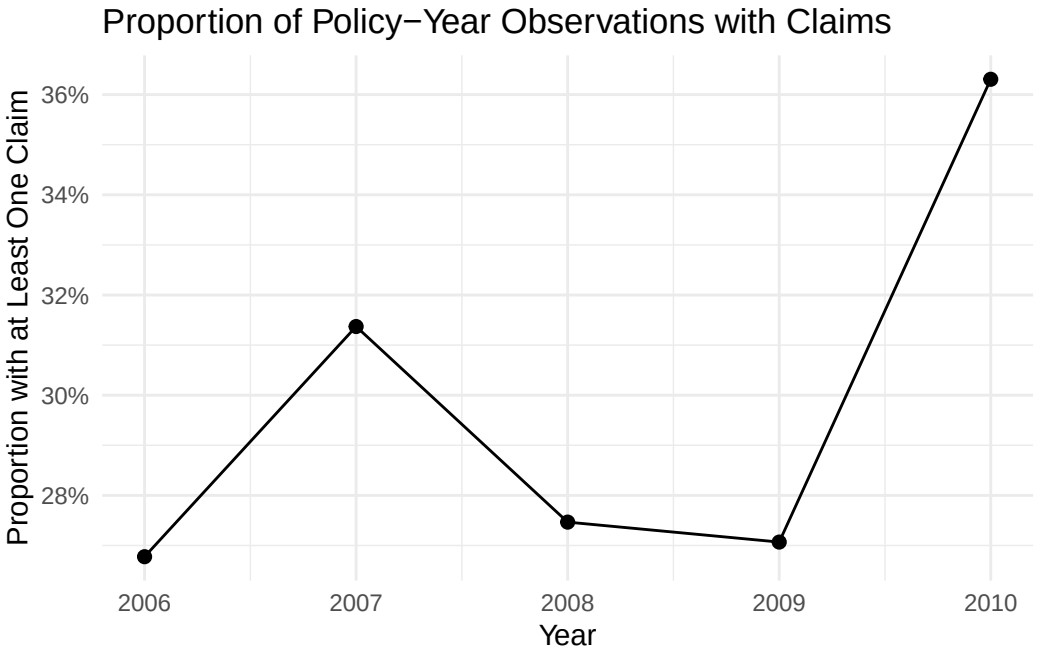


Each bar represents one year and sums to 1. The coloured segments show the proportion of policy-year observations with each claim frequency. This makes it easier to compare the distributions across years even when the total number of observations differs.

An important feature to examine is the proportion of observations with zero claims compared with those having one or more claims. We can also investigate whether the relative frequency of higher claim counts changes over time. Any apparent differences should be interpreted in the context of the portfolio and the number of observations available in each year.

For a more focused comparison, we can examine the proportion of policy-year observations with at least one claim.

```
claims %>%
  group_by(Year) %>%
  summarise(
    ProportionWithClaims = mean(Freq > 0, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  ggplot(aes(x = Year, y = ProportionWithClaims)) +
  geom_line() +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::label_percent()) +
  labs(
    title = "Proportion of Policy-Year Observations with Claims",
    x = "Year",
    y = "Proportion with at Least One Claim"
  ) +
  theme_minimal()
```



This plot focuses on claim occurrence rather than the full distribution of claim counts. It helps identify whether the proportion of policy-year observations with at least one claim appears to increase, decrease, or remain relatively stable over time.

These plots describe different aspects of claim frequency: the first shows the overall distribution, the second compares the full distribution across years, and the third focuses on whether any claim occurred. Together, they provide a useful starting point for investigating changes in insurance claim experience.

Note: These comparisons assume that each row represents a policy-year observation and that `Freq` is the claim count for that observation. Missing claim counts should be investigated rather than automatically treated as zero. If the same policy appears in multiple years, the observations are repeated policy-year records rather than independent policyholders.

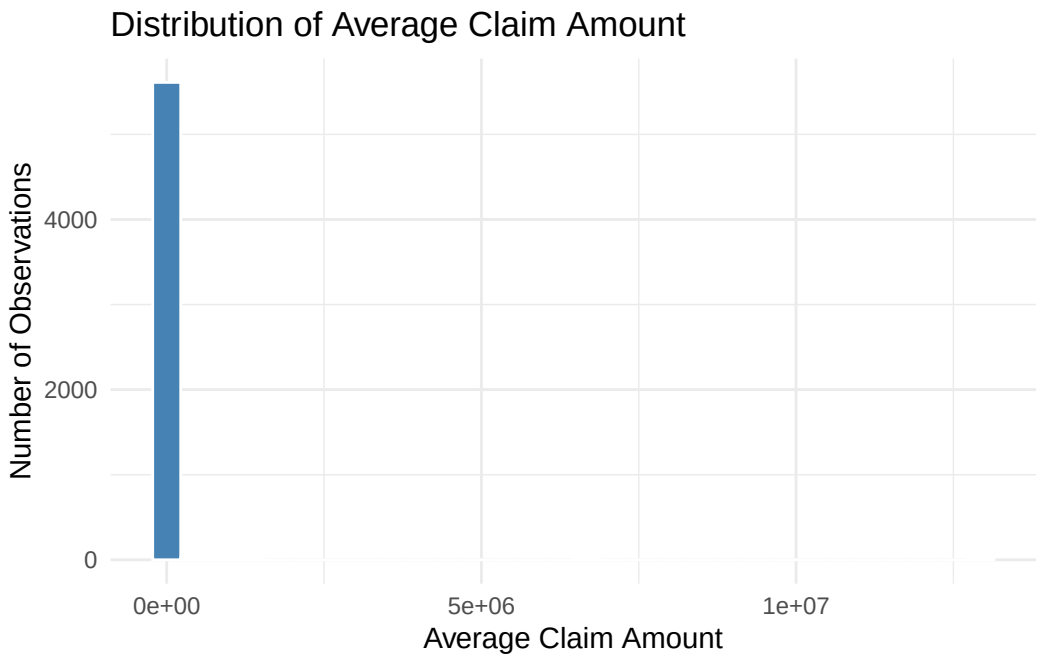
5.8.2 Exploring Claim Severity

Claim severity concerns the amount paid or incurred per claim. Unlike claim frequency, which is a non-negative count, claim amounts are quantitative monetary values and often exhibit substantial right-skewness.

We can begin by examining the distribution of `yAvg`, the average claim amount recorded in the dataset.

```
ggplot(claims, aes(x = yAvg)) +
  geom_histogram(
    bins = 30,
    fill = "steelblue",
    colour = "white"
  ) +
  labs(
```

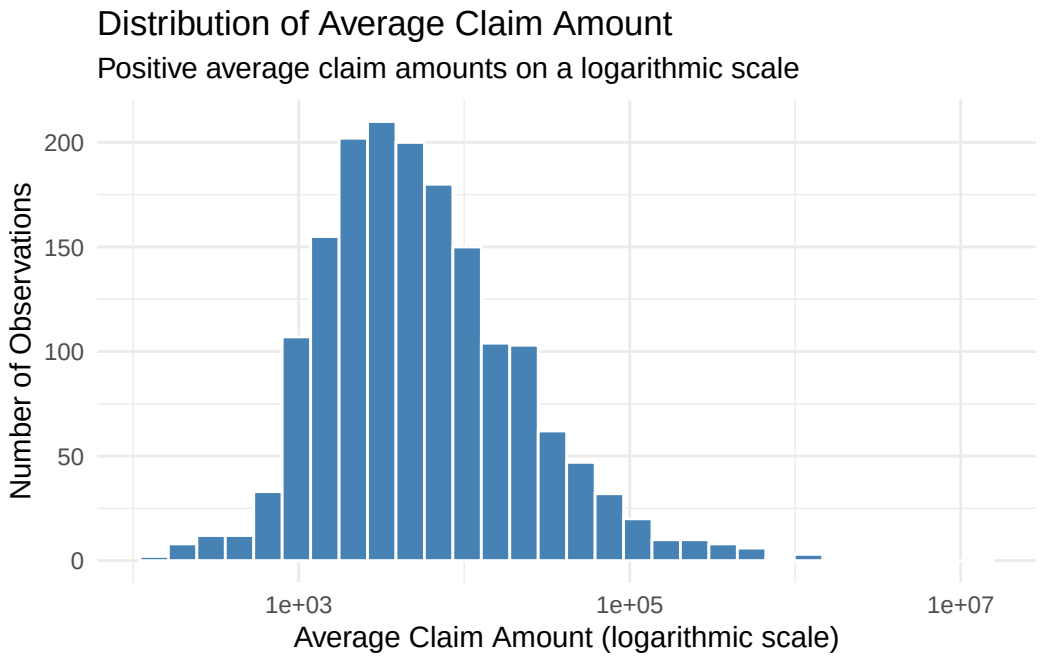
```
    title = "Distribution of Average Claim Amount",
    x = "Average Claim Amount",
    y = "Number of Observations"
  ) +
  theme_minimal()
```



A right-skewed distribution has most observations concentrated at relatively low values and a long tail extending towards larger amounts. A small number of large claims may stretch the horizontal axis, making the variation among typical claims difficult to see.

A logarithmic scale can help reveal this variation. Because a logarithmic scale requires positive values, we restrict the following plot to observations with positive average claim amounts.

```
claims %>%
  filter(!is.na(yAvg), yAvg > 0) %>%
  ggplot(aes(x = yAvg)) +
  geom_histogram(
    bins = 30,
    fill = "steelblue",
    colour = "white"
  ) +
  scale_x_log10() +
  labs(
    title = "Distribution of Average Claim Amount",
    subtitle = "Positive average claim amounts on a logarithmic scale",
    x = "Average Claim Amount (logarithmic scale)",
    y = "Number of Observations"
  ) +
  theme_minimal()
```

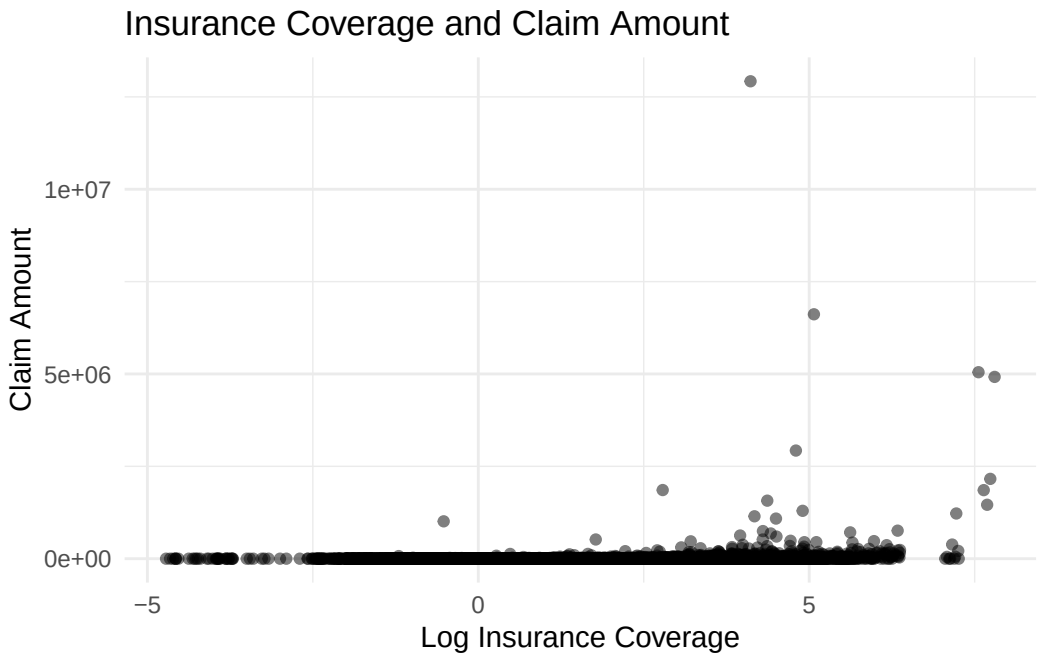


The logarithmic scale makes multiplicative differences easier to compare. However, this plot excludes zero and missing values of `yAvg`, so it should not be interpreted as the complete distribution of average claim amounts. Large claims should not automatically be treated as data errors; they may represent genuine insurance losses that deserve further investigation.

5.8.3 Exploring the Relationship Between Coverage and Claim Amount

An insurer may also wish to understand whether claim amounts vary with insurance coverage. We can explore this relationship using `LnCoverage` and `y`.

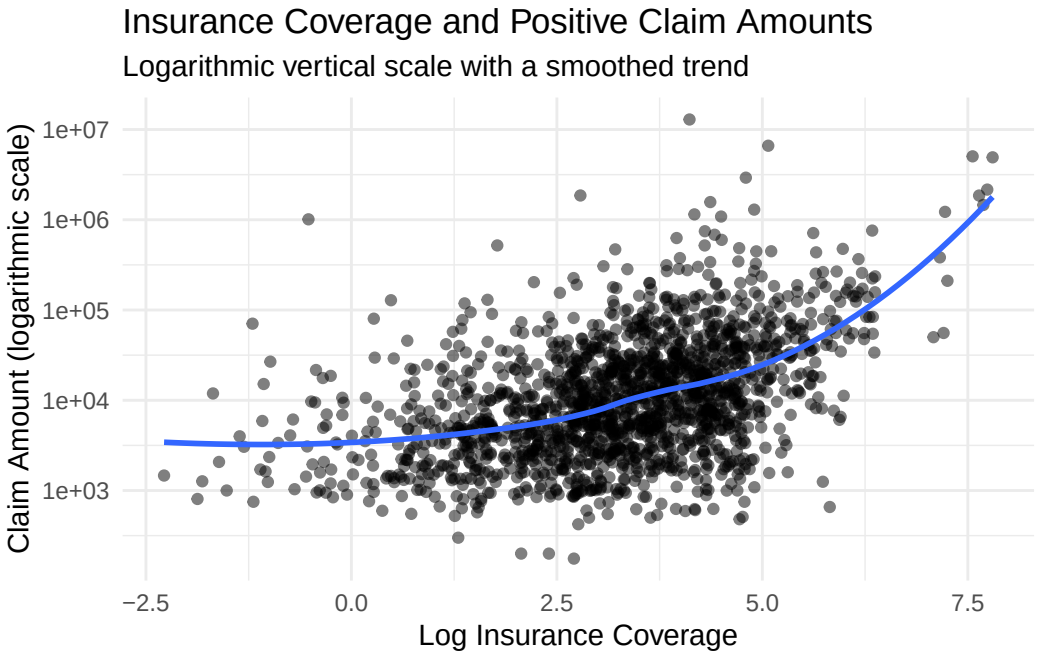
```
ggplot(claims, aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  labs(
    title = "Insurance Coverage and Claim Amount",
    x = "Log Insurance Coverage",
    y = "Claim Amount"
  ) +
  theme_minimal()
```



Each point represents an observation. Transparency (`alpha = 0.5`) helps reveal overlapping points when many observations have similar values.

Claim amounts may be highly right-skewed, so a few large claims can dominate the vertical scale. If the claim amounts are positive for the observations of interest, a logarithmic vertical scale can help us examine the relationship more closely.

```
claims %>%
  filter(!is.na(y), y > 0, !is.na(LnCoverage)) %>%
  ggplot(aes(x = LnCoverage, y = y)) +
  geom_point(alpha = 0.5) +
  scale_y_log10() +
  geom_smooth(method = "loess", se = FALSE) +
  labs(
    title = "Insurance Coverage and Positive Claim Amounts",
    subtitle = "Logarithmic vertical scale with a smoothed trend",
    x = "Log Insurance Coverage",
    y = "Claim Amount (logarithmic scale)"
  ) +
  theme_minimal()
```



The smooth curve summarizes the general pattern in the observed data. It can help identify whether claim amounts tend to increase, decrease, or show a nonlinear pattern across coverage levels. The curve is exploratory rather than a fitted actuarial model.

The plot includes only positive claim amounts with observed coverage. It therefore does not describe the probability of making a claim. Moreover, an apparent relationship between coverage and claim amount does not establish that coverage causes larger claims; other policy characteristics may also be associated with the observed pattern.

5.8.4 From Visualization to Modelling

The plots provide an initial understanding of the portfolio and suggest several questions for further investigation:

- What factors are associated with whether a policy has any claims?
- Which policy characteristics are associated with claim frequency?
- How does claim severity vary with coverage and other policy characteristics?
- Does the relationship between coverage and claim amount remain after accounting for other variables?

These questions can motivate regression models, but the appropriate response variable and modelling approach depend on the question. Claim frequency is a count outcome, whereas claim severity concerns claim amounts. A model for claim occurrence, a count model, and a severity model address different aspects of insurance experience.

Exploratory visualization is therefore an important first step in actuarial modelling. It helps formulate meaningful questions, identify unusual observations, and guide the selection of variables and modelling approaches. The next step is to use statistical models to quantify relationships and assess their predictive performance.

6 Chapter 6: Linear Regression with tidymodels

Learning Outcomes

By the end of this chapter, students should be able to:

1. Explain the importance of assessing regression model performance and distinguish between training and test data.
2. Generate predictions on test data and calculate prediction errors.
3. Calculate and interpret RMSE, MAE, R^2, and MAPE using the `yardstick` package.
4. Use `metric_set()` to evaluate regression models using multiple performance metrics.
5. Compare the predictive performance of regression models using appropriate metrics and explain the importance of evaluating models on unseen data.

6.1 Introduction

In the previous chapters, we used data science tools to **explore and understand insurance data**. We learned how to inspect a dataset, transform and summarize variables using `dplyr`, and visualize distributions and relationships using `ggplot2`. These steps are important because statistical modelling should begin with an understanding of the data rather than with fitting a model immediately.

The data science workflow can therefore be viewed as:

Inspect data → Transform and summarize → Visualize → Build a model → Evaluate and interpret.

In this chapter, we take the next step by using the patterns identified during data exploration to **build and fit a statistical model**. We will use the `tidymodels` framework to build a linear regression model and use it to describe relationships between an outcome and one or more explanatory variables.

We begin with **linear regression** because it provides a relatively simple introduction to statistical modelling. The model is easy to visualize and interpret, while introducing several ideas that are fundamental to more advanced models: response and predictor variables, model parameters, fitted values, residuals, model assumptions, prediction, and model evaluation. Linear regression also provides a useful foundation for understanding more flexible models that we will encounter later.

We will work with a dataset of **insurance charges**, containing demographic and lifestyle information such as age, sex, BMI, smoking status, and region, together with the medical charges billed to

insurance. We will first consider a simple linear regression model and then extend it to **multiple linear regression**, where several predictors are considered simultaneously.

An important distinction in this chapter is that regression can be used for different purposes. From a statistical perspective, we may be interested in understanding and quantifying **associations** between variables. From a predictive perspective, we may instead be interested in how accurately the model predicts the outcome for new observations. The same regression model can be used for both purposes, but the way we build, evaluate, and interpret the model can differ.

6.1.1 Why `tidymodels`?

`tidymodels` provides a consistent framework for modelling that follows the same general philosophy as the tidyverse tools used in the earlier chapters. Rather than treating model fitting as a single function call, it provides a structured workflow for **specifying, fitting, evaluating, and using models**. This becomes increasingly useful as models become more complicated and as we move from simple statistical analysis toward predictive modelling.

For a simple linear regression, we could use base R directly:

```
lm(charges ~ age, data = insurance)
```

This is perfectly valid and is often the simplest approach when the main goal is to fit and interpret a single statistical model. In this course, however, we use `tidymodels` because it provides a modelling workflow that can be extended naturally to model evaluation, prediction, multiple models, and more advanced modelling methods.

> **i** Why not just use `lm()`?
>
> Base R provides `lm()` for fitting linear regression models directly. It is an excellent tool for statistical analysis, and we will use it indirectly because `tidymodels` can use the `"lm"` engine. The purpose of using `tidymodels` here is not that `lm()` is inadequate. Rather, `tidymodels` introduces a **consistent modelling workflow** that will be useful when we move beyond a single linear regression model.

The chapter therefore builds on the skills developed earlier:

$$\boxed{\text{Transform} \rightarrow \text{Summarize} \rightarrow \text{Visualize} \rightarrow \text{Model} \rightarrow \text{Evaluate}}$$

The goal is not only to learn the syntax for fitting a regression model, but to understand **where modelling fits within the broader data science process** and how regression can be used to answer questions arising from insurance data.

For more information about the `tidymodels` framework and its ecosystem of packages, visit the official tidymodels website.

I would simplify this section and make it consistent with the project structure you have used in the earlier chapters. In particular, students should **not** need to modify a personal file path. Since the dataset is in the project's `data/` folder, a relative path is more appropriate.

I would also introduce `charges` explicitly as the response variable, since this becomes important in the next section.

6.2 Insurance Charges Data

We will use a dataset of **insurance charges**, which we will call `insurance`. The data are stored in the project's `data` folder in the file `insuranceKaggle.csv`.

The dataset contains information about policyholders and the medical charges billed to their insurance:

Variable	Description
`age`	Age of the policyholder, in years
`sex`	Sex of the policyholder (`male` or `female`)
`bmi`	Body mass index
`smoker`	Whether the policyholder is a smoker (`yes` or `no`)
`region`	Residential region (`northeast`, `northwest`, `southeast`, or `southwest`)
`children`	Number of children (dependents) covered by the policy
`charges`	Medical costs billed to insurance

In this chapter, `charges` will be the **response variable** (or outcome), and the other variables will be considered as potential **predictors** (or explanatory variables).

We begin by loading the data into R using `read_csv()` from the `readr` package:

```
library(readr)

insurance <- read_csv("../data/insuranceKaggle.csv")
```

The dataset can then be inspected using the functions we have already learned:

```
library(dplyr)

glimpse(insurance)
```

```
Rows: 1,338
Columns: 7
$ age      <dbl> 19, 18, 28, 33, 32, 31, 46, 37, 37, 60, 25, 62, 23, 56, 27, 1~
$ sex      <chr> "female", "male", "male", "male", "male", "female", "female",~
$ bmi      <dbl> 27.900, 33.770, 33.000, 22.705, 28.880, 25.740, 33.440, 27.74~
$ children <dbl> 0, 1, 3, 0, 0, 0, 1, 3, 2, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0~
$ smoker   <chr> "yes", "no", "no", "no", "no", "no", "no", "no", "no", "no", ~
```

```
$ region   <chr> "southwest", "southeast", "southeast", "northwest", "northwes~
$ charges  <dbl> 16884.924, 1725.552, 4449.462, 21984.471, 3866.855, 3756.622,~
```

We can also obtain a quick summary of the variables:

```
summary(insurance)
```

```
      age            sex                 bmi           children
 Min.   :18.00   Length:1338        Min.   :15.96   Min.   :0.000
 1st Qu.:27.00   Class :character   1st Qu.:26.30   1st Qu.:0.000
 Median :39.00   Mode  :character   Median :30.40   Median :1.000
 Mean   :39.21                      Mean   :30.66   Mean   :1.095
 3rd Qu.:51.00                      3rd Qu.:34.69   3rd Qu.:2.000
 Max.   :64.00                      Max.   :53.13   Max.   :5.000
    smoker             region             charges
 Length:1338        Length:1338        Min.   : 1122
 Class :character   Class :character   1st Qu.: 4740
 Mode  :character   Mode  :character   Median : 9382
                                       Mean   :13270
                                       3rd Qu.:16640
                                       Max.   :63770
```

At this stage, the objective is simply to become familiar with the data. We will use the data exploration and visualization techniques introduced in the previous chapters before deciding how to construct our regression model.

6.2.1 Data Visualization, Interpretation, and Feature Selection

Before building a model, we first explore how the response variable, `charges`, is related to the potential predictors. This continues the workflow introduced in the previous chapters:

$$\text{Transform and summarize} \rightarrow \text{Visualize} \rightarrow \text{Interpret} \rightarrow \text{Model}.$$

Visualization helps us identify patterns, trends, differences between groups, and unusual observations. These observations can guide our choice of predictors and help us decide which relationships are worth investigating further.

The type of plot we use depends on the type of predictor.

- For a **numeric predictor**, such as `age` or `bmi`, we can use a **scatter plot** with the predictor on the horizontal axis and `charges` on the vertical axis. This allows us to examine whether charges tend to increase, decrease, or follow some other pattern as the predictor changes.
- For a **categorical predictor**, such as `smoker`, `sex`, or `region`, we can use a **boxplot** to compare the distribution of charges across groups.

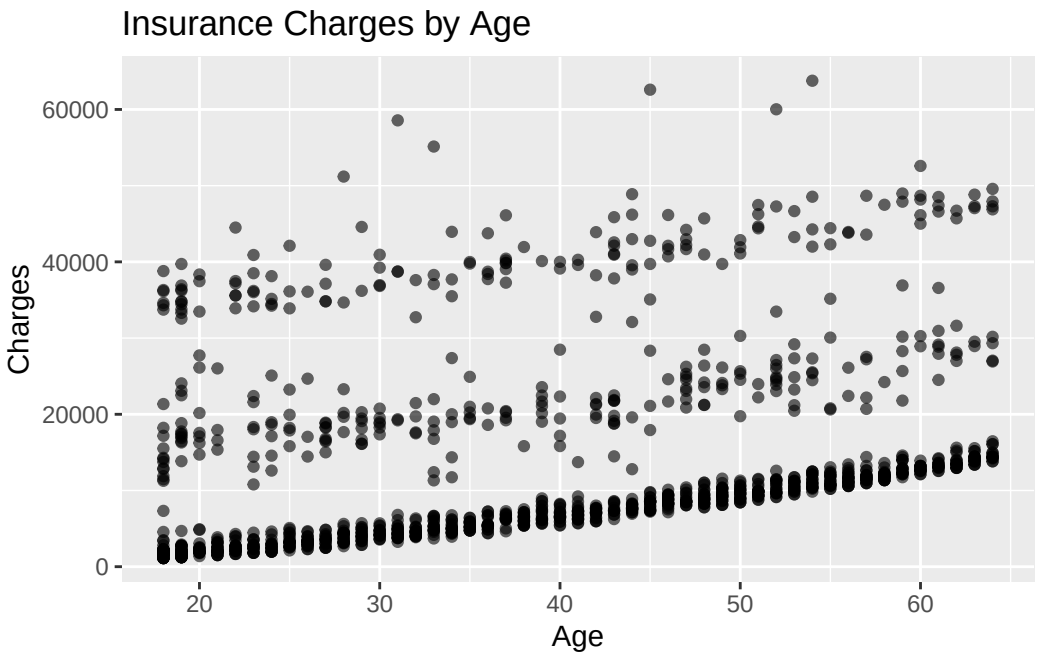
We can also use colour to examine whether the relationship between two numeric variables differs across groups. For example, colouring an age-versus-charges scatter plot by `smoker` allows us to examine whether the relationship between age and charges appears different for smokers and non-smokers.

The purpose of this exploration is **not to prove that a predictor is important or unimportant**. Instead, we are looking for patterns that may help us formulate a useful regression model. A variable that appears weak when considered by itself may still provide useful information when other predictors are included in a multiple regression model.

6.2.1.1 Age vs. Charges

`age` is a numeric predictor, so we use a scatter plot to examine its relationship with `charges`.

```
library(ggplot2)

ggplot(insurance, aes(x = age, y = charges)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by Age",
    x = "Age",
    y = "Charges"
  )
```



Interpretation: The plot suggests a positive relationship between age and charges: older policyholders tend to have higher medical charges. However, the relationship is not perfectly linear, and there is substantial variation in charges among policyholders of similar ages.

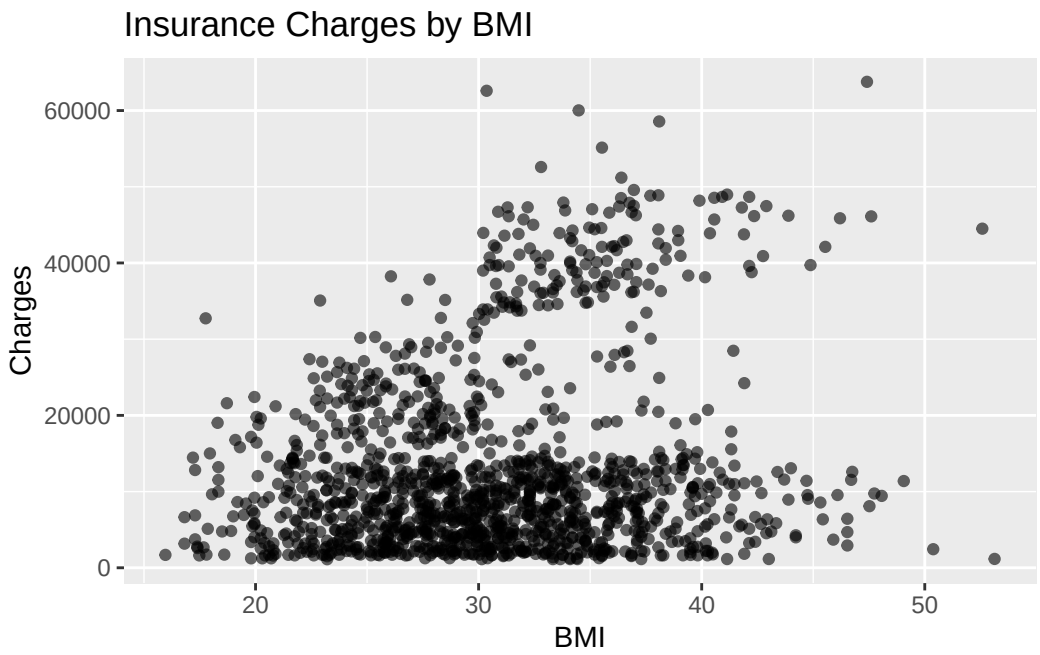
This is an important observation for regression. A predictor can have a clear overall relationship with the response even when individual observations vary considerably around that relationship.

> **Key idea**
>
> A regression model describes the **general relationship** between a predictor and a response; it does not imply that individual observations will follow that relationship exactly.
> For example, suppose two 50-year-old policyholders have medical charges of $8,000 and $25,000. This does **not** contradict a positive relationship between age and charges. It simply shows that age does not explain all of the variation in medical charges. Other factors, such as BMI, smoking status, and number of children, may also affect charges.
> Thus, when we fit a regression model, we expect observations to vary around the fitted line rather than lie exactly on it.

6.2.1.2 BMI vs. Charges

`bmi` is also a numeric predictor, so we again use a scatter plot.

```
ggplot(insurance, aes(x = bmi, y = charges)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by BMI",
    x = "BMI",
    y = "Charges"
  )
```

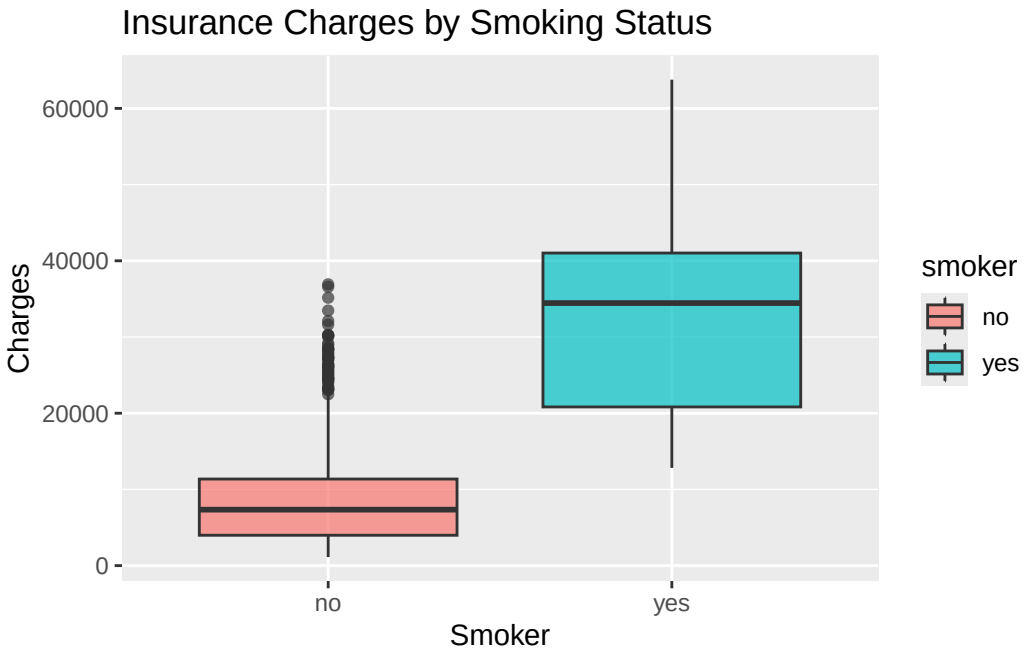


Interpretation: There appears to be a positive association between BMI and charges, although the relationship is less clear than the relationship between age and charges. The plot also suggests that the relationship may differ across groups, which we will investigate later.

6.2.1.3 Smoker vs. Charges

`smoker` is a categorical predictor with two groups. A boxplot allows us to compare the distributions of charges between smokers and non-smokers.

```
ggplot(insurance, aes(x = smoker, y = charges, fill = smoker)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Insurance Charges by Smoking Status",
    x = "Smoker",
    y = "Charges"
  )
```



Interpretation: There is a substantial difference in the distribution of charges between smokers and non-smokers. Smokers tend to have much higher charges.

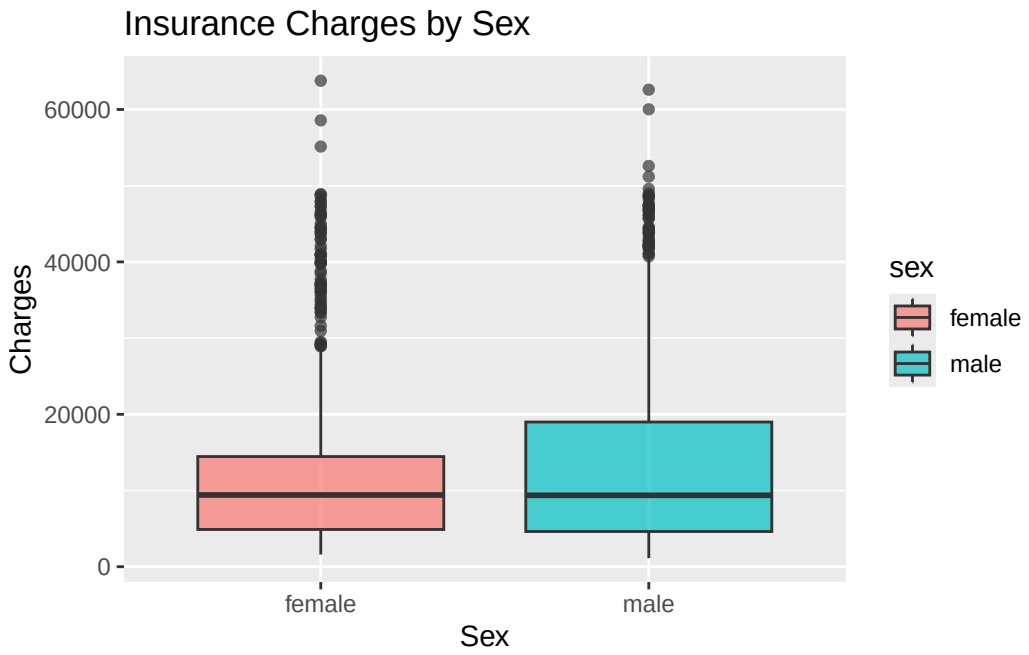
This suggests that `smoker` may be an important predictor of `charges`. Because `smoker` is categorical, it will later be represented in the regression model using indicator variables.

6.2.1.4 Sex vs. Charges

`sex` is another categorical predictor.

```
ggplot(insurance, aes(x = sex, y = charges, fill = sex)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Insurance Charges by Sex",
    x = "Sex",
```

```
    y = "Charges"
  )
```



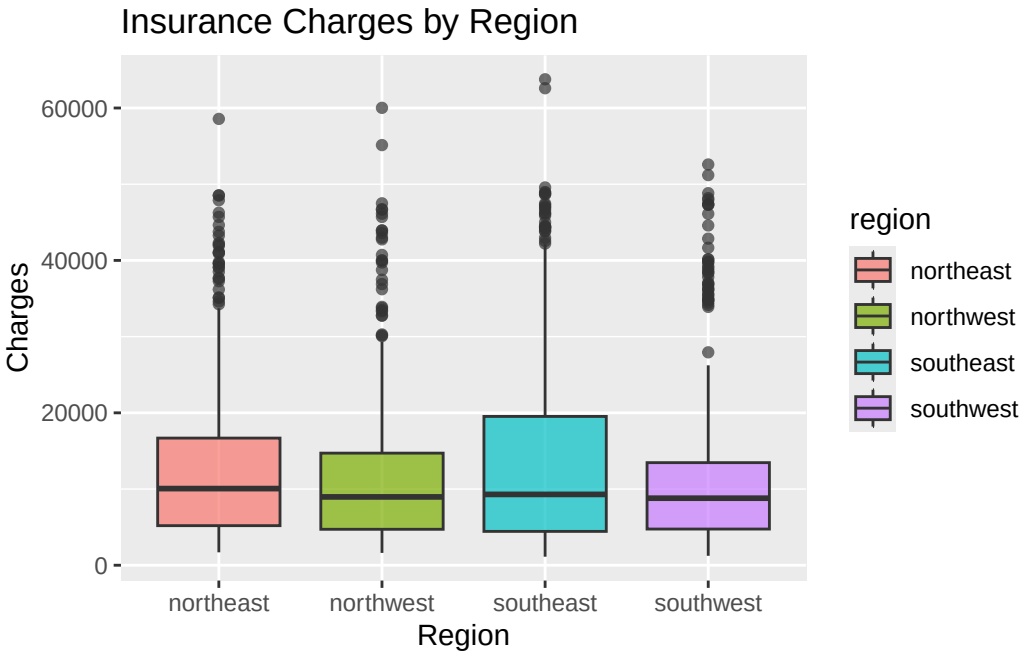
Interpretation: The distributions of charges for the two sex groups appear broadly similar. There does not appear to be a strong marginal relationship between `sex` and `charges` in this plot.

However, this does not mean that `sex` has no effect in a regression model. Its contribution may change after other predictors are taken into account.

6.2.1.5 Region vs. Charges

`region` is a categorical predictor with four groups.

```
ggplot(insurance, aes(x = region, y = charges, fill = region)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Insurance Charges by Region",
    x = "Region",
    y = "Charges"
  )
```



Interpretation: The distributions of charges across the four regions appear relatively similar, although there may be some differences in their centres and variability. The plot does not suggest a strong relationship between region and charges when considered on its own.

6.2.2 What Have We Learned?

The visual exploration gives us an initial understanding of the potential predictors:

Predictor	Type	Relationship with `charges`	Initial assessment
`age`	Numeric	Positive association	Promising
`bmi`	Numeric	Positive association	Promising
`smoker`	Categorical	Large difference between groups	Strong predictor candidate
`sex`	Categorical	Relatively small difference	Less promising on its own
`region`	Categorical	Relatively small differences	Less promising on its own

These observations provide a starting point for model building. However, **visual exploration alone is not sufficient for feature selection**. In particular, predictors can be related to one another, and the apparent relationship between a predictor and `charges` may change when other predictors are included.

In the next sections, we will first fit a simple linear regression model and then extend it to multiple linear regression. This will allow us to investigate relationships while considering several predictors simultaneously.

6.2.3 Exercises

Use the plots above as examples and create your own visualizations.

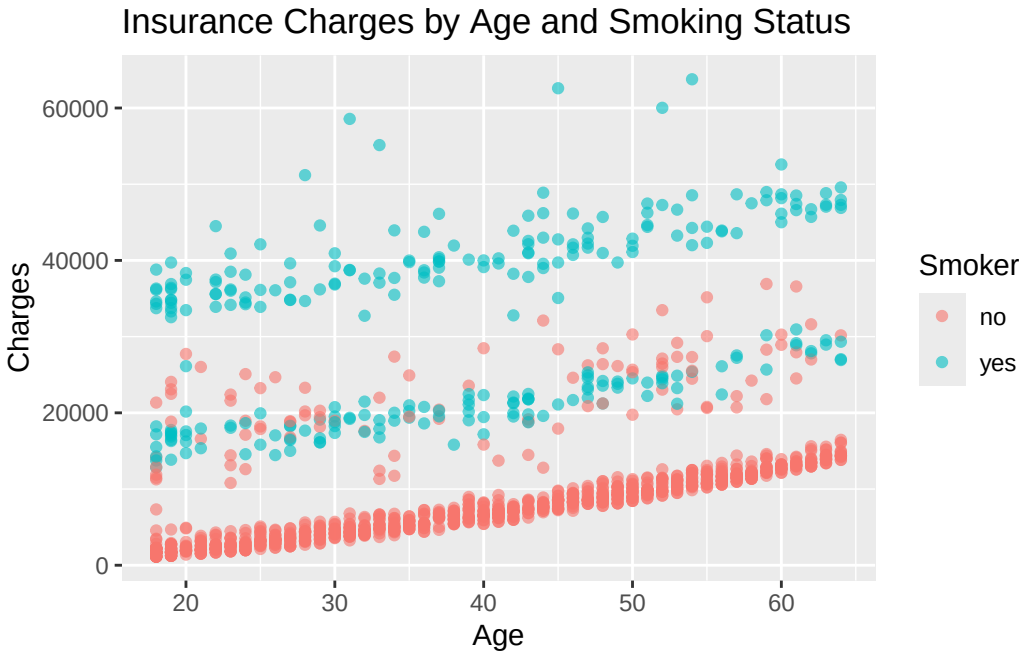
1. Create a scatter plot of `age` versus `charges`, coloured by `smoker`. What does the plot suggest about the relationship between age and charges for smokers and non-smokers?
2. Create a scatter plot of `bmi` versus `charges`, coloured by `smoker`. What pattern do you observe?
3. Create a scatter plot of `age` versus `charges`, coloured by `sex`. Does the relationship appear substantially different between the two groups?
4. Create a scatter plot of `bmi` versus `charges`, coloured by `sex`. Comment on the patterns you observe.
5. Create a scatter plot of `age` versus `charges`, coloured by `region`. Are there noticeable differences across regions?
6. Create a scatter plot of `bmi` versus `charges`, coloured by `region`. What do you observe?
7. Based on your visual exploration, which predictors would you initially consider for a regression model? Explain your reasoning.

6.2.4 Solutions to Exercises

The following solutions illustrate one reasonable approach to the exercises. The interpretation of a plot may vary slightly depending on how the relationship is described.

6.2.4.1 1: Age and Charges by Smoking Status

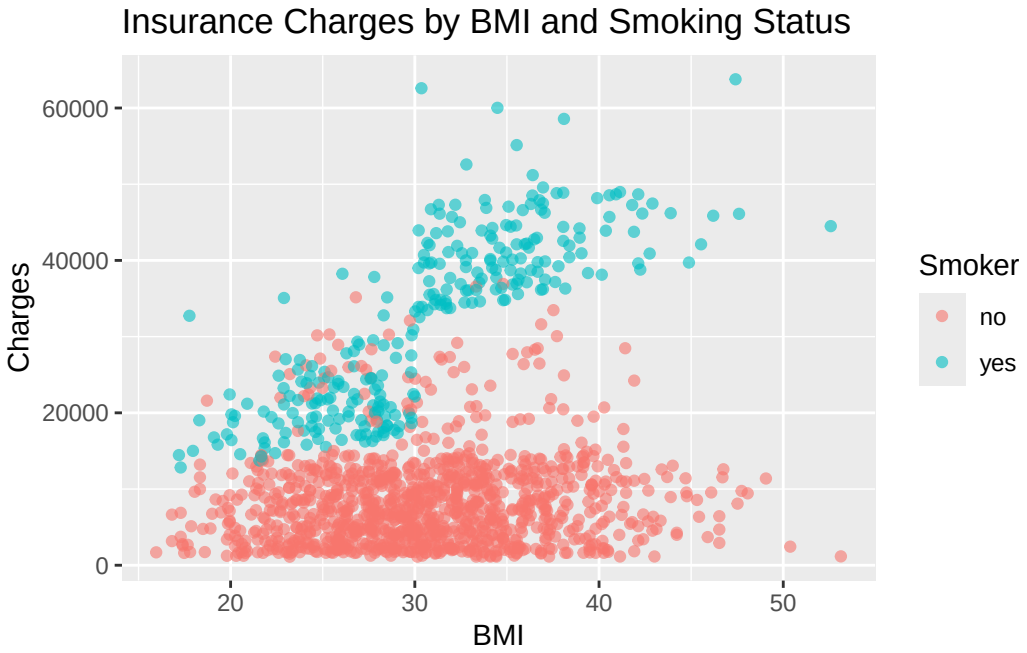
```
ggplot(insurance, aes(x = age, y = charges, colour = smoker)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by Age and Smoking Status",
    x = "Age",
    y = "Charges",
    colour = "Smoker"
  )
```



Interpretation: Charges generally increase with age. The relationship is particularly pronounced among smokers, whose charges tend to be substantially higher than those of non-smokers.

6.2.4.2 2: BMI and Charges by Smoking Status

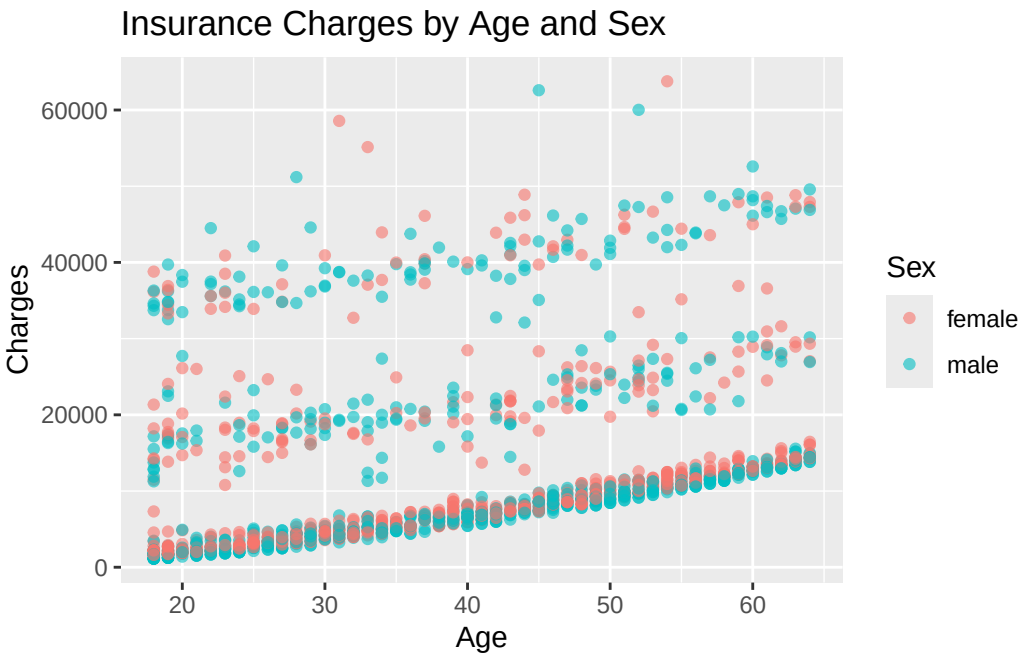
```
ggplot(insurance, aes(x = bmi, y = charges, colour = smoker)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by BMI and Smoking Status",
    x = "BMI",
    y = "Charges",
    colour = "Smoker"
  )
```



Interpretation: BMI has a positive association with charges, although the relationship is less clear than for age. Smoking status creates a substantial separation between the groups.

6.2.4.3 3: Age and Charges by Sex

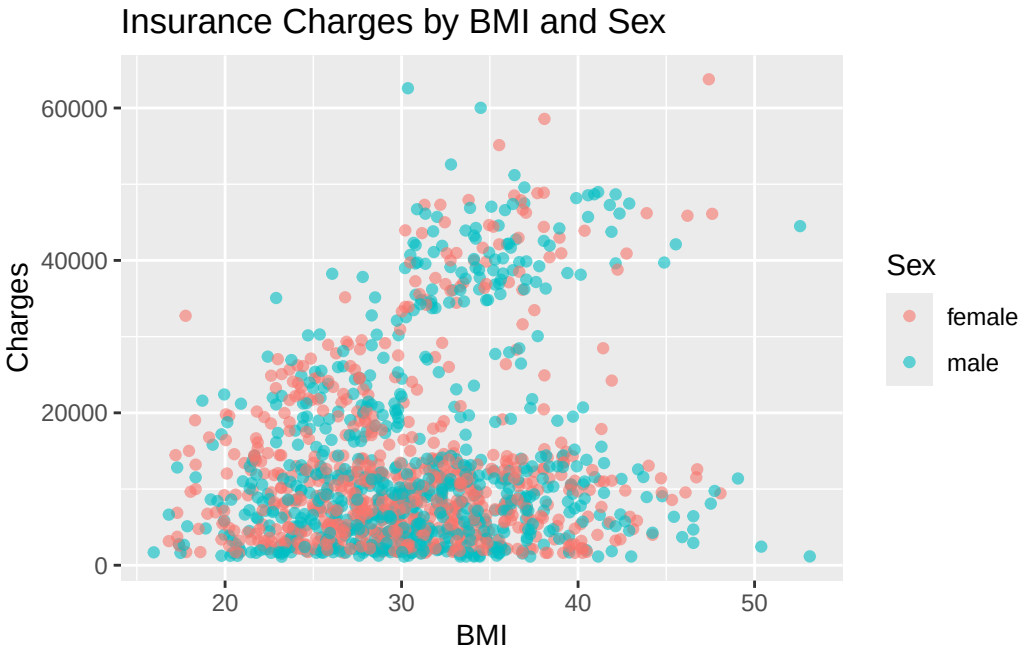
```
ggplot(insurance, aes(x = age, y = charges, colour = sex)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by Age and Sex",
    x = "Age",
    y = "Charges",
    colour = "Sex"
  )
```



Interpretation: Charges tend to increase with age for both groups. The two groups show considerable overlap, suggesting that sex has a relatively weak marginal relationship with charges.

6.2.4.4 4: BMI and Charges by Sex

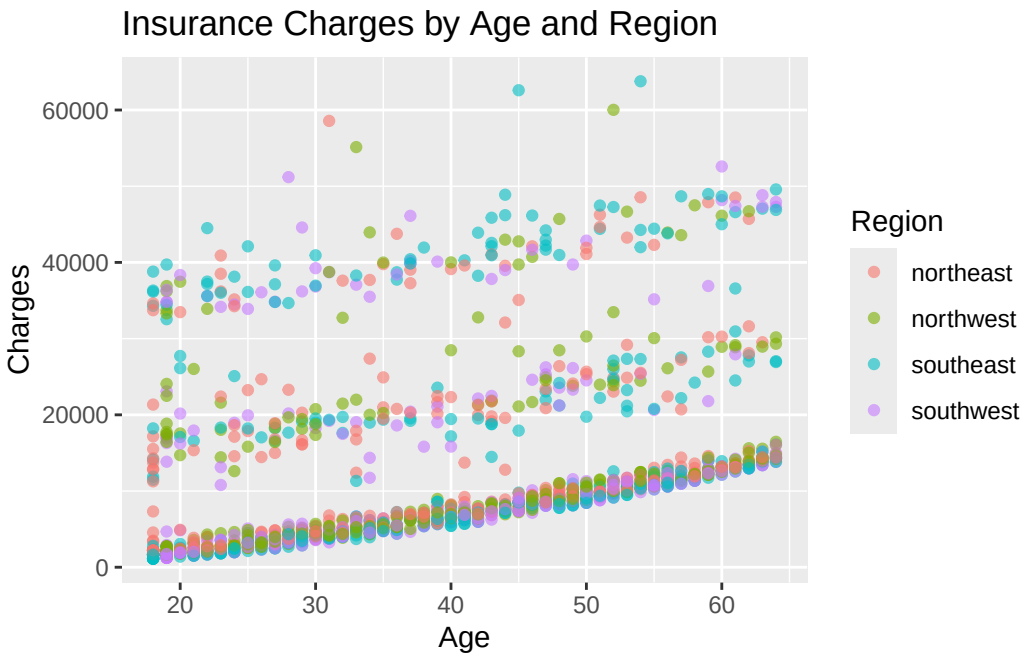
```
ggplot(insurance, aes(x = bmi, y = charges, colour = sex)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by BMI and Sex",
    x = "BMI",
    y = "Charges",
    colour = "Sex"
  )
```



Interpretation: There is some positive association between BMI and charges, but the two sex groups overlap substantially. There is no strong visual evidence of a large difference between the groups.

6.2.4.5 5: Age and Charges by Region

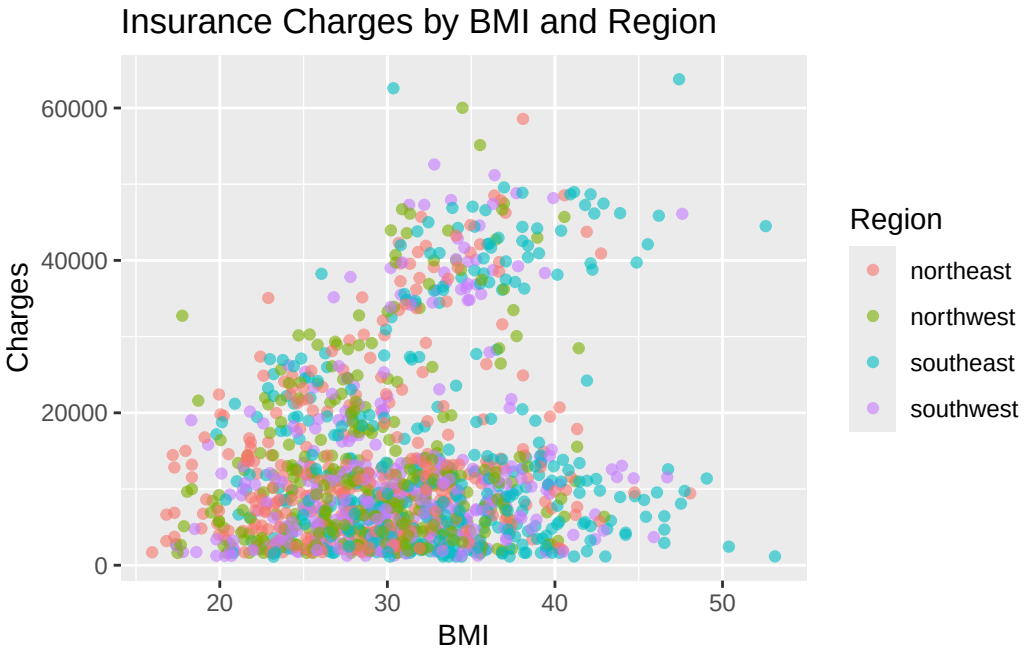
```
ggplot(insurance, aes(x = age, y = charges, colour = region)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by Age and Region",
    x = "Age",
    y = "Charges",
    colour = "Region"
  )
```



Interpretation: The positive relationship between age and charges is visible across regions. The regional groups overlap considerably, although some differences may be present.

6.2.4.6 6: BMI and Charges by Region

```
ggplot(insurance, aes(x = bmi, y = charges, colour = region)) +
  geom_point(alpha = 0.6) +
  labs(
    title = "Insurance Charges by BMI and Region",
    x = "BMI",
    y = "Charges",
    colour = "Region"
  )
```



Interpretation: There is some positive association between BMI and charges, but the relationship varies considerably among observations. The regions overlap substantially.

6.2.4.7 7: Selecting Potential Predictors

A reasonable initial choice is to consider all six available explanatory variables:

- `age`
- `sex`
- `bmi`
- `smoker`
- `region`
- `children`

The visual exploration suggests that `age`, `bmi`, and `smoker` have relatively clear relationships with `charges`. The relationships for `sex` and `region` appear weaker when examined individually. However, this does **not** mean that `sex` or `region` should automatically be excluded from the regression model. Their effects may become clearer after other predictors are taken into account.

The next step is therefore to fit a regression model and examine the results rather than selecting predictors based only on individual plots.

6.2.5 Exercise: Examining the Distribution of Insurance Charges

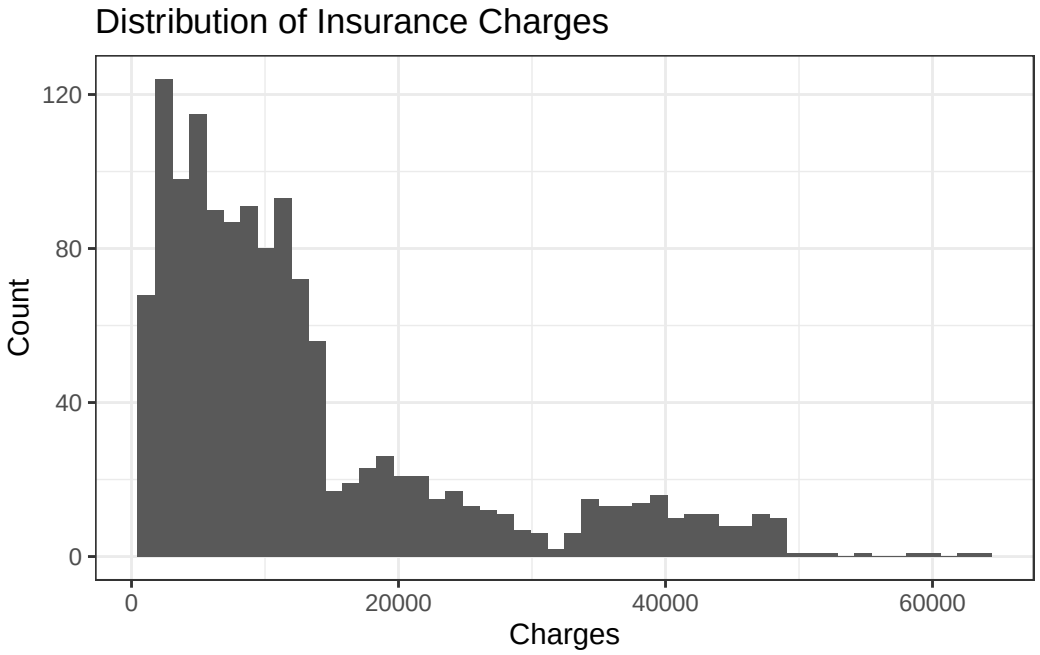
Before fitting a regression model, examine the distribution of the response variable, `charges`.

1. Create a histogram of `charges` using `ggplot2`.
2. Use an appropriate number of bins to clearly display the shape of the distribution.
3. Add appropriate axis labels and a title.

4. Examine the plot and describe the distribution of `charges`.
5. Comment on whether the data appear to contain unusually large observations.

6.2.5.1 Solution

```
ggplot(insurance, aes(x = charges)) +
  geom_histogram(bins = 50) +
  labs(
    title = "Distribution of Insurance Charges",
    x = "Charges",
    y = "Count"
  ) +
  theme_bw()
```



The distribution of `charges` is **strongly right-skewed**. Most observations have relatively low or moderate charges, while a smaller number of observations have much larger charges.

The histogram also shows some **unusually large charges** in the upper tail of the distribution. These observations are not necessarily errors. They may represent policyholders with substantial medical costs.

This pattern is important when building a regression model because unusually large observations can have a relatively strong influence on the fitted regression line and can contribute to large prediction errors.

However, the right-skewed distribution of the response variable **does not by itself violate the assumptions of ordinary least squares regression**. The normality assumption concerns the model errors (residuals), not the raw response variable. After fitting the model, we will examine the residuals to assess whether the model is appropriate.

6.3 Build and Fit a Model

In the previous section, we explored the relationship between `charges` and the potential predictors using summaries and visualizations. We now move from **exploring relationships** to **building a model** that describes these relationships and can be used to make predictions.

A linear regression model provides a simple way to relate an outcome variable to one or more predictors. In this chapter, our focus is primarily on **prediction**: we want to build a model that can make useful predictions of insurance charges for new observations.

This does not mean that the coefficients of a regression model are unimportant. They can still help us understand how the predictors are associated with the response. However, when our goal is prediction, we also need to ask an important question:

> **How well does the model predict observations that were not used to fit the model?**

This leads to a modelling workflow in which we:

1. split the data into **training and testing sets**;
2. specify and fit the model using the training data;
3. use the fitted model to make predictions; and
4. evaluate its predictive performance on the testing data.

The `tidymodels` framework provides tools for carrying out this workflow in a consistent way. It also provides a foundation for extending the workflow to other models, preprocessing steps, and evaluation methods.

We will begin with a simple linear regression model and then use the fitted model to examine what it tells us about insurance charges.

> **i** Statistical Inference and Prediction
>
> Linear regression can be used for different purposes. In **traditional statistical analysis**, the focus is often on understanding associations between variables, estimating coefficients, and conducting statistical inference using tools such as confidence intervals and hypothesis tests.
> In **prediction-focused modelling**, the emphasis is on how accurately a model can predict outcomes for new observations. Model performance on unseen data is therefore an important consideration, alongside interpretation of the model.
> The `tidymodels` framework can support both perspectives. In this chapter, however, we will primarily adopt a **prediction-focused perspective**. We will build models, use them to predict insurance charges, and evaluate how well those predictions generalize to new data.

6.3.1 Specifying and Fitting a Linear Regression Model for Insurance Data

Suppose we want to predict insurance `charges` using the policyholder's `age`. A simple linear regression model can be written as

$$\text{charges} = \beta_0 + \beta_1 \text{age} + \varepsilon,$$

where β_0 is the intercept, β_1 measures the change in expected charges associated with a one-year increase in age, and ε represents the remaining variation in charges that is not explained by age.

In R, we express this relationship using the formula

```
charges ~ age
```

The formula tells us **what we want to model**: `charges` is the response variable and `age` is the predictor.

To build this model using `tidymodels`, we use the **parsnip** package.

6.3.1.1 What is `parsnip`?

`parsnip` is the `tidymodels` package that provides a consistent way to **specify and fit different types of statistical and machine-learning models**. Instead of using a different set of functions for every modelling method, `parsnip` provides a common interface.

For example, `linear_reg()` is the `parsnip` function for specifying a linear regression model:

```
library(parsnip)

linear_reg()
```

```
Linear Regression Model Specification (regression)

Computational engine: lm
```

```
#> Linear Regression Model Specification (regression)
#>
#> Computational engine: lm
```

The output introduces two important terms: **model specification** and **model engine**.

6.3.1.2 Model Specification

A **model specification** describes the **type of model we want to use** and, when applicable, the modelling options that control how it will be fitted.

Here,

```
linear_reg()
```

specifies that we want a **linear regression model**.

At this point, the model has not been fitted to any data. We have only described the type of model we want to use.

The formula is supplied later when we fit the model:

```
charges ~ age
```

Thus, it is useful to distinguish between the two ideas:

- `linear_reg()` → **what type of model** we want;
- `charges ~ age` → **which outcome and predictors** the model uses.

Together, these tell us what model we intend to fit.

6.3.1.3 Model Engine

A **model engine** is the computational method or software implementation used to **actually fit the model**.

For linear regression, the default engine is `"lm"`. This engine uses R's standard `lm()` implementation to estimate the regression coefficients using ordinary least squares (OLS).

We can explicitly set the engine with `set_engine()`:

```
lm_mod <- linear_reg() %>%
  set_engine("lm")

lm_mod
```

```
Linear Regression Model Specification (regression)

Computational engine: lm
```

The separation between the model specification and the engine is one of the useful ideas behind `parsnip`. We can describe the model using a consistent interface and then specify which computational implementation should be used.

For the simple linear regression in this chapter, we will use the `"lm"` engine and ordinary least squares. There is therefore no advantage in changing the engine at this stage. The separation becomes more useful when we work with other modelling methods and computational engines later.

6.3.1.4 Fitting the Model

Once we have specified the model, we **fit** it to the data. Fitting means using the observed data to estimate the unknown model parameters, such as β_0 and β_1.

We use the `fit()` function:

```
lm_fit <- lm_mod %>%
  fit(charges ~ age, data = insurance)

lm_fit
```

```
parsnip model object


Call:
stats::lm(formula = charges ~ age, data = data)

Coefficients:
(Intercept)          age
     3165.9        257.7
```

The resulting `lm_fit` object contains the fitted model, including the estimated coefficients and other information needed for prediction and further analysis.

The modelling process can therefore be viewed as:

$$\boxed{\text{Model specification} \rightarrow \text{Choose engine} \rightarrow \text{Fit to data}}$$

For this example:

```
linear_reg()  →  set_engine("lm")  →  fit(charges ~ age, data = insurance)
```

The important distinction is that **specifying a model is not the same as fitting a model**. Specification describes what we want to fit, while fitting uses the data to estimate the model parameters.

6.3.1.5 Inspecting Model Estimates

After fitting the model, we can inspect the estimated coefficients to understand the fitted relationship between `age` and `charges`.

For the model

$$\widehat{\text{charges}} = \hat{\beta}_0 + \hat{\beta}_1 \text{age},$$

the fitted coefficients from our data are:

```
Call: stats::lm(formula = charges ~ age, data = data)

Coefficients:
(Intercept)          age
     3165.9        257.7
```

The fitted regression equation is therefore

$$\widehat{\text{charges}} = 3165.9 + 257.7\,\text{age}.$$

The **intercept**, 3165.9, is the predicted value of `charges` when `age = 0`. Although this value is part of the mathematical regression equation, age 0 is outside the meaningful range of policyholder ages in this dataset. Therefore, the intercept does not have a particularly useful practical interpretation here.

The **coefficient for `age`**, 257.7, is more directly useful. It means that, according to this fitted model, a one-year increase in age is associated with an estimated increase of approximately **257.7 units in insurance charges, on average**.

For example, the difference in predicted charges between a policyholder aged 40 and one aged 50 is

$$257.7(50 - 40) = 2{,}577.$$

Thus, the model predicts approximately $2,577 higher charges for a 50-year-old than for a 40-year-old, based on age alone.

The coefficient tells us about the **estimated relationship in the fitted model**. It does not mean that increasing a person's age by one year will necessarily cause their actual insurance charges to increase by exactly 257.7. Individual charges can vary substantially because other factors are not included in this simple model.

6.3.1.6 Tidying the Model Results

The model output above is useful, but it is not presented in a convenient tabular form. The **broom** package provides the `tidy()` function, which converts model results into a tidy data frame. This makes the results easier to inspect, compare, and use in subsequent analysis.

```
library(broom)

tidy(lm_fit)
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)    3166.     937.       3.38 7.51e- 4
2 age             258.      22.5     11.5  4.89e-29
```

The resulting table contains information such as:

- `term` — the model term, such as `(Intercept)` or `age`;
- `estimate` — the estimated coefficient;
- `std.error` — the standard error of the estimate;
- `statistic` — the test statistic associated with the estimate; and
- `p.value` — the p-value for testing whether the coefficient is zero.

For the prediction-focused analysis in this chapter, the **`estimate`** column is particularly important because it gives the coefficients used to construct predictions. The other columns provide additional information that can be useful when we consider statistical inference.

The next step is to visualize the fitted regression line together with the observed insurance charges.

6.3.1.7 Visualizing Regression Results

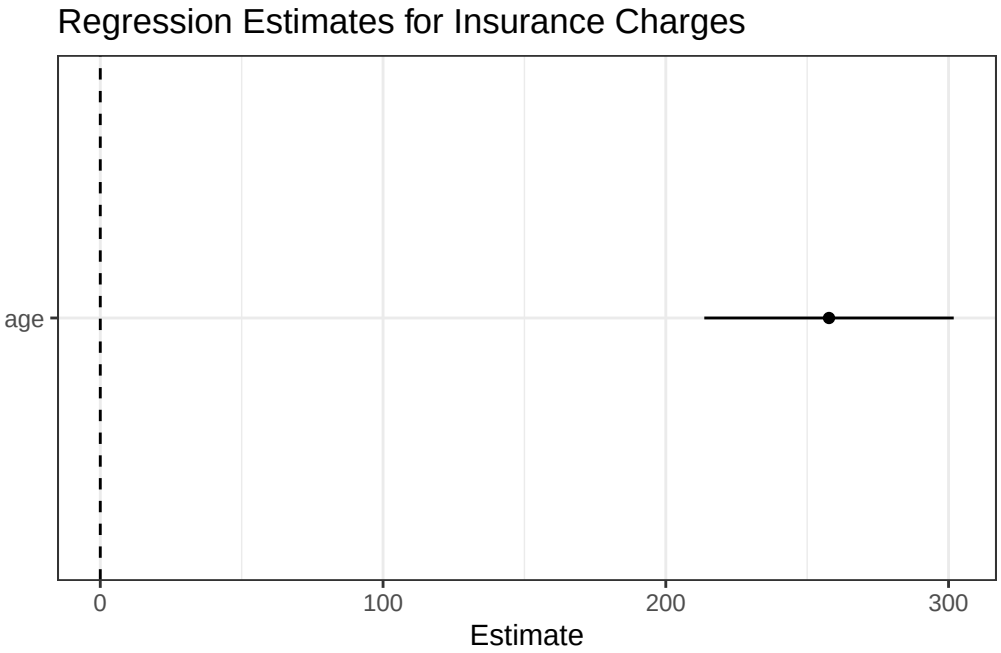
A table of regression coefficients provides precise numerical results, but a graphical representation can make the **size and uncertainty of the estimates** easier to compare. A **dot-and-whisker plot** displays each coefficient as a point together with an interval representing its uncertainty.

In the plot:

- the **dot** represents the estimated coefficient;
- the **whisker** represents an interval around the estimate; and
- the vertical line at zero provides a reference for whether the interval includes zero.

We can create a dot-and-whisker plot using the `dotwhisker` package:

```
library(dotwhisker)
library(ggplot2)

tidy(lm_fit) %>%
  dwplot(
    dot_args = list(color = "black"),
    whisker_args = list(color = "black")
  ) +
  geom_vline(xintercept = 0, linetype = 2) +
  theme_bw() +
  labs(
    title = "Regression Estimates for Insurance Charges",
    x = "Estimate",
    y = ""
  )
```



For this simple model, the estimate for `age` is positive, indicating that higher age is associated with higher predicted charges. The interval around the estimate is relatively narrow and does not include zero. This provides evidence of a positive association between age and charges in the data.

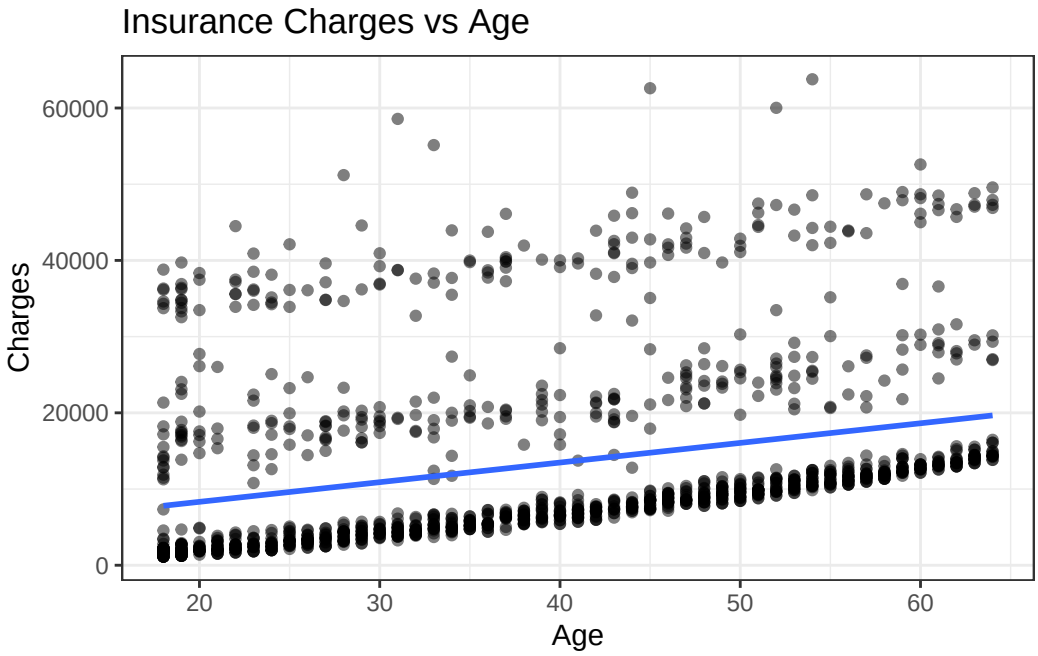
It is important to distinguish this interpretation from a causal statement. The regression coefficient describes the relationship captured by the fitted model; it does not establish that age itself causes charges to increase.

As we move to multiple regression models, a dot-and-whisker plot becomes particularly useful because it allows us to compare the estimated coefficients of several predictors in one figure.

6.3.1.8 Observed Data and Predicted Line

A regression coefficient tells us about the relationship mathematically, but we can also visualize the model directly on the original data. The following plot shows the observed insurance charges together with the fitted regression line.

```
library(ggplot2)

ggplot(insurance, aes(x = age, y = charges)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_bw() +
  labs(
    title = "Insurance Charges vs Age",
    x = "Age",
    y = "Charges"
  )
```



Each point represents the **observed charges** for one policyholder. The line represents the **predicted charges from the fitted linear regression model** for different values of age.

The upward slope of the line reflects the positive coefficient for `age`: the model predicts higher charges as age increases. The substantial vertical spread of the observations around the line also shows that age alone does not explain all the variation in insurance charges.

This distinction is important:

- the **points** are actual observed values;
- the **line** represents the model's predicted values; and
- the **distance between a point and the line** represents the model's prediction error, or **residual**, for that observation.

Thus, the plot gives us an initial visual assessment of both the relationship captured by the model and the amount of variation that remains unexplained. We will examine model predictions and residuals in more detail later.

6.3.2 Use a Model to Predict Insurance Charges with Confidence Intervals

Once a model has been fitted, we can use it to make predictions for new observations. This is one of the main purposes of building a prediction model.

The `predict()` function in `parsnip` provides a consistent interface for generating predictions from fitted models. The results are returned as a tibble, which makes them convenient to combine with the new data and use in further analysis or visualization.

Suppose we want to estimate insurance charges for policyholders aged 30, 40, and 50. We first create a dataset containing these new ages:

```r
library(tibble)

new_points <- tibble(age = c(30, 40, 50))
```

We can then use the fitted model to obtain the predicted mean charges:

```r
mean_pred <- predict(lm_fit, new_data = new_points)

mean_pred
```

```
# A tibble: 3 x 1
   .pred
   <dbl>
1 10898.
2 13475.
3 16052.
```

The `.pred` column contains the predicted charges for each age.

We can also quantify the uncertainty in these estimated mean charges. A **95% confidence interval** gives a range of plausible values for the **mean charge** associated with each age.

```r
conf_int_pred <- predict(
  lm_fit,
  new_data = new_points,
  type = "conf_int"
)

conf_int_pred
```

```
# A tibble: 3 x 2
  .pred_lower .pred_upper
        <dbl>       <dbl>
1      10156.      11639.
2      12854.      14096.
3      15270.      16834.
```

The result contains three columns:

- `.pred` — the predicted mean charge;
- `.pred_lower` — the lower limit of the 95% confidence interval; and
- `.pred_upper` — the upper limit of the 95% confidence interval.

For example, if the predicted mean charge for age 40 is approximately $10,743 and its 95% confidence interval is approximately $10,110 to $11,375, this interval describes the uncertainty in the **estimated average charge** for policyholders of age 40.

It is important not to interpret this interval as the range in which an individual policyholder's charge is expected to fall. Individual charges can vary substantially around the regression line. An interval for a future individual observation is called a **prediction interval**, which is wider because it accounts for both uncertainty in the estimated mean and the remaining individual-level variation.

In this chapter, we first use confidence intervals to illustrate uncertainty in the estimated mean response. Prediction intervals can be considered when we focus specifically on predicting an individual future observation.

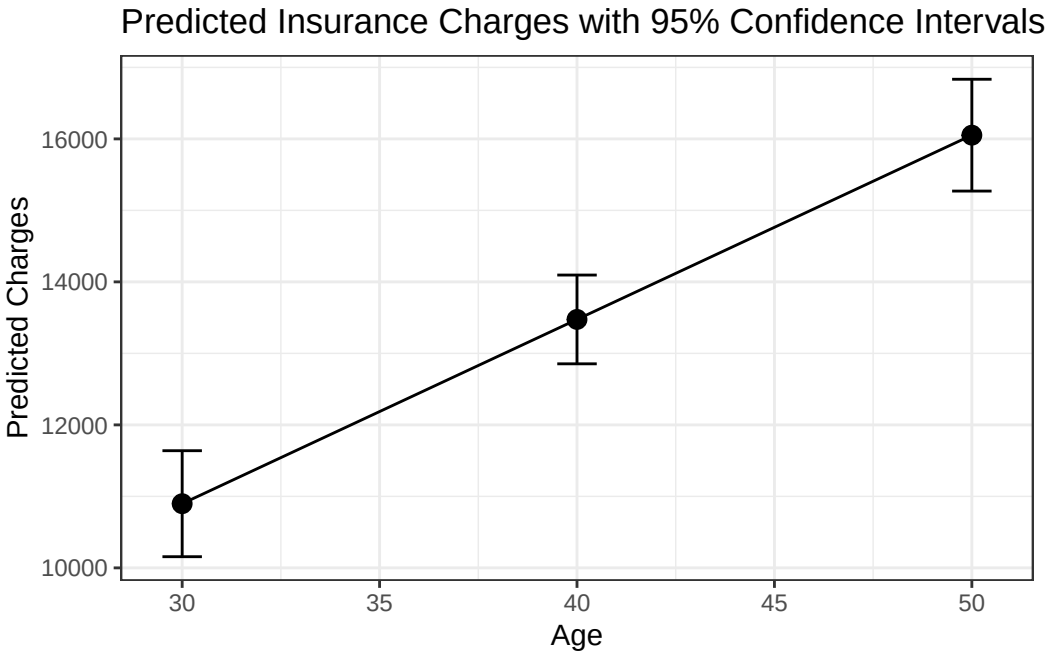
6.3.3 Visualizing Model Predictions

Numerical predictions are useful, but a graph can make the model's predictions and their uncertainty easier to understand. We can combine the new ages, predicted values, and confidence intervals into a single dataset.

```
library(dplyr)
library(ggplot2)

plot_data <- new_points %>%
  bind_cols(mean_pred) %>%
  bind_cols(conf_int_pred)
```

We can then display the predicted mean charges and their 95% confidence intervals:

```
ggplot(plot_data, aes(x = age, y = .pred)) +
  geom_errorbar(
    aes(ymin = .pred_lower, ymax = .pred_upper),
    width = 1
  ) +
  geom_point(size = 3) +
  geom_line() +
  labs(
    title = "Predicted Insurance Charges with 95% Confidence Intervals",
    x = "Age",
    y = "Predicted Charges"
  ) +
  theme_bw()
```



The points show the **predicted mean charges**, while the vertical lines show the corresponding **95% confidence intervals**. The upward pattern reflects the positive relationship between age and charges estimated by the regression model.

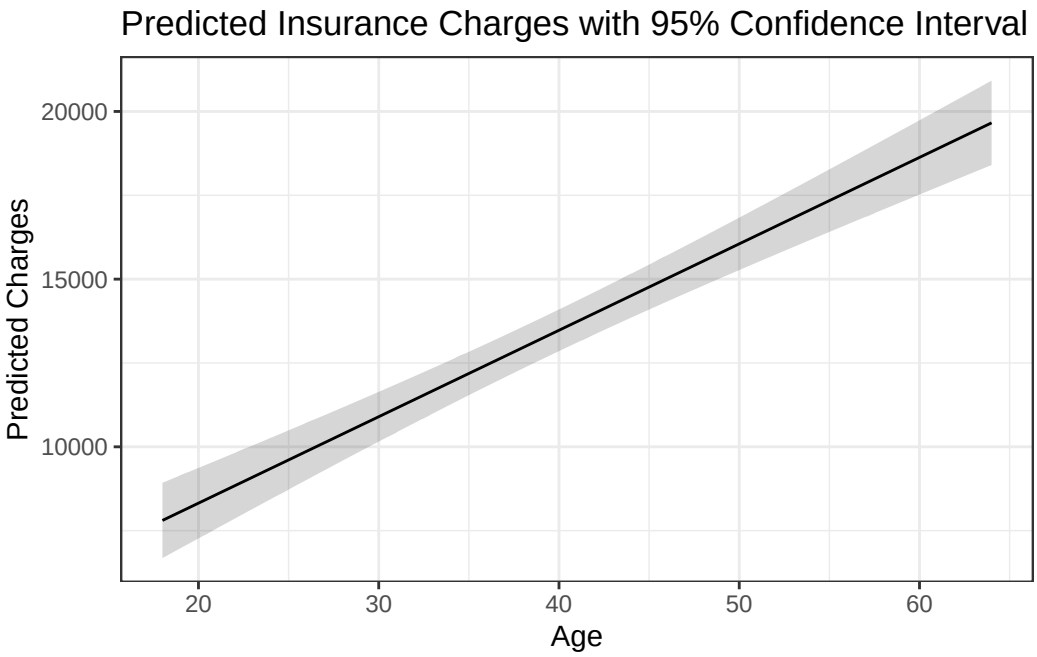
This visualization also illustrates an important idea in prediction: a model does not simply produce a single number. Predictions can also be accompanied by a measure of **uncertainty**, which helps us understand how precisely the model estimates the expected outcome.

6.3.4 Exercise: Visualizing Model Predictions

Use the fitted model `lm_fit` to visualize the predicted mean insurance charges across the observed range of `age`.

1. Create a set of 100 equally spaced age values covering the observed range of `age`.
2. Use `lm_fit` to obtain the predicted mean charges and their 95% confidence intervals.
3. Combine the results into a single data frame.
4. Create a plot showing the predicted charges as a line and the 95% confidence interval as a shaded band.
5. Add an appropriate title and axis labels.

6.3.4.1 Solution



6.3.5 Exercise: Observed and Predicted Charges

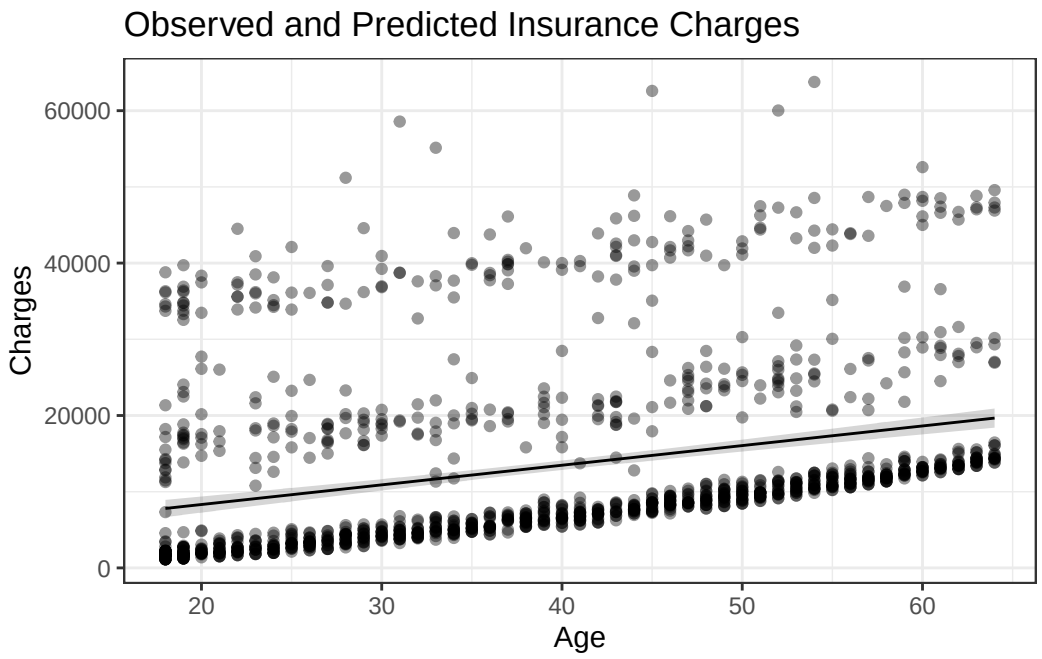
Extend the previous visualization by adding the original observations from `insurance`.

1. Plot the observed `charges` against `age`.
2. Add the predicted mean charges as a line.
3. Add the 95% confidence interval as a shaded band.
4. Add an appropriate title and axis labels.

6.3.5.1 Solution

```
ggplot(insurance, aes(x = age, y = charges)) +
  geom_point(alpha = 0.4) +
  geom_ribbon(
    data = plot_data,
    aes(
      x = age,
      ymin = .pred_lower,
      ymax = .pred_upper
    ),
    alpha = 0.2,
    inherit.aes = FALSE
  ) +
  geom_line(
```

```
  data = plot_data,
  aes(x = age, y = .pred),
  inherit.aes = FALSE
) +
labs(
  title = "Observed and Predicted Insurance Charges",
  x = "Age",
  y = "Charges"
) +
theme_bw()
```



i Why do we use `inherit.aes = FALSE`?

The main `ggplot()` call defines the aesthetics

```
aes(x = age, y = charges)
```

for the `insurance` dataset. By default, other layers inherit these aesthetics.
However, the prediction layers use `plot_data`, which contains `age`, `.pred`, `.pred_lower`, and `.pred_upper`, but does **not** contain `charges`.
Setting `inherit.aes = FALSE` tells `ggplot2` not to inherit `y = charges` from the main plot.
The prediction layers then use only the aesthetics specified in their own `aes()` calls.

6.3.6 From Simple to Multiple Linear Regression

In this chapter, we used a simple linear regression model to examine the relationship between `age` and insurance `charges`. We followed a complete modelling workflow:

Explore → Specify → Fit → Interpret → Predict → Visualize.

The fitted model provides a simple way to describe how predicted insurance charges change with age and to generate predictions for new observations. However, the observed charges show substantial variation around the fitted regression line. This indicates that `age` alone does not explain all of the differences in insurance charges.

Recall that the insurance dataset contains several other variables, including `bmi`, `smoker`, `sex`, `region`, and `children`. These characteristics may also provide information about insurance charges. A model that considers several predictors simultaneously may therefore provide a more useful representation of the relationship between policyholder characteristics and charges.

In the next chapter, we extend the simple linear regression model to **multiple linear regression**, allowing several predictors to be included in the same model. This will allow us to examine the contribution of each predictor while accounting for the others and to investigate whether a model using multiple characteristics can provide better predictions of insurance charges.

7 Chapter 7: Assessing Model Performance

7.1 Learning Outcomes

By the end of this chapter, students should be able to:

1. explain why predictive models need to be evaluated;
2. distinguish between model performance on training data and test data;
3. calculate and interpret common regression performance measures, including RMSE, MAE, R^2, and MAPE;
4. use the `yardstick` package to evaluate predictions from a fitted regression model;
5. use `metric_set()` to calculate multiple performance metrics simultaneously; and
6. compare the predictive performance of different regression models using appropriate performance metrics.

7.2 Introduction to Model Performance

Building a statistical model is only one part of the modelling process. In the previous chapter, we used linear regression to describe the relationship between insurance charges and explanatory variables and to generate predictions. Once a model has been fitted, an important question remains:

> **How well does the model predict the outcome?**

Model performance refers to how closely the predictions produced by a model agree with the actual observed outcomes. For predictive modelling, a model is useful when it can produce accurate predictions not only for the data used to fit the model, but also for new observations.

Consider a model that predicts insurance charges. For a policyholder with an actual charge of \$15,000, suppose the model predicts \$13,000. The prediction error is

$$15{,}000 - 13{,}000 = 2{,}000.$$

For another policyholder, the actual charge might be \$50,000 while the model predicts \$30,000. Although both predictions are imperfect, the second error is much larger. A model performance measure provides a systematic way to summarize such prediction errors across many observations.

Different measures emphasize different aspects of prediction accuracy. For example, some measures give greater weight to large errors, while others describe errors in relative rather than absolute terms. Therefore, evaluating a model usually involves considering more than one performance measure.

In this chapter, we focus on **predictive performance** rather than statistical inference. We will use the `yardstick` package from the `tidymodels` ecosystem to calculate and interpret common

regression performance measures. We will also consider why performance should be evaluated using observations that were not used to fit the model.

You can place this at the end of the introduction as a short forward-looking note:

> **Looking Ahead**
>
> In this chapter, we focus on evaluating the predictive performance of regression models. In the subsequent chapter, we will extend the linear regression model to **multiple regression**, allowing us to use several predictors simultaneously to explain and predict insurance outcomes.

7.3 Training Performance and Test Performance

When evaluating a predictive model, it is important to distinguish between the data used to **fit the model** and the data used to **evaluate its predictions**.

The data used to fit a model are called the **training data**. The model learns the relationship between the predictors and the outcome from these observations. If we evaluate the model using the same data, the resulting performance measures may give an overly optimistic impression of how well the model will perform on new observations.

To obtain a more realistic assessment, we can set aside part of the data before fitting the model. This is called the **test data**. The model is fitted using the training data, and its predictions are then evaluated using the test data.

The basic idea is:

$$\text{Complete dataset} \longrightarrow \begin{cases} \text{Training data} & \text{used to fit the model} \\ \text{Test data} & \text{used to evaluate predictions} \end{cases}$$

For example, suppose we have data on 1,000 insurance policies. We might use 80% of the observations for training and the remaining 20% for testing.

Dataset	Purpose	Example
Training data	Fit the model	800 policies
Test data	Evaluate predictions	200 policies

The model is fitted **only using the training data**. After fitting, we use the model to predict insurance charges for the test observations. Because these observations were not used to fit the model, the test performance provides information about how well the model may perform on new data.

7.3.1 Splitting Data with `rsample`

The **`rsample` package**, which is part of the `tidymodels` ecosystem, provides tools for creating resamples and splitting data for model evaluation.

We begin by loading the data into R using `read_csv()` from the `readr` package:

```
library(readr)

insurance <- read_csv("../data/insuranceKaggle.csv")
```

Once the data have been loaded, we can use `initial_split()` from the `rsample` package to divide the dataset into training and test sets:

```
library(tidymodels)

set.seed(123)

insurance_split <- initial_split(
  insurance,
  prop = 0.80
)

insurance_train <- training(insurance_split)
insurance_test  <- testing(insurance_split)
```

Here, `prop = 0.80` specifies that approximately 80% of the observations are allocated to the training set, with the remaining 20% allocated to the test set.

The `set.seed()` function is used so that the same random split can be reproduced when the code is run again.

We can check the number of observations in each dataset:

```
nrow(insurance_train)
```

```
[1] 1070
```

```
nrow(insurance_test)
```

```
[1] 268
```

The important point is that the **test data should remain separate from the model-fitting process**. We fit the model using `insurance_train` and use `insurance_test` only after the model has been fitted to assess its predictive performance.

> **!** Keep the Test Data Separate
>
> The test data should not be used to fit or tune the final model. If the model is fitted using the test observations and then evaluated on those same observations, the resulting performance measures no longer provide an independent assessment of prediction performance.

This separation between training and test data is fundamental to predictive modelling. In later sections, we will use the test predictions together with `yardstick` to measure how accurately the model predicts insurance charges.

7.4 Measuring Prediction Error

Once a model has been fitted, we can use it to generate predictions for observations in the test data. To evaluate the model, we compare these predictions with the actual observed outcomes.

For observation i, let

- y_i denote the actual observed outcome;
- $\widehat{y}_i$ denote the model's predicted outcome; and
- e_i denote the prediction error, or **residual**.

The prediction error is

$$e_i = y_i - \widehat{y}_i.$$

For example, suppose a policyholder has an actual insurance charge of \$15,000 and the model predicts \$13,000. The prediction error is

$$e_i = 15{,}000 - 13{,}000 = 2{,}000.$$

The model has **underpredicted** the actual charge by \$2,000.

If the model instead predicts \$17,000, then

$$e_i = 15{,}000 - 17{,}000 = -2{,}000,$$

meaning that the model has **overpredicted** the actual charge by \$2,000.

For a test dataset containing n observations, we therefore have a collection of prediction errors:

$$e_1, e_2, \ldots, e_n.$$

A model will usually make some predictions that are too high and others that are too low. Simply averaging the errors is therefore not a useful measure of overall prediction accuracy because positive and negative errors can cancel each other out.

For example, consider three prediction errors:

$$2{,}000, \quad -3{,}000, \quad 1{,}000.$$

Their average is

$$\frac{2{,}000 - 3{,}000 + 1{,}000}{3} = 0.$$

An average error of zero might suggest perfect prediction, even though all three predictions contain errors.

We therefore need measures that summarize the **magnitude** of prediction errors rather than allowing positive and negative errors to cancel out. Two common approaches are to use the **absolute error** and the **squared error**.

7.4.1 Absolute Error

The absolute error for observation i is

$$|e_i| = |y_i - \hat{y}_i|.$$

It measures how far the prediction is from the actual value, regardless of whether the model overpredicts or underpredicts.

For example, an error of $-2{,}000$ has an absolute error of

$$|-2{,}000| = 2{,}000.$$

The **Mean Absolute Error (MAE)** takes the average of these absolute errors:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

7.4.2 Squared Error

Another approach is to square each prediction error:

$$e_i^2 = (y_i - \hat{y}_i)^2.$$

Squaring removes the negative sign and gives greater weight to large errors. For example,

$$2{,}000^2 = 4{,}000{,}000$$

whereas

$$10{,}000^2 = 100{,}000{,}000.$$

The squared error from the \$10,000 error is therefore much larger relative to the \$2,000 error.

The **Root Mean Squared Error (RMSE)** is obtained by taking the square root of the average squared errors:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}.$$

Both MAE and RMSE measure prediction error on the same scale as the outcome. For insurance charges, for example, both are expressed in dollars.

The difference is that **MAE treats prediction errors proportionally, whereas RMSE places greater emphasis on large errors**. This distinction will be important when we use these measures to evaluate insurance prediction models.

7.5 Common Regression Performance Metrics

There is no single measure that completely describes the performance of a regression model. Different metrics capture different aspects of prediction accuracy. In this chapter, we focus on four commonly used regression metrics: **RMSE**, **MAE**, R^2, and **MAPE**.

7.5.1 RMSE — Root Mean Squared Error

The **Root Mean Squared Error (RMSE)** measures the typical size of prediction errors, with larger errors receiving greater weight.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}.$$

Because the errors are squared before being averaged, a large prediction error has a greater influence on RMSE than a small prediction error.

RMSE is expressed in the **same units as the outcome variable**. For example, if the outcome is insurance charges measured in dollars, RMSE is also measured in dollars.

A **smaller RMSE** indicates that the model's predictions are, on average, closer to the observed values, with particular sensitivity to large errors.

7.5.2 MAE — Mean Absolute Error

The **Mean Absolute Error (MAE)** measures the average absolute difference between the predicted and observed values.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|.$$

Unlike RMSE, MAE does not give additional weight to large errors. Each error contributes to the measure according to its absolute size.

MAE is also expressed in the same units as the outcome. For example, an MAE of \$4,000 means that the model's predictions differ from the actual charges by **\$4,000 on average in absolute terms**.

A **smaller MAE** indicates smaller prediction errors.

7.5.3 R^2 — Coefficient of Determination

The **coefficient of determination**, denoted by R^2, measures the proportion of variation in the observed outcome that is accounted for by the model.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}.$$

Here,

- y_i is the observed outcome;
- $\hat{y}_i$ is the predicted outcome; and
- $\bar{y}$ is the mean of the observed outcomes.

The denominator represents the total variation in the outcome around its mean, while the numerator represents the variation remaining after using the model's predictions.

For example, an R^2 of 0.73 indicates that **73% of the variation in the observed outcome is accounted for by the model**, according to this measure.

Unlike RMSE and MAE, R^2 does not have the units of the outcome.

7.5.4 MAPE — Mean Absolute Percentage Error

The **Mean Absolute Percentage Error (MAPE)** measures prediction error relative to the observed value.

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

MAPE expresses prediction error as a percentage. For example, a MAPE of 20 indicates an average absolute percentage error of 20%.

However, MAPE has an important limitation: it is **not defined when the observed value y_i is zero**, and it can become very large when y_i is close to zero. Therefore, MAPE should be used with care when the outcome can contain zero or very small values.

7.5.5 Comparing the Metrics

The four metrics provide different perspectives on model performance:

Metric	What it measures	Units	Effect of large errors
RMSE	Typical prediction error	Same as outcome	Greater influence
MAE	Average absolute prediction error	Same as outcome	Proportional
R^2	Proportion of variation accounted for	None	Based on squared errors
MAPE	Average absolute percentage error	Percentage	Can be sensitive to small actual values

These measures should therefore not be viewed as interchangeable. **RMSE and MAE focus directly on prediction error, R^2 describes variation accounted for by the model, and MAPE expresses error relative to the observed outcome.**

7.6 Understanding R^2

The R^2 measure can be understood by comparing two types of variation: the variation in the observed outcomes around their mean and the variation that remains after using the model's predictions.

Suppose we have observed insurance charges $y_1, \ldots, y_n$. If we did not use a regression model, a simple prediction for every policyholder would be the average charge, $\bar{y}$. The total variation in the observed charges can therefore be measured by

$$\text{Total Variation} = \sum_{i=1}^{n}(y_i - \bar{y})^2.$$

When we use a regression model, each observation has a predicted value $\hat{y}_i$. The difference between the observed and predicted values is the prediction error. The remaining, or unexplained, variation is measured by

$$\text{Unexplained Variation} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

The coefficient of determination is then

$$R^2 = 1 - \frac{\text{Unexplained Variation}}{\text{Total Variation}}.$$

This can also be written as

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}.$$

The formula compares the variation remaining after fitting the model with the variation that existed before using the model.

i Understanding R^2

Think of R^2 as describing **how much of the variation in the outcome the model can account for**.
Here, variation is measured in terms of how far the observed values are from their mean, using **squared differences**. For insurance charges, we are therefore considering how much individual charges differ from the average charge.
For example, suppose the total variation in insurance charges is represented by 100 units of squared variation. If the model leaves 27 units unexplained, then

$$R^2 = 1 - \frac{27}{100} = 0.73.$$

Thus, the model accounts for 73 units, or **73% of the variation**. The remaining 27% represents variation that is not accounted for by the model.
In simple terms, the model explains about 73% of the overall differences in insurance charges compared with simply using the average charge for everyone.
Importantly, this does **not** mean that the model predicts 73% of each individual insurance charge correctly. It describes how much of the **overall variation** in charges is accounted for by the model.
Remember that the variation is measured using **squared differences from the mean**, not by counting the percentage of individual predictions that are correct.

An R^2 value of 1 means that all observed values are exactly reproduced by the model, so there is no unexplained variation. An R^2 value of 0 means that the model does not reduce the squared variation compared with using the mean outcome as the prediction.

For example:

- $R^2 = 0$ means that the model accounts for none of the variation relative to the mean-only prediction.
- $R^2 = 0.50$ means that the model accounts for 50% of the variation.
- $R^2 = 0.73$ means that the model accounts for 73% of the variation.
- $R^2 = 1$ means that the model accounts for all of the variation.

However, R^2 should not be interpreted on its own as a measure of prediction accuracy. Two models can have similar R^2 values but different prediction errors. For this reason, measures such as **RMSE** and **MAE** are also useful when evaluating predictive performance.

In the following sections, we will examine these measures more closely and then use the `yardstick` package to calculate them for our insurance predictions.

7.7 Evaluating Model Predictions with `yardstick`

The **`yardstick` package**, which is part of the `tidymodels` ecosystem, provides functions for measuring how well a model's predictions agree with the observed outcomes.

To evaluate a regression model, we need two pieces of information:

- **truth** — the actual observed outcome; and
- **estimate** — the value predicted by the model.

In our insurance data, the actual outcome is `charges`, while the predictions generated by the model are stored in `.pred`.

Suppose we have already fitted a linear regression model using the training data. We can then use the fitted model to generate predictions for the test data. The predictions can be combined with the observed `charges` in a single dataset:

```
# Fit the model using the training data
lm_model <- linear_reg() %>%
  set_engine("lm")

lm_fit <- lm_model %>%
  fit(charges ~ age, data = insurance_train)

# Generate predictions for the test data
insurance_test_results <- insurance_test %>%
  bind_cols(
    predict(lm_fit, new_data = insurance_test)
  )
```

```r
# View the actual and predicted values
insurance_test_results %>%
  select(charges, .pred) %>%
  head()
```

```
# A tibble: 6 x 2
  charges  .pred
    <dbl>  <dbl>
1  11091. 17566.
2  39612. 10612.
3  13229. 18525.
4   4150. 11331.
5  14452. 19244.
6   4688.  8694.
```

The resulting `insurance_test_results` dataset contains both the **actual insurance charges** (`charges`) and the **predicted charges** (`.pred`). These two columns can then be supplied to `yardstick` functions to evaluate the model's predictive performance.

The resulting dataset contains the actual `charges` and the model's `.pred` values.

We can then use `yardstick` to calculate individual performance metrics.

7.7.1 Calculating RMSE

The `rmse()` function calculates the Root Mean Squared Error:

```r
rmse(
  insurance_test_results,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard      10253.
```

The arguments specify which columns contain the actual values and the predictions:

- `truth = charges` identifies the observed insurance charges.
- `estimate = .pred` identifies the model's predictions.

The result gives the RMSE for the model's predictions on the test data.

7.7.2 Calculating MAE

Similarly, we can calculate the Mean Absolute Error using `mae()`:

```
mae(
  insurance_test_results,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 mae     standard       8509.
```

The result represents the average absolute difference between the actual charges and the predicted charges.

7.7.3 Calculating R^2

The `rsq()` function calculates the coefficient of determination:

```
rsq(
  insurance_test_results,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rsq     standard       0.176
```

The resulting value describes the proportion of variation in `charges` accounted for by the model.

7.7.4 Calculating MAPE

We can also calculate the Mean Absolute Percentage Error using `mape()`:

```
mape(
  insurance_test_results,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 mape    standard        135.
```

The result is expressed as a proportion or percentage depending on how it is displayed. As discussed earlier, MAPE should be interpreted with care when observed outcomes are zero or close to zero.

These functions follow the same basic structure:

```
metric(
  data,
  truth = actual,
  estimate = predicted
)
```

This consistent structure makes it straightforward to evaluate different aspects of prediction performance using `yardstick`. In the next section, we will see how several metrics can be calculated together rather than calling each metric separately.

7.8 Computing Multiple Metrics with `metric_set()`

When evaluating a model, it is often useful to examine several performance measures rather than relying on a single metric. For example, RMSE and MAE both measure prediction error, while R^2 provides information about the proportion of variation accounted for by the model.

Calling each metric separately works well for a small number of calculations, but `yardstick` also provides `metric_set()` to combine several metrics into a single set.

We can create a set containing the regression metrics introduced earlier:

```
regression_metrics <- metric_set(
  rmse,
  mae,
  rsq,
  mape
)
```

The object `regression_metrics` can then be applied to the test results in the same way as an individual metric:

```
regression_metrics(
  insurance_test_results,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 4 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   10253.
2 mae     standard    8509.
3 rsq     standard       0.176
4 mape    standard     135.
```

The result reports all four performance measures together.

This is useful because each metric provides different information about the model:

- **RMSE** shows the typical prediction error while giving greater weight to large errors.
- **MAE** shows the average absolute prediction error.
- R^2 describes the proportion of variation in the outcome accounted for by the model.
- **MAPE** describes prediction error relative to the observed outcome.

Using a metric set also provides a convenient way to evaluate and compare models consistently. For example, when we fit another regression model, we can apply the same `regression_metrics` object to its test predictions rather than writing separate calls for each metric.

Reading the `yardstick` Output

The output from `metric_set()` includes three important columns:

- `.metric` identifies the performance measure;
- `.estimator` identifies the method used to calculate the measure; and
- `.estimate` gives the value of the performance measure.

For example:

.metric	.estimator	.estimate
rmse	standard	10,253.22
mae	standard	8,509.31
rsq	standard	0.176
mape	standard	134.92

We can interpret these results as follows:

- **RMSE = 10,253.22**: the model's prediction errors are relatively large, with RMSE placing greater weight on large errors.
- **MAE = 8,509.31**: the model's predictions differ from the actual charges by about \$8,509 on average in absolute terms.
- R^2 **= 0.176**: the model accounts for about **17.6% of the variation** in insurance charges.
- **MAPE = 134.92%**: the average absolute percentage error is about **134.92%**.

The value `"standard"` in `.estimator` indicates the standard calculation used for this regression metric.
The metrics should be interpreted according to what each measure represents rather than compared directly with one another, since they measure different aspects of model performance.

7.9 Interpreting Model Performance

A model's performance should be interpreted by considering the performance measures together. Each metric describes a different aspect of prediction error, so no single measure provides a complete picture of how well a model performs.

For example, suppose the `charges ~ age` model produces the following results on the test data:

Metric	Value
RMSE	10,253.22
MAE	8,509.31
R^2	0.176
MAPE	134.92%

The **RMSE of \$10,253.22** indicates that the model has relatively large prediction errors, with larger errors receiving greater weight. The **MAE of \$8,509.31** indicates that the predictions differ from the actual insurance charges by about \$8,509 on average in absolute terms.

The R^2 **value of 0.176** indicates that the model accounts for about 17.6% of the variation in insurance charges. Thus, age alone accounts for only part of the differences in charges among individuals.

The **MAPE of 134.92%** indicates a large average percentage error. Percentage-based measures should be interpreted carefully because they can become large when actual outcome values are relatively small.

These results suggest that the `charges ~ age` model does not provide highly accurate individual predictions. However, this does not mean that the model is useless. A simple model with one predictor can be useful for demonstrating the modelling process and for understanding the relationship between age and insurance charges. Prediction performance may improve when additional relevant predictors are included.

> **Important:** Performance measures should be interpreted in the context of the modelling objective and the scale of the outcome. A metric value is not meaningful by itself without considering what is being predicted and how the model will be used.

When comparing models, the same performance measures should be calculated on the **same test data**. This allows us to determine whether a more complex model provides better predictive performance than a simpler model.

In the next section, we will compare two regression models using the same test data and performance measures.

7.10 Comparing Two Models

Model performance becomes more useful when we compare different models for the same prediction task. Suppose we want to predict insurance charges using age. We can first consider a simple model using only `age`, and then examine whether adding `bmi` improves predictive performance.

The two models are

$$\text{Model 1:} \qquad \text{charges} = \beta_0 + \beta_1 \text{age} + \varepsilon$$

and

$$\text{Model 2:} \qquad \text{charges} = \beta_0 + \beta_1 \text{age} + \beta_2 \text{bmi} + \varepsilon.$$

Both models are fitted using the same training data. We then use each model to generate predictions for the same test data.

```
# Model 1: age only
model_age <- linear_reg() %>%
  set_engine("lm") %>%
  fit(charges ~ age, data = insurance_train)

# Model 2: age and BMI
model_age_bmi <- linear_reg() %>%
  set_engine("lm") %>%
  fit(charges ~ age + bmi, data = insurance_train)
```

We can generate predictions for the test data:

```
test_age <- insurance_test %>%
  bind_cols(
    predict(model_age, new_data = insurance_test)
  )

test_age_bmi <- insurance_test %>%
```

```
  bind_cols(
    predict(model_age_bmi, new_data = insurance_test)
  )
```

We can then use the same `regression_metrics` object introduced earlier to evaluate both models:

```
regression_metrics(
  test_age,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 4 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   10253.
2 mae     standard    8509.
3 rsq     standard       0.176
4 mape    standard     135.
```

```
regression_metrics(
  test_age_bmi,
  truth = charges,
  estimate = .pred
)
```

```
# A tibble: 4 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   10157.
2 mae     standard    8351.
3 rsq     standard       0.178
4 mape    standard     132.
```

To make the comparison easier, we can combine the results into a single table:

```
model_comparison <- bind_rows(
  regression_metrics(
    test_age,
    truth = charges,
    estimate = .pred
  ) %>%
    mutate(model = "Age only"),

  regression_metrics(
```

```
    test_age_bmi,
    truth = charges,
    estimate = .pred
  ) %>%
    mutate(model = "Age + BMI")
)

model_comparison %>%
  select(model, .metric, .estimate)
```

```
# A tibble: 8 x 3
  model     .metric .estimate
  <chr>     <chr>       <dbl>
1 Age only  rmse    10253.
2 Age only  mae      8509.
3 Age only  rsq         0.176
4 Age only  mape      135.
5 Age + BMI rmse    10157.
6 Age + BMI mae      8351.
7 Age + BMI rsq         0.178
8 Age + BMI mape      132.
```

The resulting table allows us to compare the predictive performance of the two models on the same test data. The model using `age + bmi` has a slightly lower RMSE and MAE than the model using `age` alone. Its R^2 is also slightly higher, while its MAPE is lower.

Metric	Age only	Age + BMI
RMSE	10,253.22	10,156.70
MAE	8,509.31	8,351.08
R^2	0.176	0.178
MAPE	134.92%	132.23%

These results show that adding `bmi` produces a **small improvement in predictive performance** on the test data. The RMSE decreases from \$10,253.22 to \$10,156.70, while the MAE decreases from \$8,509.31 to \$8,351.08. The R^2 increases from 0.176 to 0.178, indicating that the model accounts for a slightly greater proportion of the variation in insurance charges. MAPE also decreases from 134.92% to 132.23%.

Although the second model performs better according to these measures, the differences are relatively small. This illustrates an important point: **adding a predictor does not necessarily produce a substantial improvement in predictive performance**. The usefulness of an additional predictor should therefore be assessed using model performance on data that were not used to fit the model.

> **Key idea:** When comparing predictive models, use the **same test data** and the **same performance measures** for each model. This provides a consistent basis for evaluating whether a change in the model improves its predictions.

This comparison also prepares us for the next chapter, where we will extend the linear regression model to include several predictors simultaneously. Before doing so, it is important to understand why performance on test data is more informative about prediction on new observations than performance on the training data alone.

In the next section, we examine **why test performance matters**.

7.11 Why Test Performance Matters

A model is fitted using the training data, so it is natural for the model to perform well on those observations. However, good performance on the training data does not necessarily mean that the model will make accurate predictions for new observations.

The purpose of the test data is to provide an independent assessment of predictive performance. Because the test observations were not used to fit the model, their outcomes provide information about how well the model generalizes beyond the data used for fitting.

For example, suppose a model has an RMSE of \$8,000 on the training data but an RMSE of \$12,000 on the test data. The difference indicates that the model performs better on the data used to fit it than on previously unseen observations.

This distinction can be illustrated as follows:

Data	Purpose	Performance
Training data	Fit the model	Training performance
Test data	Evaluate predictions on unseen observations	Test performance

Training performance is still useful for understanding how well the model fits the data. However, when the objective is prediction, **test performance is more informative about how the model may perform on new data**.

A large difference between training and test performance can indicate that the model has fitted patterns in the training data that do not generalize well to new observations. This issue is commonly referred to as **overfitting**.

> **Key idea:** A predictive model should not be judged only by how well it fits the data used to build it. Test data provide a more independent assessment of how well the model can generalize to new observations.

The separation between training and test data should therefore be maintained throughout the modelling process. The training data are used to fit the model, while the test data are kept separate until the model is ready to be evaluated.

The next section brings these ideas together by considering how model evaluation fits into the overall modelling workflow.

7.12 From Model Evaluation to a Modelling Workflow

Model evaluation is not a separate step that is performed only after a model has been built. It is part of an overall modelling workflow. A useful predictive modelling process can be summarized as

$$\text{Explore} \rightarrow \text{Prepare} \rightarrow \text{Fit} \rightarrow \text{Predict} \rightarrow \text{Evaluate} \rightarrow \text{Compare}.$$

First, we **explore** the data to understand the variables, distributions, relationships, and unusual observations. We then **prepare** the data for modelling. The model is fitted using the training data, and predictions are generated for observations that were not used to fit the model.

The predictions are then **evaluated** using appropriate performance measures. For regression models, measures such as RMSE, MAE, R^2, and MAPE provide different information about prediction performance. When several models are available, their performance can be **compared** using the same test data and performance measures.

For the insurance example in this chapter, the process can be represented as follows:

Step	Example
Explore	Examine `charges`, `age`, and `bmi`
Prepare	Split the data into training and test sets
Fit	Fit a regression model using the training data
Predict	Predict `charges` for the test data
Evaluate	Calculate RMSE, MAE, R^2, and MAPE
Compare	Compare the performance of different models

The workflow is iterative. Evaluation may show that a model does not provide sufficiently accurate predictions, or that adding another predictor produces only a small improvement. We can then return to the modelling process, consider alternative predictors or model specifications, and evaluate the revised model again.

> **Key idea:** Model evaluation helps us determine whether a model provides useful predictions on data that were not used to fit it. It is therefore an essential part of the modelling process, not simply a final calculation.

In this chapter, we focused on evaluating the performance of regression models using test data. We examined prediction errors, introduced RMSE, MAE, R^2, and MAPE, used `yardstick` to calculate these measures, and compared the predictive performance of two models.

The models considered so far have used only one or two predictors. In the next chapter, we extend linear regression to **multiple regression**, where several predictors can be used simultaneously to explain and predict an insurance outcome.

8 Chapter 8: Preprocessing and Recipes

8.1 Introduction to Preprocessing and Recipes

Before fitting a predictive model, we need to ensure that the data are suitable for modelling. Preprocessing involves preparing the predictors and outcome so that they can be used consistently by a modelling algorithm. Common tasks include handling missing values, encoding categorical predictors, and scaling numeric predictors. Depending on the modelling problem, preprocessing may also involve creating new features, such as interaction terms or polynomial terms.

The preprocessing requirements depend on the model being fitted. For example, ridge regression generally benefits from scaling numeric predictors, whereas regression trees and random forests generally do not require scaling. Many modelling engines also require categorical predictors to be represented numerically.

In this chapter, we use the `recipes` package from the `tidymodels` framework to define preprocessing steps. A recipe describes a sequence of transformations that can be applied consistently during model fitting and prediction. Rather than manually transforming the training and test datasets separately, we specify the required steps in a recipe and allow tidymodels to manage their application.

8.1.1 The Insurance Dataset

We will use the `insurance` dataset introduced in the previous chapter on regression modelling. It contains information about insurance policyholders and their medical insurance charges. The response variable is `charges`, while the remaining variables are potential predictors.

Variable	Description	Type
`age`	Age of the policyholder	Numeric
`sex`	Sex of the policyholder	Categorical
`bmi`	Body mass index	Numeric
`children`	Number of children covered by the insurance	Numeric count
`smoker`	Smoking status	Categorical
`region`	Residential region	Categorical
`charges`	Medical insurance charges	Numeric outcome

If you are continuing from the previous chapter, the `insurance` dataset may already be available in your R environment. Otherwise, load it from the project's `data` folder:

```
library(tidyverse)
library(tidymodels)

insurance <- read_csv("../data/insuranceKaggle.csv")
```

The file path assumes that the CSV file is stored in the project's `data` folder. Adjust the path if your project uses a different directory structure.

We can inspect the dataset before defining a recipe:

```
glimpse(insurance)
```

```
Rows: 1,338
Columns: 7
$ age      <dbl> 19, 18, 28, 33, 32, 31, 46, 37, 37, 60, 25, 62, 23, 56, 27, 1~
$ sex      <chr> "female", "male", "male", "male", "male", "female", "female",~
$ bmi      <dbl> 27.900, 33.770, 33.000, 22.705, 28.880, 25.740, 33.440, 27.74~
$ children <dbl> 0, 1, 3, 0, 0, 0, 1, 3, 2, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0~
$ smoker   <chr> "yes", "no", "no", "no", "no", "no", "no", "no", "no", "no", ~
$ region   <chr> "southwest", "southeast", "southeast", "northwest", "northwes~
$ charges  <dbl> 16884.924, 1725.552, 4449.462, 21984.471, 3866.855, 3756.622,~
```

The first step in preprocessing is to understand the data and identify any problems that need to be addressed. For example, we may need to investigate duplicate observations or missing values before fitting a model. We should not automatically remove observations or impute values without first considering the nature of the data.

8.1.2 Why Use Recipes?

The `recipes` package provides a structured way to specify preprocessing operations. Each operation is represented by a step, such as `step_impute_median()` for imputing missing numeric values, `step_dummy()` for creating dummy variables, or `step_normalize()` for standardizing numeric predictors.

A recipe separates the **specification of preprocessing steps** from their estimation and application. The `prep()` function estimates quantities required by the steps using the training data, while `bake()` applies the prepared transformations to data. This distinction is particularly important when evaluating predictive models: preprocessing parameters must be estimated using the training data only and then applied consistently to the test data.

Recipes also integrate with the `workflows` package, which allows preprocessing steps and a model specification to be combined into a single object. This makes the modelling process easier to reproduce and helps prevent inconsistencies between the data used for training and the data used for prediction.

In the following sections, we will develop recipes for handling missing values, encoding categorical predictors, and normalizing numeric predictors. We will then combine these steps into a complete recipe and use it in a tidymodels workflow.

8.2 Preparing Data Before Creating a Recipe

Before defining preprocessing steps, we should inspect the dataset and identify data-quality issues that need to be addressed. Two important checks are duplicate observations and missing values.

Some data-cleaning decisions, such as removing erroneous duplicate records, are made before creating a recipe. Other operations, such as imputing missing values, can be specified within a recipe so that they are estimated using the training data and applied consistently to new data.

8.2.1 Removing Duplicate Observations

Duplicate observations may arise when data are combined from multiple sources or when records are accidentally entered more than once. If duplicate records are included unintentionally, they can give certain observations excessive weight in model fitting.

We can check for completely duplicated rows using `duplicated()`:

```
duplicated_rows <- insurance[duplicated(insurance), ]
duplicated_rows
```

```
# A tibble: 1 x 7
    age sex     bmi children smoker region    charges
  <dbl> <chr> <dbl>    <dbl> <chr>  <chr>       <dbl>
1    19 male   30.6        0 no     northwest   1640.
```

The function `duplicated()` returns `TRUE` for each row that duplicates an earlier row. The first occurrence is not marked as a duplicate.

Alternatively, we can use `dplyr` to inspect the duplicated rows:

```
library(dplyr)

insurance |>
  filter(duplicated(insurance))
```

```
# A tibble: 1 x 7
    age sex     bmi children smoker region    charges
  <dbl> <chr> <dbl>    <dbl> <chr>  <chr>       <dbl>
1    19 male   30.6        0 no     northwest   1640.
```

If the duplicated records are confirmed to be erroneous, we can remove them using `distinct()`:

```
insurance <- insurance |>
  distinct()
```

The function `distinct()` retains unique rows by default, keeping the first occurrence of each completely duplicated row.

It is important to distinguish exact duplicate rows from records that share some characteristics. For example, two policyholders may have the same age, BMI, smoking status, and insurance charges without representing duplicate records. Duplicate removal should therefore be based on an understanding of the data, rather than on the assumption that similar observations are necessarily errors.

Note: Removing duplicate observations is a data-cleaning operation rather than a standard recipe step. It should be handled separately before creating a recipe when the records are confirmed to be duplicates. In addition, if the dataset contains a unique identifier, checking duplicates using that identifier may be more appropriate than checking entire rows.

8.2.2 Checking for Missing Values

Missing values can prevent some modelling algorithms from fitting a model or require additional handling during preprocessing. Before deciding how to handle missing values, we should determine which variables contain missing observations and how many values are missing.

We can count missing values in each column using base R:

```
colSums(is.na(insurance))
```

```
     age      sex      bmi children   smoker   region  charges
       0        0        0        0        0        0        0
```

Alternatively, `dplyr` provides a convenient way to summarize missing values across all columns:

```
insurance |>
  summarise(
    across(everything(), ~ sum(is.na(.)))
  )
```

```
# A tibble: 1 x 7
    age   sex   bmi children smoker region charges
  <int> <int> <int>    <int>  <int>  <int>   <int>
1     0     0     0        0      0      0       0
```

The expression `across(everything(), ...)` applies the specified function to every column. Within each column, `is.na(.)` identifies missing values, and `sum()` counts them.

To examine the percentage of missing values in each column, we can use:

```
insurance |>
  summarise(
    across(everything(), ~ mean(is.na(.)) * 100)
  )
```

```
# A tibble: 1 x 7
    age   sex   bmi children smoker region charges
  <dbl> <dbl> <dbl>    <dbl>  <dbl>  <dbl>   <dbl>
1     0     0     0        0      0      0       0
```

Because `TRUE` is treated as 1 and `FALSE` as 0 in arithmetic operations, the mean of `is.na(.)` gives the proportion of missing values. Multiplying by 100 converts the proportion to a percentage.

After identifying missing values, we must decide how to handle them based on the variable type, the amount of missing data, and the modelling objective. Common approaches include removing observations when appropriate, imputing numeric values with the mean or median, and imputing categorical values with the most frequent category or a separate missing category.

In this chapter, we will use the `recipes` package to implement imputation. This allows the required quantities, such as a training-set median or most frequent category, to be estimated from the training data and then applied consistently to test data and future observations.

The next section introduces how to define a recipe and how to prepare and apply it.

8.3 Creating and Understanding Recipes

The `recipes` package allows us to specify preprocessing steps in a structured and reproducible way. A recipe describes how the data should be transformed before model fitting, including operations such as imputing missing values, encoding categorical predictors, and normalizing numeric predictors.

8.3.1 Specifying a Recipe

We begin by defining a recipe for the `insurance` dataset. Recall that `charges` is the response variable, while `age`, `bmi`, `children`, `sex`, `smoker`, and `region` are potential predictors.

```
insurance_recipe <- recipe(
  charges ~ age + bmi + children + sex + smoker + region,
  data = insurance
)

insurance_recipe
```

The `recipe()` function specifies the relationship between the outcome and predictors, as well as the data used to define the recipe. At this stage, no preprocessing has been performed.

The formula follows the same general structure used in linear regression:

$$\text{response} \sim \text{predictors}.$$

For example, `charges ~ age + bmi` specifies `charges` as the outcome and `age` and `bmi` as predictors. The formula `charges ~ .` can be used when all remaining variables in the dataset should

be predictors. However, explicitly listing the variables can make the intended model easier to understand.

A recipe can also include transformations and interactions in its formula. For example, `age * bmi` represents the main effects of `age` and `bmi`, together with their interaction. In this chapter, we will focus on preprocessing steps rather than extending the regression formula.

8.3.2 Understanding `prep()` and `bake()`

After specifying a recipe, we need to prepare it before applying its transformations. The `prep()` function estimates the quantities required by the preprocessing steps, using the training data. For example, it can estimate the median used for imputing missing numeric values or the mean and standard deviation used for normalization.

The `bake()` function applies the prepared recipe to a dataset. It uses the quantities learned during preparation to perform the specified transformations.

Consider a simple recipe that imputes missing numeric predictors with their medians and missing categorical predictors with their most frequent categories:

```
insurance_recipe <- recipe(
  charges ~ .,
  data = insurance
) |>
  step_impute_median(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors())
```

The code specifies the preprocessing operations, but the imputation values have not yet been estimated. We prepare the recipe and apply it to the training data as follows:

```
insurance_prep <- prep(
  insurance_recipe,
  training = insurance
)

insurance_baked <- bake(
  insurance_prep,
  new_data = NULL
)
```

Here, `prep()` estimates the imputation values from `insurance`. The argument `new_data = NULL` tells `bake()` to apply the prepared recipe to the training data used during preparation.

In a predictive modelling exercise, we would instead prepare the recipe using the training dataset and then apply that same prepared recipe to both the training and test datasets. This ensures that the imputation values and other preprocessing parameters are not estimated from the test data.

> **Note**
>
> **The roles of `recipe()`, `prep()`, and `bake()`**
>
> - `recipe()` specifies the preprocessing steps to be performed.
> - `prep()` estimates and stores the quantities required by those steps using the training data.
> - `bake()` applies the prepared transformations to a dataset.
>
> The key distinction is that **a recipe defines the preprocessing plan, `prep()` learns from the training data, and `bake()` applies what has been learned**.

8.3.3 Selecting Variables in Recipes

Recipes often apply different transformations to different types of variables. For example, numeric predictors may require imputation or normalization, whereas categorical predictors may require imputation or dummy encoding.

Selectors allow us to choose the variables to which a preprocessing step applies. Common selectors include:

Selector	Variables selected
`all_predictors()`	All predictor variables
`all_outcomes()`	All outcome variables
`all_numeric_predictors()`	Numeric predictors
`all_nominal_predictors()`	Nominal or categorical predictors
`starts_with("...")`	Variables whose names begin with a specified string
`ends_with("...")`	Variables whose names end with a specified string
`contains("...")`	Variables whose names contain a specified string
`matches("...")`	Variables whose names match a regular expression

For example:

```
recipe(charges ~ ., data = insurance) |>
  step_impute_median(all_numeric_predictors())
```

The selector `all_numeric_predictors()` ensures that median imputation is applied to numeric predictors rather than to every numeric column. In particular, the outcome `charges` is excluded because it is not a predictor.

Selectors make recipes more flexible and reduce the need to list each variable individually. They are especially useful when the same preprocessing operations need to be applied to several predictors.

Tip

Example: Selecting Variables by Name
The `starts_with()`, `ends_with()`, `contains()`, and `matches()` selectors can select predictors based on their variable names. For example, suppose a dataset contains variables such as `age_policyholder`, `age_spouse`, `region_code`, and `smoker_status`. We can use these selectors in a recipe as follows:

```
recipe(charges ~ ., data = insurance) |>
  step_normalize(starts_with("age"))
```

This selects numeric predictors whose names begin with `"age"`.
Other examples include:

```
all_nominal_predictors() |>
  # Not used as a selector pipeline; see examples below
```

```
step_dummy(contains("region"))
```

This selects categorical predictors whose names contain `"region"` for dummy encoding.

```
step_normalize(ends_with("_score"))
```

This selects numeric predictors whose names end with `"_score"`.

```
step_normalize(matches("^age|bmi$"))
```

This selects predictors whose names match the regular expression: names beginning with `"age"` or ending with `"bmi"`.
These selectors are useful when datasets contain many variables with systematic naming conventions. However, the selected variables must be appropriate for the preprocessing step. For example, `step_normalize()` should be applied to numeric predictors, whereas `step_dummy()` is intended for categorical predictors.

In the following sections, we will examine specific preprocessing steps in more detail, beginning with imputation for missing values.

8.4 Handling Missing Values

Missing values are common in insurance data. For example, a policyholder's BMI or smoking status may not have been recorded. Many modelling algorithms cannot handle missing values directly, so these observations must be addressed before model fitting.

A common approach is **imputation**, which replaces missing values with estimated or specified values. The appropriate method depends on the variable and the modelling objective. For example,

numeric variables may be imputed using the mean or median, while categorical variables may be imputed using the most frequent category.

In this section, we use the `recipes` package to perform imputation. An important advantage is that imputation values are estimated from the training data and then applied consistently to new data.

8.4.1 Creating a Dataset with Missing Values

To demonstrate imputation, we create a small synthetic dataset containing missing values in numeric and categorical predictors. The dataset resembles the insurance data used in this course but is created specifically for illustration.

```
insurance_demo <- tibble(
  age = c(25, 32, NA, 45, 52, 29, 40, NA),
  bmi = c(22.5, 27.8, 31.2, NA, 26.5, 24.1, NA, 28.3),
  smoker = factor(
    c("yes", "no", "no", "yes", NA, "no", "yes", "no"),
    levels = c("no", "yes")
  ),
  region = factor(
    c(
      "northwest", "southeast", "southwest", NA,
      "northeast", "southwest", "northwest", "southeast"
    ),
    levels = c("northeast", "northwest", "southeast", "southwest")
  ),
  charges = c(3200, 4100, 3900, 5800, 6000, 4500, 5200, 4300)
)

insurance_demo
```

```
# A tibble: 8 x 5
    age   bmi smoker region    charges
  <dbl> <dbl> <fct>  <fct>       <dbl>
1    25  22.5 yes    northwest    3200
2    32  27.8 no     southeast    4100
3    NA  31.2 no     southwest    3900
4    45  NA   yes    <NA>         5800
5    52  26.5 <NA>   northeast    6000
6    29  24.1 no     southwest    4500
7    40  NA   yes    northwest    5200
8    NA  28.3 no     southeast    4300
```

The dataset contains four predictors: `age` and `bmi` are numeric, while `smoker` and `region` are categorical. The response variable is `charges`.

We can count the missing values in each variable:

```
insurance_demo |>
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
# A tibble: 1 x 5
    age   bmi smoker region charges
  <int> <int>  <int>  <int>   <int>
1     2     2      1      1       0
```

This confirms that some predictors contain missing values. In this example, we will use mean imputation for numeric predictors and mode imputation for categorical predictors.

8.4.2 Specifying Imputation Steps in a Recipe

The `recipes` package provides several imputation steps. Two useful functions are:

- `step_impute_mean()` replaces missing numeric values with the mean estimated from the training data.
- `step_impute_mode()` replaces missing categorical values with the most frequent category estimated from the training data.

We specify these steps in a recipe as follows:

```
rec_demo <- recipe(
  charges ~ age + bmi + smoker + region,
  data = insurance_demo
) |>
  step_impute_mean(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors())

rec_demo
```

Recipe

Inputs
Number of variables by role

- outcome: 1
- predictor: 4

Operations

- Mean imputation for: `all_numeric_predictors()`
- Mode imputation for: `all_nominal_predictors()`

The recipe specifies the preprocessing operations but has not yet estimated the imputation values or changed the data. The selectors `all_numeric_predictors()` and `all_nominal_predictors()` ensure that each imputation step is applied to the appropriate type of predictor.

Notice that `charges` is the outcome and is not included in either imputation step. We are imputing the predictors used to model charges, not the response variable.

8.4.3 Preparing the Recipe with `prep()`

The next step is to prepare the recipe. The `prep()` function estimates the quantities required by each preprocessing step using the training data.

```
rec_prep <- prep(
  rec_demo,
  training = insurance_demo
)

rec_prep
```

Prepared Recipe

Inputs
Number of variables by role

- outcome: 1
- predictor: 4

Training information
Training data contained 8 data points and 5 incomplete rows.
Operations

- Mean imputation for: `age`, `bmi` — *Trained*
- Mode imputation for: `smoker`, `region` — *Trained*

The prepared recipe records the information learned from the training data. In this example, it estimates the means for `age` and `bmi` and the most frequent categories for `smoker` and `region`.

We can inspect the actual values learned by each step using `tidy()`:

```
tidy(rec_prep, number = 1)
```

```
# A tibble: 2 x 3
  terms value id
  <chr> <dbl> <chr>
1 age    37.2 impute_mean_eUPFX
2 bmi    26.7 impute_mean_eUPFX
```

```
tidy(rec_prep, number = 2)
```

```
# A tibble: 2 x 3
  terms  value     id
  <chr>  <chr>     <chr>
1 smoker no        impute_mode_83cEs
2 region southwest impute_mode_83cEs
```

The first call displays the mean values learned by the numeric imputation step, while the second displays the modes learned by the categorical imputation step.

For example, the observed values of `age` are 25, 32, 45, 52, 29, and 40. Their mean is approximately 37.17. Thus, the missing `age` values will be replaced with this estimated mean.

For `smoker`, the category `"no"` occurs more frequently than `"yes"` among the observed values. Therefore, missing values of `smoker` will be replaced with `"no"`.

This illustrates that `prep()` estimates and stores the values needed for imputation; it does not itself return the final imputed dataset.

8.4.4 Applying Imputation with `bake()`

Once the recipe has been prepared, we use `bake()` to apply the learned transformations.

```
insurance_baked <- bake(
  rec_prep,
  new_data = NULL
)

insurance_baked
```

```
# A tibble: 8 x 5
    age   bmi smoker region    charges
  <dbl> <dbl> <fct>  <fct>       <dbl>
1  25    22.5 yes    northwest    3200
2  32    27.8 no     southeast    4100
3  37.2  31.2 no     southwest    3900
4  45    26.7 yes    southwest    5800
5  52    26.5 no     northeast    6000
6  29    24.1 no     southwest    4500
7  40    26.7 yes    northwest    5200
8  37.2  28.3 no     southeast    4300
```

The resulting dataset contains imputed values in place of the missing predictor values. For example, the missing `age` values are replaced with the mean estimated from the observed ages, and the missing `smoker` value is replaced with the mode.

Why do we use `new_data = NULL`?

When `new_data = NULL`, `bake()` applies the prepared recipe to the **training data used in `prep()`**. This is useful when we want to inspect the training data after preprocessing.
If we have separate training and test datasets, we should apply the same prepared recipe to both datasets:

```
insurance_train_baked <- bake(rec_prep, new_data = insurance_train)
insurance_test_baked  <- bake(rec_prep, new_data = insurance_test)
```

The recipe is prepared using the training data only. The same imputation values learned from the training data are then applied to the test data, preventing information from the test data from leaking into the preprocessing process.

We can verify that no missing values remain in the predictors:

```
insurance_baked |>
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
# A tibble: 1 x 5
    age   bmi smoker region charges
  <int> <int>  <int>  <int>   <int>
1     0     0      0      0       0
```

In this example, `new_data = NULL` tells `bake()` to process the training data used when preparing the recipe. If we had a separate test dataset, we would pass it to `bake()` instead.

8.4.5 Applying the Same Recipe to Test Data

In predictive modelling, preprocessing must be estimated using the training data only. The prepared recipe can then be applied to the test data without recalculating the imputation values.

For illustration, we split the synthetic dataset into training and test observations:

```
set.seed(124)

demo_split <- initial_split(
  insurance_demo,
  prop = 0.75
)

demo_train <- training(demo_split)
demo_test <- testing(demo_split)
```

We then prepare the recipe using only the training data and apply it to the test data:

```
rec_train_prep <- prep(
  rec_demo,
  training = demo_train
)

demo_test_baked <- bake(
  rec_train_prep,
  new_data = demo_test
)

demo_test_baked
```

```
# A tibble: 2 x 5
    age   bmi smoker region    charges
  <dbl> <dbl> <fct>  <fct>       <dbl>
1    45  26.8 yes    southeast    5800
2    52  26.5 no     northeast    6000
```

The imputation values used for the test observations come from `demo_train`, not from `demo_test`. This is essential for a fair assessment of predictive performance because the test data must not influence the preprocessing parameters.

Applying the Recipe to Test Data

Before preprocessing, `demo_test` contains missing values in three predictors: `bmi`, `smoker`, and `region`.

```
demo_test
```

```
# A tibble: 2 x 5
    age   bmi smoker region    charges
  <dbl> <dbl> <fct>  <fct>       <dbl>
1    45  NA   yes    <NA>         5800
2    52  26.5 <NA>   northeast    6000
```

The prepared recipe replaces these missing values using the imputation values learned from the training data. Numeric predictors are imputed using their training-data means, while categorical predictors are imputed using their training-data modes. The observed values remain unchanged, and `charges` is retained without imputation because it is the outcome variable.

In practice, when preprocessing is included in a tidymodels workflow, the workflow manages these steps during model fitting and prediction. We will introduce workflows later in this chapter.

8.5 Preprocessing Numeric Predictors

Numeric predictors can have very different scales. In the insurance dataset, for example, `age` is measured in years, `bmi` typically takes values around 15–50, and `children` is a small nonnegative integer. Other datasets may contain predictors measured in thousands or millions.

Differences in scale are not necessarily a problem for every statistical model. However, some modelling methods are sensitive to the scale of the predictors. Before fitting these models, it may be necessary to transform numeric predictors so that they are on comparable scales.

8.5.1 Why Normalize Numeric Predictors?

One common preprocessing technique is **normalization**, also called *standardization*. It transforms a numeric variable by subtracting its mean and dividing by its standard deviation:

$$x^* = \frac{x - \bar{x}}{s_x},$$

where x is the original value, $\bar{x}$ is the sample mean, and s_x is the sample standard deviation.

After normalization, the variable has mean approximately zero and standard deviation approximately one, provided its standard deviation is nonzero. The transformation changes the scale of a variable but preserves the relative ordering of its observations.

For example, age and BMI are measured in different units. Normalization puts both variables on a comparable scale, making the numerical size of their values less dependent on their original units.

The main motivation is that **some models use the numerical scale of predictors when estimating their parameters**. If predictors have very different scales, a model may assign disproportionate importance to variables because of their units or may be harder to fit reliably. Normalization can address these issues for models that are sensitive to predictor scale.

8.5.2 Which Models Need Normalization?

Normalization is particularly important for methods that penalize model coefficients or rely on distances between observations.

- **Ridge and lasso regression:** These methods penalize the sizes of regression coefficients. Without normalization, predictors measured on different scales can be penalized unevenly, making the penalty depend partly on their units rather than solely on their predictive contribution. Normalization helps ensure that the regularization penalty treats predictors more comparably.
- **Distance-based methods:** Methods such as k-nearest neighbors use distances between observations. A predictor with a much larger numerical range can dominate these distances if the predictors are not appropriately scaled.

- **Ordinary least squares regression:** Normalization is generally not required to obtain fitted values from a standard linear regression model. However, scaling may still be useful for numerical stability or when comparing coefficient magnitudes, provided the coefficients are interpreted on the transformed scale.
- **Regression trees and random forests:** These methods generally do not require normalization. Their splits depend on the ordering of predictor values rather than the numerical magnitude of the units. A simple change of scale, such as standardization, usually does not change the possible splits or the resulting predictions.

> **!** Important
>
> Normalization is not automatically necessary for every model. Its usefulness depends on the modelling method. In particular, it is important for ridge and lasso regression, whereas tree-based models generally work well with numeric predictors on their original scales.

8.5.3 Normalizing Predictors with `step_normalize()`

In `recipes`, the function `step_normalize()` adds normalization to a preprocessing recipe. The selector `all_numeric_predictors()` applies the step to all numeric predictors while excluding the outcome variable.

```
library(tidymodels)

insurance_recipe <- recipe(charges ~ ., data = insurance) |>
  step_normalize(all_numeric_predictors())

insurance_recipe
```

The recipe specifies the preprocessing operation but does not immediately calculate the means and standard deviations. These quantities are estimated when the recipe is prepared using the training data.

```
insurance_prep <- prep(
  insurance_recipe,
  training = insurance
)

insurance_normalized <- bake(
  insurance_prep,
  new_data = NULL
)

insurance_normalized |>
  select(age, bmi, children, charges) |>
  summary()
```

```
      age                bmi               children          charges
 Min.   :-1.51108   Min.   :-2.41022   Min.   :-0.90889   Min.   : 1122
 1st Qu.:-0.87025   1st Qu.:-0.71690   1st Qu.:-0.90889   1st Qu.: 4746
 Median :-0.01582   Median :-0.04319   Median :-0.07941   Median : 9386
 Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.00000   Mean   :13279
 3rd Qu.: 0.83862   3rd Qu.: 0.66168   3rd Qu.: 0.75007   3rd Qu.:16658
 Max.   : 1.76426   Max.   : 3.68276   Max.   : 3.23852   Max.   :63770
```

The resulting data contain normalized values for the numeric predictors `age`, `bmi`, and `children`. The outcome `charges` remains on its original scale because it is not a predictor. Categorical predictors such as `sex`, `smoker`, and `region` are also unchanged by this step.

8.5.4 Why Must Normalization Be Learned from the Training Data?

The means and standard deviations used for normalization must be calculated from the training data and then applied to new observations. This is handled by `prep()` and `bake()`.

```
set.seed(123)

insurance_split <- initial_split(insurance, prop = 0.75)
insurance_train <- training(insurance_split)
insurance_test <- testing(insurance_split)

insurance_recipe <- recipe(charges ~ ., data = insurance_train) |>
  step_normalize(all_numeric_predictors())

insurance_prep <- prep(
  insurance_recipe,
  training = insurance_train
)

train_processed <- bake(insurance_prep, new_data = NULL)
test_processed <- bake(insurance_prep, new_data = insurance_test)
```

The recipe estimates the normalization parameters using `insurance_train`. The same parameters are then applied to both the training and test data. Consequently, the test data do not influence the preprocessing parameters before model evaluation.

If normalization were calculated separately for the test data, its values would be transformed using different means and standard deviations. This would make the training and test data inconsistent and could introduce information from the test set into the modelling process.

> **Note**
>
> In a predictive modelling workflow, preprocessing steps must be learned from the training data and then applied consistently to new data. This principle helps prevent data leakage and supports a fair assessment of predictive performance.

Normalization is one example of numeric preprocessing. Other transformations may be appropriate when predictors are highly skewed, contain extreme values, or require a different representation. The appropriate preprocessing steps should be selected according to the data and the modelling method rather than applied automatically to every model.

8.6 Preprocessing Categorical Predictors

Categorical predictors, such as `smoker` and `region` in the `insurance` dataset, represent groups rather than numerical quantities. Before fitting a model, we need to consider how the modeling algorithm handles these variables.

Some algorithms, including ordinary linear regression implemented by `lm`, require categorical predictors to be represented numerically. Other algorithms, such as many tree-based implementations, can handle categorical predictors directly. Therefore, dummy encoding is not always necessary; the appropriate preprocessing depends on the model and its requirements.

8.6.1 Creating Dummy Variables with `step_dummy()`

A common approach is to convert categorical predictors into **dummy variables**, also called indicator variables. Each dummy variable takes the value 1 when an observation belongs to a particular category and 0 otherwise.

For example, the `smoker` variable has two levels, `yes` and `no`. With the default treatment contrast, `step_dummy()` creates one dummy variable, typically `smoker_yes`, which equals 1 for smokers and 0 for non-smokers. The non-smoker category serves as the reference group.

We first split the data into training and testing sets, then define a recipe using the training data.

```
library(tidymodels)

set.seed(123)
insurance_split <- initial_split(insurance, prop = 0.8)

insurance_train <- training(insurance_split)
insurance_test  <- testing(insurance_split)

insurance_recipe <- recipe(
  charges ~ age + bmi + smoker,
  data = insurance_train
) %>%
  step_dummy(all_nominal_predictors())
```

The selector `all_nominal_predictors()` identifies categorical predictors in the recipe. Consequently, `step_dummy()` converts `smoker` into a numeric dummy variable while leaving `age` and `bmi` unchanged.

We then prepare the recipe using the training data.

```
insurance_prepped <- prep(
  insurance_recipe,
  training = insurance_train
)
```

The `prep()` function learns the information needed to apply the preprocessing steps, including the categorical levels observed in the training data. It does not fit the predictive model; it prepares the recipe for subsequent data transformation.

We can apply the prepared recipe to both datasets:

```
insurance_train_processed <- bake(
  insurance_prepped,
  new_data = insurance_train
)

insurance_test_processed <- bake(
  insurance_prepped,
  new_data = insurance_test
)
```

Both datasets are transformed using the same learned preprocessing rules. This consistency is important because a predictive model must receive predictors with the same structure during training and prediction.

8.6.2 Handling New Categorical Levels

A categorical predictor may contain a level in new data that was not observed when the recipe was prepared. For example, suppose the training data contain both `yes` and `no` in `smoker`, but a new dataset contains a previously unseen level, `unknown`.

By default, `step_dummy()` does not automatically create a new dummy variable for an unseen level. When `bake()` encounters a level that was not represented in the learned encoding, it will generally produce an error. The model cannot simply accept a new category without a defined preprocessing rule.

One solution is to use `step_novel()` before `step_dummy()`. This step maps previously unseen categorical levels to a designated novel level, which can then be represented by the dummy encoding.

```
insurance_recipe_novel <- recipe(
  charges ~ age + bmi + smoker,
  data = insurance_train
) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

insurance_prepped_novel <- prep(
```

```
  insurance_recipe_novel,
  training = insurance_train
)
```

Here, `step_novel()` creates a designated level for previously unseen categories before dummy variables are generated. The novel level is not learned as an ordinary observed category from the training data; its purpose is to provide a defined representation for new levels during preprocessing.

However, creating a dummy variable for a novel level does not teach the predictive model how that category relates to the outcome. If the novel level never occurs in the training data, its dummy column will be zero throughout model fitting, so its effect cannot be estimated from the training data alone. Novel-level handling prevents a preprocessing failure, but it does not guarantee reliable predictions for genuinely new categories.

8.6.3 Practical Considerations

- Use `step_dummy()` when the chosen model requires numeric predictors or when a consistent numeric representation is otherwise appropriate.
- Use `step_novel()` when new data may contain categorical levels that were absent from the training data.
- Keep preprocessing inside a recipe so that training and new data are transformed consistently.
- Check the requirements of the chosen modeling engine. Dummy encoding is not universally necessary, and some models can handle categorical predictors directly.

The key principle is that preprocessing should be determined by the modeling method and the data expected at prediction time, rather than applied automatically to every categorical variable.

8.7 Combining Preprocessing Steps into a Recipe

In practice, a model may require several preprocessing steps. For example, when predicting insurance charges using age, BMI, and smoking status, we may need to impute missing values, convert the categorical predictor `smoker` into dummy variables, and normalize the numeric predictors.

The `recipes` package allows us to combine these operations into a single recipe. The following example illustrates the complete preprocessing workflow.

```
library(tidymodels)

insurance_recipe <- recipe(
  charges ~ age + bmi + smoker,
  data = insurance
) %>%
  step_impute_median(all_numeric_predictors()) %>%
  step_impute_mode(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())
```

The steps are applied in the order in which they appear in the recipe:

1. **Imputation:** Replace missing numeric predictor values with their medians and missing categorical predictor values with their most frequent levels.
2. **Dummy encoding:** Convert `smoker` into a numeric indicator variable so that it can be used by models requiring numeric predictors.
3. **Normalization:** Standardize the numeric predictors using their estimated means and standard deviations.

The response variable `charges` is not imputed, dummy-encoded, or normalized because the recipe steps target predictors only.

8.7.1 Preparing and Applying the Recipe

The recipe is prepared using the training data. During this stage, the required imputation values and normalization parameters are estimated.

```
insurance_prep <- prep(insurance_recipe, training = insurance)

insurance_processed <- bake(insurance_prep, new_data = NULL)
```

Here, `bake()` applies the prepared recipe to the training data and returns the processed dataset. The same prepared recipe can also be applied to validation, test, or future data, using the same imputation values and normalization parameters estimated from the training data.

Key point: Preprocessing must be learned from the training data and then applied consistently to other datasets. This prevents information from the evaluation data from influencing the preprocessing process.

8.8 Exercise: Preprocessing with Recipes

In this exercise, you will use the `insurance` dataset to create a recipe for predicting `charges` from `age`, `bmi`, and `region`. Unlike the previous example, which included `smoker`, this exercise focuses on converting `region` into dummy variables.

8.8.1 Tasks

1. Split the `insurance` dataset into training and testing sets, using 70% of the observations for training and 30% for testing.
2. Define a recipe for predicting `charges` using `age`, `bmi`, and `region`.
3. Use `step_dummy()` to convert the categorical predictor `region` into dummy variables.
4. Use `prep()` to estimate the preprocessing steps from the training data.
5. Use `bake()` to obtain the preprocessed training data.
6. Use `bake()` to apply the same preprocessing to the testing data.

8.8.2 Expected Outcome

- The categorical predictor `region` is represented by dummy variables in both datasets.
- The numeric predictors `age` and `bmi` remain unchanged.
- The same preprocessing learned from the training data is applied to the testing data, avoiding the need to estimate preprocessing steps separately for each dataset.
- The resulting data can be used to fit models whose input requirements are compatible with these predictors.

8.9 Building Workflows with tidymodels

In `tidymodels`, a **workflow** combines preprocessing steps, defined in a recipe, with a model specification in a single object. This makes the modelling process easier to manage and helps ensure that preprocessing is applied consistently during model fitting and prediction.

In the previous section, we learned how to prepare data using `recipes`. We can now combine a recipe with a model to create a complete modelling workflow.

8.9.1 Why Combine Preprocessing and Modelling?

Suppose we want to predict insurance charges using `age`, `bmi`, and `smoker`. Before fitting a linear regression model, we must ensure that the predictors are represented appropriately. For example, the categorical variable `smoker` must be converted into numeric indicator variables for this modelling approach.

Without a workflow, we would need to prepare the data separately and ensure that the same transformations are applied whenever we use the model to make predictions. A workflow keeps the preprocessing instructions and model specification together, reducing the risk of inconsistent data preparation.

A workflow is particularly useful when a modelling process involves several preprocessing steps, multiple candidate models, or repeated predictions on new data.

8.9.2 Creating a Workflow

The `workflows` package provides three functions for constructing a workflow:

- `workflow()` creates an empty workflow.
- `add_recipe()` adds a recipe containing the preprocessing steps.
- `add_model()` adds a model specification.

We can combine these functions using the pipe operator (`|>` or `%>%`) to create a workflow object.

8.9.3 Example: Building a Linear Regression Workflow

We will use the `insurance` dataset and predict `charges` from `age`, `bmi`, and `smoker`. To focus on workflows, we assume that the dataset has already been imported and that any required data cleaning has been completed.

First, define a recipe for the predictors. Because `smoker` is categorical, we use `step_dummy()` to create indicator variables.

```
library(tidymodels)

insurance_recipe <- recipe(
  charges ~ age + bmi + smoker,
  data = insurance
) %>%
  step_dummy(all_nominal_predictors())
```

Next, specify the linear regression model. The function `linear_reg()` defines the model, while `set_engine("lm")` specifies that the model will be fitted using R's standard linear modelling function, `lm()`.

```
lm_model <- linear_reg() %>%
  set_engine("lm")
```

We can now combine the recipe and model specification into a workflow.

```
insurance_workflow <- workflow() %>%
  add_recipe(insurance_recipe) %>%
  add_model(lm_model)

insurance_workflow
```

```
== Workflow ====================================================================
Preprocessor: Recipe
Model: linear_reg()

-- Preprocessor ----------------------------------------------------------------
1 Recipe Step

* step_dummy()

-- Model -----------------------------------------------------------------------
Linear Regression Model Specification (regression)

Computational engine: lm
```

The object `insurance_workflow` contains both the preprocessing instructions and the model specification. At this stage, the model has not yet been fitted.

8.9.4 Fitting the Workflow

We fit the workflow using `fit()`, supplying the data on which the model should be trained.

```
lm_fit <- insurance_workflow %>%
  fit(data = insurance)
```

During fitting, the workflow prepares the recipe using the supplied data, applies the resulting preprocessing steps, and fits the linear regression model to the processed data. The fitted workflow, `lm_fit`, retains the trained model and the estimated preprocessing information.

We can inspect the estimated regression coefficients using `extract_fit_parsnip()` and `broom::tidy()`.

```
lm_fit %>%
  extract_fit_parsnip() %>%
  tidy()
```

```
# A tibble: 4 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)  -11672.     938.      -12.4 1.07e-33
2 age             259.      11.9      21.7 9.36e-90
3 bmi             323.      27.5      11.7 2.51e-30
4 smoker_yes    23822.     413.       57.7 0
```

The output contains the estimated coefficients for the fitted linear regression model. Their interpretation follows the same principles as in the earlier linear regression chapter.

Notice that we do not need to create dummy variables manually before fitting the model. The workflow applies the transformations defined in the recipe automatically.

8.9.5 9.5 Generating Predictions with `predict()`

Once the workflow has been fitted, we can use `predict()` to generate predictions for new observations. The argument `new_data` specifies the data for which predictions are required.

The new data should be treated as unseen data: these observations should not have been used to fit the model when the objective is to evaluate its predictive performance. This allows us to assess how well the model predicts outcomes for observations it has not encountered during training.

For illustration, we can generate predictions for the first five observations in the dataset. However, if these observations were also used to fit the model, the predictions would not represent a genuine assessment of out-of-sample predictive performance. This distinction is important when using training and test sets to assess predictive performance.

```r
new_observations <- insurance %>%
  slice(1:5)

predictions <- predict(
  lm_fit,
  new_data = new_observations
)

predictions
```

```
# A tibble: 5 x 1
   .pred
   <dbl>
1 26081.
2  3894.
3  6240.
4  4215.
5  5948.
```

The output is a tibble containing the predicted values in a column named `.pred`.

The workflow automatically applies the preprocessing steps learned during fitting before generating predictions. For example, the categorical variable `smoker` is converted into the dummy variables required by the fitted model. We do not need to repeat these transformations manually.

In practice, predictions should generally be generated for observations that were not used to fit the model when the objective is to assess predictive performance. We will examine this process in the next section.

8.9.6 Key Idea

A workflow combines the recipe and model specification into a single object. After fitting, the resulting object retains both the trained model and the preprocessing information needed to generate predictions consistently.

This provides a reproducible modelling process: define the preprocessing steps, specify the model, combine them into a workflow, fit the workflow, and use the fitted workflow for prediction.

8.10 9. Using Workflows for Predictive Modelling

In practice, a model is often used to predict outcomes for observations that were not available during model fitting. To assess how well a model generalizes to new observations, we divide the data into a **training set** and a **test set**.

The training set is used to estimate the preprocessing parameters and fit the model. The test set is kept separate until prediction, allowing us to assess the model using unseen observations.

8.10.1 Fitting a Workflow on Training Data

We begin by splitting the `insurance` dataset into training and test sets, using 80% of the observations for training and the remaining 20% for testing.

```
library(tidymodels)

set.seed(123)

insurance_split <- initial_split(insurance, prop = 0.8)
insurance_train <- training(insurance_split)
insurance_test  <- testing(insurance_split)
```

Next, we define the recipe and linear regression model, as in the previous section. The recipe converts the categorical predictor `smoker` into dummy variables.

```
insurance_recipe <- recipe(
  charges ~ age + bmi + smoker,
  data = insurance_train
) %>%
  step_dummy(all_nominal_predictors())

lm_model <- linear_reg() %>%
  set_engine("lm")
```

We combine the recipe and model into a workflow and fit it using only the training data.

```
insurance_workflow <- workflow() %>%
  add_recipe(insurance_recipe) %>%
  add_model(lm_model)

lm_fit <- insurance_workflow %>%
  fit(data = insurance_train)
```

During fitting, the workflow estimates the required preprocessing parameters from the training data and fits the linear regression model using the processed predictors. The test data are not used in this process.

8.10.2 Generating Predictions on Test Data

Once the workflow has been fitted, we use `predict()` to generate predictions for the test observations.

```r
insurance_test_predictions <- predict(
  lm_fit,
  new_data = insurance_test
)

insurance_test_results <- insurance_test %>%
  bind_cols(insurance_test_predictions)

head(insurance_test_results)
```

```
# A tibble: 6 x 8
    age sex      bmi children smoker region    charges  .pred
  <dbl> <chr>  <dbl>    <dbl> <chr>  <chr>       <dbl>  <dbl>
1    56 female  39.8        0 no     southeast  11091. 15538.
2    27 male    42.1        0 yes    southeast  39612. 33083.
3    60 female  36.0        0 no     northeast  13229. 15300.
4    30 female  32.4        1 no     southwest   4150.  6665.
5    63 female  23.1        0 no     northeast  14452. 11869.
6    19 female  28.6        5 no     southwest   4688.  2698.
```

The resulting dataset contains the original test observations and a `.pred` column containing the predicted insurance charges.

Notice that we do not need to apply `bake()` or create dummy variables manually. The fitted workflow automatically applies the preprocessing steps learned from the training data before generating predictions for the test set.

This is an important advantage of workflows: the same preprocessing instructions are applied consistently to both training and new observations, while information from the test set is kept out of model fitting.

8.10.3 Assessing Predictive Performance

We can compare the predicted charges with the actual charges in `insurance_test_results` to assess predictive performance. For example, we can calculate RMSE, MAE, and (R^2) using the methods introduced in the previous chapter.

```r
insurance_test_results %>%
  metrics(truth = charges, estimate = .pred)
```

```
# A tibble: 3 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard    5554.
2 rsq     standard       0.760
3 mae     standard    3874.
```

These metrics summarize how well the model predicts charges for observations that were not used to fit it. Their interpretation is the same as discussed in the chapter on model performance.

The key distinction is that the metrics are calculated using the test set rather than the training data. This provides a more appropriate assessment of how the model may perform on future observations drawn from a similar population.

8.10.4 Exercise: Fit a Workflow and Predict Test Data

Using the `insurance` dataset, complete the following tasks.

1. Split the data into training and test sets using an 80:20 ratio and `set.seed(123)`.
2. Define a recipe to predict `charges` using `age`, `bmi`, and `smoker`. Convert categorical predictors into dummy variables.
3. Specify a linear regression model and combine it with the recipe using `workflow()`.
4. Fit the workflow using only the training data.
5. Generate predictions for the test set using `predict()`.
6. Combine the predictions with the actual charges and inspect the results.
7. Calculate RMSE, MAE, and (R^2) for the test set. Interpret the results using the concepts from the model-performance chapter.